NPTEL Cloud Computing

IIT Kharagpur · Prof. Soumya Kanti Ghosh

Complete exam notes — 60 lectures, 12 weeks

Topic-wise notes · formula sheet · classification sheet · high-yield list · your practice sets (400 Q + answer keys) · practice MCQs · last-minute revision

Built from the lecture transcripts

Contents

Week 1 — Overview, Introduction and Cloud Architecture (Lectures 1–5)

1.0 Course roadmap — what L1 previews (L1)
1.1 What “computing” means and how cloud evolved (L1–L2)
1.2 NIST definition and characteristics of cloud (L2–L3)
1.3 Service-Oriented Architecture view (L3–L5)
1.4 Service models — IaaS, PaaS, SaaS, XaaS (L3–L5)
1.5 Deployment models (L3; detailed again in Week 2, L6)
1.6 Virtualization basics (L3; detailed in Week 2, L7)
1.7 Cloud computing architecture (L4–L5)
Week 1 — High-yield takeaways

Week 2 — Deployment Models, Virtualization, XML, Web Services & SOA (Lectures 6–10)

2.1 Deployment models in detail (L6)
2.2 IaaS and virtualization architecture (L7)
2.3 XML basics (L8–L9)
2.4 Web services and Service-Oriented Architecture (L10)
Week 2 — High-yield takeaways

Week 3 — SLA, Cloud Economics, Managing Data, MapReduce, OpenStack (Lectures 11–15)

3.1 Service Level Agreement (L11)
3.2 Cloud economics — “cloudonomics” (L12)
3.3 Managing data in the cloud (L13)
3.4 MapReduce (L14)
3.5 OpenStack (L15; demo in Week 4, L16)
Week 3 — High-yield takeaways

Week 4 — OpenStack Demo, Microsoft Azure, Google Cloud Platform (Lectures 16–20)

4.1 OpenStack demo — Meghamala, IIT Kharagpur (L16)
4.2 Microsoft Azure (L17–L18)
4.3 Google Cloud Platform (L19–L20)
Week 4 — High-yield takeaways

Week 5 — Tutorials (SLA, Economics, MapReduce) and Resource Management (Lectures 21–25)

5.1 SLA tutorial (L21)
5.2 Economics tutorial (L22)
5.3 MapReduce tutorial (L23)
5.4 Resource management in the cloud (L24–L25)
Week 5 — High-yield takeaways

Week 6 — Cloud Security I–IV and Broker for Cloud Marketplace (Lectures 26–30)

6.1 Security fundamentals (L26)
6.2 What is different about cloud security (L27)
6.3 Case study: “Hey, You, Get Off of My Cloud” (L28)
6.4 Security in collaborative SaaS clouds (L29)
6.5 Broker for cloud marketplace (L30)
Week 6 — High-yield takeaways

Week 7 — Mobile Cloud Computing, Fog Computing, Geospatial Cloud (Lectures 31–35)

7.1 Mobile Cloud Computing — basics (L31)
7.2 Mobile Cloud Computing — partitioning and offloading (L32)
7.3 Fog computing — concept (L33)
7.4 Fog computing — comparison, use cases, resource management (L34)
7.5 Use case: Geospatial cloud (L35)
Week 7 — High-yield takeaways

Week 8 — Docker Introduction, Green Cloud, Sensor Cloud, IoT Cloud, Course Summary (Lectures 36–40)

8.1 Introduction to Docker containers (L36)
8.2 Green Cloud (L37)
8.3 Sensor Cloud (L38)
8.4 IoT Cloud (L39)
8.5 Course summary and research areas (L40)
Week 8 — High-yield takeaways

Week 9 — Cloud–Fog–Edge Paradigm, Resource Management, Cloud Federation, VM Migration Basics (Lectures 41–45)

9.1 Cloud–Fog computing overview and health case study (L41)
9.2 Resource management in cloud–fog–edge — I (L42)
9.3 Resource management in cloud–fog–edge — II: service placement (L43)
9.4 Cloud federation (L44)
9.5 VM migration — basics and strategies (L45)
Week 9 — High-yield takeaways

Week 10 — VM Migration Analysis, Containers & Kubernetes, Docker Architecture and Demo (Lectures 46–50)

10.1 VM migration — analysis and multi-VM strategies (L46)
10.2 Containers and Kubernetes (L47)
10.3 Docker — overview, benefits, architecture (L48)
10.4 Docker demo — install, build, ship, run (L49–L50)
Week 10 — High-yield takeaways

Week 11 — Dew Computing, Serverless Computing I–II, Sustainable Cloud Computing I–II (Lectures 51–55)

11.1 Dew computing (L51)
11.2 Serverless computing — concepts (L52)
11.3 Serverless computing — platforms (L53)
11.4 Sustainable cloud computing — I (L54)
11.5 Sustainable cloud computing — II: taxonomy (L55)
Week 11 — High-yield takeaways

Week 12 — Cloud in the 5G Era, CPS & Cloud, Case Studies: Spatial Cloud and Internet of Health Things (Lectures 56–60)

12.1 Cloud computing in the 5G era (L56)
12.2 Cyber-Physical Systems and cloud computing (L57)
12.3 Case Study I — Spatial cloud computing (L58)
12.4 Case Study II — Internet of Health Things (IoHT) (L59–L60)
Week 12 — High-yield takeaways

Formula Sheet — all numerical tools in the course

A. SLA and availability (Weeks 3, 5)
B. Cloud economics (Weeks 3, 5)
C. Reliability and distributed storage (Week 3, 5)
D. Parallel computing and MapReduce (Weeks 3, 5)
E. VM migration (Weeks 9–10)
F. Energy and green cloud (Weeks 5, 8, 11)
G. Mobile cloud computing — offloading (Week 7)
H. Sensor/IoHT signal processing (Week 12)
I. Quick unit conversions

Classification & Comparison Sheet

1. NIST model and cloud basics
2. Distributed paradigms
3. Service models
4. Deployment models
5. Virtualization
6. XML and web services
7. SLA
8. Data management systems
9. Platforms and their components
10. Security
11. Cloud vs fog vs edge vs dew
12. MCC
13. Resource management
14. Federation
15. VM migration, containers
16. Serverless
17. Sensor cloud, IoT, 5G, CPS
18. Green / sustainable cloud

High-Yield Revision List

1. Numbers and counts to remember
2. One-line distinctions
3. “If the question mentions X → think Y”

Your practice sets — 400 questions checked against the lectures

Week 1 · L1 — Cloud Computing — Overview your file w10 · 50 questions
Week 2 · L6 — Deployment Models your file w11 · 50 questions
Week 3 · L11 — Service Level Agreement your file w12 · 50 questions
Week 4 · L16 — OpenStack demo (Meghamala) your file w1 · 50 questions
Week 5 · L21 — SLA tutorial your file w2 · 50 questions
Week 6 · L26 — Security I your file w3 · 50 questions
Week 7 · L31 — Mobile Cloud Computing I your file w4 · 50 questions
Week 8 · L36 — Introduction to Docker your file w5 · 50 questions

Practice MCQs — check yourself (answers with explanations)

GAAND FATT RAHI HAII😬 .....

Course at a glance
Week 1 — must-remember
Week 2 — must-remember

Week 3 — must-remember
Week 4 — must-remember
Week 5 — must-remember
Week 6 — must-remember
Week 7 — must-remember
Week 8 — must-remember
Week 9 — must-remember
Week 10 — must-remember
Week 11 — must-remember
Week 12 — must-remember
Recurring numerical patterns
Formula strip
Top traps

Week 1 — Overview, Introduction and Cloud Architecture (Lectures 1–5)

> **Source:** L1 Cloud Computing – Overview · L2 Overview (contd.) · L3 Cloud Computing – Introduction · L4 Cloud Computing Architecture · L5 Cloud Computing Architecture (contd.)
>
> Week 1 is almost entirely conceptual. Most questions here test **definitions, lists (NIST characteristics, service/deployment models), classification of examples, and "where is X useful / not useful"**.

1.0 Course roadmap — what L1 previews (L1)

> 🎯 **Your practice set for L1** (your file *w10*): 50 questions · 12 easy · 28 medium · 10 hard. Answers checked against this lecture.
> Full set and answer key: chapter "Your practice sets", Week 1.

> Added after checking your practice set for L1 (file *w10*). About a third of that set asks about this roadmap, so it is worth one read.

- **Course objective:** give an **overall picture of what cloud computing is, its major components and the recent trends**. The first lectures introduce the **NIST model, properties and characteristics**. Then come architecture, service models (**IaaS, PaaS, SaaS — "anything as a service", XaaS**) and **deployment models**, with the stack "moving towards **SOA**".
- **SLA** = agreement between **service provider and consumer**.
  - The **consumer** wants a *guarantee of minimal services*.
  - The **provider** must work out the *back-end resources* needed to honour it.

    - Example: during an **exam**, availability must be **≈ 100 %** (redundant services); during a **practice session** the requirement may come down. The provider sizes resources, and **charges**, according to the availability promised. **Higher availability → more resources → higher price.**
- **Cloudonomics** (economics of cloud) asks: is the cloud *always* beneficial, i.e. **buy or hire** resources?
    - Analogy: if you commute a long way **every day**, buying a car pays off. If you drive 50–100 km **once a month**, **hiring** is cheaper.
    - So for occasional or spiky demand, hire (cloud).
- **Resource management** is mainly a **provider-side** problem. Resources: **CPU, RAM, disk, network connectivity**.
    - **Over-provisioning** → idle resources → extra **cost**.
    - **Under-provisioning** → **starvation**.
    - Analogy: a stationery shop must neither run out of stock nor overflow. Getting this right is a "serious challenge".
- **Data management:** is the **conventional DBMS approach still good for cloud**? Storage as a service, e.g. **Dropbox**: what has to be managed in the background.
- **Security** — the data and the computation sit **in someone else's domain**.
    - **"You can reinstall an application or OS, but you cannot reinstall your data."**
    - Issues: infrastructure security, data/storage security, **identity and access management (IAM)**, **access control** (e.g. **role-based access control**).
    - **Trust, reputation and risk** are the **"three nodes of a triangle"**: they are interlinked.
- **Case studies/demos: open-source and commercial clouds**, possibly a **cloud simulator**.
- **Recent trends** (fog computing etc.) and research opportunities: pointers for **UG/PG projects and research**.
- **VANET** (example of an ad-hoc distributed system): vehicles form a short-lived network for **safety-related and infotainment** services.

1.1 What "computing" means and how cloud evolved (L1–L2)

Core concept

- **Computing (ACM Computing Curricula, 2005):** any *goal-oriented activity* requiring, benefiting from, or creating computers — hardware/software systems for a wide range of purposes, making computing systems intelligent, using communications, finding and gathering information.
- Cloud computing is **not a brand-new invention**; it **evolved** from earlier paradigms that each solved a need:

`Centralized (mainframe) → Distributed computing → Grid computing / Cluster computing → Utility computing (concept) → Cloud computing (practical realization of utility computing)`

> The lecture stresses these did **not** appear strictly one after another; they are different aspects that together made cloud feasible. Distributed computing is called the "mother" of them all.

Distributed computing

- **Definition:** a field of computer science that studies distributed systems; use of distributed systems to solve computational problems.
- A distributed system = **several autonomous computational entities, each with its own local memory**, that **communicate by message passing** over a communication network.
- OS view: processors communicate over communication lines (high-speed buses, telephone lines); each processor has its own local memory.
- **Examples:** the Internet, ATM/bank networks, intranets/workgroups, ad-hoc networks such as **VANETs** (vehicular ad-hoc networks), ubiquitous network-connected devices.
- **Typical nodes:** *workstations* (end-user computing), *server systems* (provide resources/services), *personal assistance devices* (handheld devices on wireless networks).

Common properties / benefits of distributed systems

Property	Meaning
Fault tolerance	Works (maybe at lower performance) even if some nodes fail — unlike a centralized system where one failure stops everything
Each node plays a partial role	Work is divided across nodes
Resource sharing	Nodes share resources
Load sharing / load balancing	Work can be distributed among nodes
Easy to expand	Add more systems to the network as needed
Performance	Parallel computing is considered a **subset** of distributed computing

Why distributed systems are needed: (1) nature of the application demands it, (2) performance — compute-intensive or data-intensive workloads, (3) **robustness — no single point of failure**; tasks of a failed node can be executed by other nodes.

Distributed applications: a set of processes distributed across a network that work together as an ensemble to solve a common problem (needs *orchestration*). Forms: **client–server** (resource management centralized at the server; a server can itself be a client of another server) and **peer-to-peer** (a move toward truly distributed applications built from peer processes).

Grid computing

Three definitions given in the lecture (all examinable):

1. A form of networking that, unlike conventional networks focused on communication among devices, **harnesses unused processing cycles of all computers in a network** to solve problems too intensive for a standalone machine.
2. Grid computing **enables virtualization of distributed computing and data resources** (processing, network bandwidth, storage) **to create a single system image**, giving users and applications seamless access to vast IT capabilities. *(This is a key stepping stone to cloud.)*
3. A computing infrastructure that provides **dependable, consistent, pervasive and inexpensive access** to computational capabilities.

- **Analogy: the electrical power grid** — users tap computing resources (processors, storage, data, applications) as needed, with little or no knowledge of where they are located.
- **Characteristics:** efficient use of (often under-utilized) resources across organizations — e.g., UG/PG lab PCs pooled after lab hours for research via a middleware layer; joining local communities (biology, genetics etc.); interaction with underlying layers must be **transparent and seamless** to the user; get resources **without having to purchase, install and maintain** them.
- **Who uses it:** mainly the scientific community — physics, weather, material science, reactor applications; data analysis/visualization, simulation and modelling, collaboration across continents; exploiting under-utilized resources.

Types of grids

Grid type	Focus
Computational grid	Computing power (processing cycles)
Data grid	Data storage/access
Collaborative grid	Collaboration among users/groups
Network grid	Fault-tolerant, high-performance communication services
Utility grid	Not only data and cycles but **software or just about any resource** is shared

Grid players: the grid itself, users, groups, heterogeneous sites (resource locations), and policies (what can be shared, where things run).

Cluster computing

- **Definition:** a type of parallel or distributed system consisting of a **collection of interconnected stand-alone computers working together as a single integrated computing resource**.
- **Key components:** stand-alone computers (PCs, workstations, SMPs), operating systems, **high-performance interconnects**, middleware, parallel programming environments, applications.
- Deployed to **improve speed and/or reliability** over a single computer.
- **Typical properties:** network is faster/closer than a typical LAN; **low-latency** communication protocols; **more loosely coupled than SMPs**.

Cluster type	Purpose
High-availability (failover) cluster	If a node fails, others take over
Load-balancing cluster	Distributes load across nodes
Parallel/distributed processing cluster	Runs parallel and distributed jobs

Operational benefits: system availability, fault tolerance, scalability, OS and application reliability, high performance.

Utility computing

- **Utility** = get something as and when you need it, without caring how it is produced (railway/flight ticket via an agent, plumbing via an office).
- **Definition:** a **service provisioning model** in which a service provider makes computing resources and infrastructure management available to the customer **as needed**, and **charges for specific usage rather than a flat rate**.
- Named by analogy with **electricity** (metered; you don't care how power is generated) and **telecom/mobile** services.
- Pricing: **pay-per-use / pay-as-you-go**, metered.
- Utility computing is "purely a **concept** which **cloud computing practically implements**."
- Viewed as a form of **virtualization**: storage/computing available appears much larger than any single system ("open the tap when needed, close it when not").
- **Data-center virtualization and provisioning:** carve customized virtual instances (e.g., a VM with 16 GB RAM) from a large pool; solves the resource-utilization problem, enables outsourcing and service-level delivery, and allows **automation** (workflows).
- Shift noted in the lecture: from a **data-driven** architecture to a **service-driven** architecture.
- **Example — on-demand cyber infrastructure:** temporary infrastructure for an online exam, a conference/event, or disaster management (real time) — provision when needed, release after.

Provider–consumer issues that tie them together: pricing, SLA (availability, downtime), security (incl. DoS protection), access mechanisms, **application sizing** (size the application to *my* need).

Payment models: flat rate, tiered, subscription, and customized pricing per customer based on scale, commitment, payment frequency. *"The pricing model is simply an expression of the provider's costs."*

Risks of utility computing (all five are list-type MCQ material):

1. **Data backup** — what if the provider crashes or goes out of business?
2. **Data security** — is someone else reading my data?
3. **Competence** — is the provider competent?
4. **Defining SLA** — each side wants it in their favour; no standard approach.
5. **Getting value from charge-back.**

> These evolved into the full cloud computing model.

MCQ traps (1.1)

- Grid = harnessing **unused cycles** / **single system image** across organizations; Cluster = **interconnected stand-alone computers acting as one integrated resource**, usually tightly managed, low latency. Don't swap the definitions.
- "Clusters are more **loosely** coupled than SMPs" — true (an SMP is tightly coupled shared memory).
- Utility computing is a **concept/business model**; cloud computing is its **practical realization**.
- Parallel computing is a **subset** of distributed computing (lecture's view), not the other way round.
- Distributed-system nodes have **their own local memory** and communicate by **message passing** (not shared memory).

Quick revision (1.1)

- Evolution: distributed → grid & cluster → utility → cloud.
- Grid analogy = electric grid; cluster types = HA/failover, load balancing, parallel processing.
- Utility computing = metered, pay-as-you-go, charges for usage not flat rate.
- Five risks of utility computing: backup, security, competence, SLA definition, charge-back value.

1.2 NIST definition and characteristics of cloud (L2–L3)

NIST definition (memorize the structure)

> *Cloud computing is a model for enabling* ***ubiquitous, convenient, on-demand network access*** *to a* ***shared pool of configurable computing resources*** *(e.g., networks, servers, storage, applications, services) that can be* ***rapidly provisioned and released with minimal management effort or service provider interaction****.*

Key phrases tested: *shared pool*, *configurable*, *rapidly provisioned and released*, *minimal management effort or service-provider interaction* → this is what makes pay-as-you-go possible (provision when needed, de-provision when not).

Five essential characteristics (NIST)

Characteristic	What it means (lecture wording)
On-demand self-service	Consumer provisions resources as needed, by itself
Broad network access	Resources are geographically distributed and pooled, so good network access is essential; accessed through standard clients
Resource pooling	Resources pooled to serve many consumers — a **multi-tenant model**, physical and virtual resources **dynamically assigned** (e.g., one user gets a Windows VM, another an Ubuntu VM from the same provider)
Rapid elasticity	Scale up *and* down quickly (e.g., grow a VM from 4 GB to 8 GB RAM while a simulation needs it, then shrink)
Measured service	Usage is metered — "I pay for what I use"

> **NIST model also has 3 service models (SaaS, PaaS, IaaS) and 4 deployment models (private, public, hybrid, community).** *(Standard NIST structure; the lecture presents all of these.)*

Common characteristics (beyond NIST) listed in L3

Massive scale · resilient computing · homogeneity (user sees homogeneous resources even if back end is heterogeneous) · geographic distribution · **virtualization** · **service orientation** · low-cost software (multi-tenancy lowers cost per consumer) · advanced security (access control, attack blocking, no data loss).

Advantages of cloud (L3 list)

Lower computing cost · improved performance · reduced software costs · instant software updates · improved document-format compatibility · (theoretically) unlimited storage · increased data reliability (distributed with redundancy) · universal information access · latest versions available · easier group collaboration · device independence.

Disadvantages of cloud (L3 list)

Requires a **constant Internet connection** · does not work well with **low-speed connections** · features may be limited · can be slow (provisioning takes time) · stored data might not be secure · stored data can be lost (crash, provider exit) · **cloud is not truly a high-performance computing (HPC) system** · general concerns (APIs, database connectivity).

Why cloud-sourcing is becoming important

High-scale/low-cost providers · anytime-anyplace access via web browser · rapid scalability, **incremental cost** and load sharing · (but you still need to focus on local IT).

Concerns with cloud-sourcing: performance, reliability and SLAs · control of data and service parameters (how is "95 % availability" audited?) · application features and choices (fixed, not customized) · interaction between cloud providers · **no standard API** (SOAP vs REST mix) · security, privacy, compliance, trust, competence.

Cloud storage: data stored remotely and **temporarily cached** on desktops, phones and other Internet-linked devices (sync); examples cited: Amazon **EC2/S3**, Dropbox.

MCQ traps (1.2)

- There are **five** NIST essential characteristics. "Virtualization", "multi-tenancy" and "service orientation" are **not** among the five (multi-tenancy is how *resource pooling* is achieved; virtualization is a common characteristic/enabler).
- **Elasticity ≠ scalability alone:** elasticity includes scaling **down/release** as well as up.
- "Cloud is a high-performance computing system" → **False** per lecture.
- "Cloud works well with low-speed connections" → **False**.

Quick revision (1.2)

- NIST: 5 characteristics + 3 service models + 4 deployment models.
- Mnemonic for the 5 characteristics: **"O-B-R-R-M"** → On-demand self-service, Broad network access, Resource pooling, Rapid elasticity, Measured service.

1.3 Service-Oriented Architecture view (L3–L5)

- Cloud's major aim is to **realize a service-oriented architecture (SOA)**.
- **SOA triangle:** *service provider* publishes to a **registry/catalogue** → *service consumer* looks up (finds) the service in the registry → consumer **binds/invokes** the provider. Registry is distributed but **logically centralized**. Analogy: **telephone directory / yellow pages**.
- **Client–server (legacy) vs SOA:** client–server is tightly bound (every service needs its matching client and server program); SOA lets heterogeneous services talk to each other with more flexibility. (Detailed SOA/Web services: Week 2, L10.)

Client–server model details (L5)

- Server process **listens on a well-known port** (e.g., `httpd` on **port 80**; also `ftpd`, `telnetd`); client connects to that port. Client and server can be the same or different machines.
- **Concurrent server** (serves many clients at once) vs **iterative server** (one after another).
- Request–response messaging; clients often thin; server may be a load-balanced cluster or standalone.
- Realizes a **three-tier architecture**: presentation (top) → logic → data (bottom).

Client–server vs cloud (L5 comparison — classic MCQ)

Aspect	Client–server	Cloud
Service model	Simple: server serves client requests	Variety of complex models (IaaS, PaaS, SaaS…)
Load balancing	May or may not be load balanced	Load balanced
Scalability	Scalable to some extent (in a cluster)	**Theoretically infinitely scalable**
Virtualization	**No concept of virtualization**	Virtualization is the **core concept**

1.4 Service models — IaaS, PaaS, SaaS, XaaS (L3–L5)

Core concept

Model	What the consumer gets	Typical components in the lecture's stack
IaaS	Raw infrastructure — servers, storage, network, OS — **on demand**, usually as VMs	Virtual machines, servers, storage, load balancers, network
PaaS	A virtualized **development and runtime platform** to build, test, deploy, host apps	Execution runtime, database, web server, development tools, sandbox
SaaS	Ready-to-use application over the Internet, pay-as-you-go/subscription	CRM, email, virtual desktop, games, word processing, spreadsheets
BPaaS	Horizontal or vertical **business processes** on a subscription basis	Orchestrated processes/workflows (e.g., BPEL)

- **Transportation analogy (L5):** roads = **infrastructure**; cars and trucks = **platform**; goods and people = **software/information**.
- **Client access:** web browser (universal), mobile apps, thin clients, terminal emulators.

SaaS — characteristics, where useful, where not (L5)

Characteristics: web access to commercial software · software managed from a **(logically) central location** · delivered in a **one-to-many** model · **users not required to handle upgrades/patches** · APIs allow integration between pieces of software.

Useful when: significant interplay between the organization and the outside world (e.g., **email, newsletter campaign** software) · need for web and mobile access · software needed only for **short-term** use (e.g., a one-off online recruitment test) · **demand spikes** significantly (e.g., **tax, billing** at year end). Example: **Salesforce CRM**.

Not the best option when: extremely fast processing / real-time data is required (e.g., disaster management) · **legislation or regulation does not permit data to be hosted externally** · an **existing on-premise solution** already fulfils all needs.

PaaS — characteristics, where useful, where not (L5)

Characteristics: services to **develop, test, deploy, host and maintain** applications in the **same integrated development environment** · web-based UI tools to create/modify/test/deploy · **multi-tenant architecture** (many concurrent users use the same development application) · **built-in scalability** · integration with web services and databases · **support for development-team collaboration** · tools for billing and subscription management.

Useful when: multiple developers work on a project (possibly across countries) · **external parties** need to interact with the development process · developers want to **automate testing and deployment** · **agile** (iterative, incremental) development. Examples cited: **Microsoft Azure, Google App Engine**.

Not ideal when: the application must be **highly portable** in terms of where it is hosted · **proprietary languages/approaches** would impact development · a proprietary language would **hinder later moves to another provider → vendor lock-in** · application performance needs **customization of underlying hardware and software**.

IaaS — characteristics, where useful, where not (L5)

Characteristics: resources distributed **as a service** · **dynamic scaling** · **variable cost** (utility pricing) · **multiple users on a single piece of hardware**.

Useful when: demand is **very volatile** (spikes) · **new organizations without capital** to invest · pressure to **limit capital expenditure** · specific line-of-business, **trial or temporary** infrastructure needs (e.g., proof of concept).

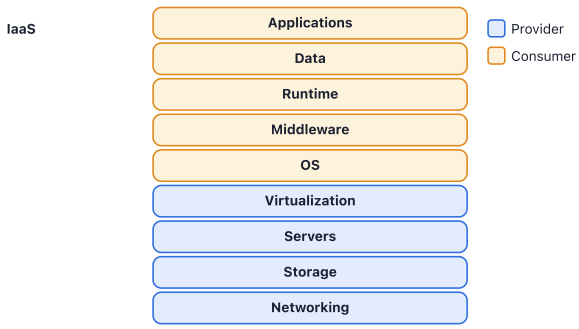
Not the best option when: regulatory compliance makes outsourcing of storage/processing difficult · **highest levels of performance** are required and on-premise/dedicated infrastructure can meet them (virtualization layers consume some resources).

Provider comparison tables shown in L5 (recognition level)

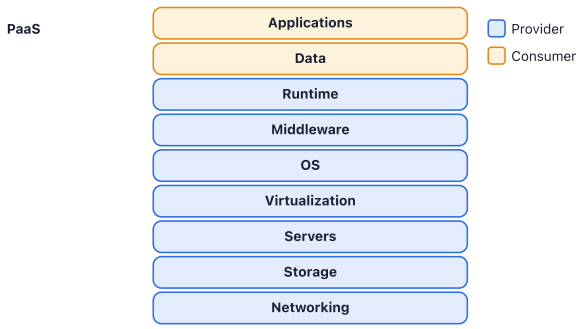
- **SaaS providers** differ in **pricing model** — some **pay-as-you-go**, some **free**, some **subscription** (e.g., pay per year).
- **PaaS providers** differ in **target use** (e.g., .NET platform, web development, enterprise applications) and **database** offered (e.g., BigTable-based).
- **IaaS providers** differ in **hardware capacity**, **operating systems** supported and **billing mechanism**.

Who manages what (responsibility split, L5)

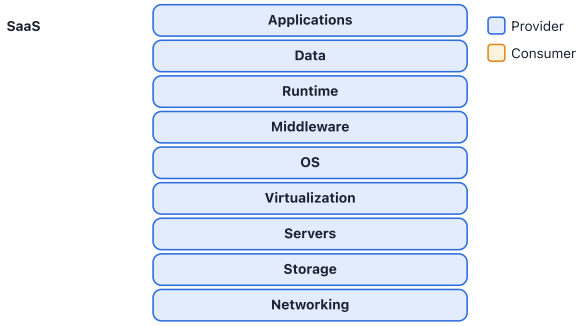
Who manages which layer? diagram sequence



1. IaaS: provider manages up to **virtualization** (VMs). Guest OS may be supplied or chosen; OS, middleware, runtime, data and apps are the consumer's.



2. PaaS: provider manages up to the **runtime/middleware**; the consumer manages only **data and applications**.



3. SaaS: provider manages the **whole stack up to the application**; the consumer just uses it.

Layer (bottom → top)	IaaS	PaaS	SaaS
Networking, storage, servers	Provider	Provider	Provider
Virtualization	Provider	Provider	Provider
OS	Consumer (provider may supply guest OS or let you choose)	Provider	Provider
Middleware, runtime	Consumer	Provider	Provider
Data, applications	Consumer	**Consumer**	Provider

- SaaS: provider responsible **up to the application**.
- PaaS: provider responsible **up to the runtime**; data and applications are the user's.
- IaaS: provider responsible **up to virtualization/providing VMs**.

Networking responsibility on the OSI stack (L5)

Model	Provider responsible up to...
IaaS	**Data link layer** (rest is consumer's)
PaaS	**Presentation** (sometimes session) layer
SaaS	**Whole stack**

XaaS — "anything as a service"

- XaaS = generalization of cloud services delivered over the Internet rather than locally; **XaaS is the essence of cloud computing**.
- XaaS = **Service-Oriented Infrastructure (SOI) + cloud computing**.
- Instances named in L4: IaaS, PaaS, SaaS, **BPaaS**, storage-aaS, security-aaS, database-aaS, monitoring/management-aaS, communication/content/computing-aaS, identity-aaS, backup-aaS, desktop-aaS; also data-aaS and "science as a service" (L3).
- XaaS fulfils the four demands of a service provider: **broad network access, resource pooling, business-driven infrastructure on demand, service orientation**.
- **BPaaS:** a business process = orchestration/integration of sub-processes with **timing relationships**, forming a **process execution graph/tree** (e.g., P1, P2 → P3; P3, P4 → final output). Sub-processes may come from **different heterogeneous providers**. Language mentioned: **BPEL (Business Process Execution Language)**.

MCQ traps (1.4)

- **Lecture examples:** Amazon (EC2) → IaaS; **Microsoft Azure → PaaS**; Google Docs/spreadsheets → SaaS; **Google App Engine → PaaS**; Salesforce → SaaS (CRM). Use these classifications.

  > ⚠️ *Lecture vs today:* Azure today also offers IaaS (VMs) and SaaS; for the exam follow the lecture's classification (Azure = PaaS example).

- "Users must handle upgrades in SaaS" → **False** (provider handles them).
- Vendor lock-in is a **PaaS** concern listed in the lecture.
- "IaaS is best when highest performance is needed" → **False** (dedicated/on-premise is better then).
- Real-time/extremely fast processing → SaaS **not** suitable.
- In PaaS, the **consumer** manages data and applications.

Quick revision (1.4)

- Stack bottom→top: IaaS → PaaS → SaaS (→ BPaaS).
- SaaS = one-to-many, no upgrades for user; PaaS = develop/test/deploy in one IDE, multi-tenant, team collaboration; IaaS = dynamic scaling, variable cost, multiple users on one hardware.
- Each model has a "not suitable when" list — know them.

1.5 Deployment models (L3; detailed again in Week 2, L6)

Model	Description	Lecture example
Private	Built and used by one organization	**Meghamala** — IIT Kharagpur's private cloud on open-source software, mainly **IaaS** for researchers; accessible only inside the IIT KGP network; **no charge** (institute funded)
Public	Open to the public	Amazon (IaaS), Microsoft Azure (PaaS), Google Docs (SaaS)
Community	Shared by **several organizations with common goals** ("semi-public/semi-private")	**Banking cloud** (RBI as nodal agency), government-funded institutional cloud, **geospatial cloud**
Hybrid	Mix — private + public, or with community	—

1.6 Virtualization basics (L3; detailed in Week 2, L7)

- **Virtualization stack:** bare-metal hardware → **VMM (Virtual Machine Monitor) / hypervisor** → multiple **VMs**, each with a **guest OS** and applications.
- The VMM **emulates the hardware**; one physical machine (or several machines pooled together) can host multiple VMs of different specifications. (Pooled capacity cannot be split arbitrarily — virtualization has its own overhead/requirements.)
- **Virtual workspace:** an abstraction of an execution environment that can be **made dynamically available to authorized clients using well-defined protocols** (e.g., a 14-day project environment).
- Meghamala provisions **three categories of VMs** based on availability.
- **Popular VMMs/hypervisors cited: Xen, VMware**.
- **Advantages of VMs:** run an OS where physical hardware is unavailable · easier to create new machines and backups · software testing with clean installs · emulate more machines than are physically available · test/debug on many OSs.
- Cloud and virtualization "come hand in hand"; performance of para-virtualization can be close to raw physical performance (Week 2).

1.7 Cloud computing architecture (L4–L5)

Three-level architectural view

`Business goals → Quality attributes → Architectural tactics`

Level	Items (lecture lists)
Business goals (CSP's view)	**TCO** (total cost of ownership/to the organization), stakeholder satisfaction, compliance with standards, market share, flexibility
Quality attributes	Availability, **elasticity**, interoperability, security, adaptability, performance, usability, maintainability
Architectural tactics	**Stateless design**, **loose coupling**, heterogeneity handled via interconnection/homogeneity, **caching**, **claims-based authentication**

Major building blocks

- **Technical architecture:** structuring according to the **XaaS stack**; adopting cloud paradigms; structuring cloud services and components; **middleware and communication**; **management and security**.
- **Deployment/operation architecture: geo-location check** (legal/political — e.g., government data and mail servers must stay within the country's territory); **operation and monitoring**.

Layered view

- Bottom: **storage and infrastructure** → middle: middleware/platform (cloud runtime, libraries) → top: **applications** → client infrastructure interacts with applications.
- **Stakeholder views:** IaaS → **IT architects/IT operations**; PaaS → **software architects/developers**; SaaS → **end users/application administrators** (e.g., CRM).
- **Microsoft Azure vs Amazon EC2 mapping:** both split into infrastructure (Azure: Windows servers; Amazon: Windows/Linux + **S3, SimpleDB**), middleware/platform, and applications; client via browser (and Silverlight for Azure — historical).
- **Management, security (and QoS) are *vertical* stacks** — they cut across all layers, so they need **cross-layer** consideration. How much the provider manages depends on the service model (IaaS: up to infrastructure; SaaS: up to the application).

Classic XaaS stack (bottom → top)

`Networking · Storage · Servers (bare metal)` → `Virtualization (of machines, network, storage)` → `OS` → `Middleware` → `Runtime` → `Data` → `Applications`

- This is a **logical** representation (like OSI/TCP-IP), not a strict physical layering. Storage is often treated separately from other infrastructure because storage management is different.

Elasticity: scale up vs scale out (L4)

	Vertical scale-up	**Horizontal scale-out**
How	Add **more resources to the same box** (more blades in a chassis, more RAM/CPU)	Add **more boxes/machines**
Cost	Probably **cheaper for small scenarios**	**Only solution for large scenarios** — *one 64-way server is much more expensive than 64 one-way servers*
Failure	Single box can be a **single point of failure**	Failure of some nodes → work continues at lower performance

Legacy IT vs simplified (cloud) IT stack

- **Legacy:** end-user devices, applications and **dedicated infrastructure** tightly coupled (application sized for the infrastructure).
- **Simplified/cloud:** loosely bound layers → better resource utilization, better **ROI**, many services on a common resource base (e.g., pool idle lab PCs after hours).
- **Classical model:** licensed software, hardware products, people-based services → **XaaS model**.

Broader cloud-service view (L5)

Infrastructure/operations (compute, network, storage) → **services** (IaaS/PaaS/SaaS) → **business processes** → **presentation** (user interfaces, APIs). Cross-cutting: **integration** (service orchestration/execution engine), **QoS**, **information** (service dissemination), **governance** (management + legal/policy compliance with federal laws).

Networking in cloud (L5)

- Network is **critical** — blade/rack servers rely on highly redundant, high-speed backbone networks.
- Network resources can be **provisioned dynamically**; the whole network can be **virtualized** (e.g., set up a virtual network for an event, then release it).
- Concepts forming the core: **VLAN, VPN, protocol layers**; tools: **OpenSSH, OpenVPN**.
- **Network Function Virtualization (NFV):** implement network functions **in software** running on **industry-standard servers/switches/storage** (possibly in data centers) that can be **moved or instantiated at various locations without installing new equipment** — consolidates many network equipment types.

MCQ traps (1.7)

- Management and security are **vertical/cross-layer**, not confined to one layer.
- "Scale-up is the only solution for large-scale scenarios" → **False** (scale-out is).
- Geo-location check belongs to the **deployment/operation** architecture, not the technical architecture.

- Claims-based authentication, stateless design, loose coupling, caching = **architectural tactics**, not quality attributes. Elasticity, availability, interoperability = **quality attributes**. TCO, market share = **business goals**.
- NFV = network functions in software on **standard hardware**, not on new proprietary appliances.

Quick revision (1.7)

- Business goals → quality attributes → architectural tactics.
- Scale-up (vertical, cheaper small) vs scale-out (horizontal, only solution large; 64×1-way ≪ one 64-way).
- Provider responsibility on OSI: IaaS → data link; PaaS → presentation/session; SaaS → all.
- NFV = network functions as software on commodity servers.

Week 1 — High-yield takeaways

1. NIST: **5 characteristics** (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service), **3 service models**, **4 deployment models**.
2. Utility computing = concept (metered, pay-per-use); cloud = practical implementation.
3. Cluster = interconnected stand-alone computers as one resource; Grid = harness unused cycles / single system image; analogy = electric grid.
4. Client–server has **no virtualization**; cloud's core is virtualization and it is (theoretically) infinitely scalable.
5. SaaS not for real-time/extremely fast processing; PaaS has vendor lock-in risk; IaaS not when highest dedicated performance or regulation forbids outsourcing.
6. Responsibility: IaaS provider up to virtualization; PaaS up to runtime; SaaS up to application.
7. Scale-out beats scale-up for large systems; management and security are vertical stacks.

Week 2 — Deployment Models, Virtualization, XML, Web Services & SOA (Lectures 6–10)

> **Source:** L6 Architecture – Deployment Models · L7 Virtualization · L8 XML Basics · L9 XML Basics (contd.) · L10 Web Services, Service Oriented Architecture
>
> High-yield areas: **pros/cons tables of each deployment model**, **Popek–Goldberg**, **full vs para vs hardware-assisted virtualization**, **IaaS 3-level manager hierarchy**, **well-formed vs valid XML**, **SAX vs DOM**, **SOAP/WSDL/UDDI and publish–find–bind**.

2.1 Deployment models in detail (L6)

> 🎯 **Your practice set for L6** (your file *w11*): 50 questions · 11 easy · 27 medium · 12 hard · 1 with ⚠️ notes. Answers checked against this lecture.
> Full set and answer key: chapter "Your practice sets", Week 2.

Public cloud

- Infrastructure provisioned for **open use by the general public**; owned/managed/operated by a business, academic or government organization, or a combination.
- **Exists on the premises of the cloud provider (CSP).**
- Provider's computing and storage resources are **potentially large**; communication is over the **public Internet**; serves a diverse pool of clients — some may be **attackers**.
- Examples (lecture): **Google App Engine, Microsoft Azure, IBM Cloud, Amazon EC2**.

Characteristics / considerations of public cloud | Point | Explanation | |—|—| | Workload locations hidden from clients | You don't know where your VM/data reside or with whom | | **Risks from multi-tenancy** | A single machine may be shared by workloads of any combination of subscribers — your workload may be **co-resident with a competitor or adversary** → **reliability and security risk** | | Network dependency | Always-available connectivity required | | Limited visibility and control over data (security) | You rely on SLA/MoU promises | | **Elasticity: illusion of unlimited resource availability** | Theoretically infinite scale up/down | | **Low up-front cost** to migrate into the cloud | Pay and use; nothing to buy/install | | **Restrictive default SLAs** | Small users must accept provider's standard terms (large deployments may negotiate) |

Private cloud

- Provisioned for **exclusive use by a single organization comprising multiple consumers (e.g., business units)**.

- May be owned, managed and operated by the organization, **a third party**, or a combination; **may exist on or off premises**.
- *"Contrary to popular belief, a private cloud may exist off premises and can be managed by a third party."*
- Examples / software (lecture): **Eucalyptus, OpenStack, Ubuntu Enterprise Cloud, Amazon VPC (Virtual Private Cloud), VMware Cloud Infrastructure Suite, Microsoft ECI data center**; IIT KGP's **Meghamala**.
- Two scenarios: **on-site private cloud** (at the customer's premises — the "de facto" one) and **outsourced private cloud** (server side outsourced to a hosting company).

On-site private cloud

- **Security perimeter extends around both the subscriber's on-site resources and the private cloud's resources.** The perimeter does not *guarantee* control over private-cloud resources, but the subscriber can exercise control over them.
- Considerations: network dependency (on-site) · **subscriber still needs IT skills** · workload locations hidden from clients · risk from multi-tenancy (among the organization's own units) · **data import/export and performance limitations** (bulk import/export limited by on-site network capacity; real-time critical processing may be problematic) · **potentially strong security from external threats** (inherits the organization's network security, e.g., Meghamala inside IIT KGP's network) · **significant-to-high up-front costs** to migrate · **limited resources** — a fixed computing and storage capacity sized for anticipated workloads (e.g., buy 1.5× expected need and you are capped at 1.5×).

Outsourced private cloud

- Has **two security perimeters** — one implemented by the **subscriber**, one by the **provider** — joined by a **protected communication link**. Security depends on the **strength and availability of both perimeters and of the link**.
- Considerations: network dependency · workload locations hidden · risk from multi-tenancy (provider may host others' private clouds) · data import/export & performance limitations · potentially strong security from external threats · **modest-to-significant up-front costs** · **SLA must be negotiated** with the provider · **extensive resource availability** (provider has a large back-end pool).

Community cloud

- Provisioned for **exclusive use by a specific community of consumers from organizations with shared concerns** (mission, security requirements, policy, compliance considerations).
- Owned/managed/operated by **one or more community organizations, a third party, or a combination**; may be on or off premises.
- An organization can belong to **several communities** (like being in both an office group and a residential community); communities can form and dissolve over time.

- Participant organizations are connected via **links between boundary controllers**, allowing access through their security perimeters.
- **Access policy can be complex** (prevent leakage between communities) → policy specification techniques: **RBAC (role-based access control)** and **ABAC (attribute-based access control)**.
- On-site community considerations: network dependency · subscribers still need IT skills · workload locations hidden · data import/export & performance limitations · potentially strong security from external threats · **highly variable up-front costs**.
- Outsourced community considerations: network dependency · workload locations hidden · risk from multi-tenancy · data import/export & performance limitations · potentially strong security from external threats · **modest-to-significant up-front costs** · **extensive resource availability**.

Hybrid cloud

- **Composition of two or more distinct cloud infrastructures (private, community or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability.**
- Why: classify data/applications by sensitivity — e.g., keep **student records/exam data private**, run **practice lab sessions on public cloud**; or temporarily rent public capacity when the private cloud is short (buying for a private cloud takes a long time). *(The second pattern is commonly called "cloud bursting" — clarification, term not used in lecture.)*
- **Portability/interoperability** of data and applications across the constituents must be handled (applications may need resizing).
- Examples (lecture): **Windows Azure** (capable of hybrid), **VMware vCloud**.
- Considerations: **significant variation in performance, reliability and security** depending on constituent clouds · **can be extremely complex** · **may change over time** as constituent clouds join or leave (subscriptions end, pricing changes, providers disappear).

Choosing a deployment model

Depends on requirements: small organizations/individuals → public cloud is often fine; mission-critical clients (financial, defence) may demand private infrastructure; or classify applications/data into categories and place each in private/public/community accordingly (managing this is itself a challenge).

Comparison table (high-yield)

Model	Up-front cost to migrate	Resource availability	Security from external threats	Typical location
Public	**Low**	Illusion of unlimited (elastic)	Multi-tenancy risk; limited visibility	Provider's premises
On-site private	**Significant to high**	**Limited** (fixed capacity)	**Potentially strong**	Customer's premises
Outsourced private	**Modest to significant**	**Extensive**	Potentially strong (two perimeters + link)	Hosting company
On-site community	**Highly variable**	Limited	Potentially strong	Participant premises
Outsourced community	Modest to significant	Extensive	Potentially strong	Third party
Hybrid	Depends on constituents	Mix	Varies — can be extremely complex	Mix

MCQ traps (2.1)

- "A private cloud must be on-premises and managed by the organization" → **False**.
- "Extensive resource availability" is a property of **outsourced** private/community clouds, **not on-site** private clouds (those have **limited** resources).
- **Low** up-front cost belongs to **public** cloud; on-site private has **significant-to-high**.
- Outsourced private cloud = **two** security perimeters.
- Workload locations are hidden from clients in **all** these models (even on-site private — hidden from the organization's own client units).
- RBAC/ABAC are mentioned for **community** cloud access policies.
- Hybrid constituents **remain unique entities** — they are not merged into one cloud.

2.2 IaaS and virtualization architecture (L7)

What an IaaS subscriber gets and pays for

- Gets: **access to virtual computers**, network-accessible **virtual storage**, **network infrastructure components** (e.g., firewalls, configuration services).
- Pays (typically): **per CPU-hour**, **per GB of data stored per hour**, **network bandwidth consumed**, **network infrastructure used** (e.g., IP addresses), and **value-added services** (monitoring, automatic scaling).
- Providers keep a large pool of VMs in a few **flavours** (e.g., Meghamala offers three: *IITKGP* — 4 GB RAM, 20 GB disk; *IITKGP Large* — 8 GB RAM; *IITKGP Extra Large* — larger RAM). The mix of flavours

depends on back-end capacity and the type of requests received. Different clients get disjoint VMs (A: VM1–VM2, B: VM3, C: VM4–VM6; provider retains VM7–VMn for others).

IaaS component stack and scope of control

- Stack: **hardware → operating system → middleware → applications**.
- The OS level is split into two: the **lower, most privileged layer is the VMM/hypervisor**; the **higher layer is the guest OS** running inside a VM.

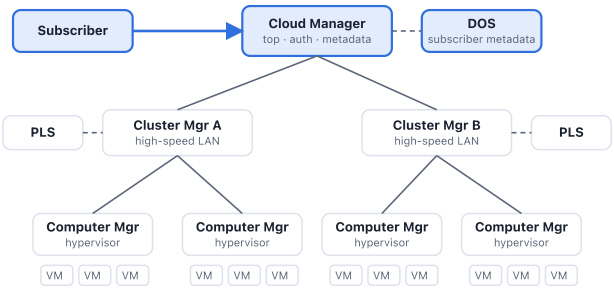
Layer	Cloud provider	Cloud subscriber
Hardware	**Total control**	No control
Hypervisor (VMM)	**Administrative control**	Makes requests (e.g., for VMs)
Guest OS	No control	**Total control** (can load any OS the provider supports)
Middleware (e.g., Java)	No control	Total control
Applications (mail, CRM…)	No control	Total control

Rule: **control of upper layers moves to the subscriber; control of lower layers stays with the provider.**

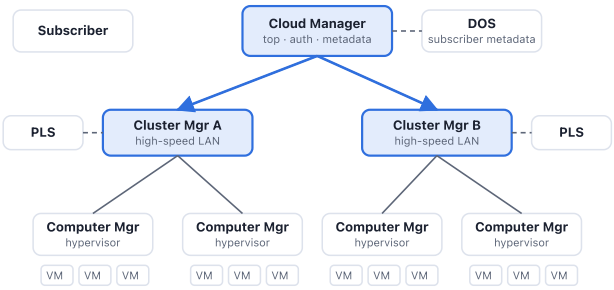
- The hypervisor uses the hardware to **synthesize one or more VMs** — each an **"efficient, isolated duplicate of a real machine."** To the subscriber the VM appears as actual hardware that can be powered on/off and have peripherals configured via commands sent over the network.

IaaS cloud architecture — 3-level hierarchy (very examinable)

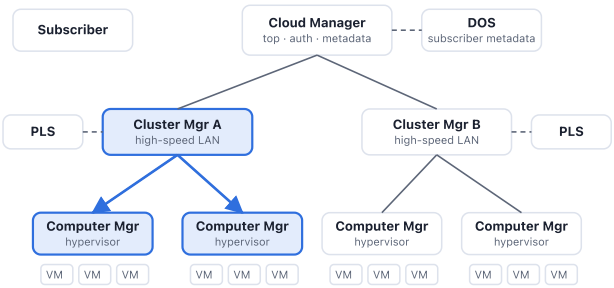
IaaS 3-level hierarchy: request flow diagram sequence



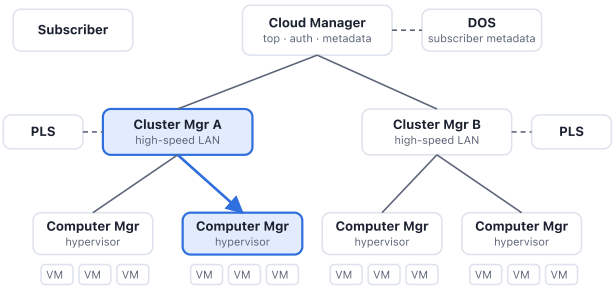
1. Subscriber's request reaches the **cloud manager** — the public access point: it **authenticates** the subscriber, issues credentials, consults **DOS** metadata and checks whether the cloud has free resources.



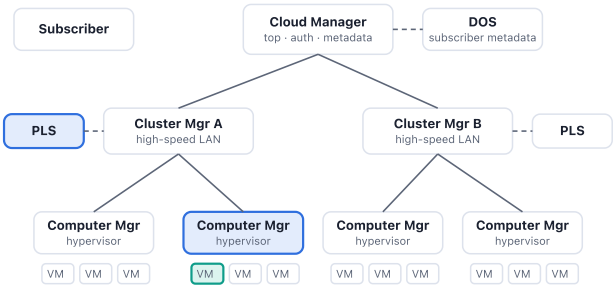
2. Top-level resource management: the request is forwarded down to the **cluster managers**.



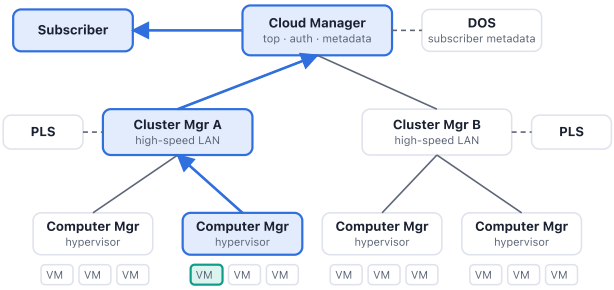
3. Cluster manager **queries its computer managers** and calculates whether part or all of the request can be satisfied in its cluster.



4. Cluster manager **instructs** a computer manager to allocate (or reconfigure/de-allocate) resources.



5. The **computer manager** (one per host) uses the hypervisor's command interface to **start the VM**; **PLS** gives it persistent disk-like storage.



6. Status flows back up; the subscriber gets the VM's access credentials. Queries/commands go **top → down**, answers come back up.

`Cloud Manager (top) → Cluster Managers (middle) → Computer Managers (bottom, one per computer, running the hypervisor + VMs)` Supporting stores: **Data Object Storage (DOS)** at the cloud-manager level; **Persistent Local Storage (PLS)** at the cluster-manager level.

Component	Level	Responsibilities
Cloud Manager (CLM)	Top — **central control**	**Public access point** of the cloud; subscribers sign up for accounts and manage rented resources; **authenticates subscribers**; generates/validates **access credentials** used to communicate with VMs; **top-level resource management** — decides whether the cloud has enough free resources for a request; maintains metadata. Can be geographically distributed.
Data Object Storage (DOS)	With cloud manager	Stores **subscriber metadata** (user credentials, OS images, etc.); **usually a single DOS service per cloud** — acts like the cloud's registry/catalogue
Cluster Manager (CM)	Middle	Operates a **collection of computers connected by a high-speed LAN** (clusters may be geographically distant from each other); receives allocation commands/queries from the cloud manager and **calculates whether part or all of a command can be satisfied** by its cluster; **queries computer managers** for availability and reports back; **instructs computer managers** to allocate, reconfigure, de-allocate resources
Persistent Local Storage (PLS)	Connected to cluster manager	Provides **persistent, disk-like storage to VMs** — data survives VM shutdown/de-provisioning
Computer Manager	Bottom — runs on **each computer**	Uses virtualization to **provide VMs** to subscribers; **maintains status** (e.g., how many VMs are running); uses the **command interface to the hypervisor to start, suspend and reconfigure VMs**

Flow: subscriber queries/commands enter at the **top** and are **forwarded down** the layers, which answer queries or execute commands.

> ⚠️ *Transcript slip:* one line of L7 says "cloud manager runs on each computer system"; from context (and the slide it describes) this is the **computer manager**. Use: *computer manager runs on each computer.*

Virtualization — core concept

- Virtualization is **not new** (VLAN, VPN, running Linux on Windows); cloud **exploits** it.
- **Definition (lecture):** virtualization allows **one computer to do the job of multiple computers by sharing the resources of a single hardware across multiple environments**.
- **Non-virtualized:** a single OS controls all hardware platform resources. **Virtualized:** multiple **virtual containers** run on a single physical platform.
- **Virtualization vs dual boot:** in dual boot only one OS runs at a time; with virtualization **all OSs run simultaneously** and are **completely isolated** from each other.
- A hypervisor runs the guest OS **directly on the CPU** — this works if the **guest uses the same instruction set as the host**; otherwise **instruction translation** is needed.

Popek and Goldberg (1974) — requirements for virtualization

Three goals of a virtual machine architecture: | Property | Meaning | |—|—| | **Equivalence** | The VM should be **indistinguishable from the underlying hardware** | | **Resource control** | The VM (VMM) should be in **complete control of the virtualized resources** | | **Efficiency** | **Most VM instructions should execute directly on the underlying CPU** without involving the hypervisor |

Instruction classification:

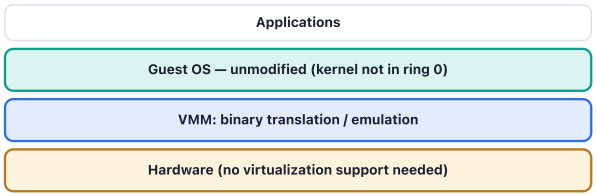
- **Privileged instructions** — **cause a trap if executed in user mode**.
- **Sensitive instructions** — change underlying resources (e.g., do I/O, change page tables) **[control-sensitive]**, or observe information indicating the current privilege level / that the guest isn't on bare metal **[behaviour-sensitive]**.

Theorem (key result): *A VMM with all three properties can be built if the set of sensitive instructions is a subset of the set of privileged instructions.*

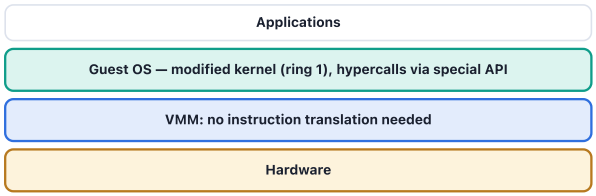
- Intuition: every sensitive instruction then traps to the VMM, which can emulate it safely; everything else runs directly (efficiency).
- Lower privilege rings (ring 0) are closer to bare metal; the higher the level at which virtualization operates, the more translation and latency.

Approaches to (server) virtualization — the three generations

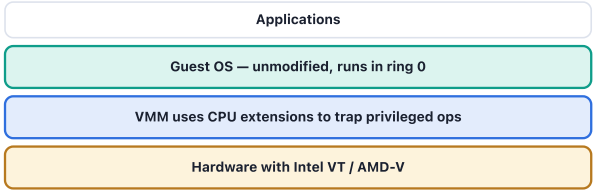
Three generations of server virtualization diagram sequence



1. Full virtualization (1st gen): dynamic binary translation; all hardware emulated (VMware, Microsoft; QEMU, Bochs). ✓ total VM portability ✘ performance price.



2. Para-virtualization (2nd gen): cooperative; guest kernel recompiled with para-virtualized drivers (Xen, VMware). ✓ faster ✘ OS must be modified.



3. Hardware-assisted (3rd gen): silicon support (Intel VT, AMD-V). ✓ unmodified OS ✘ speed/flexibility issues, VMs less portable.

	Full virtualization (1st gen)	**Para-virtualization** (2nd gen)	**Hardware-assisted virtualization** (3rd gen)
Technique	**Binary rewriting / dynamic binary translation**; software-based; **all hardware emulated, including the CPU**	**Cooperative** virtualization; **guest OS is modified**	**Silicon-based**; VMM uses **processor extensions (Intel VT, AMD-V)** to intercept privileged operations
Guest OS	Unmodified	**Modified** — kernel recompiled; para-virtualized drivers installed; guest runs kernel at **ring 1 (or 3)** and is **aware** of virtualization; uses a **specialized API to talk to the VMM**	**Unmodified**; guest OS runs at **ring 0**
Examples	**VMware, Microsoft**; emulators **QEMU, Bochs**	**VMware, Xen**	VMware and Xen on virtualization-aware hardware
Pros	Emulation layer isolates VMs; **total VM portability** (only the VMM needs to understand it)	**Better performance** — privileged-instruction translation by VMM no longer necessary	Runs **unmodified OS**; removes many problems of writing a VMM
Cons	**Performance price** of hardware emulation; x86 kernels expect ring 0	Requires **modifying/recompiling the OS kernel**	**Speed and flexibility** issues; **loses VM portability** (VM tied to hardware that supports the extensions)

Network virtualization (L7)

- Emulate one or more **virtual networks over a given physical network**.
- **Why:** the Internet is "ossified" (hard to change); hard to design a **one-size-fits-all** architecture → instead let **"all sizes fit into one"**; provides a **test bed** for future networking research.
- Not new: builds on **VPNs, active and programmable networks, overlay networks**.
- Characterization: networking technology (IP-based, ATM-based), **layer** of virtualization, **architectural domain** (network resource management, spawning networks), **level** of virtualization (node level, full virtualization).
- **Business model players: infrastructure providers, service providers, end users, brokers** (complex relationships; same model applies to other virtualization). Several service providers (SP1, SP2) can create different virtual networks on one infrastructure provider's network.
- Server-level (IaaS) virtualization + network virtualization together → a **virtualized IT infrastructure** (e.g., a geographically spread test with its own network).

MCQ traps (2.2)

- **Para-virtualization modifies the guest OS; hardware-assisted does not; full virtualization does not** (it translates binaries instead).
- Popek–Goldberg condition is **sensitive ⊆ privileged** (not the reverse).
- Equivalence = indistinguishable from hardware; **efficiency** = most instructions run **directly** on the CPU.
- QEMU and Bochs are **emulators / full-virtualization** tools (lecture).
- Full virtualization gives **total VM portability**; hardware-assisted **loses** portability.
- In IaaS the provider has **administrative** control over the hypervisor and **total** control over hardware; the subscriber has **total** control over guest OS and above.
- **DOS** = metadata store, usually one per cloud (at cloud-manager level); **PLS** = persistent disk-like storage for VMs (at cluster level). Don't swap them.

Quick revision (2.2)

- IaaS billing: CPU-hour, GB-stored-hour, bandwidth, infrastructure (IPs), value-added services.
- Hierarchy: Cloud manager (authenticate, credentials, top-level allocation, DOS) → Cluster manager (cluster-level feasibility, instructs computer managers, PLS) → Computer manager (one per machine; starts/suspends/reconfigures VMs via hypervisor).
- Generations: full (binary translation) → para (modified guest, faster) → hardware-assisted (Intel VT/AMD-V, unmodified guest, ring 0).

2.3 XML basics (L8–L9)

Why XML matters for cloud

Cloud services (XaaS) must interoperate regardless of where/how they are hosted → **web services/SOA**, whose basic building block is **XML**. SOAP, WSDL and UDDI are **all XML documents**.

Core properties of XML (eXtensible Markup Language)

Property	Meaning
Data transformation / exchange language	Unlike **HTML (data display/presentation)**, XML represents and transforms data to achieve **interoperation**
Text-based syntax	Written in **printable Unicode characters**; **no explicit binary data**
Extensible	**You define your own elements/tags** (HTML has a fixed, predefined tag set) — within syntax rules
Universal format	Syntax rules ensure **all XML processing software handle a given XML data identically** — if you can read/process it, so can everyone else
Hierarchical, structured	Forms an **XML tree**
Strict syntax	To eliminate syntax errors
Self-describing data	Tags describe the data
Designed for distributed environments	Data from many sources can be **mixed** (e.g., combine academic-section and activity data to find the best all-rounder)

Parsing: well-formed vs valid

- An **XML parser** extracts the information for the XML-based application (parser → parser interface → application).
- **Well-formed** = **syntactically correct**. This is the **minimum requirement** to be XML. If the data is not well-formed, the parser **must stop processing and throw an exception** to the application.
- **Valid** = **well-formed AND conforms to a DTD or XML Schema**. → **All valid XML documents are well-formed; not all well-formed documents are valid.**
- **DTD (Document Type Declaration/Definition):** declares the document's structure (like a relational table schema).
- **Internal entities** (defined in the document's DTD) vs **external entities** (referenced by a **URL**). The parser checks each entity is well-formed, then **replaces every entity reference by the referenced entity, recursively**, and hands the resolved data to the application.
- What if an external entity can't be found (e.g., firewall blocks it)? Depends on the parser type:

Parser type	Behaviour
Validating parser	**Must retrieve all entities and process all DTD content**; **stops and indicates failure** if it cannot
Non-validating parser	Tries to retrieve entities, but **ceases processing DTD content at the first entity it cannot find**

- Validating parsers are **slower** than non-validating ones (must check conformance first).
- Character-set rules: XML specifies which characters may be whitespace (e.g., **EBCDIC NEL cannot be used as whitespace**); mechanisms exist to include characters outside the encoding.

DTD vs XML Schema (XSD)

	DTD	XML Schema (XSD)
Origin	Part of core XML spec	**New specification (2001)**
Written in	Non-XML syntax	**Written in XML itself** — schema handled like XML data
Power	Structure only	**More powerful** — data types (integer, date, real…), type validation, relating **database schemas** to XML models
Entities	**Can declare entities**	**Cannot declare entities** (only DTD can)

- Both specify **allowed elements and attributes, hierarchical nesting rules, element content and type restrictions**. **They contain no XML data** — only structure.
- **Exchanging structure, not data:** two organizations (e.g., IIT KGP and a bank) first **agree on the schema** — no privacy/maintenance issues of sharing data — then interoperate (even auto-create DB tables from the schema).

XML namespaces

- Because tags are user-defined, the same name can mean different things — "table" (furniture vs word-processing), "key" (**security key vs database key**).
- A **namespace** is a **mechanism for identifying different spaces for XML names (element and attribute names)** — a way of identifying different language dialects with specific semantics.
- Declared with the special `xmlns` **attribute**; the namespace is given as a **URL string, but the URL does not reference anything in particular** (may point to nothing).
- **Prefixes** let you mix dialects in one document (e.g., `db:key` vs `sec:key`; a prefix bound to the **MathML** namespace to embed MathML elements).

XML parser APIs (L9)

Parser	Model	Advantages	Disadvantages
SAX (Simple API for XML)	**Event-based**: reports events (tags, attributes, text nodes, unresolved external entities); programmer attaches **event handlers**	Simple; **very fast** (as fast as you can stream data); **low memory footprint**; widely available	Doesn't do much for you; **not useful for dynamically modifying a document in memory**
DOM (Document Object Model)	**Object/tree-based**: builds an **in-memory tree** of the whole document; methods to access and modify it	Dynamic modification and access; querying that depends on tree structure; **same interface for many languages (C++, Java)**	**Slow; memory-intensive**; programming interface not straightforward
JDOM (Java DOM)	Java-based, object-oriented in-memory tree	Like DOM, plus a **much nicer OO interface**	Can be slow; **Java only**
DOM4J	Java framework to read, write, navigate, edit XML	Access to SAX, DOM, JDOM, XSLT; **mixed SAX/DOM parsing**; open-source (Apache-style licence)	Java only; open-source concerns

Others: **Xerces**, XML Toolkit, XML for C++. Parser benchmarks compare speed, memory, stream processing.

XSLT (Extensible Stylesheet Language Transformations)

- An **XML language for processing XML data — tree transformation**: takes **XML + XSLT stylesheet** → produces **a new XML document with a different structure** (filtering, extracting a sub-tree).
- XML + styling ≈ displayable output (even HTML).
- Pros: very useful for tree transformations. Cons: **slow for large documents/stylesheets**, **difficult to debug**, **poor error detection**.

XML messaging

- Use XML as the **format for messages between systems**. Pros: common syntax, self-describing, easier to parse, **can use existing transport mechanisms (HTTP, HTTPS, SMTP)**.
- XML needs a **carrier protocol**; "transport" here means the carrier (e.g., HTTP), **not the TCP/IP transport layer**.
- The XML message is **embedded in an HTTP POST** (HTTP is the envelope, XML the payload).
- Message format standards: **XML-RPC** — very simple encoding of function/method calls; **SOAP** — more complex wrapper: specify **schemas for interfaces**, rules for handling and **proxying** messages; primarily used in web services/SOA.
- **XML family:** XSL, XSLT, **XPath**, **XPointer**; dialects such as **MathML**, **SVG** (graphics); web-service protocols SOAP, WSDL, UDDI.

MCQ traps (2.3)

- XML is **not** primarily a display language (that's HTML); XML tags are **user-defined**.
- **Valid ⇒ well-formed**, not the converse.
- **XSD is written in XML; DTD is not.** XSD **cannot declare entities**.
- A namespace URL **need not point to any real resource**.
- **SAX = event-based, fast, low memory, no in-memory tree; DOM = in-memory tree, slow, memory-hungry, good for modification.**
- Non-validating parser **stops processing DTD content at the first unresolvable entity**; validating parser **fails**.
- XSLT produces **XML** (a transformed tree), not binary.

Quick revision (2.3)

- Well-formed (syntax) → valid (+ DTD/XSD).
- DTD (entities, non-XML) vs XSD (2001, XML, data types, no entities).
- `xmlns` + prefixes resolve name clashes.
- SAX (events) vs DOM (tree) vs JDOM (Java OO tree) vs DOM4J (Java, mixed).

2.4 Web services and Service-Oriented Architecture (L10)

Definitions of a web service (all three are used in MCQs)

1. **W3C:** *a software application identified by a URI, whose interfaces and bindings are capable of being defined, described and discovered as XML artifacts.*
2. **Microsoft:** *programmable application logic accessible using standard Internet protocols.*
3. *An interface that describes a collection of operations that are network-accessible through standardized XML messaging; software components that can be spontaneously discovered, combined and recombined to provide a solution to the user's problem/request.*

Common thread: **XML-based, message-exchange, accessible by URI, loosely coupled** (client–server is more tightly bound). The caller only cares *where* the service is and *how to talk to it*, not how it works internally — which also lets **legacy applications** talk to others.

Genesis

Structured programming → **object-oriented programming**; **distributed computing**; **Electronic Data Interchange (EDI)**; **World Wide Web**.

- Problem in distributed computing: **interoperability** (communicate and share data with software from different vendors/platforms) was limited in conventional proprietary technologies.

EDI (Electronic Data Interchange)

- **Computer-to-computer exchange of business data and documents between companies using standard formats recognized nationally and internationally.** The format is **agreed by both companies beforehand**.
- Advantages: **lower operating cost**, less human error/**more accuracy** (no re-entry), **increased productivity**, **faster trading cycle**.

What web services add

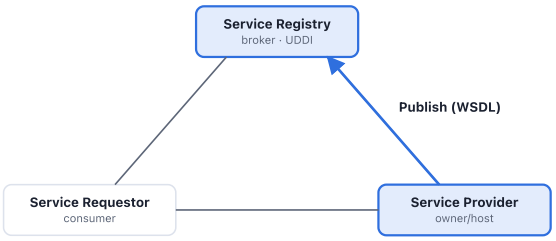
- Exploit OOP: **build applications from existing software components in a modular way**; transform the Internet into a **library of programmatic components**.
- **Example — online travel booking:** the travel portal (a broker) owns no flights or hotels; it calls airline services, shows options, then calls card/net-banking services — multiple services **choreographed** to execute one job; neither the airline nor the bank knows the whole picture.
- Improve interoperability through **open, non-proprietary standards**.
- **Like EDI but simpler and less expensive** to implement; can be configured to **work with EDI** or to **phase it out**.
- **Unlike the WWW, separate visual from non-visual components**; interaction via browser or desktop clients (Java Swing, Python, Windows).

Three problems web services address | Problem | How | |—|—| | **Interoperability** | Lack of interoperability standards in distributed object messaging → open XML standards | | **Firewall traversal** | Runs over **HTTP (port 80)**, which most firewalls allow; **CORBA and DCOM used non-standard ports** | | **Complexity** | Developer-friendly, open **text-based** standards; components in different languages/platforms can communicate; can be **implemented incrementally** (lower cost, less disruption); legacy software keeps running behind a WS interface |

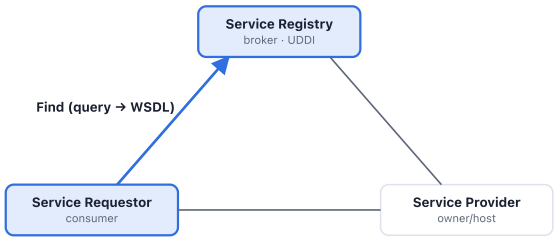
Summary of how web services work: communicate via open standards (**HTTP, SMTP**, other TCP/IP application protocols) · process **XML messages framed using SOAP** · describe messages using **XML Schema** · provide an endpoint description using **WSDL** · **publish and discover using UDDI**.

SOA — the service triangle

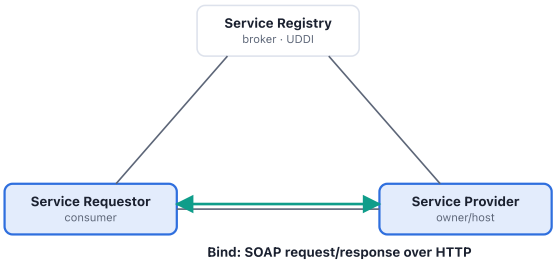
SOA triangle (IBM): publish – find – bind diagram sequence



1. Publish: the provider publishes its service description (**WSDL**) in the registry (**UDDI**).



2. Find: the requestor queries the registry and gets the WSDL describing how to call the service.



3. Bind: the requestor invokes the provider directly with **SOAP** messages carried over **HTTP** (port 80, firewall-friendly).

- **IBM** created the model showing web-service interaction, called **Service-Oriented Architecture**, as a relationship between three entities:
  - **Service provider** — owner of the service; platform hosting access to it.
  - **Service requestor (consumer)** — business requiring the service.
  - **Service registry (broker)** — searchable registry of service descriptions where providers publish.
- **Three operations: Publish** (provider → registry), **Find** (requestor → registry, using WSDL and UDDI), **Bind** (requestor ↔ provider).
- Analogy: **telephone directory / yellow pages**.

Interaction sequence: client **queries the registry** to locate services → registry **refers to the WSDL document** → client **accesses the WSDL** (data needed to interact) → client **sends a SOAP request** to the provider → service **returns a SOAP response**.

Web-service stack (bottom → top): network (TCP/IP, HTTP) → **XML-based messaging (SOAP)** → **service description (WSDL)** → **service publication & discovery (UDDI** — static or dynamic) → **service flow (WSFL)**. **Vertical: Quality of service, management, security.**

The core standards

Standard	Full form	Role
XML	eXtensible Markup Language (evolved from **SGML**)	Uniform data representation and exchange; separates content from presentation; structure via DTD/XSD
SOAP	**Simple Object Access Protocol**	**Messaging protocol** — format for sending messages over the Internet; **XML-based, W3C compliant, stateless, one-way**; uses mainly **HTTP as the (carrier) transport**
WSDL	**Web Services Description Language**	XML-based, W3C-compliant **meta-language to describe the services offered** — port types, messages, **bindings**
UDDI	**Universal Description, Discovery and Integration**	Framework to define **XML-based registries** — mechanism to **register and locate** web-service applications

SOAP details

- Building blocks (outside → in): **transport protocol → MIME header → SOAP envelope → SOAP header + SOAP body** (+ fault handling). All XML-based.
- The SOAP envelope **piggybacks as the payload** of the transport (HTTP POST for request; HTTP response for reply — e.g., "get stock price of IBM").
- **Why SOAP: platform-neutral**; simple XML **wire format**; **places no restriction on endpoint implementation technology** (legacy systems can participate).
- Characteristics: **extensible, neutral, independent**.
- Usage models: **RPC-like message exchange**; or any kind of body content (document style).

UDDI benefits: registry data in a standard, searchable format; **discover the right business** among millions online; define how to enable connectivity/commerce with the preferred business.

Web-service security (vertical): WS-Policy, WS-Trust, WS-Privacy, WS-SecureConversation, WS-Federation, WS-Authorization. Management and QoS are also vertical.

Link to cloud: IaaS/PaaS/SaaS/XaaS are all built on the web-service **publish–find–bind** idea (who provides, where the service is, how to bind) — web services and SOA made cloud computing viable.

MCQ traps (2.4)

- **SOAP = messaging; WSDL = description; UDDI = registry/discovery.** Classic matching question.
- SOAP is **stateless and one-way**; "transport" for SOAP means **HTTP/SMTP as carrier**, not TCP.
- Web services traverse firewalls because they use **HTTP/port 80**; **CORBA/DCOM** used non-standard ports.
- SOA operations are **publish, find, bind** — not "register, request, respond".
- The **registry** is the broker; the **consumer** does *find*, the **provider** does *publish*.
- EDI requires a **pre-agreed format**; web services are a **simpler, cheaper** alternative that can coexist with EDI.

Quick revision (2.4)

- W3C definition: URI + interfaces/bindings defined, described, discovered as XML artifacts.
- SOA triangle (IBM): provider, requestor, registry; publish–find–bind.
- XML → SOAP → WSDL → UDDI → WSFL; QoS/management/security vertical.

Week 2 — High-yield takeaways

1. Public: low up-front cost, multi-tenancy risk, restrictive default SLAs, illusion of unlimited resources.
2. On-site private: limited resources, significant–high up-front cost, strong external security; outsourced private: two perimeters, extensive resources, negotiated SLA.
3. Community → RBAC/ABAC; hybrid → constituents remain unique, portability, complexity, members join/leave.
4. IaaS hierarchy: Cloud manager (+DOS) → Cluster manager (+PLS) → Computer manager (per machine).
5. Popek–Goldberg: equivalence, resource control, efficiency; VMM possible if **sensitive ⊆ privileged**.
6. Full (binary translation, QEMU/Bochs, portable, slow) / Para (modified guest, ring 1, Xen) / Hardware-assisted (Intel VT, AMD-V, unmodified guest, ring 0, less portable).
7. XML: well-formed vs valid; DTD vs XSD; namespaces via `xmlns`; SAX (event) vs DOM (tree); XSLT transforms trees.
8. Web services: SOAP (messaging, stateless, one-way, over HTTP), WSDL (description), UDDI (registry); SOA publish–find–bind.

Week 3 — SLA, Cloud Economics, Managing Data, MapReduce, OpenStack (Lectures 11–15)

> **Source:** L11 Service Level Agreement (SLA) · L12 Economics · L13 Managing Data · L14 Introduction to MapReduce · L15 OpenStack
>
> **Numerical-heavy week.** Formulas you must be able to apply: coefficient of variation and the $1/\sqrt{n}$ smoothing rule, utility-pricing condition $U < P/A$, penalty $\int |D(t) - R(t)|\, dt$, system-failure probability $1 - (1-p)^N$, Dynamo quorum $R + W > N$, parallel efficiency $T_1/(p\,T_p)$, word-count efficiencies $1/p$ vs $1/(1 + 2p/nf)$, MapReduce efficiency $1/(1 + 2c\sigma/w)$.
>
> Worked tutorial problems on SLA, economics and MapReduce are in **Week 5 (L21–L23)**.

3.1 Service Level Agreement (L11)

🎯 **Your practice set for L11** (your file *w12*): 50 questions · 14 easy · 26 medium · 10 hard · 2 with ⚠️ notes. Answers checked against this lecture.
Full set and answer key: chapter “Your practice sets”, Week 3.

Core concept

- **SLA = a formal contract between the service provider and the service consumer.**
- It is **the foundation of the consumer’s trust in the provider**, and lets consumers **compare providers**.
- **Purpose:** to define a **formal basis for the performance and availability the provider guarantees to deliver** (e.g., “uptime ≥ 95 %”).
- There is still **no standard format** for SLAs; consumers usually click-accept the provider’s terms.
- Applies to every service model (IaaS, PaaS, SaaS, data services…).

Terminology hierarchy: KPI → SLO → SLA

Term	Meaning	Examples
KPI (Key Performance Indicator)	**Low-level resource metric**, measured directly from the system	downtime, uptime, bytes in, bytes out, packet size
SLO (Service Level Objective)	**Objectively measurable condition for the service**; encompasses multiple QoS parameters. **Multiple KPIs are composed, aggregated or converted into high-level SLOs**	availability of service = 99.9 %; response time of a DB query between 3 and 5 s; throughput of a server at peak = 0.875
SLA	**Multiple SLOs are integrated** into the agreement	the whole contract
Service Level Management	Monitoring and managing service performance based on SLOs	Provider: decisions from business objectives + technical realities; Consumer: decisions about how to use cloud services

- SLOs **vary from consumer to consumer** (an academic institute may accept 95 % uptime but want high data persistence; another may need ≥ 99 % uptime but keep its own backups) and **over time** (≥ 99 % at peak hours, ≥ 90 % off-peak).
- QoS parameters inside SLOs: **availability, serviceability, billing, throughput, response time, quality**.

KPI → SLO mapping example given in the lecture:

$$\text{Availability} = 1 - \frac{\text{downtime}}{\text{uptime}}$$

> ⚠ *Flag:* the lecture slide writes downtime/**uptime**. The standard definition is $\text{Availability} = \frac{\text{uptime}}{\text{uptime} + \text{downtime}} = 1 - \frac{\text{downtime}}{\text{total time}}$. For small downtimes the two are almost equal. If an MCQ reproduces the slide formula, choose it; for numerical availability problems use downtime/total time (as in the SLA tutorial, Week 5).

Typical contents of an SLA (list question)

1. **Set of services** the provider will deliver.
2. **Complete, specific definition of each service.**
3. **Responsibilities of the provider and the consumer.**
4. **Set of metrics** to measure whether the provider is offering the services as guaranteed.
5. **Auditing mechanism** to monitor the services (possibly a third-party auditor).
6. **Remedies** available to consumer and provider if the terms are not satisfied (penalty/compensation for not providing guaranteed service).
7. **Whether/how the SLA will change over time.**

Web-service SLAs (cloud SLAs evolved from these)

- **WS-Agreement:** an **XML-based language and protocol for negotiating, establishing and managing service agreements at runtime**; specifies the nature of the **agreement template**; **facilitates discovering compatible providers**; interaction is usually **request–response**; **SLA violations dynamically managed and verified**.
- **WSLA (Web Service Level Agreement framework): formal XML-schema-based language to express SLAs + a runtime interpreter**; **measures and monitors QoS parameters and reports violations**; weakness: **lack of formal definitions for the semantics of the metrics**.

Traditional web-service SLA vs cloud SLA (classic comparison)

Aspect	Traditional web services	Cloud
QoS parameters	Response time, SLA violation rate for reliability, availability, cost of service	**Security, privacy, trust management** have more importance
Automation	SLA negotiation, provisioning, service delivery, monitoring are **not automated**	**SLA automation required** for highly dynamic, scalable services
Resource allocation	**UDDI** used for advertising and discovering services	Resources **allocated and distributed globally without any central directory**

Types of SLA

Type	Description
Off-the-shelf / non-negotiable / "direct" SLA	Provider creates an **SLA template** defining all criteria (contract period, billing, response time, availability…). **Followed by present-day (public) clouds. Not conducive for mission-critical data or applications.** Suits small requirements.
Negotiable SLA	Negotiated **via an external agent**, or **via multiple external agents** (when combining several services). Suits large requirements (like booking a whole hotel vs one room).

Considerations for SLAs (L11 lists)

- **Business-level objectives.**
- **Responsibilities of provider and consumer** — balance varies with service type (e.g., consumer must not send data faster than the agreed rate, or it may cause violations of the provider's other SLAs).
- **Business continuity and disaster recovery** — consumer should ensure adequate protection against natural/man-made disasters and system failure.
- **System redundancy** — many providers use massively redundant systems.
- **Maintenance** of cloud infrastructure (affects every offering, hardware and software).
- **Location of data** — if regulations require data to stay in a jurisdiction (e.g., within India), the **consumer must be able to audit** the provider to prove compliance.

- **Seizure of data** — if law enforcement targets one consumer's data, **multi-tenancy** makes it likely that **other consumers are affected** too.
- **Failure of the provider.**
- **Jurisdiction** (where litigation is handled).

Further SLA requirements:

- **Security** — data encryption, **key management** (where keys reside, how communicated).
- **Privacy** — **isolation of consumer data in a multi-tenant environment**.
- **Data retention and deletion.**
- **Hardware erasure/destruction** — provider must **zero out memory** when a consumer powers off a VM, and **zero out disk platters** before disposal/recycling.
- **Regulatory compliance, transparency, certification.**
- **Monitoring** — to eliminate conflict of interest, a **neutral third-party organization** is the best solution (hard to implement in practice).
- **Auditability** — consumers are liable for breaches, so they must be able to audit provider systems/procedures; the SLA should say how and when. **Audits are disruptive and expensive**, so providers will **limit** them or **charge** for them.

Metrics for monitoring and auditing

Throughput · availability · reliability · load balancing (when elasticity boots/terminates VMs) · **durability** (how likely data is to be lost) · **elasticity · linearity** (how the system performs as load increases — linear growth is easy to chase; non-linear/exponential is hard) · **agility · automation · customer service response time · service level violation · transaction time · resolution time.** Different metrics matter for IaaS vs PaaS vs SaaS.

Example provider SLAs cited

Provider	Service	Availability in SLA (as cited)
Amazon EC2	IaaS	**99.5 %** (service year 365 days; annual percentage uptime; region)
Amazon S3	Storage	~99 % (defined via error rate, monthly uptime percentage)
Microsoft (Azure)	IaaS/PaaS	**99.9 %** (monthly connectivity uptime service level)
Google, Salesforce, Zoho, Rackspace, Terremark	Various	listed

> ⚠️ *Lecture vs today:* these figures are historical and differ from current SLAs. Use them only if an MCQ quotes the lecture table.
>
> ⚠️ *Practice-set clash:* your L11 practice set (file w12, Q40) keys **"S3 — storage as a service, 99.9 %"**. The lecture actually says **99 % for S3**; 99.9 % is the Microsoft/Google/Salesforce line. If both values appear as options, go with the lecture's 99 %.

- **Availability ≠ durability:**
  - **Availability** asks whether the service is reachable *now*.
  - **Durability** asks how likely the stored data is to be **lost**.
  - So 99 % availability with 99.9999 % durability = "may be down for a while, but the data is almost never lost".
- **Who is at fault?** The SLA divides responsibilities. Downtime from the consumer's own code, unpatched OS (IaaS) or bad configuration is **not** a provider violation.

MCQ traps (3.1)

- **KPI is the lowest level**; SLOs are built from KPIs; the SLA integrates SLOs. Don't reverse it.
- Off-the-shelf SLAs are **not** suitable for mission-critical applications.
- In **cloud** SLAs, security/privacy/trust gain importance and **automation is required**; in traditional WS SLAs, monitoring/negotiation is **not automated**.
- **WSLA** = XML-schema language + runtime interpreter; its weakness = no formal **semantics** of metrics.
- Best monitor = **neutral third party** (not the provider, not the consumer).

Quick revision (3.1)

- SLA = formal contract; foundation of trust.
- KPI → SLO → SLA; SLM manages it.
- 7 SLA contents; 2 SLA types; cloud vs WS SLA on QoS, automation, resource allocation.

3.2 Cloud economics — “cloudonomics” (L12)

Economic properties of cloud

Property	Economic meaning
Common infrastructure	**Pooled, standardized resources**, with benefits generated by **statistical multiplexing**
Location independence	Ubiquitous availability meeting performance requirements; benefits from **latency reduction** and better user experience
Online connectivity	An enabler of the other attributes; affects service cost and performance via network architecture
Utility pricing	**Pay-as-you-go** (like electricity)
On-demand resources	Scalable, elastic resources **provisioned and de-provisioned without delay or costs** associated with the change

Why owning is wasteful: we size for **peak** load (e.g., a lab of 55 PCs for 50 M.Tech seats, used ~8 h/day; a 24-port 100 Mbps switch with a 1 Gbps uplink is “blocking” but a 2.4 Gbps uplink would be over-provisioning, because all users rarely peak together). Overheads (AC, power/UPS, AMC, staff) can exceed equipment cost, and hardware becomes **obsolete within 2–3 years (certainly within 5)**.

Value of common infrastructure:

- **Economies of scale:** reduced overhead costs; buyer power through volume purchase.
- **Statistics of scale:**
  - For infrastructure **built to peak**: multiplexing demand → **higher utilization** → lower cost per delivered resource than unconsolidated workloads.
  - For infrastructure **built to less than peak**: multiplexing demand → **fewer unserved demands** → lower loss of revenue / fewer SLA-violation payouts.

Coefficient of variation — the smoothness measure

Coefficient of variation & aggregation calculator

σ (std. dev.)	μ (mean)	n workloads
40	**100**	**100**

C_v of one workload	**0.4**
n independent → $C_v/\sqrt{n}$	**0.04**
n perfectly correlated	**0.4 (unchanged)**
negatively correlated pair (X, 1–X)	**0**

Worked example with the default values (interactive in the HTML version).

Formula:

$$C_v = \frac{\sigma}{|\mu|}$$

- **Symbols:** σ = standard deviation of demand; μ = mean demand.
- **Purpose:** statistical measure of the **dispersion of a data series around its mean**; lets you compare the variability of two series **even if their means differ greatly**. Used in investing to compare volatility (risk) against expected return.
- **Interpretation: lower C_v ⇒ smoother demand ⇒ better risk–return trade-off**; a smoother curve has a **large mean for a given σ** or a **small σ for a given mean**.
- **Not** the same as variance or correlation coefficient.
- **Importance of smoothness:** *a facility with fixed assets servicing highly variable demand achieves lower utilization than a similar one servicing relatively smooth demand.*

Multiplexing independent demands (key result): If $X_1, \ldots, X_n$ are **independent** demands each with mean μ and s.d. σ:

- Aggregate mean $= n\mu$; aggregate variance $= n\sigma^2$ ⇒ aggregate s.d. $= \sqrt{n}\,\sigma$

$$C_v(\text{aggregate}) = \frac{\sqrt{n}\,\sigma}{n\mu} = \frac{1}{\sqrt{n}}\,C_v$$

- **Adding n independent demands reduces C_v by a factor $1/\sqrt{n}$** → penalty for insufficient/excess resources shrinks.
- **Example:** aggregating **100** independent workloads → $1/\sqrt{100} = 0.1$ → the penalty falls to **10 %** of the original (the lecture phrases this as “bring the penalty down by 10 percent”; mathematically it is *reduced to one-tenth*).

Other correlation cases: | Demand relationship | Result | |—|—| | **Negatively correlated** (e.g., X and $1 - X$; day-time vs night-time customers) | Sum is constant → $C_v = 0$; choose customer segments with complementary demand | | **Perfectly correlated** (n copies of X) | Mean $n\mu$, variance $n^2\sigma^2$, s.d. $n\sigma$ → C_v **unchanged** (no smoothing) | | **All peak at the same time** | Congestion (e.g., all classes ending on the hour) — worst case |

Implications:

- With **correlated** demands, private, mid-size and large providers see **similar** statistics of scale.
- With **independent** demands, a **mid-size provider can achieve statistical economies similar to an infinitely large provider** (benefit grows like $\sqrt{n}$ and saturates).
- Evidence on economies of scale for large providers is **mixed**: all use the same **COTS** components; large ones locate **near cheap power**; early entrants’ **automation tools** soon become available from third parties.

Value of location independence

- Earlier we went to the computer; now applications/services come to us (thin client + network).
- **Latency** matters: human response **10–100 ms**; latency grows with distance and hops; speed of light in fibre ≈ **124 miles per millisecond**; VoIP becomes hard beyond **200 ms**; users are unhappy if search takes more than a couple of seconds.
- Serving a **global user base requires a dispersed service architecture** — with coordination issues of **consistency, availability, partition tolerance** — which has investment implications.

Utility pricing — when is cloud cheaper? (core numerical)

Own vs cloud (utility pricing) calculator

Peak demand P	Average demand A	Owned unit cost B	Cloud unit cost C
100	**25**	**0.1**	**0.3**

Period T

720

Utility premium U = C/B	**3**
Peak-to-average P/A	**4**
Owned cost B_T = P·B·T	**7200**
Cloud cost C_T = U·B·A·T	**5400**

Cloud is cheaper (U < P/A)

Worked example with the default values (interactive in the HTML version).

"Cloud services do not need to be cheaper to be economical." Car analogy: buying/leasing vs renting — renting is cheaper if you need the car only occasionally; buying is cheaper for daily long commutes.

Symbols | Symbol | Meaning | |—|—| | $D(t)$ | Demand for resources over $0 \le t \le T$ | | $P = \max_t D(t)$ | **Peak** demand | | $A = \frac{1}{T}\int_0^T D(t)\,dt$ | **Average** demand | | B | **Baseline (owned) unit cost** | | C | **Cloud unit cost** | | $U = C/B$ | **Utility premium** (how much more a unit of cloud costs than an owned unit) |

Costs

$$\text{Cloud cost } C_T = \int_0^T U \cdot B \cdot D(t)\,dt = U \cdot B \cdot A \cdot T$$

$$\text{Owned (baseline) cost } B_T = P \cdot B \cdot T \quad \text{(you must own enough for the peak)}$$

Cloud is cheaper when $C_T < B_T \iff U\,B\,A\,T < P\,B\,T \iff$

$$\boxed{U < \frac{P}{A}}$$

Utility premium less than the peak-to-average ratio ⇒ cloud is cheaper than owning.

- **Interpretation:** spiky demand (high P/A) favours cloud; flat demand ($P/A \to 1$) favours owning unless $U < 1$.
- **Car example (lecture numbers):** own/lease ≈ 500 per day, rent ≈ 5000 per day ⇒ $U = 5000/500 = 10$. Demand when needed = 1 car, so $P = 1$; if you need it on a fraction f of days, $A = f$ ⇒ rent if $1/f > 10$, i.e., **need the car on fewer than 10 % of days**.
- **Worked example:** peak $P = 100$ servers, average $A = 25$ ⇒ $P/A = 4$. If cloud costs $0.30/server-h and owning $0.10/server-h, $U = 3 < 4$ ⇒ **cloud cheaper** (cloud: $0.30 \times 25 = 7.5$/h vs own: $0.10 \times 100 = 10$/h).
- **Real world:** demand is spiky (news stories, promotions, product launches, Internet flash crowds, seasonal — Christmas, tax). **A hybrid model is often best** — own a car for the daily commute, rent one for trips (own the baseline, rent the peaks). The key factor is still **peak-to-average ratio**, but also consider **network costs (fixed and usage), interoperability overhead, reliability, accessibility**.
- **Common mistakes:** using $U = B/C$ (wrong — it's cloud ÷ owned); comparing U with A/P; forgetting owned cost scales with **peak**, cloud cost with **average**.

Value of on-demand services — penalty of mis-provisioning

When you own resources, you pay a penalty whenever resources $R(t)$ don't match demand $D(t)$: either **pay for unused resources** or suffer the **penalty of missing service delivery**.

$$\text{Penalty} \;\propto\; \int_0^T \big|D(t) - R(t)\big|\, dt$$

Demand pattern	Consequence
Flat	Penalty = 0 (provision exactly)
Linear	**Periodic provisioning is acceptable** (gap stays bounded)
Exponential $D(t) = e^t$	With provisioning delay t_p, $R(t) = D(t - t_p) = e^{t - t_p}$, so $D(t) - R(t) = e^t(1 - e^{-t_p}) = k_1 e^t$ ⇒ **penalty cost** $c\,k_1 e^t$ **grows exponentially** — you fall exponentially behind unless you over-provision (also hard)

- **Lecture exercise (solved in Week 5, L22):** peak demand 120 units, given $D(t)$, $R(t) = D(t - \delta)$ with provisioning delay δ, cost 0.9 units per resource-unit-time — compute penalty.

MCQ traps (3.2)

- Cloud cheaper iff $U < P/A$ (peak/average), *not* average/peak.
- C_v reduction is $1/\sqrt{n}$, not $1/n$.
- Perfectly correlated demands **do not** smooth (C_v constant); negatively correlated demands can give $C_v = 0$.

- "Cloud must be cheaper per unit to be economical" → **False** (it can cost more per unit, $U > 1$, and still win if demand is spiky).
- Exponential demand + provisioning delay ⇒ **exponentially growing** penalty; linear ⇒ periodic provisioning OK.

Quick revision (3.2)

- 5 economic properties: common infrastructure, location independence, online connectivity, utility pricing, on-demand resources.
- $C_v = \sigma/|\mu|$; n independent ⇒ $C_v/\sqrt{n}$.
- $U = C/B$; cloud cheaper iff $U < P/A$; owned cost $= PBT$, cloud cost $= UBAT$.
- Penalty $\propto \int |D - R|\, dt$.

3.3 Managing data in the cloud (L13)

Why a new approach

Conventional relational/object models work for small instances, but **large-scale read/write and huge volumes** need parallel, distributed, fault-tolerant designs. Suitable for **large-volume massively parallel text processing** and **enterprise analytics** on distributed data stores (retail chains, banks, meteorological/climatological data). (Book followed: Gautam Shroff, *Enterprise Cloud Computing*.)

Relational database recap

- Applications interact via **SQL**; the **RDBMS parser** transforms queries into memory/disk operations and **optimizes execution time** (e.g., doing selections before a join shrinks tuples from millions to thousands — same result, far less time).
- **Disk space management layer:** stores records on **pages of contiguous memory blocks** to minimize disk movement; pages fetched with **pre-fetching and page-replacement** policies.
- **Database file system layer: independent of the OS file system** (full control of pages in memory); files may span multiple disks; uses **parallel I/O, RAID** (RAID-1, 5, 6…) for redundancy/fault tolerance.

Row vs column storage | | Row-oriented | Column-oriented | |—|—|—| | Stores | Tuples (rows of same schema) in contiguous pages | Columns (projections) stored separately, sorted by dimension values | | Best for | **Write-oriented transaction processing (OLTP)** | **Data-warehouse workloads** — aggregation of measure columns by dimension columns | | Access | Indexes, **B+-tree** on primary key | **Join indexes** needed to link projections |

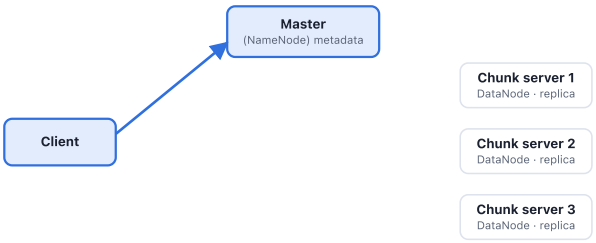
Parallel database architectures

Architecture	Description
Shared memory	Servers with multiple CPUs share one memory address space, managed by an **SMP OS**; schedules processes in parallel
Shared disk	**Hybrid** — independent servers share storage via a high-speed network (**NAS or SAN**) over Ethernet, Fibre Channel, InfiniBand
Shared nothing	**Cluster of independent servers, each with its own disk**, connected by a high-speed network

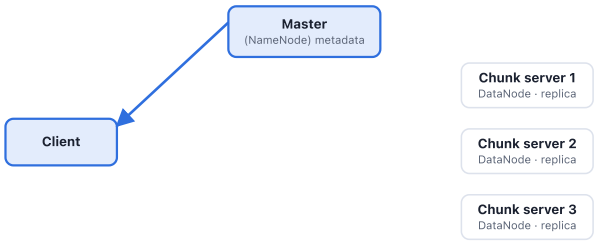
Parallel relational DB features: efficient SQL execution over multiple processors; in shared-nothing, **tables partitioned and distributed** across processors; **SQL optimizer handles distributed joins**; **distributed two-phase commit (2PC) locking** for transaction isolation; **fault tolerance by transferring control to a standby system**. Examples: **Oracle, DB2, SQL Server** (transaction processing); **Vertica, Teradata, Netezza** (data warehouses).

Cloud file systems: GFS and HDFS

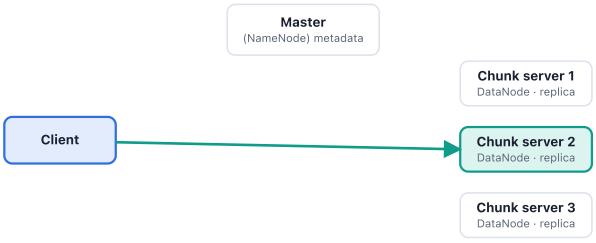
GFS / HDFS read and append paths diagram sequence



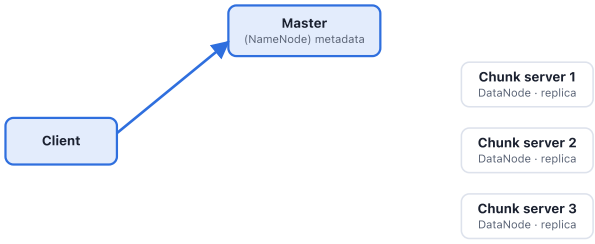
1. READ 1: client sends the **full path + offset** to the master.



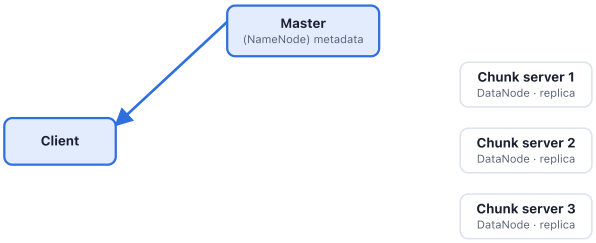
2. READ 2: master replies with metadata for **one** of the replicas; the client **caches** it.



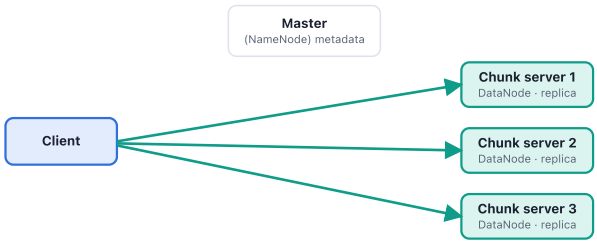
3. READ 3: client reads the data **directly from that chunk server** (master is not in the data path).



4. APPEND 1: client sends the **full path** to the master.



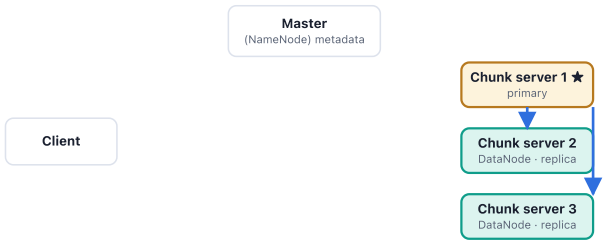
5. APPEND 2: master replies with metadata for **all replicas**.



6. APPEND 3: client **pushes the data to all chunk servers**; each acknowledges receipt.



7. APPEND 4: master **designates one chunk server as primary**.



8. APPEND 5: the **primary chooses the offset** (may be beyond EOF for concurrent writers) and tells the replicas. If any replica fails at that offset, the **client retries**. Replicas: **3** on different racks; heartbeats detect failures.

- **GFS (Google File System):** manages **relatively large files** using **very large distributed clusters of commodity (Linux) servers** connected by high-speed networks; handles **failures even during**

reads/writes of individual files; supports **parallel reads, writes and appends by multiple simultaneous clients**.

- **HDFS (Hadoop Distributed File System): open-source implementation of the GFS architecture** (available on Amazon EC2).

Why fault tolerance is a necessity — the formula:

$$P(\text{system failure}) = 1 - \big(1 - P(\text{component failure})\big)^N$$

- N = number of components; system fails if **any** component fails.
- As N grows this probability **approaches 1** — e.g., $p = 0.001$, $N = 1000$: $1 - 0.999^{1000} \approx 1 - e^{-1} \approx 0.63$.

> ⚠️ *Flag:* the transcript says "if N is pretty large … the risk of failure is minimum", which is the opposite of what the formula shows. With large N, some failure is almost certain, so **fault tolerance is mandatory**. Answer numerical MCQs using the formula.

GFS vs HDFS terminology (matching question) | GFS | HDFS | Role | |—|—|—| | **Master** | **NameNode** | **Single (logical) master controls the namespace** (metadata) | | **Chunk server** | **DataNode** | Commodity Linux servers storing the data | | **Chunk** | **Block** | Large files are **broken into** these | | Replication | Replication | Each chunk/block **replicated 3 times on different physical racks and network segments** |

Read operation (GFS): `Client → master: full path + offset` → `master → client: metadata for ONE of the replicas` → `client caches metadata` → `client reads directly from that chunk server`.

Write/append operation (GFS):

1. Client sends full path to master.
2. Master replies with **metadata for ALL replicas**.
3. Client **sends the data to be appended to all chunk servers**; they acknowledge receipt.
4. Master **designates one chunk server as primary**.
5. Primary **chooses an offset** and appends its copy (may append **beyond EOF** to account for multiple simultaneous writers).
6. Primary sends the offset to the replicas; **if not all replicas succeed at that offset, the client retries** (so all replicas hold the data at the same offset).

Fault tolerance in GFS: master exchanges **heartbeat messages** with chunk servers; on failure, chunk-server metadata is updated; if the **primary fails, the master assigns a new primary**; clients occasionally retry the failed server, update metadata from the master, and retry.

BigTable (Google)

- **Distributed structured storage system built on GFS.**
- Data accessed by **row key, column key, timestamp**.
- Each column stores arbitrary **name–value pairs** in the form **column-family:label**.
- **Set of column families is fixed when the table is created**; **labels within a family can be created dynamically at any time**.
- Each cell can store **multiple versions** of data in **decreasing timestamp order**.
- Tables are split into row ranges called **tablets**, each managed by a **tablet server**, which stores each column family for a row range in a separate distributed file called an **SSTable**.
- A **single metadata table** is managed by metadata servers; because metadata can be very large it is itself split into tablets, with a **root tablet** pointing to the other metadata tablets.
- Supports **large parallel reads and inserts, even simultaneously on the same table**; inserts are done in **sorted** fashion → more work than a simple append.

Dynamo (Amazon)

Dynamo quorum checker calculator

N replicas	R (read quorum)	W (write quorum)
3	**2**	**2**

R + W	**4**
N	**3**
Guaranteed overlap (R + W − N)	**1**

Quorum-consistent: every read sees the latest write

Small W ⇒ fast writes but large R; small R ⇒ fast reads but large W.

Worked example with the default values (interactive in the HTML version).

- Supports **large volumes of concurrent updates, each small in size** (contrast: BigTable supports **bulk** reads/writes).
- **Data model: simple key–value pairs** (values are arbitrary byte arrays) — well suited to **web-based e-commerce**.
- **Not dependent on an underlying distributed file system** — handles failure, conflict resolution, etc. itself.
- Uses **MD5** to generate a **128-bit hash**; the hash range is mapped to **virtual nodes arranged in a ring**; each physical node manages several virtual nodes at distributed ring positions (load balancing for transient failures, network partitions).
- An object is replicated at its **primary virtual node plus $N-1$ additional virtual nodes** (lecture: "N is the number of physical nodes"; effectively N = number of replicas).

- A write executes at one virtual node, which **forwards the request to all nodes holding replicas**; a **quorum protocol maintains eventual consistency** under many concurrent reads/writes.
- **Distributed object versioning:** each write creates a new version with its local timestamp.

Quorum consistency rule (numerical!):

$$R + W > N \;\Rightarrow\; \text{quorum-consistent}$$

- R = number of replicas read; W = number of replicas that must acknowledge a write; N = number of replicas.
- Intuition: every read set overlaps every write set, so a read sees the latest write.
- **Trade-off:** efficient writes (small W) ⇒ a **large number of replicas must be read** (large R); efficient reads (small R) ⇒ **large number written** (large W).
- Example: $N = 3$: $(R, W) = (2, 2)$ ✓ consistent; $(1, 3)$ ✓ (fast reads); $(3, 1)$ ✓ (fast writes); $(1, 1)$ ✗ (may read stale).
- Node-level storage engines: **Berkeley DB (used by Amazon)**, MySQL.

Datastores (Google App Engine Datastore, Amazon SimpleDB)

- Simple **key–value** database stores; **all entities (objects) reside in one big table**.
- Exploit **column-oriented storage** — store data as **column families** (vs row/tuple-based RDBMS).
- **Multiple index tables** support efficient queries; the big table is **horizontally partitioned ("sharded")** across disks, stored **lexicographically by key** → enables **prefix and range queries**.
- Entities are **grouped for transactions**.
- Indexes: automatically created **single-property indexes** and a **"kind" index** (efficient lookup like "select all of a kind"); **configurable (composite) indexes**; query execution chooses the **index with the highest selectivity**.

MCQ traps (3.3)

- **GFS read: metadata for ONE replica; write: metadata for ALL replicas.**
- **Master = NameNode, chunk server = DataNode, chunk = block**; default replication **3**.
- In GFS append, the **primary chunk server** chooses the offset (not the master, not the client).
- BigTable: column **families fixed at creation**; **labels dynamic**; versions in **decreasing** timestamp order; tablets → tablet servers → SSTables.
- **Dynamo = key–value, small concurrent updates, MD5 ring, quorum, eventual consistency, no dependence on a DFS**; **BigTable = built on GFS, bulk reads/writes**.
- Quorum consistency requires $R + W > N$ (strictly greater).
- Row-store → OLTP (write-heavy); column-store → warehouse analytics.

Quick revision (3.3)

- $P(\text{fail}) = 1 - (1 - p)^N$ → fault tolerance necessary.

- GFS/HDFS: master/NameNode, chunk servers/DataNodes, 3× replication on different racks, heartbeats.
- BigTable: (row, column, timestamp); column-family:label; tablets & SSTables; root tablet.
- Dynamo: key–value, MD5 128-bit, ring of virtual nodes, N replicas, $R + W > N$.
- Datastore: one big table, sharded, lexicographic, kind index, highest-selectivity index.

3.4 MapReduce (L14)

Core concept

- **Programming model developed at Google** to implement **large-scale search and text processing on massively scalable web data** stored in **BigTable and GFS**.
- Designed for **processing and generating large volumes of data via massively parallel computation** using **tens of thousands of processors**.
- **Fault tolerant:** computation progresses **even if processors or the network fail**.
- **Hadoop:** open-source implementation of MapReduce, **initially developed at Yahoo**; available as pre-packaged AMIs on Amazon EC2.
- Parallel computing began for compute-intensive science, later used in databases; MapReduce programs run over shared-memory / shared-disk / shared-nothing configurations.

Parallel efficiency (needed for all MapReduce numericals)

- Ideal: a task taking time T on one processor takes T/p on p processors — only if the work has **inherent parallelism**.
- **Sources of inefficiency: synchronization** among processors, **message-communication overhead**, **imbalance in work distribution**.

$$\varepsilon = \frac{T}{pT_p}$$

- **Scalable** if ε **remains constant** as data size grows **with a proportional increase in processors**.
- For a scalable algorithm, ε **increases with data size for a fixed number of processors**.

Word-count efficiency analysis (classic derivation — know the results)

Setup: n documents, m distinct words; each word occurs f times per document on average; time to read a word = time to send a word to another processor = c; adding to running totals is negligible. Distributed memory with a shared disk (no contention).

- Single processor: $T_1 = nmfc$.

Approach	Work per processor	Parallel efficiency	Scalable?
1. Split the words: each processor counts m/p words over **all** n documents	Reads up to $n\,m\,f$ words (the whole collection)	$\varepsilon = \frac{nmfc}{p \cdot nmfc} = \boxed{\frac{1}{p}}$	**No** — falls as p grows; most reads are wasted (no hit)
2. Split the documents: each processor counts **all** m words over n/p documents, then partial counts are exchanged and summed	Read: $\frac{n}{p}mfc$; write partial counts: cm; communicate to $p-1$ processors & add p sub-vectors (to build $1/p$ of the final vector): $\approx cm$	$\varepsilon = \frac{1}{1+\frac{2p}{nf}}$	**Yes** — constant when n and p grow proportionally; **→ 1 for fixed p as n grows**; since $p \ll nf$, $\varepsilon \approx 1$

- **Intuition:** in approach 2 **every read is useful** (contributes to the result); in approach 1 each processor scans all words but only $1/p$ of them matter.
- **Numerical check:** $n = 10^4$ docs, $f = 1$, $p = 10$: approach 1 → $\varepsilon = 0.1$; approach 2 → $\varepsilon = 1/(1 + 20/10^4) \approx 0.998$.
- Approach 2 is exactly the MapReduce pattern (local map counts → shuffle → reduce/sum).

The MapReduce model

MapReduce word count — data flow

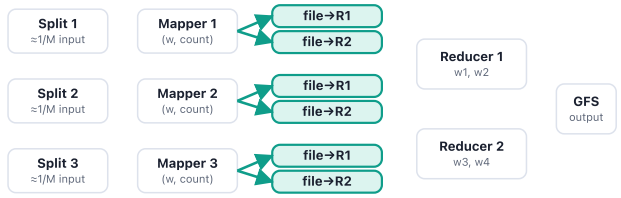
diagram sequence



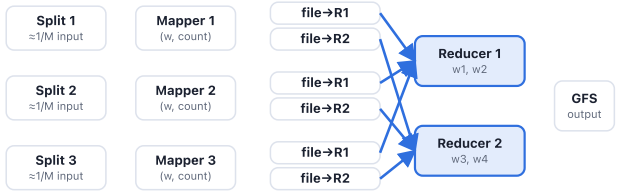
1. The **single master** assigns each of the **M mappers** about **1/M** of the input (read from GFS).



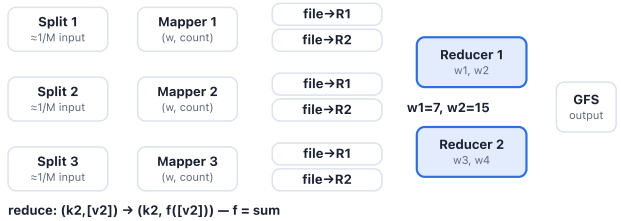
2. Map: each mapper transforms key–value pairs, e.g., emits **(word, count)** for its documents.



3. Each mapper writes **one key-sorted file per reducer** to its **local disk**; the master records where the files are.



4. The master tells reducers where their partitions are; reducers **pull them via RPC** from every mapper (shuffle).



5. Reduce: each reducer groups by key and applies f (sum) — e.g., reducer 1 owns w1, w2.



6. Final results are written back to **GFS**. Mapper failure ⇒ redo its whole task; reducer failure ⇒ only unfinished tasks.

- **Two phases: Map and Reduce.** A **configurable** number of M **mapper** and R **reducer** processors; computation coordinated by a **single master** process.

Map phase

- Each mapper reads **≈** $1/M$ **of the input** from the global file system.
- **Map transforms one set of key–value pairs into another:** $\text{map} : (k_1, v_1) \rightarrow [(k_2, v_2)]$.
- Each mapper writes its results into **one file per reducer**, **sorted by key**, on its **local** file system. The **master keeps track of the locations** of these files.

Reduce phase

- The **master informs reducers** where the partial results are stored (on mappers' local disks).
- Reducers make **remote procedure calls (RPC) to the mappers** to fetch the files.

- Each reducer **groups results by key** and applies a function f: $(k_2, [v_2]) \to (k_2, f([v_2]))$ (e.g., f = sum for word count).
- **Final results are written back to GFS.**

`Input (GFS) → M mappers [(k1,v1)→(k2,v2), one sorted file per reducer, local disk] → master tells reducers file locations → R reducers fetch via RPC, group by key, apply f → output (GFS)`

Word-count example (3 mappers, 2 reducers): each mapper emits $(w_i, \text{count in its documents})$ and writes one file for each reducer (e.g., reducer 1 handles w_1, w_2; reducer 2 handles w_3, w_4); reducers sum counts, e.g., $w_1 = 7$, $w_2 = 15$.

Fault tolerance in MapReduce

- **Heartbeat messages** from workers.
- If communication exists but **no progress**, the master **duplicates those tasks** onto processors that have finished or are free.
- **Mapper fails:** the master **reassigns its key-range to another node for re-execution** (its local outputs are lost).
- **Reducer fails: only the remaining (unfinished) tasks are reassigned**, because completed reduce outputs are **already written to GFS**.

Efficiency of MapReduce (formula)

Setup: uniprocessor time wD for data volume D (reading, computing, writing); c = time to read/write one word.

- Map stage: time $c_m D$, output σD; Reduce stage: time $c_r \sigma D$, output $\sigma \mu D$.
- No decomposition overhead: $wD = c_m D + c_r \sigma D$.
- With P processors acting as both mappers and reducers, the extra cost is that each mapper **writes** $\sigma D/P$ **to local disk** and each reducer **reads** $\sigma D/P$ **remotely** (it reads $\sigma D/P^2$ from each of the P mappers). This adds $2c\sigma D/P$ per processor.

$$\varepsilon_{MR} = \frac{wD}{P\left(\frac{wD}{P} + \frac{2c\sigma D}{P}\right)} = \boxed{\frac{1}{1 + \frac{2c}{w}\sigma}}$$

- **Interpretation:** efficiency is **independent of** $P \Rightarrow$ **scalable**; it decreases as the map output ratio σ or I/O cost c grows relative to the work w.

Applications

- **Indexing a large collection of documents** (Google's main motivation): **map** emits (word, document/record-id) pairs for each word; **reduce** groups pairs by word and creates an index entry (list of documents).

- **Relational operations** with MapReduce: execute SQL-like **joins** and **group-by** on large data — adding large-scale fault tolerance to parallel-database-style processing.

MCQ traps (3.4)

- **Each mapper produces one output file per reducer** (not one file overall); files are **sorted by key** and stored **locally**, not in GFS.
- **Reducers pull** data via **RPC** after the master tells them where it is.
- **Mapper failure ⇒ re-execute its whole task; reducer failure ⇒ only unfinished tasks** (finished output already in GFS).
- Approach 1 (split words) has efficiency $1/p$ — **not scalable**; approach 2 (split documents) → $1/(1+2p/nf)$ — **scalable**.
- MapReduce efficiency $1/(1+2c\sigma/w)$ does **not** depend on P.
- Hadoop came from **Yahoo** (lecture), MapReduce from **Google**.

Quick revision (3.4)

- $\varepsilon = T/(pT_p)$; scalable = constant ε when data and processors grow together.
- map: $(k_1, v_1) \to [(k_2, v_2)]$; reduce: $(k_2, [v_2]) \to (k_2, f([v_2]))$.
- One single master; M mappers; R reducers; mapper output per reducer, local, sorted.
- $\varepsilon_{MR} = 1/(1+2c\sigma/w)$.

3.5 OpenStack (L15; demo in Week 4, L16)

Core concept

- **OpenStack is a cloud operating system that controls large pools of compute, storage and networking resources throughout a data center, all managed through a dashboard** that gives administrators control while **empowering users to provision resources through a web interface**.
- **Open source**; primarily used for **IaaS** (can be installed on blade servers or even a couple of PCs/laptops — good for lab experiments in resource/power management, green cloud).
- **IIT KGP's Meghamala** is built on OpenStack (blade servers).
- Capabilities: **VMs on demand (provisioning and snapshotting)**, networking, **storage for VMs and arbitrary files**, **multi-tenancy** (quotas per project; a user can belong to multiple projects).
- **History:** started as a collaboration between **NASA and Rackspace** (~**2010**); regular releases named alphabetically — **Austin** (first) … Newton, Pike, Queens.

Components (very high-yield matching question)

Service	Project name	Role
Compute	**Nova**	**Manages the life cycle of compute instances — spawning, scheduling, decommissioning VMs** on demand; VM flavours; ephemeral + persistent storage attached
Networking	**Neutron**	**Network connectivity as a service** for other OpenStack services (e.g., Nova); **API for users to define networks and attachments**; **pluggable architecture** supporting many vendors/technologies; internal (cloud-internal) and external networks
Object storage	**Swift**	**Stores/retrieves arbitrary unstructured data objects via a RESTful, HTTP-based API**; highly fault tolerant via **replication and scale-out**; **not a file server with mountable directories**; writes objects to multiple devices across the cluster
Block storage	**Cinder**	**Persistent block storage for running instances**; **pluggable driver architecture** for creating/managing block devices
Identity	**Keystone**	**Authentication and authorization** for other OpenStack services; **catalogue of endpoints** of all services
Image	**Glance**	**Stores and retrieves VM disk images**; used by Nova during instance provisioning
Telemetry	**Ceilometer**	**Monitors and meters** the cloud for **billing, benchmarking, scalability and statistical** purposes (realizes “measured service”; helps plan capacity)
Dashboard	**Horizon**	**Web-based self-service portal** — launch instances, assign IP addresses, configure access control; admin front end

> ⚠️ *Transcript slip:* L15 says “project newton … networking” — the networking project is **Neutron** (Newton is a release name).

Sub-components: Glance — database, registry, API, storage adapter. Cinder — database, scheduler, API, volume, backup service. Keystone — policy backend, token backend, catalogue service, identity service, assignment backend, credentials backend. Neutron — API, scheduler, plugins, queues. Nova — API, scheduler, conductor, compute nodes, message queue. Clients: CLI (nova, neutron, swift…), cloud management tools (e.g., RightScale), GUI tools (dashboard, Cyberduck).

Storage types in OpenStack (classic comparison)

	Ephemeral	Block	Object
Persists	**Until the VM is terminated**	**Until specifically deleted by the user**	**Until specifically deleted**
Accessed	From within the VM as a **local file system**	Within the VM as a **block device**	**From anywhere** (REST/HTTP)
Used for	Running the **OS** and **scratch space**	**Adding additional persistent storage** to a VM	**Storing files, including VM images**
Managed by	**Nova**	**Cinder**	**Swift**

Workflow — launching a VM (sequence question)

1. User logs in to **Horizon**, specifies VM parameters (name, **flavor**, keys…) and clicks *create*.
2. Horizon sends an HTTP request to **Keystone** (credentials in header); **Keystone authenticates** and returns a **temporary token**.
3. Horizon sends a POST request to the **Nova API**; Nova API **validates the token with Keystone**.
4. Nova initiates provisioning and **saves state to the database**; nova-api makes an **RPC cast to the scheduler** via the **message queue**.
5. **Nova scheduler** fetches cluster info from the DB, **filters and selects a compute node (host)**, publishes to the compute queue.
6. **nova-compute** picks up the message, makes RPC calls to **nova-conductor** (DB access).
7. **Neutron** configures networking; **Cinder** provides block devices.
8. **Image URI looked up through Glance; image retrieved via Swift.**
9. **The VM is rendered by the hypervisor.**

Short form: *Horizon → Keystone (auth) → Nova (provision, DB) → Nova scheduler (host) → Neutron (network) → Cinder (block) → Glance (image URI) → Swift (image) → hypervisor (VM)*. Token flow: user → Keystone → Nova → Glance → Neutron.

MCQ traps (3.5)

- **Swift = object storage; Cinder = block storage** (commonly swapped).
- **Glance = images** (not storage of user files); **Ceilometer = telemetry/metering**; **Keystone = identity**; **Horizon = dashboard**; **Nova = compute**; **Neutron = networking**.
- Ephemeral storage **disappears with the VM** (managed by Nova).
- OpenStack began with **NASA + Rackspace**.
- Swift is **not** a file server with mountable directories.

Quick revision (3.5)

- Nova, Neutron, Swift, Cinder, Keystone, Glance, Ceilometer, Horizon — know each role.
- Ephemeral (Nova, dies with VM) / Block (Cinder, persistent device) / Object (Swift, anywhere, VM images).
- VM launch: Horizon → Keystone → Nova → scheduler → Neutron → Cinder → Glance/Swift → hypervisor.

Week 3 — High-yield takeaways

1. KPI → SLO → SLA; off-the-shelf vs negotiable SLA; cloud SLA needs automation, security/privacy/trust focus; neutral third-party monitoring.
2. $C_v = \sigma/|\mu|$; n independent workloads → $C_v/\sqrt{n}$ (100 workloads → 10 %); negative correlation → 0; perfect correlation → unchanged.
3. Cloud cheaper iff $U = C/B < P/A$; penalty $\propto \int |D - R|$; exponential demand → exponential penalty.
4. $P(\text{fail}) = 1 - (1-p)^N$; GFS/HDFS master/NameNode, chunk server/DataNode, 3× replicas, read = one replica's metadata, write = all replicas, primary picks offset.
5. BigTable (row, column, timestamp; tablets; SSTable) vs Dynamo (key–value, MD5 ring, quorum $R + W > N$, eventual consistency).
6. Parallel efficiency $T/(pT_p)$; word count: $1/p$ (not scalable) vs $1/(1 + 2p/nf)$ (scalable); MapReduce $1/(1 + 2c\sigma/w)$.
7. Mapper → one sorted local file per reducer; reducer pulls via RPC; master tracks; mapper failure = redo all, reducer failure = redo remaining.
8. OpenStack: Nova/Neutron/Swift/Cinder/Keystone/Glance/Ceilometer/Horizon; NASA + Rackspace.

Week 4 — OpenStack Demo, Microsoft Azure, Google Cloud Platform (Lectures 16–20)

> **Source:** L16 Open Source Cloud: OpenStack Demo · L17 Case study with a commercial cloud: Microsoft Azure · L18 Azure Demo · L19 Case Study: Google Cloud Platform (GCP) · L20 Demo on GCP
>
> This week is mostly **demonstrations**. Click-by-click steps are omitted; what is kept is what MCQs can test: **which service does what, which model (IaaS/PaaS) each product belongs to, the order of steps, key terms (flavor, security group, floating IP, web role/worker role, resource group, app service plan, bucket, app.yaml)**.

4.1 OpenStack demo — Meghamala, IIT Kharagpur (L16)

> 🎯 **Your practice set for L16** (your file *w1*): 50 questions · 16 easy · 23 medium · 11 hard · 11 with ⚠️ notes. Answers checked against this lecture.
> Full set and answer key: chapter “Your practice sets”, Week 4.

What Meghamala is

- IIT KGP’s **experimental private IaaS cloud built on OpenStack** (on blade servers), used by faculty and research scholars; reachable only inside the IIT KGP network.
- **Other services built on top of Meghamala: Meghadoop** — a **Hadoop cluster** service; **Meghadata** — a **personal cloud storage** service (“Dropbox-like”).
- **Three VM flavours: IITKGP Regular** (2 vCPU, 4 GB RAM, ephemeral disk ≈ 45 GB as stated in the demo), **IITKGP Large**, **IITKGP Extra Large**; persistent volumes (e.g., 20 GB or 60 GB) on request.
- **Three OS images offered: Ubuntu, CentOS, Fedora**.
- Users request a VM through a request form; the admin provisions it.

What the OpenStack dashboard (Horizon) shows

Dashboard item	OpenStack service behind it
Overview — vCPUs, RAM, disk in use, vCPU-hours (e.g., 134 vCPUs, 316 GB RAM, 2945 GB disk at demo time)	Nova (compute) + Ceilometer (usage)
Instances — running VMs	**Nova**
Volumes and snapshots	**Cinder**
Images	**Glance**
Networks (IPv4; internal and external)	**Neutron**
Hypervisors — physical compute nodes (demo: 3 physical machines, 48 vCPUs each)	Nova compute
Access & Security → Security groups	Firewall rules for VMs

- **Security group = a firewall-like set of rules** defining **which incoming and outgoing traffic (by port/service) is allowed** for a VM. Assign a security group when creating a VM and its firewall rules apply automatically — no per-VM firewall configuration needed.
- The **admin is also a tenant**: the dashboard has a **Project** view (tenant) and an **Admin** view (e.g., hypervisors list).

Launching and using a VM (sequence)

1. **Instances → Launch Instance.**
2. **Availability zone** (only one in Meghamala: **"nova"**), **instance name**, **flavor** (e.g., IITKGP Regular), **instance count**.
3. **Boot source = image** (e.g., CentOS) — fetched via Glance.
4. **Security group** (one default group), **network** (choose the **internal** network).
5. Optional **post-creation script**; disk partition automatic → **Launch**.
6. Status goes **Build → Active**; power state **Running**.
7. The VM first gets an **internal IP**; to reach it from outside the cloud, **allocate and associate an external ("floating") IP** from the pool.
8. Connect through a GUI client (**x2go**, XFCE desktop) using the public IP and credentials; the VM behaves like any normal machine (Internet access, YouTube, terminal).
9. **Terminate instance** → deletes the VM and **releases all its resources**. (Other admin actions: shut down, view log.)

MCQ traps (4.1)

- Security groups control **both incoming and outgoing** traffic; they are assigned per VM to avoid configuring each firewall individually.
- A VM gets an **internal IP** automatically; **external access requires associating an external (floating) IP**.

- **Terminate** ≠ shut down: terminate **deletes** the VM and frees resources.
- Meghadoop = Hadoop service; Meghadata = personal storage (Dropbox-like).

Demo details your practice set asks about (4.1)

- **Who and what:** the demo is run by **Rajesh, the Meghamala administrator**.
  - The VM created is named **"nptel"**, uses the **CentOS** image and goes on the **internal** network.
  - The single **availability zone is "nova"**. Status goes **Build → Active**, with power state **Running**.
  - Login is through **X2Go** (GUI remote-desktop client, XFCE session). He then opens a terminal and a browser, checks the internet, and plays YouTube.
- **Admin overview** shows usage graphs at a glance (vCPU-hours etc.). The **Hypervisors** tab is admin-only: 3 physical machines, **48 vCPUs** each, e.g. **14 instances** on one. An **instance** = a VM.
- **Ephemeral vs persistent:**
  - The ephemeral disk comes with the **flavour** and is **deleted when the VM is terminated**.
  - A **persistent volume** (Cinder, typically **20 GB** on request) **survives** termination.
  - **vCPU** = virtual CPU.
- **Terminate vs shut down:** *shut down* only powers the VM off, and it keeps its allocation. *Terminate* deletes the VM and **frees** its CPU, RAM and disk.
- **External IPs come from a pre-allocated pool.** Associate one so that people **outside** the cloud can reach the VM.
- **Professor's takeaway:** set up your **own small OpenStack cloud**. It is a good exercise for seeing both the admin and the user side. "Realised" = how compute, storage and image services are **actually implemented** in a working system.

> **Beyond the lecture (general OpenStack facts the practice set also tests):**
>
> - A **snapshot** of a VM is stored as a new **Glance image**.
> - A VM can **boot from an image or from a volume**.
> - **nova-scheduler** picks the hypervisor; if no host has room for the flavour, the launch **fails (ERROR state)**.
> - A released floating IP goes **back to the pool**.
> - A security group that allows only port 22 inbound lets **SSH** in and blocks everything else inbound (outbound is allowed by default).
> - **View Log** shows the VM's console output.
> - Real OpenStack can also **resize** a VM to another flavour. The lecture's route is simply to request a VM with a bigger flavour.

4.2 Microsoft Azure (L17–L18)

Core concept

- Described in the lecture as **popular mainly as a PaaS**, maintained by Microsoft with **data centers across the globe**.
- **Definition (Microsoft):** *a growing collection of integrated cloud services which developers and IT professionals use to build, deploy and manage applications through a global network of data centers.* Developers can build and deploy wherever they want with **tools, applications and frameworks of their choice**; can run in the cloud or on-premises (**hybrid**).

Azure platform components (lecture version)

Component	Role
Windows Azure	**Windows-based environment for running applications and storing data** on servers in Microsoft data centers
SQL Azure	**Data services in the cloud based on SQL Server**
AppFabric	**Cloud services for connecting applications running in the cloud or on premises**

Windows Azure: compute, storage, fabric

Part	Role
Compute	Runs applications; supports apps with a **very large number of simultaneous users** that **scale out**
Storage	Storing and accessing data — simple **blobs**, more structured **tables**, and **queues** for exchanging data between parts of an application
Fabric	**Manages the underlying resources** (bare metal across data centers) and gives a common way to **manage and monitor** applications

Web role and worker role (very examinable)

Role	Purpose
Web role	A **running environment that users access over the Internet** — the **web application service** (front end, behind a **load balancer**)
Worker role	An environment for **running processing** — handles **distributed or complex tasks** in the background

- **Any service must include at least one role of either type**, but may have **any number of web roles or worker roles**.
- **Worker roles communicate with web roles through Azure storage queues.**
- **Each VM contains an agent** that lets the application **interact with the Azure fabric**.
- Flow: user → Internet → load balancer → web role → (queue) → worker role → results back.

Azure offerings highlighted

- **Deploy anywhere with your choice of tools; consistent hybrid capabilities** — build apps your own way, **connect on-premises data and apps**, **extend the cloud on-premises**; uses open-source technologies too.
- **Protect assets:** security, privacy, compliance and transparency.
- **Accelerate app innovation:** build simple-to-complex projects in a consistent portal; **build quickly, manage proactively, deliver mobile apps seamlessly**.
- **Analytics and data services:** detect anomalies, predict behaviours, recommend actions.

Azure demo — deploying a Python (Flask) web app (L18)

1. Log in at **portal.azure.com** → open **Cloud Shell** (terminal in the browser).
2. `git clone` a sample "Hello World" Flask app; install **Flask** (Python micro web framework); run locally and test at **localhost:5000**.
3. **Create a deployment user** (`az webapp deployment user set`) — needed for **FTP and local Git deployment**.
4. **Create a resource group** (`az group create`) — **a logical container into which Azure resources (web apps, databases, storage accounts) are deployed and managed**.
5. **Create an App Service plan** (`az appservice plan create`) — specifies the **location, size and features of the web server** that hosts the app.
6. **Create the web app** (`az webapp create`) — provides **hosting space and a URL**: `<appname>.azurewebsites.net` .
7. **Configure Python version** (`az webapp config set …`) — uses the platform's default container.
8. **Configure local Git deployment**, `git remote add azure <url>` , then **`git push azure master`** (asks for the deployment user's password).
9. Edit `main.py` → `git commit` → `git push azure master` → refresh to see the change ("Welcome to the NPTEL course on cloud computing").

- App Service supports deployment via **FTP, local Git, GitHub, Visual Studio Team Services**, etc.

MCQ traps (4.2)

- **Web role = user-facing front end; worker role = background processing.** They communicate via **storage queues**.
- A service needs **at least one** role (of either type), not one of each.
- **Resource group** = logical container; **App Service plan** = location/size/features of the hosting server; **web app** = hosting space + URL.
- **AppFabric** connects cloud and on-premises applications; **SQL Azure** = SQL Server data services.
- The lecture classes Azure as **PaaS**.

4.3 Google Cloud Platform (L19–L20)

Core concept

- **GCP = a set of (modular) cloud services that enables developers to build, test and deploy applications on Google's reliable infrastructure** — from a simple website to complex applications.
- Infrastructure is spread over **regions**, each with **zones**; resources can be **global, regional or zonal**.
- Motto: **Build, Store, Analyze.**
- Runs on the **same infrastructure** Google uses for its own products (lecture figures: billions of search results in milliseconds, ~**6 billion hours of YouTube per month**, ~**425 million Gmail users**) — globally connected, highly redundant (fault tolerant).
- **Developer focus:** rapidly develop, deploy and iterate **without worrying about system administration**; Google manages application, database and storage servers; **managed services, developer tools/SDKs, console and administration**.
- **Mix and match services:** VMs, managed platform, blob storage, block storage, NoSQL datastore, MySQL databases, big-data analytics.
- **Scale to millions of users:** apps **scale up automatically** under heavy load and **scale down** when traffic subsides; **pay only for what you use**. **Managed services such as App Engine and Cloud Datastore provide auto-scaling.**
- **Performance:** consistent CPU, memory and disk performance; global network and edge caches; transparent maintenance; support from community, partners and premium packages.

GCP services (lecture grouping)

Category	Services	Model
Compute	**App Engine**	**PaaS** — focus on your code, no patching/maintenance
	Compute Engine	**IaaS** — raw, flexible VMs
Storage	**Cloud Storage**	Flexible **object storage** with global edge caching
	Cloud SQL	**MySQL** (relational)
	Cloud Datastore	**NoSQL**
	Persistent disk	Block storage for VMs
App services	**BigQuery**, **Cloud Endpoints**, caching, queues	APIs to add functionality quickly instead of building from scratch

- Migrate web apps to App Engine; scale apps using **Cloud Endpoints**; integrate Google services using **Google APIs**.

GCP demo 1 — hosting a static website on Cloud Storage (L20)

1. Log in to the **GCP console**.

2. **Storage → Create bucket** (give a name, choose **storage class**, e.g., **Regional, asia-east1**).
3. **Upload** the site's files/folders — a site needs **`index.html` (home page)** and a **`404.html` (not-found page)**.
4. **Share the files publicly** (tick public link for every file, including those inside folders).
5. Open the public link — served from **`storage.googleapis.com/<bucket>/index.html`**. → Uses only GCP **storage services**.

GCP demo 2 — building a web app with App Engine (L20)

1. **Create a new project** (gets a **globally unique project ID**).
2. Activate **Cloud Shell**; `git clone` Google's sample apps.
3. Two key files:
   - **`app.yaml`** — **configuration file**: runtime **python27**, **`threadsafe: true`** (the app can handle **many simultaneous requests**), URL **handlers** (any URL matching the regex is handled by `main.app`).
   - **`main.py`** — the app (Flask), prints "Hello World".
4. Start the **local development server** → check via **Web preview** → edit the message → restart and re-check.
5. **Deploy:** **`gcloud app deploy`** (choose the region) → files uploaded to Cloud Storage → app deployed to App Engine.
6. **View:** **`gcloud app browse`** (URL contains the project ID). The project can later be shut down.

MCQ traps (4.3)

- **App Engine = PaaS; Compute Engine = IaaS.**
- **Cloud SQL = MySQL; Cloud Datastore = NoSQL; Cloud Storage = object storage.**
- Static website hosting uses **Cloud Storage buckets** (not App Engine) and needs files to be **publicly shared**.
- `app.yaml` is the **configuration** file; `threadsafe: true` ⇒ concurrent requests.
- Deployment command: **`gcloud app deploy`**; open in browser: **`gcloud app browse`**.

Week 4 — High-yield takeaways

1. OpenStack dashboard maps to services: instances→Nova, volumes→Cinder, images→Glance, networks→Neutron; security groups = per-VM firewall rules (in/out); external access needs a floating IP; terminate releases resources.
2. Meghamala: 3 flavours (Regular/Large/Extra Large), 3 OSs (Ubuntu/CentOS/Fedora), Meghadoop (Hadoop), Meghadata (personal storage).
3. Azure (PaaS in lecture): Windows Azure + SQL Azure + AppFabric; compute/storage/fabric; web role (front end) vs worker role (processing) via storage queues; agent in each VM talks to fabric.

4. Azure deploy flow: deployment user → resource group → App Service plan → web app → config Python → git push.
5. GCP: App Engine (PaaS), Compute Engine (IaaS), Cloud Storage (object), Cloud SQL (MySQL), Datastore (NoSQL), BigQuery/Endpoints; auto-scaling via managed services.
6. GCP demos: bucket + index.html/404.html + public sharing; App Engine with app.yaml + main.py → `gcloud app deploy`.

Week 5 — Tutorials (SLA, Economics, MapReduce) and Resource Management (Lectures 21–25)

> **Source:** L21 SLA – Tutorial · L22 Economics Tutorial · L23 MapReduce – Tutorial (titled "Lecture 24" in the transcript) · L24 Resource Management – I · L25 Resource Management – II
>
> **This is the most numerical week of the course.** Each tutorial problem below is solved step by step, followed by a **reusable template** and the **traps**. Theory for these tools is in Week 3 (§3.1 SLA, §3.2 economics, §3.4 MapReduce).

5.1 SLA tutorial (L21)

> 🎯 **Your practice set for L21** (your file *w2*): 50 questions · 10 easy · 27 medium · 13 hard · 7 with ⚠️ notes. Answers checked against this lecture.
> Full set and answer key: chapter "Your practice sets", Week 5.

Availability formulas used in the tutorial

$$\text{Availability (\%)} = \left(1 - \frac{\text{Downtime}}{\text{Expected service time}}\right) \times 100 \qquad \left(\text{Azure: } \frac{\text{Max available minutes} - \text{Downtime}}{\text{Max available minutes}} \times 100\right)$$

- **Expected service time** = days in period × service hours per day (only the hours the service is *supposed* to run).
- SLA is **violated** if measured availability < guaranteed availability.
- **Service credit** = percentage of the bill refunded, chosen from the credit table using the measured availability.
- **Effective cost payable** = total cost – service credit.

> ⚠️ *Flag on the formula:* in Problem 1 the lecture divides downtime by the **uptime** (360 – 10.75 h), giving 96.92 %; in Problem 2 it divides by the **total expected time** (360 h), giving 97.385 %. The total-time version is the standard one (and Azure's definition). **The verdict is the same either way in both problems.** In an MCQ, compute both if options are close and pick the one that matches.

Problem 1 — Was the availability guarantee violated?

Given: guaranteed availability 99 %; application runs **12 h/day**; over **one month (30 days)** total outage = **10.75 h**.

1. Expected run time = 12 × 30 = **360 h**.

2. Service (up) duration = 360 – 10.75 = 349.25 h.
3. Lecture: availability = (1 – 10.75/349.25) × 100 = **96.92 %**. (Standard: (360 – 10.75)/360 = 97.01 %.)
4. 96.92 % < 99 % ⇒ **the provider violated the SLA.**

Problem 2 — Effective cost after service credits

Given: | SLA term | Value | |—|—| | Availability guarantee | **99.95 %** of service period | | Service period | **30 days** | | Max service hours per day | **12 h** | | Cost | **$50 per day** | | Service credit | **10 %** if 99 % ≤ monthly uptime < 99.95 %; **25 %** if uptime < 99 % | | Outages observed (different days) | 5 h 30 min, 1 h 30 min, 15 min, 2 h 25 min |

Solution (lecture):

1. Expected service time = 30 × 12 = **360 h**; total cost = 50 × 30 = **$1500**.
2. Total downtime — lecture total = **9 h 25 min** (= 9.4167 h).
3. Availability = (1 – 9.4167/360) × 100 = **97.385 %**.
4. 97.385 % < 99 % ⇒ credit = **25 %** of $1500 = **$375**.
5. **Effective cost payable = 1500 – 375 = $1125.**

> ⚠️ *Arithmetic flag:* the four listed outages add up to **9 h 40 min**, not 9 h 25 min (the lecture total equals the sum without the 15-minute outage). With 9 h 40 min, availability = 1 – 9.667/360 = **97.31 %**. This is still < 99 %, so the **credit (25 %) and the payable amount ($1125) are unchanged**.

Reusable template — SLA availability / service-credit questions

SLA availability, credit & payable calculator

Days in period	Service hours / day	Total outage (minutes)	Guaranteed availability %
30	**12**	**565**	**99.95**

Cost per day	Lower band limit %	Credit % (b ≤ A < g)	Credit % (A < b)
50	**99**	**10**	**25**

Expected service time T	**360 h**
Downtime d	**9.417 h**
Availability (1 – d/T)	**97.384 %**
Lecture form (1 – d/uptime)	**97.314 %**
Allowed downtime at guarantee	**10.8 min**
Bill	**1500**
Credit	**25 % = 375**
Payable	**1125**

SLA violated

Worked example with the default values (interactive in the HTML version).

1. Expected time T = days × hours/day (convert to minutes if the table uses minutes).
2. Downtime d = sum of outages (watch h/min conversion: 25 min = 0.4167 h).
3. Availability $= (1 - d/T) \times 100$.
4. Compare with guarantee → violated or not.
5. Find the **credit band** that contains the measured availability → credit % × bill.
6. Payable = bill – credit.

- **Variant to practise:** guarantee 99.9 %, 24 h/day, 30 days ⇒ $T = 720$ h; outages total 2 h ⇒ availability = 1 – 2/720 = 99.72 % → falls in the "≥ 99 % but < 99.9 %" band.
- **Allowed downtime** for a target availability: $d_{\max} = (1 - A)\,T$ — e.g., 99.95 % over 360 h ⇒ 0.18 h ≈ **10.8 min**; 99.9 % over 30 days of 24 h ⇒ 0.72 h = **43.2 min**.

Time-varying availability (L21)

Availability requirements can **vary over time** with business needs — e.g., a bank needs 99.9 % during peak hours (9:00–17:00), 99 % in shoulder hours, 97 % overnight; an institute needs high availability during lab hours. **More guaranteed availability ⇒ higher price.**

SLA definitions used by commercial clouds (Microsoft Azure example)

Term	Meaning
Applicable monthly period	Calendar month for which a service credit is owed (days you were a subscriber)
Applicable monthly service fees	Fees paid for the service in that month
Downtime	Defined per service in service-specific terms
Error code	Indication that an operation failed, e.g., **HTTP 5xx**
Success code	e.g., **HTTP 2xx**
External connectivity	**Bidirectional network traffic** over supported protocols (HTTP/HTTPS) that can be sent/received from a **public IP**
External connectivity incident	Any single incident or set of incidents resulting in downtime
Management portal	Web interface through which the customer manages the service
Service credit	% of fees credited back when the service level is not met
Service level	Performance metric(s) the SLA sets out
Support window	Period during which a service feature/compatibility with a separate product is supported
Availability set	**Two or more VMs deployed across different fault domains** to avoid a single point of failure
Fault domain	**Collection of servers sharing common resources such as power and network connectivity**
Update domain	Set of instances to which **platform updates are applied concurrently**
Cloud services	Compute resources used for web and worker roles
Tenant	One or more roles, each with one or more role instances, **deployed in a single package**
VNet	Virtual (private) network

- **Monthly uptime % = (Maximum available minutes − Downtime) / Maximum available minutes × 100**, where maximum available minutes = total accumulated minutes in the billing period (for cloud services: all Internet-facing roles with ≥ 2 instances in different update domains). Credit rules like **< 99.95 % → 10 %; < 99 % → 25 %** (same pattern as Problem 2).

CSCC — 7 steps for evaluating cloud SLAs (Cloud Standards Customer Council, April 2012)

1. **Identify the cloud actors** — per NIST: **consumer, provider, carrier, broker, auditor** (5 actors).
2. **Evaluate business-level policies** — data policies, guarantees, services not covered, excess usage, payment and penalties, sub-contracted services, licensed software, industry-specific standards.

3. **Understand service and deployment model differences** — SaaS/PaaS/IaaS and public/private/hybrid; SLA terms depend on the complexity of the **control variables** the provider exposes.
4. **Identify critical performance objectives** — which **metrics** (availability, response time…) and how often they are collected.
5. **Evaluate security and privacy requirements** — asset sensitivity, legal/regulatory requirements (e.g., data must stay within a geographic boundary), provider's security capabilities.
6. **Identify service management requirements** — what to monitor and report (load, application performance), what to **meter**, how rapid provisioning should be (speed, testing, demand flexibility), how resource changes are managed.
7. **Prepare for service failure management** — remedies (service credits), **liability limitations**, how the **disaster recovery plan** works, and an **exit clause** (every cloud SLA should have one, for either party to terminate).

MCQ traps (5.1)

- Use **only scheduled service hours** (12 h/day × 30 days = 360 h), not 24 × 30.
- Credit is chosen by **which band** the measured availability falls in — 97 % is < 99 %, so it gets the larger (25 %) credit, not 10 %.
- NIST cloud actors are **five**: consumer, provider, carrier, broker, auditor.
- Fault domain = shared power/network; update domain = updated together; availability set = VMs across fault domains.

5.2 Economics tutorial (L22)

Problem 1 — Penalty with provisioning delay

Given: peak demand 120 units;

$$D(t) = \begin{cases} 50 \sin t, & 0 \le t \le \pi/2 \\ 20 \sin t, & \pi/2 < t \le \pi \end{cases}$$

Resources provisioned with delay: $R(t) = D(t) + \delta \dfrac{dD(t)}{dt}$, delay $\delta = \pi/12$; cost of one unit of cloud resource per unit time = 0.9; minimum demand 0. Find the penalty.

Lecture solution:

1. $\int_0^{\pi} D\,dt = 50[-\cos t]_0^{\pi/2} + 20[-\cos t]_{\pi/2}^{\pi} = 50(1) + 20(1) = 70.$
2. $\int_0^{\pi} \delta\, D'(t)\,dt = \delta\left(\int_0^{\pi/2} 50\cos t\,dt + \int_{\pi/2}^{\pi} 20\cos t\,dt\right) = \delta(50 - 20) = 30\delta.$
3. $\int R = 70 + 30\pi/12$, so $\left|\int R - \int D\right| = 30\pi/12 \approx$ **7.85** (lecture: **7.86**) — reported as the **penalty** (resource provisioned exceeds demand, so it is paid-for unused resource).

- Multiplying by the unit cost (0.9) would give ≈ **7.07 cost units** — the lecture stops at 7.86.

> ⚠ *Method flag:* the formula $\int |D - R|\, dt$ takes the absolute value **inside** the integral. Here $R - D = \delta D'$ is **positive on** $[0, \pi/2]$ (over-provisioning) and **negative on** $(\pi/2, \pi]$ (under-provisioning, i.e., missed demand). The strict value is $\delta(50 + 20) = 70\pi/12 \approx 18.33$ (×0.9 ≈ 16.5). The lecture's 7.86 nets the two parts against each other. **For assignment/exam questions modelled on this tutorial, expect the lecture's answer (≈ 7.86).** Know that the strict method gives 18.33.

Problem 2 — Is cloud cheaper than owning over 100 time units?

Given: peak demand **100** units; demand $D(t) = 50(1 + e^{-t})$ (so $D(0) = 100$ = peak, decaying towards 50); baseline (owned) unit cost $B = 120$; cloud unit cost $C = 200$; period $T = 100$.

1. **Owned:** $B_T = P \cdot B \cdot T = 100 \times 120 \times 100 = 1{,}200{,}000$.
2. **Cloud:** $C_T = \int_0^{100} C\, D(t)\, dt = 200 \times 50\, [t - e^{-t}]_0^{100} = 10{,}000 \times (100 - e^{-100} + 1) \approx$ **1,010,000**.
3. Ratio $C_T / B_T \approx 1{,}010{,}000 / 1{,}200{,}000 \approx$ **0.84 < 1 ⇒ cloud is cheaper.**

- Cross-check with the rule: $U = C/B = 200/120 = 1.67$; $A = 5050/100 = 50.5$; $P/A = 100/50.5 = 1.98$; $U < P/A$ ✓.

> ⚠ *Flags:* (i) the transcript calls the 0.84 ratio the "utility premium" — strictly, the utility premium is $U = C/B = 1.67$; 0.84 is the **cost ratio** C_T/B_T. (ii) The slide had a typo and the transcript reads the demand once as $50(1 - e^{-t})$; only $50(1 + e^{-t})$ matches the stated peak of 100 and the lecture's 1,010,000. With $1 - e^{-t}$ you get $C_T \approx 990{,}000$ (ratio 0.825), and the cloud is still cheaper.

Problem 3 — In-house server vs cloud server (core-hour economics)

In-house vs cloud server (core-hour economics) calculator

In-house purchase cost	Life (years)	In-house cores	In-house efficiency %
600000	**3**	**12**	**40**
Power & cooling / h	In-house mgmt / h	Cloud cost / h	Cloud cores
22	**6**	**42**	**8**
Cloud efficiency %	Cloud mgmt / h		
80	**1**		

In-house cost / h	**22.83**
In-house per core-h	**1.9**
Cloud per core-h	**5.25**
Per effective h (in-house / cloud)	**57.08 / 52.5**
Total per effective h (in-house / cloud)	**85.078 / 53.5**
Ratio in-house : cloud	**1.59**
Break-even in-house efficiency	**89.5 %**

Cloud is cheaper

Worked example with the default values (interactive in the HTML version).

Given: | | In-house server | Cloud server | |—|—|—| | Purchase cost | **₹6,00,000** (over a 3-year life) | Nil | | CPU cores | **12** | **8** | | Cost per hour (3-year span) | — | **₹42** | | Efficiency (utilization) | **40 %** | **80 %** | | Power & cooling per hour | **₹22** | Nil | | Management cost per hour | **₹6** | **₹1** |

(a) Price of a core-hour

- In-house cost per hour = 600000 / (3 × 365 × 24) = 600000/26280 = **₹22.83/h** → per core = 22.83/12 = **₹1.90**.
- Cloud per core = 42/8 = **₹5.25**.

(b) Cost per effective hour (divide by efficiency)

- In-house = 22.83/0.40 = **₹57.08** (lecture: 57.075).
- Cloud = 42/0.80 = **₹52.50**.

(c) Total cost per effective hour and ratio

- In-house = 57.075 + 22 (power & cooling) + 6 (management) = **₹85.075**.
- Cloud = 52.5 + 1 = **₹53.50**.
- Ratio in-house : cloud = 85.075/53.5 = **1.59** → cloud is cheaper.

> ⚠️ *Transcript slips:* it writes 87.075 once (the correct sum is 85.075) and 53.25 once (it should be 53.5).

(d) What if in-house efficiency rises?

- At **70 %**: 22.83/0.7 = 32.61 → total = 32.61 + 28 = **₹60.61** > 53.5 → **cloud still cheaper**.
- At **90 %**: 22.83/0.9 = 25.37 → total = 25.37 + 28 = **₹53.37** < 53.5 → **in-house becomes (slightly) cheaper**.
- **Break-even efficiency** (my extension): $22.83/e + 28 = 53.5 \Rightarrow e \approx 0.895$ (≈ **89.5 %**).
- **Lesson:** under-utilized in-house infrastructure loses to cloud; **heavily utilized** (≈ 90 %) in-house infrastructure can beat cloud. It depends on demand pattern and duration.

Reusable templates — economics numericals

- **Own vs cloud (utility pricing):** compute $U = C/B$ and P/A; cloud cheaper iff $U < P/A$. Or compute $B_T = PBT$ and $C_T = C \int D\,dt$ directly.
- **Core-hour comparison:** hourly cost = capex/(years × 8760); per core = hourly ÷ cores; per effective hour = hourly ÷ efficiency; total = per-effective-hour + power/cooling + management (per hour); compare.
- **Penalty with delay:** $R(t) = D(t) + \delta D'(t) \Rightarrow \int (R - D) = \delta\,[\text{net change in } D]$ — lecture nets the pieces; strict = $\delta \int |D'|$.

MCQ traps (5.2)

- Owned cost uses **peak** P; cloud cost uses **actual demand** (average).
- Effective cost divides by **efficiency** (40 % → ×2.5).
- Remember to add **power/cooling and management** to in-house per-hour cost; cloud has only management.
- 3 years = 3 × 365 × 24 = **26,280 h**.

5.3 MapReduce tutorial (L23)

Problem 1 — HDFS blocks with replication

HDFS blocks & replicated storage calculator

Block size (MB)
64

Replication factor
3

File sizes (e.g. 64KB, 65MB, 127MB, 1GB)
64KB, 65MB, 127MB

64KB	**1 block(s)**
65MB	**2 block(s)**
127MB	**2 block(s)**
Total blocks	**5**
Stored block replicas (× 3)	**15**
Raw storage ≈ size × r	**576.19 MB**

Worked example with the default values (interactive in the HTML version).

Given: HDFS block size **64 MB**, replication factor **3**; files of **64 KB, 65 MB, 127 MB**. | File | Blocks = ⌈size / 64 MB⌉ | |—|—| | 64 KB | **1** | | 65 MB | **2** (64 + 1) | | 127 MB | **2** (64 + 63) | | **Total** | **5 blocks** |

- With 3 replicas: **5 × 3 = 15 blocks** stored.
- *(Clarification:)* a block smaller than the block size (e.g., the 64 KB file) occupies only its actual size on disk; the count of **blocks** is still 1.

> ⚠️ *Lecture vs today:* the lecture uses **64 MB** (Hadoop 1.x / GFS chunk size). Hadoop 2.x+ defaults to **128 MB**. Use whatever block size the question states. With 128 MB: 64 KB → 1, 65 MB → 1, 127 MB → 1 ⇒ 3 blocks, 9 with replication.

Template: blocks per file = ⌈file size / block size⌉ (convert units first: 1 MB = 1024 KB in HDFS); total blocks = Σ blocks × replication factor; raw storage ≈ Σ file sizes × replication factor.

Problem 2 — Average of integers with MapReduce

Given: A = {10, 20, 30, 40, 50}; **3 mappers, 1 reducer**. Master splits: M1 = {10, 20}, M2 = {30, 40}, M3 = {50}.

- **Map:** each mapper emits **(local average, count)** to its local file system: M1 → (15, 2), M2 → (35, 2), M3 → (50, 1).
- **Reduce:** total = Σ(avg × count) = 15×2 + 35×2 + 50×1 = **150**; count = 2 + 2 + 1 = **5** ⇒ average = 150/5 = **30**. The result is written back to GFS/HDFS.

- **Trap:** the **average of the mappers' averages** (15 + 35 + 50)/3 = 33.3 is **wrong**. Mappers must emit the **count** (or the sum and count) so the reducer can weight correctly.
- Pseudocode idea: `map(l)` : sum the chunk, emit `sum*1.0/len(l), len(l)` (use float division). `reduce` : accumulate `avg*count` and `count` , then emit `total/count` .

Problem 3 — Total and average salary grouped by gender

Input tuples: (name, gender, salary), e.g., (John, M, 10000), (Martha, F, 15000).

- **Map:** split each line on commas; **drop the name**; emit **key = gender, value = salary** → (M, 10000), (F, 15000).
- **Reduce:** group by key (dictionary keyed by gender); for each gender, **total = Σ salaries**, **average = total / count**; emit (gender, total, average) to HDFS/GFS.
- Pattern = **SQL GROUP BY with SUM and AVG** done in MapReduce.

General MapReduce-design checklist (from the tutorial)

1. Identify the **key** you group by (word, gender, "all" for a global aggregate).
2. Mapper: transform input records into **(key, value)** pairs; emit enough info for the reducer (e.g., counts).
3. Master assigns $\approx 1/M$ of the input to each mapper and tracks the intermediate files.
4. Reducer: apply the aggregate function f (sum, count, average…) per key.
5. Number of mappers ≈ number of input splits/blocks; number of reducers is chosen by the developer.

5.4 Resource management in the cloud (L24–L25)

Why resource management

- Consumer's view: the cloud is an **infinite, scalable resource pool**, pay-as-you-go.
- Provider's view: **finite resources** that must serve as many consumers as possible, **maximize profit without compromising QoS or violating SLAs**, under **energy and environmental (carbon footprint)** limits.
- **Definition:** *resource management refers to the operations used to control how capabilities provided by cloud resources and services are made available to other entities — users, applications, services — in an efficient manner.* ("Efficient" may mean profit, energy, SLA or QoS.)

Types of resources | Physical | Logical | |—|—| | CPU/compute, memory, storage/disk, databases (hardware), network, scientific instruments, workstations, sensors/actuators | OS, applications, communication protocols, **energy management, network throughput/bandwidth, load balancing mechanisms, information security, delays, APIs**, monitoring and management tools |

- Resources exist at every layer — **infrastructure, platform, application** — but **IaaS (infrastructure) is the main focus**, because other layers are dictated by the underlying hardware. Physical and logical resources are **intermingled** (a fast CPU with little memory, or fast storage with a slow network, doesn't help).

Data-center power — the motivation for green cloud (L24)

- Servers consume about **0.5 % of the world's electricity** (lecture's figure; likely higher now).
- A data center = **compute infrastructure** (servers, storage, network) + **environment/logistics** (power supply/UPS, **air conditioning/cooling**, space). The **environment part consumes roughly as much power as the compute part** ⇒ total data-center consumption can exceed **1 %** of world electricity.
- Server energy demand is **doubling every ~5–6 years**; large **CO_2 footprint** (mostly fossil fuel power); **sustainable sources not yet ready** for such loads.
- **Economic aspects:** new data centers run at **megawatt scale** and cost millions to operate; even with money, the **electricity supplier may not have surplus power** (e.g., state electricity board) ⇒ peak capacity can't simply be expanded.
- **Performance per watt does not follow Moore's law.**
- **Goal: reduce energy use with minimal (or no) performance loss — "green data centers."**

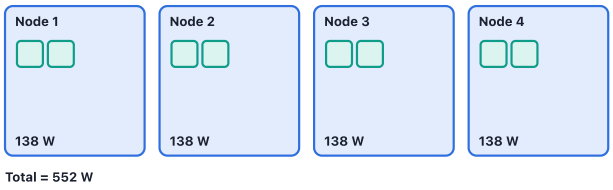
How to conserve energy — Green Cloud framework

Area	Techniques
VM control — scheduling	**Power-aware** and **thermal-aware** VM scheduling
VM control — management	**VM image design** (minimal images), **VM migration/consolidation**, **dynamic shutdown** of idle servers
Data-center design	**Server and rack design**, **air-conditioning recirculation**, confined/integrated rack cooling (cool only the rack enclosure, not the whole room)

- Also: **minimize operating inefficiencies for non-essential tasks** (don't run unneeded packages/services); optimize data-center design.

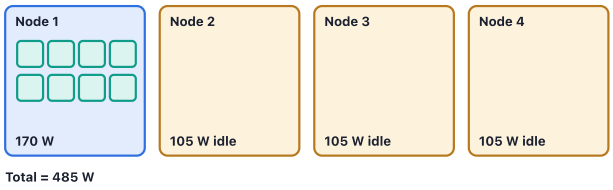
Power-aware VM consolidation (worked example, L24)

Power-aware VM consolidation (L24 example) diagram sequence



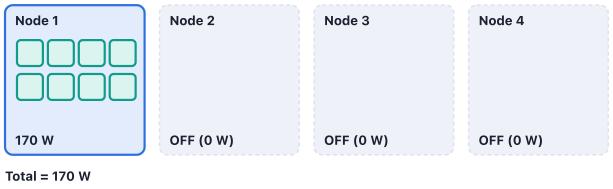
Total = 552 W

1. 8 VMs spread **2 per node**: 4 × 138 W = **552 W** (looks balanced, but every node burns base power).



Total = 485 W

2. Consolidate all 8 VMs on one node: 170 + 3 × 105 idle = **485 W** — power vs load is **non-linear**; idle ≈ 62–66 % of peak.



Total = 170 W

3. Consolidate **and switch the idle nodes off/hibernate**: **170 W**. When load rises, **Wake-on-LAN** brings nodes back. Caveat: migration has a cost.

Key observation: the relationship between the **number of processors (cores) in use and power consumption is non-linear** — power rises steeply from idle and then flattens. An idle server still burns a large **base power**. ⇒ It is better to **pack VMs onto as few servers as possible and put the rest into low-power/hibernate mode**.

Example: 4 identical nodes; **idle = 105 W**; **fully loaded with 8 VMs = 170 W**; 8 VMs to run. | Placement | Power | |—|—| | All 8 VMs on **one** node, other 3 idle | 170 + 3 × 105 = **485 W** | | Spread **2 VMs per node** (each node ≈ 138 W) | 4 × 138 = **552 W** | | All 8 on one node **and the 3 idle nodes shut down/hibernated** | **170 W** |

- **Greedy scheduling:** schedule as many VMs as possible on a multi-core node, keep track of the free cores on each node, and match each VM's requirements to node capacity.
- **Catch: migration has a cost**, and VMs differ in size, so not all VMs can be packed onto one node — monitor load before consolidating.
- **Operating policy:** monitor cloud usage and load → **when load decreases, live-migrate VMs to more utilized nodes and shut down unused nodes** → **when load increases, use Wake-on-LAN to start waiting nodes** and schedule new VMs on them. (Many hypervisors support **live migration**; it also helps when a server is failing.)
- *(Clarification:)* a commonly used linear approximation is $P(u) = P_{idle} + (P_{max} - P_{idle})\,u$. With this example's figures, 2/8 load would give 121 W, but the lecture's measured **138 W** shows the curve is **non-linear (concave)**, which makes consolidation even more attractive.

Minimizing VM images (L24)

- Default VM images carry **unwanted packages and services**; clouds are service-oriented ⇒ build **customized lightweight (Linux) VM images** per service; **smaller images reduce network latency** (and migration cost).
- Example procedure: start from **Ubuntu 9.04** → remove unneeded packages (X11, window manager…) → **read-ahead profiling** (reorder the boot sequence and prefetch boot files to cut I/O wait) → **optimized Linux kernel for Xen DomU** (no 3D graphics, no sound).
- Results: **boot time 38 s → 8 s**; **image size 4 GB → ~600 MB**. Small per-VM savings add up to a large impact (a cloud creating 100 or 1000 images per hour).
- **Summary:** green cloud framework = power-aware VM scheduling + advanced VM/infrastructure management + specialized VM images; **combining methods has a larger impact than each alone**. Research directions: combined power- and thermal-aware scheduling; integrated server/rack/cooling; better VM image minimization; customer profiling.

Resource-management goals and challenges (L25, based on a survey paper)

Broad objectives: preserve **scalability** · preserve **QoS** · **optimal utility** · **reduced overhead** (heavy RM algorithms hurt performance) · **improved throughput** · **reduced latency** · **specialized environments** · **cost effectiveness** (for provider and consumer) · **simplified interface** (ease of use).

IaaS resources include compute (VMs), raw storage, firewalls, load balancers and network devices. **Key challenge:** manage them optimally with **minimal energy** while maximizing provider profit and utilization and still meeting **QoS/SLA**.

Resource-management aspects — definitions (classic matching question)

Aspect	Definition
Resource provisioning	**Allocation of the provider's resources to a customer** (user or process)
Resource allocation	**Distribution of resources economically among competing groups** of people, programs or processes
Resource adaptation	Ability/capacity of the system to **adjust resources dynamically** to fulfil users' requirements
Resource mapping	**Correspondence between the resources required by users and the resources available with the provider**
Resource modelling	Based on detailed information about network elements, resources and entities; describes the **attributes, states, transitions and outputs** of resource management in a given environment (e.g., state charts)
Resource estimation	**A close guess of the actual resources an application requires**, usually with some thought or calculation (a priori/meta information)
Resource discovery and selection	**Identify the list of authenticated resources available for job submission and choose the best** among them
Resource brokering	**Negotiation of resources through an agent** to ensure the necessary resources are available at the right time (and price) to complete the objective
Resource scheduling	A **timetable of events and resources** — resources available at certain times, events planned for those times (durations, predecessor relationships, start/end)

These aspects are **inter-linked**; their emphasis depends on the application.

Approaches named in the lecture (recognition level)

Aspect	Approaches
Provisioning	**Nash equilibrium / game theory** (runtime IaaS allocation considering heterogeneous resources, rational selfish users, incomplete information, successive allocation) · **network queuing model** (queues represent application **tiers**; session workloads, concurrency limits, caching) · **prototype provisioning** using **k-means clustering** to find the workload mix + queuing model to predict server capacity · **VM provisioning** on Xen with an **earliest-deadline-first (EDF) scheduler** implementing **weighted fair sharing of CPU** among VMs · **adaptive resource provisioning** (detect and resolve bottlenecks) · **SLA-oriented** methods · **dynamic/automated** frameworks · **optimal cloud provisioning** under demand and price uncertainty
Allocation	**Market-oriented** allocation (model predictive control) · **intelligent multi-agent** model (virtualization for mobile devices) · **energy-aware** allocation · measurement-based performance analysis · **dynamic** and **real-time** allocation (helps small/medium IaaS providers) · **dynamic scheduling and consolidation**
Mapping	**Symmetric mapping pattern** (users and providers match and enter resource-supply agreements; users place tasks in subscribed **resource containers**; providers place supplied containers on physical resources) · **load-aware mapping** (simplify VM image management, multicast file transfer, image caching) · **iterated-local-search request partitioning** across eligible cloud providers · mapping a **virtual network onto a substrate network**
Adaptation	**Reinforcement-learning-guided control policy** · web-service-based prototypes · virtual networks · **DNS-based load balancing** · hybrid approaches

Performance metrics for judging resource-management techniques

Reliability · ease of deployment · QoS · delay · control overhead. Different applications weight them differently (some are delay-sensitive, some accuracy-sensitive). A bad RM technique can undermine cloud's basic benefits (scalability, the "infinite resources" illusion).

MCQ traps (5.4)

- **Provisioning** = giving resources to a customer; **allocation** = distributing among competing users economically; **mapping** = matching required to available; **brokering** = negotiation via an agent; **discovery** = find and choose authenticated resources; **estimation** = guess requirements; **adaptation** = dynamic adjustment.
- Power vs processors is **non-linear** ⇒ **consolidate** VMs and **shut down/hibernate** idle servers. Spreading VMs evenly is **less** energy-efficient.
- Migration **has a cost**; VMs are heterogeneous.
- The environment/cooling side of a data center uses about **as much power as the compute side**.
- Performance per watt does **not** follow Moore's law.
- k-means → **workload mix**; Nash equilibrium → **game theory provisioning**; EDF → **VM CPU sharing on Xen**.

Week 5 — High-yield takeaways

1. **SLA numericals:** Availability = 1 – downtime/expected time (12 h × 30 d = 360 h); 10.75 h outage → 96.92 % < 99 % ⇒ violated; 9 h 25 min → 97.385 % → 25 % credit → **$1125** payable on $1500.
2. CSCC 7 steps; NIST 5 actors (consumer, provider, carrier, broker, auditor); fault domain vs update domain vs availability set.
3. **Economics numericals:** penalty example ≈ **7.86** (lecture; strict 18.33); $50(1 + e^{-t})$ example: B_T = 1.2 M vs C_T ≈ 1.01 M → cloud cheaper (ratio 0.84); core-hour: ₹1.90 vs ₹5.25; effective ₹57.08 vs ₹52.5; totals ₹85.075 vs ₹53.5 (ratio **1.59**); in-house wins only at ≈ 90 % efficiency.
4. **HDFS:** 64 KB, 65 MB, 127 MB with 64 MB blocks → **5 blocks**, ×3 = **15**.
5. **MapReduce average:** mappers emit (avg, count); reducer Σ(avg×count)/Σcount = 150/5 = 30. Salary by gender: key = gender, value = salary.
6. **Green cloud:** consolidate VMs (485 W vs 552 W; 170 W if idle nodes off); Wake-on-LAN; minimal VM images (38 s → 8 s, 4 GB → 600 MB).
7. RM aspects: provisioning, allocation, adaptation, mapping, modelling, estimation, discovery, brokering, scheduling; metrics: reliability, ease of deployment, QoS, delay, control overhead.

Week 6 — Cloud Security I–IV and Broker for Cloud Marketplace (Lectures 26–30)

> **Source:** L26 Cloud Security – I (security basics) · L27 Cloud Security – II (cloud-specific issues) · L28 Cloud Security – III (case study: "Hey, You, Get Off of My Cloud") · L29 Cloud Security – IV (collaborative SaaS cloud security) · L30 Broker for Cloud Marketplace
>
> High-yield: **CIA triad; the 4 attack types; 4 threat classes; passive vs active attacks; policy vs mechanism; secure/precise/broad; Gartner's 7 cloud risks; RPO vs RTO; co-tenancy; the EC2 co-residency case study (placement + extraction, Dom0, co-residency checks); IDRM, SoD, cyclic inheritance; fuzzy broker.**

6.1 Security fundamentals (L26)

> 🎯 **Your practice set for L26** (your file *w3*): 50 questions · 11 easy · 27 medium · 12 hard · 1 with ⚠️ notes. Answers checked against this lecture.
> Full set and answer key: chapter "Your practice sets", Week 6.

CIA triad

Component	Meaning
Confidentiality	Keeping data and resources **hidden** from unauthorized parties
Integrity	**Data integrity** (what A sends is what B receives) and **origin/source integrity** (authentication of the source)
Availability	Enabling **access to data and resources** — most attacks (DoS/DDoS) target this

A **security attack** = any action that compromises the security of information (violates CIA).

Four basic attack types (source S → destination D, intruder I)

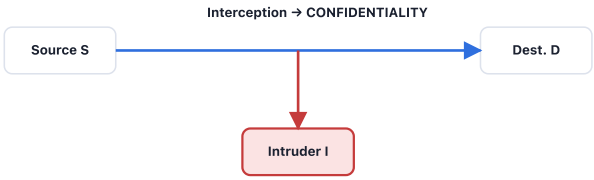
Four basic attacks and the property each violates diagram sequence



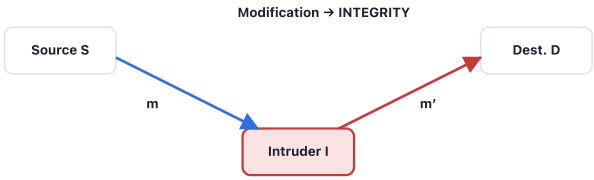
1. Normal flow: S sends message m to D.



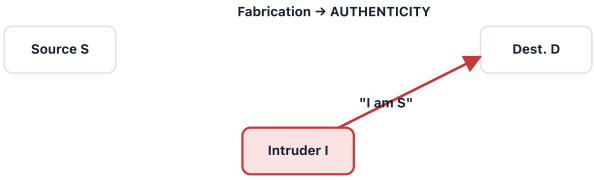
2. Interruption: the path is broken (e.g., DoS) — attack on **availability**.



3. Interception: I silently listens — attack on **confidentiality** (passive; hard to detect).



4. Modification: I changes m to m′ and forwards it; D thinks it came from S — attack on **integrity**.



5. Fabrication: I pretends to be S — attack on **authenticity**.

Attack	What happens	Attack on
Interruption	Communication path is broken	**Availability**
Interception	I listens to the message	**Confidentiality**
Modification	I intercepts, changes message $m \rightarrow m'$, forwards it; D thinks it came from S	**Integrity**
Fabrication	I pretends to be the source	**Authenticity**

Classes of threats (a threat is a *possible* violation, not an attack)

Threat class	Examples
Disclosure	**Snooping**
Deception	Modification, **spoofing**, **repudiation of origin**, **denial of receipt**
Disruption	Modification
Usurpation	Modification, spoofing, **delay**, **denial of service**

Policies and mechanisms

- **Policy says what is and what is not allowed**; it defines the security of a site/system. Policies are usually **formulated centrally**, but **implemented in a distributed way** (switches, routers, firewalls across departments) — the challenge is to guarantee the implementation **conforms exactly** to the policy (an active research problem).
- **Mechanisms enforce policies.**
- **Composition of policies:** conflicts/discrepancies (one allows a traffic class, another denies an overlapping class; local vs global policies) can **create security vulnerabilities**.

Goals of security

Goal	Meaning
Prevention	Prevent attackers from violating security policies
Detection	Detect attackers' violations of policy — the **earlier** the better (late detection = post-mortem)
Recovery	**Stop the attack, assess and repair damage, continue to function correctly even if the attack succeeds** (redundancy, logging) — at a cost

Trust and assumptions

- Underlie **all** aspects of security. Assumptions: **policies unambiguously partition system states** (secure vs non-secure) and **correctly capture security requirements**; **mechanisms are assumed to enforce policies**; **support mechanisms work correctly**.
- **Types of security mechanisms (set view):** let R = set of reachable states, Q = set of secure states.
  - **Secure:** $R \subseteq Q$ — every reachable state is secure.
  - **Precise:** $R = Q$.
  - **Broad:** some reachable states are **not** secure.

Assurance, operational and human issues

- **Assurance** = **specification** (requirements analysis, statement of desired functionality) → **design** (how the system will meet the specification) → **implementation** (programs/systems that carry out the design).
- **Operational issues: cost–benefit analysis** (is it cheaper to prevent or to recover? e.g., a teaching lab that stores no data can just be reimaged; research data must be protected) · **risk analysis** (should we protect something, and how much?) · **laws and customs** (are desired measures legal? will people follow them?).
- **Human issues:** organizational problems (responsibility, authority) and people problems. Example: an employee tricked by a **phishing** mail into revealing a password (social engineering).
- **Why this matters for cloud:** mission-critical data, e.g. **banking data, defence data, even academic data such as students' results**. The lecture says these generic security concepts **carry**

over to cloud in the same or different forms.

- **Goals of security = prevention, detection, recovery** (only three; "retaliation" is a common distractor).
- Tying it together: **threats → policy → specification → design → implementation → operation**, with operational issues fed back into each stage.

Passive vs active attacks

	Passive	Active
Nature	**Obtain information being transmitted (eavesdropping)**	**Modify the data stream or create a false stream**
Types	**Release of message contents**; **traffic analysis** (infer activity from volume/timing — e.g., heavy traffic ⇒ video call)	**Masquerade** (one entity pretends to be another) · **Replay** (passive capture and later **retransmission** to produce an unauthorized effect) · **Modification** of a legitimate message · **Denial of service**
Detection	**Difficult to detect** (no direct harm)	Easier to detect, hard to prevent

Security services / role of security infrastructure

Confidentiality (protect against loss of privacy) · **integrity** (against alteration/corruption) · **availability** (against DoS) · **authentication** (identify legitimate users) · **authorization** (whether an operation is allowed for a user) · **non-repudiation** ("the order is final" — you can't later deny an online fund-transfer instruction you gave) · **access control** (prevent misuse of resources; e.g., RBAC) · **safety** (against tampering, damage, theft). Common attacks listed: social engineering, phishing, password attacks, buffer overflow, command injection, viruses deleting files (non-availability).

Network security process

1. **Determine the security policy** — a full security roadmap; network design must reflect it.
2. **Implement the policy** — install/configure **firewalls, IDS**, **DMZ**, **NAT**, dual firewalls; **honeypots/honeynets** (deliberately vulnerable systems that attract attacks so their **signatures** can tune IDS/firewall rules).
3. **Reconnaissance** — learn about the network (host IPs, key servers with critical data): **passive** (undetectable) or **active** (often not detected by IDS).
4. **Vulnerability scanning** — e.g., **Nessus, nmap** (open ports, possible vulnerabilities); exploitation frameworks such as **Metasploit**; vulnerability databases such as **NVD (National Vulnerability Database)**; scanners must be **updated** like antivirus signatures.
5. **Penetration testing** — **safe "self-attack"** to find how far the system can be penetrated, then patch.
6. **Post-attack investigation** — **forensics**, guided by law; retain the **chain of evidence**.

MCQ traps (6.1)

- Interception ⇒ **confidentiality**; modification ⇒ **integrity**; fabrication ⇒ **authenticity**; interruption ⇒ **availability**.
- **Snooping = disclosure**; **delay/DoS = usurpation**; **repudiation of origin, denial of receipt = deception**.
- **Traffic analysis is passive**; **replay is active**.
- Passive attacks are **difficult to detect**.
- **Policy = what is allowed; mechanism = enforcement.**
- Secure: $R \subseteq Q$; precise: $R = Q$; broad: $R \not\subseteq Q$.

6.2 What is different about cloud security (L27)

Why security is the top barrier

- Cloud = **elastic (theoretically infinite) resource scalability + on-demand, just-in-time provisioning + pay-as-you-go**. Economic benefits: consumer has no upfront cost and scales as required; provider increases data-center utilization — a **win-win**.
- So why isn't everyone on cloud? Clouds are **still subject to traditional confidentiality, integrity, availability and privacy issues, plus some additional attacks.**
- **IDC Enterprise Panel (2008): security** was the **top** challenge/issue, followed by **performance** and **availability**. It's not only that cloud is insecure — it's hard to **define** security precisely and to **trust** the provider.

New threats introduced by cloud

- **Traditional security** focuses on **keeping attackers out**; an attacker must compromise authentication/access control or impersonate a user.
- **In cloud:**
  - **Co-tenancy: multiple independent users share the same physical infrastructure** → an **attacker can legitimately use the same physical machine as the target**.
  - **Customer's lack of control over its own data and applications** (they sit on the provider's premises).
  - **Reputation fate-sharing:** tenants sharing infrastructure share each other's fate (e.g., one tenant's misbehaviour gets a shared IP blacklisted).
- **Responsibility shift:** IaaS → **consumer** bears most security responsibility (everything above the infrastructure); moving towards SaaS → **provider's** responsibility increases.
- Scenarios: in a **private** cloud, business units A and B may share a physical host; in a **public** cloud, different customers share hosts and a channel between them may exist (e.g., via a compromised VMM) — beyond the consumer's control, who can only rely on SLAs and provider reports.

Gartner's seven cloud computing security risks (list question)

Risk	Key point
1. Privileged user access	Sensitive data processed outside the enterprise bypasses the organization's physical, logical and personnel controls → ask providers about **hiring and oversight of privileged administrators** and controls over their access
2. Regulatory compliance and audit	Traditional services undergo external audits/certification; a provider **refusing scrutiny** is fit only for trivial functions; usually **no customer-side audit facility**; a trusted third-party auditor may be needed
3. Data location	Customer often has no clue where data is stored; data centers in different **jurisdictions** → whose law applies?
4. Data segregation	Data sits in a **shared environment** with other customers' data; **encryption is effective but not a cure-all**; data in transit must be decrypted for processing; need a **secure key store**; encryption schemes should be designed and tested by experts
5. Recovery	What happens to data and services in a disaster/outage; replication and redundancy for fault tolerance
6. Investigative support	Investigating inappropriate or illegal activity **may be impossible** in cloud (little customer-side control/monitoring)
7. Long-term viability	How do you get data back (in an importable format) if the provider goes bankrupt, shuts the service, raises prices, degrades quality or has a business dispute? Main obstacle to switching: **vendor lock-in**

RPO and RTO (recovery metrics)

Metric	Definition
RPO — Recovery Point Objective	**Maximum amount of data that may be lost** following an interruption or disaster
RTO — Recovery Time Objective	**Time period allowed for recovery** — time allowed to elapse **between the disaster and activation of the secondary site**

Additional cloud security issues

(a) Virtualization (VMM/hypervisor is the key):

- Recap: full virtualization — VMs run on the hypervisor, which interacts with the hardware; para-virtualization — VMs interact with the host more directly (modified guest). Main functionality: **resource isolation**.
- **Hypervisor vulnerabilities: shared clipboard** technology can transfer malicious programs from VMs to the host; **keystroke logging** — some VM technologies log keystrokes and screen updates passed across virtual terminals.
- **Hypervisor risks: rogue hypervisor rootkits** (initiate a rogue hypervisor, **hide from normal malware detection**, create a **covert channel** to dump unauthorized code); **external modification** of the hypervisor; **VM escape**; **improper VM configuration**; **denial of service**.

- Threats: **unauthorized access to virtual resources; loss of confidentiality, integrity, availability**. *If the VMM is compromised, all VMs on it are potentially compromised.*

(b) Access control and identity management: proper access control (RBAC, MAC, DAC) for CIA; prevention of **identity theft** — privacy threats via **massive data mining**; **identity management** authenticates users and services based on **credentials and characteristics**.

(c) Application security: cloud apps are mostly **web-based** → **injection attacks, XML signature element wrapping, cross-site scripting (XSS), flooding, DNS poisoning, phishing, metadata (WSDL) spoofing**; **insecure communication channels** → interruption, eavesdropping.

(d) Data life-cycle management:

- **Confidentiality** — will sensitive data stay confidential? Will the cloud leak client data? Will the **provider itself** be honest and not peek?
- **Integrity** — how do I know the provider **computed correctly** and **stored data without tampering**?
- **Availability** — will critical systems go down if the provider suffers **DoS**, or goes **out of business**?
- **Data location** — all copies/backups only where the contract/SLA/regulation allows.
- **Archive access latency.**

(e) Inter-cloud communication: a process in cloud 1 → cloud 2 → cloud 3 → back may let a user **access data that would otherwise be denied** — violating access control.

(f) Choosing providers: trust, competence, risk of each provider.

MCQ traps (6.2)

- Gartner list has **seven** items (privileged user access, regulatory compliance & audit, data location, data segregation, recovery, investigative support, long-term viability). "Pricing" or "performance" is **not** one of them.
- **RPO = data loss tolerance; RTO = time to restore.**
- Co-tenancy lets an attacker be on the same machine **legitimately** (no break-in needed).
- IaaS consumer has **more** security responsibility than a SaaS consumer.
- Encryption is "effective but **not a cure-all**" (data must be decrypted for processing).

6.3 Case study: "Hey, You, Get Off of My Cloud" (L28)

Paper

Ristenpart et al., ACM CCS 2009 — "Hey, You, Get Off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds." Claimed to be the **first work on cloud cartography**; experiments on **Amazon EC2**; achieved about **40 % success** in getting **co-resident** with a target VM. (Purpose in the course: show *new* cloud risks, not to criticize a provider.)

New risks highlighted

- **Trust and dependency** — customer must trust the provider to respect **data privacy** and **integrity of computations**.
- **Multi-tenancy** — **multiplexing VMs of disjoint customers on the same physical machine**; you **cannot prevent** your instance from being co-resident with an adversary.
- **Side-channel exploitation** — **cross-VM information leakage** due to shared physical resources; potential to extract **RSA and AES secret keys**.
- **Vulnerable VM isolation** — a vulnerability may allow an **escape to the hypervisor**.
- **Lack of control** over **who you share the server with**.

Attack model — two steps

1. **Placement:** the adversary arranges to **place a malicious VM on the same physical machine as the target** customer.
2. **Extraction: extract confidential information via a side-channel attack**.

Threat model (assumptions)

- **Provider and infrastructure are trusted.** No attacks that subvert administrative functions; **no exploitation of VMM vulnerabilities**.
- **Adversaries are non-provider-affiliated malicious parties** (ordinary customers) who can **run and control many instances**.
- **Victims** run confidentiality-requiring services.
- Two attack kinds: on **some known hosted service**, or on **a particular victim's service** (targeted).

The four questions the study answers

1. **Can one determine where in the cloud infrastructure an instance is located?** (cartography)
2. **Can one easily determine if two instances are co-resident** on the same physical machine?
3. **Can an adversary launch instances that will be co-resident** with a target's instances?
4. **Can an adversary exploit cross-VM information leakage** once co-resident?

EC2 background (as of 2009)

- **Three degrees of freedom** when launching: **instance type, region, availability zone**.
- Instance types: m1.small, c1.medium (32-bit), m1.large, m1.xlarge, c1.xlarge (64-bit). Regions: US, EU, Asia; regions split into **availability zones** with **separate power and network** (independent failures).
- Customers are assigned to a physical machine based on instance type, region and zone choices; an instance stays on **one physical machine for its lifetime**.
- EC2 uses the **Xen** hypervisor. **Dom0** = the **privileged VM** that manages guest images, provisions physical resources and access-control rights, **routes packets to guests** and **appears as a hop in**

traceroute.

- Each instance has an **internal and an external IP address and domain name**; **inside the cloud both names resolve to the internal IP**; outside, the external name maps to the external IP.

Q1 — Cloud cartography

- Amazon doesn't disclose placement; map EC2 to find where potential targets are and which **creation parameters** (zone, type) are needed for co-residence.
- **Hypothesis:** different **availability zones and instance types correspond to different internal IP address ranges**.
- **Network probing** with open-source tools **nmap, hping, wget** on **ports 80 (HTTP) and 443 (HTTPS)**. **External probe:** from outside EC2 to an EC2 instance. **Internal probe:** from one EC2 instance to another. **DNS lookups** translate public IPs/names to **internal** IPs (~14,000 unique internal IPs found).
- Findings: EC2's **internal address space is cleanly partitioned between availability zones**, and instance types occupy disjoint ranges; assumption: **internal IPs are statically assigned to physical machines** (eases routing). So picking the same zone/type as the target likely lands you in the same IP block.

Q2 — Checking co-residency

Network-based checks — instances are likely co-resident if they have:

1. **Matching Dom0 IP address** (first traceroute hop);
2. **Small packet round-trip times**;
3. **Numerically close internal IP addresses (typically within 7).**

- Verified using a **hard-disk-based covert channel** (if two self-controlled instances can communicate over it, they are co-resident).
- **Efficient check against a target:** compare internal IPs; do a **TCP SYN traceroute** to an open port and see if there's a **single hop (Dom0)**. Needs **at most two TCP SYN packets, no full TCP connection** → very **quiet** (victim unaware).

Q3 — Achieving co-residency

- **Brute-force placement:** launch numerous probe instances and keep those that land co-resident.
- **Target recently launched instances:** EC2 tends to assign fresh instances to a **small set of machines**.
- **Placement locality:**
  - Instances launched **simultaneously from the same account do *not* run on the same machine**.
  - **Sequential placement locality:** two instances run sequentially are often assigned the same machine.

- **Parallel placement locality:** two instances (different accounts) run at **roughly the same time** are often on the **same machine**.
- **Instance flooding:** launch many instances in parallel in the right zone and type.

Q4 — Exploiting co-residency (cross-VM attacks)

- **Learn about resource usage** of other instances; **create/use covert channels** to leak information intentionally.
- **Load-based co-residence check** via **cache usage** (no network needed once co-resident); **estimating traffic rates** of a co-resident web server; **keystroke timing attacks** (infer passwords from timing).

Countermeasures discussed

Attack step	Countermeasure	Catch
Mapping	**Randomize IP allocation**; block probing tools (nmap, traceroute)	Randomization may hurt the provider's resource optimization
Co-residency checks	**Prevent identification of Dom0**	—
Co-location	**Don't allow co-residence at all**	Good for users, **inefficient for providers**
Side channels	—	**Information leakage via side channels remains a big challenge**

Summary: shared physical infrastructure creates **new risks** absent from traditional computer/network security; practical attacks were demonstrated, along with countermeasures.

MCQ traps (6.3)

- The two steps are **placement** then **extraction** (side channel).
- The threat model **trusts the provider** and **does not exploit VMM bugs**; adversaries are **ordinary customers**.
- Co-residency indicators: **same Dom0 IP, small RTT, internal IPs within ~7**.
- **Simultaneous launches from the same account → different machines**; **parallel launches by different accounts at the same time → often the same machine**.
- Probing used ports **80 and 443**, tools **nmap, hping, wget**.
- Disallowing co-residence is **inefficient for the provider**.

6.4 Security in collaborative SaaS clouds (L29)

Context

- Cloud-unique issues: **co-tenancy** and **lack of control over outsourced data and applications**; general concerns: **inadequate policies and practices**, **insufficient security controls**.
- **Trust must go both ways** — the provider must also be wary of **malicious customers** who use the platform to attack or peep into others' data.
- Provider security responsibility: **IaaS → up to the hypervisor**; **PaaS → up to the platform/solution stack**; **SaaS → up to the application interface** (SaaS consumers depend most on the provider).
- **Collaboration types: tightly coupled (federated)** vs **loosely coupled**. **Existing security mechanisms are mainly for tightly coupled systems**; existing authentication/authorization in cloud is restrictive → securing **loosely coupled** collaboration is a major problem.
- **Motivation:** SaaS has the **maximum lack of control**; **no active data streams, audit trails or outage reports** directly available; **cloud marketplaces** with many providers, **inconsistent service guarantees** and **non-standard SLA clauses** → choosing an ideal provider is hard.
- Goal: **select an ideal SaaS provider** and **secure loosely coupled collaboration** (dynamic information sharing among e.g., sellers, payment providers, couriers).

The four-part framework (Dr. Nirnay Ghosh's work)

Objective	Approach	Key points
1. Select a trustworthy and competent collaboration service provider (SelCSP)	Model **trust, reputation and competence** of providers (interaction ratings + temporal metrics → trust & reputation → **trustworthiness**; recommendations + **standard security controls** in SLAs → **competence**; plus **risk** estimation)	Present-day SLAs mostly guarantee **availability only** and have **non-standard clauses** on assurance and compensation → need a **standard set of SLA parameters** to reduce the perceived risk of outsourcing
2. Recommend access requests from anonymous users for authorization	**Risk-based access control** using a **fuzzy inference system** (inputs: requester's **reputation**, local **object security level**) → grants a **subset** of the requested permissions	Gives access even when subjects **lack proper permissions** (with bounded risk); **balances access risk vs security uncertainty** due to information sharing; **more flexible than binary MLS**; challenge: computing security uncertainty; quantifying operational need
3. Map authorized permissions into local roles	**Inter-Domain Role Mapping (IDRM)** — find a **minimal set of roles that encompasses the requested permission set**	**No polynomial-time solution (NP-hard)** → **greedy search heuristics** (sub-optimal); **multiple minimal role sets** may exist, or **none** may match exactly; variants **IDRM-safety** and **IDRM-availability**; aim: **minimal excess permissions** (e.g., give read only if only reading is needed)
4. Dynamically detect and remove access conflicts	**Distributed** secure collaboration framework using **only local information**	Conflicts: **cyclic inheritance conflict** (a role gains write access via a cycle across domains that it should not have) and **SoD (separation of duty) constraint violation** (e.g., the person who issues a demand draft must not verify it); removal: if an **exactly matching role set exists** → RBAC hybrid hierarchy; otherwise create a **virtual/collaborating role**

MCQ traps (6.4)

- Existing security mechanisms target **tightly coupled (federated)** systems; the hard problem is **loosely coupled** collaboration.
- **IDRM** = map requested permissions to a **minimal** set of local roles; it is **NP-hard**, so use **greedy heuristics**.
- **SoD** = one person must not perform two conflicting duties (issue and verify).
- Risk-based access control is **more flexible than binary MLS**.
- The provider's SaaS responsibility extends **up to the application interface**.

6.5 Broker for cloud marketplace (L30)

Why a broker

- Rapid growth of cloud services: **many providers with varying QoS** and **many customer use cases** with different requirements (e.g., Amazon VMs created per day grew enormously 2007–2011).
- Need a **middleman — an (intelligent) agent/broker** that **suggests the best provider** for a customer and **safeguards the customer's interest**.
- **Motivation:** flexible provider selection · **trustworthiness** of providers · **monitoring** of services · avoiding **vendor lock-in**.

Objectives

1. **Select the most suitable provider satisfying the customer's QoS requirements.**
2. **Calculate the degree of SLA satisfaction and the provider's trustworthiness.**
3. **Decision support for dynamic service migration** (move the application to another provider) based on **experienced QoS**.
4. Handle the **fuzziness** of user requirements (they are rarely crisp).

Related approaches: CloudCmp (a tool that **compares cloud providers' QoS** to help users choose) · **fuzzy provider selection** · a framework measuring **satisfaction with a provider** given fuzzy requirements · selection based on **trustworthiness and competence**.

QoS parameters: IaaS — **24×7 availability, bandwidth**; SaaS — **uptime, response time**; also storage, security, trust, competence, risk, cost. Providers publish **promised QoS values** (e.g., 99.99 % uptime). **Trust values are kept per parameter** (e.g., trust 0.8 on availability), computed from **history** or **third-party monitoring**.

Marketplace broker architecture

Module	Role
Provider selection decider	**Fuzzy inference engine**: **inputs = QoS offered by a provider + its trustworthiness**; **output = suitability of the provider** for the customer; request dispatched to the provider with **maximum suitability**
Monitoring module	Monitors **QoS experienced by VMs** (availability, bandwidth vs promised values) → computes **SLA satisfaction per VM per parameter** → feeds the **provider trust calculator**
Migration module (migration decider)	Fuzzy inference engine: **output = degree of SLA satisfaction** for a service instance; **if below a threshold → migrate**; target provider chosen like in provider selection

- **Customer side: input membership functions built on the fly from user requirements**, plus a **rule list**. **Provider side:** offerings are **fuzzified → rules evaluated → defuzzified** → provider fitness.

- Linguistic levels: **extremely poor, very poor, poor, medium, good, very good, excellent**.

Evaluation (simulation)

- **10 providers**, **500 requests**, simulated over **365 days**; some providers degrade, with degraded QoS following a **Gaussian distribution**; compared with a **conventional minimum-cost crisp broker**.
- **Result:** the **fuzzy broker tracks user requirements better** than the crisp broker on average availability, bandwidth, cost per VM-hour (IaaS) and response time, cost (SaaS) — "likely better, not always".
- Future scope: flexible QoS specification, production workloads, customer classes, extend to XaaS.

MCQ traps (6.5)

- The fuzzy **selection** engine's inputs are **QoS offered + trustworthiness**; its output is **suitability**.
- The **migration** engine outputs the **degree of SLA satisfaction**; migration happens when it is **below a threshold**.
- **CloudCmp** compares providers' QoS.
- The baseline compared against is a **minimum-cost crisp broker**.

Week 6 — High-yield takeaways

1. CIA; interruption→availability, interception→confidentiality, modification→integrity, fabrication→authenticity.
2. Threat classes: disclosure (snooping), deception (spoofing, repudiation, denial of receipt), disruption, usurpation (delay, DoS).
3. Passive (release of contents, traffic analysis; hard to detect) vs active (masquerade, replay, modification, DoS).
4. Policy vs mechanism; goals prevention–detection–recovery; secure $R \subseteq Q$, precise $R = Q$, broad otherwise.
5. Cloud-specific: co-tenancy, lack of control, reputation fate-sharing; IDC 2008: security is the top concern.
6. Gartner's 7 risks; RPO (data loss) vs RTO (recovery time); hypervisor risks (rootkits, VM escape, shared clipboard, keystroke logging).
7. EC2 study: placement + extraction; Dom0; co-residency = same Dom0, small RTT, IPs within 7; ~40 % success; countermeasures and their costs.
8. Collaborative SaaS: SelCSP, risk-based access (fuzzy), IDRM (NP-hard, greedy), conflicts (cyclic inheritance, SoD).
9. Broker: fuzzy selection (QoS + trust → suitability), monitoring → trust, migration when SLA satisfaction < threshold; beats crisp min-cost broker.

Week 7 — Mobile Cloud Computing, Fog Computing, Geospatial Cloud (Lectures 31–35)

> **Source:** L31 Mobile Cloud Computing – I · L32 Mobile Cloud Computing – II · L33 Fog Computing – I · L34 Fog Computing – II · L35 Use Case: Geospatial Cloud
>
> High-yield: **MCC definition, profiler/solver/synchronizer, MAUI vs COMET, cloudlet, the offloading energy formula (numerical!), cloud vs fog comparison table, fog enablers, fog use cases, geospatial cloud architecture and SDI.**

7.1 Mobile Cloud Computing — basics (L31)

> 🎯 **Your practice set for L31** (your file *w4*): 50 questions · 14 easy · 25 medium · 11 hard. Answers checked against this lecture.
> Full set and answer key: chapter "Your practice sets", Week 7.

Why MCC

- Huge growth of **smartphones and smart applications**; devices more resourceful; easy, **low-cost high-bandwidth** Internet access.
- But **resource challenges** remain for resource-hungry apps: **battery life, storage, bandwidth** (and processing).
- Cloud offers infrastructure, platform and software **at low cost, elastically, on demand** → combine them: **Mobile Cloud Computing (MCC)**.

Definitions

- **MCC = a combination of cloud computing, mobile computing and wireless networks to bring rich computational resources to mobile users.**
- MCC **provides mobile users with data storage and processing services in the cloud — moving computing power and data storage away from the mobile device**.
- Benefits **mobile users, network operators and cloud providers** (a "win" for all).
- **Mobile Backend as a Service (MBaaS):** gives app developers a way to connect their apps to **backend cloud storage and processing**, **abstracting away the complexity** of launching and managing their own infrastructure, so they **focus on front-end development**; supports multiple apps, backends, developers, platforms, third-party integrations (APIs, SDKs, enterprise connectors).
- **Examples of mobile + cloud: Amazon Silk** (a **split browser**), **Apple Siri**, **Apple iCloud**, **image-recognition apps** for **augmented reality**.

Why MCC — three reasons

Reason	Meaning
Speed and flexibility	Mobile apps can be built or revised quickly using cloud services; e.g., a new shortest-path heuristic is changed in the cloud, front end unchanged
Shared resources	Apps running in the cloud are **not constrained by device storage and processing**; data-intensive processing runs in the cloud while user engagement continues on the device
Integrated data	Quickly and securely collect and integrate data from various sources wherever it resides (e.g., disaster management with mobile sensors)

Key features: delivers apps quickly and securely with capabilities beyond local resources; **uses fewer device resources** (less battery, heat); supports many development approaches and devices (reusability/standardization); devices connect through an **API architecture**; **improves reliability** — information backed up in the cloud.

Architecture and offloading

`Smart Mobile Device (SMD) ⇄ wireless network (service provider) ⇄ computational cloud`

- **MCC = a framework to augment resource-constrained mobile devices to execute parts of a program on cloud-based servers.**
- **Pros: saves battery**; **makes execution faster**. **Cons:** must **send program state/data to the cloud server — consumes battery**; **network latency can delay execution**.
- **Network side:** mobile devices connect to mobile networks via **base stations**; requests go to the operator's central processors/servers; subscriber requests reach the cloud **through the Internet**; in the cloud, **cloud controllers process requests** and provide the services.

Dynamic runtime offloading — tricky issues:

1. **Dynamic application profiling and solver** on the SMD.
2. **Runtime application partitioning.**
3. **Migration of computation-intensive components.**
4. **Continuous synchronization** for the entire duration of runtime execution.

Key components (very examinable) | Component | Job | |—|—| | **Profiler** | **Monitors application execution** to collect data on **time to execute, power consumption and network traffic** | | **Solver** | **Selects which parts of an app run on the mobile device and which on the cloud** | | **Synchronizer** | **Collects the results of split execution, combines them**, and makes execution details **transparent** to the user | (Also: SMD synchronizer, mobile load manager, cloud data center.) **Requirements: simple APIs** (no knowledge of underlying network technology needed), **web interface**, **Internet access to remotely stored applications**.

Advantages of MCC

1. **Extending battery lifetime** — computation offloading of heavy work to resourceful servers; remote execution can save significant energy.
2. **Improving data storage capacity and processing power.**
3. **Improving reliability and availability** — data/apps in the cloud reduce loss; comprehensive data security model possible (e.g., protecting copyrighted digital content, **virus scanning, malicious-code detection** in the cloud); always available while moving.
4. **Dynamic provisioning · scalability · multi-tenancy · ease of integration** (services from different providers integrated through the cloud).

Challenges

- **Security and privacy:** two categories — **security of mobile users** and **securing data in the cloud**. Mobile devices face malicious code; **GPS/location-based services** leak private info (location, mobility footprints). Approaches: **move threat detection to the cloud** — a **host agent** on the device inspects file activity while **attack detection runs on a remote server**; the smartphone records only a **minimal execution trace** and sends it to the cloud security server.
- **Context-aware mobile cloud services** — use local context (**data types, network status, device environment, user preferences**) to improve QoS.
- **Network access management** — improve link performance and **optimize bandwidth usage**.
- **Quality of service.**
- **Pricing** — **multiple parties** (mobile service provider + cloud provider) → business model and price sharing are tricky.
- **Standard interfaces.**
- **Service convergence** — services differentiated by type, cost, availability, quality; mobile users should be able to use **multiple clouds in a unified fashion**.

7.2 Mobile Cloud Computing — partitioning and offloading (L32)

Key challenge

- **Dynamically partition an application** to **optimize energy saving and execution time**; middleware must decide **which parts run locally and which remotely** — a classic optimization problem; can be **static** (a priori) or **dynamic** (data/condition dependent), and needs synchronization, scheduling and resource management across **different authorities** (device owner, network operator, cloud provider).

MAUI vs COMET (classic comparison)

	MAUI — Mobile Assistance Using Infrastructure	**COMET — Code Offload by Migrating Execution Transparently**
Granularity	**Method-level**	**Thread-level**
Source code	**Programmer annotates** each method as **remoteable or not** (needs source)	**Works on unmodified applications — no source code needed, only program binaries**
Rules	**Native methods cannot be remoteable**	Executes **multi-threaded programs** correctly
Mechanism	MAUI framework uses annotations to decide at runtime whether a method runs on the **cloud server** to **save energy and time**; **MAUI client (mobile)** + **MAUI server (cloud)**; profiler + solver; **RPC**	**Distributed Shared Memory (DSM)** in the runtime engine lets **threads migrate between machines** depending on workload; **DSM = physically separate memories addressed as one logically shared address space**; sends **only modified parts of the heap** to reduce traffic

Task-partitioning problem (formalization)

- **Input:** a **call graph** of the application's methods (edges = call/execution sequence).
- **Node attributes: energy to execute the method on the mobile device**, and **energy to transfer the program state** to the remote server. (Energy for executing in the cloud is treated as negligible from the device's view.)
- **Output:** partition methods into **two sets** — executed **on the mobile** vs **on the cloud**.
- **Goal: minimize energy consumed** (maximize energy saved = local energy avoided – transfer energy).
- **Constraints: total execution time/latency ≤ threshold** (T or L); **some (native) methods must execute on the device** (highlighted nodes v_1, v_4, v_9 in the lecture's example); **total monetary cost** limit; **dependencies** respected.
- **Static partitioning:** when the app is launched, invoke an **ILP solver** (or heuristics) to decide where each method runs. **Dynamic/adaptive partitioning:** for **long-running programs**, conditions vary (bandwidth, remote availability, input-dependent energy) → re-partition.
- Related approaches: estimate energy before execution; dynamic optimal partitioning from the **communication vs computation** trade-off; **profiling at procedure-call level → cost graph → branch-and-bound** to minimize total energy and data.

Challenges

- **Mobile communication issues: low bandwidth** (radio resources are scarcer than wired; also varies over time), **service availability** (congestion, network failure, weak signal), **heterogeneity** (always-on connectivity, on-demand scalability, energy efficiency across heterogeneous networks).
- **Computing/offloading issues: offloading is not always effective in saving energy**; it is **critical to decide whether to offload and which portion** of the code.

Cloudlet

- **A cloudlet is a trusted, resource-rich computer or cluster of computers that is well connected to the Internet and available for use by nearby mobile devices.**
- **Middle tier of a 3-tier architecture: mobile device → cloudlet → cloud.**
- **Decreases latency** by using a **single-hop** network and **can lower battery consumption** using Wi-Fi/short-range radio (vs multi-hop WAN to a distant cloud).

When to offload — the energy formula (numerical!)

Should the mobile offload? (energy) calculator

Instructions C (×10⁹)	Mobile speed M (×10⁹/s)	Cloud speed-up F	P computing (W)
2	**1**	**20**	**1**

P idle (W)	P transmit (W)	Data D (MB)	Bandwidth B (MB/s)
0.2	**1.5**	**4**	**4**

Local compute energy $P_c \cdot C/M$	**2 J**
Idle while cloud computes $P_i \cdot C/S$	**0.02 J**
Transmission $P_{tr} \cdot D/B$	**1.5 J**
Energy saved	**0.48 J**

Offload — saves energy

Worked example with the default values (interactive in the HTML version).

Symbols | Symbol | Meaning | |—|—| | P_c | Power (energy per second) of the mobile while **computing** (W) | | P_i | Power of the mobile while **idle** (waiting for the cloud) (W) | | P_{tr} | Power of the mobile while **transmitting** data (W) | | C | Number of **instructions** of the computation | | M | **Speed of the mobile** (instructions/s) | | S | **Speed of the cloud** (instructions/s); $S = F \cdot M$ | | F | **Speed-up factor** — how many times faster the cloud is | | D | **Data to transmit** (bytes/bits) | | B | **Bandwidth** (same unit per second) |

Energy saved by offloading:

$$E_{saved} = P_c \frac{C}{M} - P_i \frac{C}{S} - P_{tr} \frac{D}{B}$$

Substituting $S = FM$:

$$\boxed{E_{saved} = \frac{C}{M}\left(P_c - \frac{P_i}{F}\right) - P_{tr}\frac{D}{B}}$$

- **Interpretation:** term 1 = energy you would spend computing locally; term 2 = energy spent **idling** while the cloud computes; term 3 = energy spent **sending data**.
- **Offload only if $E_{saved} > 0$.** This happens when D/B **is sufficiently small compared to** C/M and F **is sufficiently large ⇒ offloading is beneficial when a large amount of computation** C **is needed with relatively small data communication** D**.**
- **Not all applications are energy-efficient when migrated to the cloud.** Mobile cloud services must differ from desktop cloud services because they must save **energy**. Energy overheads of **privacy, security, reliability, data communication** are not included in this simple model.

Worked example (practice): $C = 10^9$ instructions, $M = 10^8$ instr/s (10 s locally), $P_c = 0.9$ W, $P_i = 0.3$ W, $P_{tr} = 1.3$ W, $F = 10$, $B = 1$ MB/s.

- Local compute energy = 0.9 × 10 = **9 J**; idle while cloud computes = 0.3 × (10/10) = **0.3 J**.
- If $D = 10$ MB: transmit energy = 1.3 × 10 = 13 J ⇒ $E_{saved} = 9 - 0.3 - 13 = -4.3$ J → **don't offload**.
- If $D = 1$ MB: transmit = 1.3 J ⇒ $E_{saved} = 9 - 0.3 - 1.3 = 7.4$ J → **offload**. **Effect of parameters:** ↑C or ↑F or ↑B ⇒ more favourable; ↑D or ↑P_{tr} ⇒ less favourable; if $F \to \infty$, the idle term → 0.

MCQ traps (7.1–7.2)

- **Profiler** measures time/power/traffic; **solver** decides placement; **synchronizer** merges results.
- **MAUI needs annotations (source code) at method level; COMET needs only binaries, migrates threads via DSM.**
- In MAUI, **native methods are never remoteable**.
- Cloudlet = **single-hop**, nearby, trusted, resource-rich; it is the **middle** tier.
- Offloading helps for **compute-heavy, data-light** tasks; it is **not always** beneficial.
- In the formula, the **idle** term uses cloud time C/S; the **transmit** term uses D/B.

7.3 Fog computing — concept (L33)

Problem with cloud-only computing

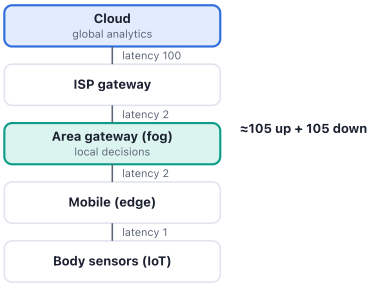
- All data must be **uploaded to the cloud** for processing → huge volumes (IoT, sensors, multimedia), much **redundant data** (10 sensors in a lab all reporting ≈ 20 °C).
- **Congestion in the core network** (especially during exigencies like disasters) and **long response time** — bad for tasks needing **very low response time** (e.g., accident prevention).
- Meanwhile, **intermediate/edge devices** (sensor sink nodes, routers, gateways, base stations) are **increasingly resourceful** but used only to store-and-forward.
- Example: a **sink node** of a lab's 10 temperature sensors can send the **average** (or just "within 18–22 °C: yes/no") instead of all 10 readings.
- Must keep the cloud's key features: **dynamic scalability, (almost) no infrastructure management at the user end, metered pay-as-you-go service**.

Definition

- **Fog computing (also called “fogging” or edge computing) is a model in which data, processing and applications are concentrated in devices at the network edge rather than existing almost entirely in the cloud.**
- **The term was first introduced by Cisco** as a new model to ease wireless data transfer to distributed devices in the **IoT** paradigm.
- **Cisco’s vision:** enable applications on **billions of connected devices** to **run directly at the network edge**; users can develop, manage and run software on **Cisco network devices** (hardened routers, switches) — Cisco brings **open-source Linux and the network OS together in a single network device**.
- **Fog = a paradigm that extends the cloud and its services to the edge of the network**; it provides **data, compute, storage and application services** to end users (like a small, local instance of the cloud).
- Fog performs some transactions/resource use **at the edge** instead of just channelling everything to cloud storage; it **reduces bandwidth need by aggregating data at certain access points** → lowers cost, improves efficiency.
- Locality matters: Kolkata’s traffic lights have nothing to do with Delhi’s — decide locally, send aggregates upward.

Three-layer view

Cloud vs fog: where the decision is made diagram sequence



1. Cloud-only: every reading travels all hops to the cloud and back (IoHT case latencies 1 + 2 + 2 + **100**) — high latency, high core traffic, higher cloud cost.



2. Fog/edge: filtering, processing and event handling at the **area gateway** (one hop from users) — low, stable latency; only candidate events/aggregates go up to the cloud (confirmatory module).

Layer	Role
Cloud (data centers)	Massive parallel processing, big-data mining, machine learning — **more compute power and storage**
Fog (cellular base stations, routers, Wi-Fi gateways, local servers)	**Real-time data processing, data caching, computation offloading** — closer to users, **more interactive and responsive**
Edge/end devices (sensors, cameras, smart vehicles, phones)	**Basic data processing**: pre-processing, compression; human–computer interface

- Processing close to the devices **lowers response time**, enabling **real-time applications** (e.g., traffic signals timed by camera-measured flow).
- Motivating developments: **smart grid, smart traffic lighting, connected vehicles, software-defined networks** — all generate huge data.

Fog enablers (list question)

Enabler	Role
Virtualization	VMs can run on edge devices
Containers	**Lightweight virtualization** reduces resource-management overhead; e.g., **Docker** (dependencies travel with the app)
Service-Oriented Architecture (SOA)	Services provided to components via a communication protocol over a network; provider–consumer–registry; loosely coupled heterogeneous parties
Software-Defined Networking (SDN)	Uses **open protocols (e.g., OpenFlow)** to apply **globally aware software control at the network edge** to access switches/routers that would typically use **closed, proprietary firmware**

Fog is not a replacement for cloud

- Fog **offloads part of the workload** from the cloud to edge devices; **handshaking/synchronization between fog and cloud is needed**.

Benefits/characteristics of fog: low latency and location awareness · widespread geographical distribution · mobility · very large number of nodes · predominant role of wireless access · strong presence of streaming and real-time applications · heterogeneity (different sink nodes, but aggregated data in a standard format). **Distinguished from cloud** by **proximity to end users, dense geographical distribution and support for mobility**; improves **QoS** for real-time apps.

Security issues in fog

- Devices are **dispersed** → maintaining security protocols across many fog devices is hard (cloud is centrally managed).
- **Man-in-the-middle attacks.**
- **Privacy:** intermediate processing can track vehicles or a household's consumption.

- **Smart grid example:** each smart meter/appliance has an **IP address** → a malicious user can **tamper with his own meter, report false readings, or spoof IP addresses**.

7.4 Fog computing — comparison, use cases, resource management (L34)

Why fog (recap)

Cloud helps IoT through **scalability, resource provisioning and data intelligence**, but has limits for **real-time, latency-sensitive applications** (responses in seconds/ms/µs — e.g., car collision alerts in a VANET). **Fog computing was coined to serve real-time, latency-sensitive applications faster**, leveraging **local knowledge** of data. Fog devices are **not as resource-rich** → **resource management** is central.

Cloud vs fog (classic comparison table)

Requirement	Cloud computing	Fog computing
Latency	**High**	**Low**
Delay jitter	High	**Very low**
Location of server nodes	**Within the Internet**	**At the edge of the local network**
Distance between client and server	**Multiple hops**	**One hop**
Security	**Undefined** (user has little control)	**Can be defined** (local authority control)
Attack on data en route	**High probability**	**Very low probability**
Location awareness	**No**	**Yes**
Geographical distribution	**Centralized**	**Distributed**
Number of server nodes	**Few**	**Very large**
Support for mobility	**Limited**	**Supported**
Real-time interactions	Supported	Supported
Type of last-mile connectivity	**Leased line**	**Wireless**

They are **companions, not competitors** — the right **orchestration/synchronization** between them is the key.

Use cases

- **Emergency evacuation systems** — real-time info on affected areas/buildings, **dynamic exit-route planning** (blocked exits after fire/earthquake).
- **Natural disaster management** — real-time notifications about **landslides, flash floods** to potentially affected areas.

- **Large sensor deployments** — pre-process and summarize data before sending to cloud (reduce congestion).
- **IoT/big-data applications** — connected vehicles, smart cities, wireless sensor and actuator networks.
- **Applicability:** smart traffic lighting, **connected vehicles**, **smart grid**, wireless sensor networks, IoT, **SDN**.

Connected vehicles: rich connectivity — **V2V, V2I, I2I**; **road-side units (RSUs)**/access points; applications: **safety**, **infotainment**, alerts, traffic support, analytics; local fog decisions (a crash alert to approaching vehicles) instead of a round trip to the cloud; **VANET**. Fog attributes that make it ideal: **geo-distribution, mobility, location awareness, low latency, heterogeneity**. A **fog orchestration and network management layer** coordinates fog nodes with the cloud; **large analytics go to the cloud**. **Smart grid:** smart meters in homes → fog/micro-grid level decisions (e.g., campus-wide consumption) → aggregated data to the cloud for long-term analytics and provisioning. **IoT stack:** embedded systems & sensors → **multi-service edge (fog)** → core network → cloud data center.

Resource management in fog — challenges

- **Proper resource allocation among applications while ensuring end-to-end latency**; increase **throughput** with **high availability and scalability**.
- Aspects: **utilization of idle fog nodes**, more **parallel operations**, **load balancing**, **meeting delay requirements** of real-time apps, **crash fault tolerance**, **scalability**.
- Specific issues: **data may not be available at the executing fog node** (must fetch from sensor/data source); an **executing node may become unresponsive** under heavy load (latency compromised); **choosing a new node** for **micro-service migration**; **migrating partially processed persistent data**; delivering the **final result to the client/actuator in time**; deploying application components on different fog nodes meeting latency; **multiple applications co-located on one fog node** → one app's data may be compromised by another (**multi-tenancy**, worse because fog has fewer resources).
- **Approaches in literature:** migrate to the **nearest available** node (may not be the most suitable); **minimize carbon footprint** (e.g., video streaming); **resource prediction, estimation, (advance) reservation and pricing** for new IoT apps; **Docker as an edge-computing platform** (fast deployment, elasticity, good performance); pricing based on customers' fluctuating **relinquish probability**; joint **base-station association, task distribution and VM placement** as an **LP formulation** + heuristics.
- **Security:** fog devices **can't run resource-hungry security measures**; they may be **compromised and used as a platform to launch attacks**.

MCQ traps (7.3–7.4)

- Fog term introduced by **Cisco**.
- Fog: **one hop, low latency, location-aware, distributed, very many nodes, mobility supported, wireless last mile**; cloud: **multi-hop, leased line, centralized, few nodes, limited mobility**.

- **Both** cloud and fog support real-time interactions (per the table).
- Fog **complements** cloud; it does not replace it.
- SDN enabler uses **OpenFlow**; containers enabler = **Docker**.
- Security in fog "**can be defined**"; in cloud "**undefined**".

7.5 Use case: Geospatial cloud (L35)

Geospatial data — characteristics

- **Information explicitly linked to a location on the Earth's surface** (coordinate-based). Over **90 % of data relates to some location** (e.g., student records: hall, department, home PIN code).
- **Static** (rarely changes — state/district boundaries, a building's footprint, lakes) or **dynamic** (traffic, city population by day/night, footfall in a building, electricity demand).
- **Scale** varies from **metres to the globe**; **scale vs detail** (a building is a point from the sky but a polygon close up); ecological fallacy.
- Types: legal/political boundaries, cultural, climatic, economic data. **Sources:** social surveys, national surveys (Survey of India maps), **remote sensing** (air photos, satellite images), weather stations, **GPS**, pollution/air-quality/temperature sensors — maintained by **different organizations** in **proprietary** formats.
- **GIS** = tools/platform for **capturing, storing, querying, analyzing and displaying geospatial data**; components: **hardware, software, data (management), people (specialists)**.

Why move geospatial work to the cloud

- **Data-intensive** and **computation-intensive** (shortest path on huge road graphs, map overlays).
- **Variable load on GIS servers ⇒ dynamic scaling in/out** (utility computing fits).
- **Network-intensive web services** — data lies with many organizations; users want **services, not raw data** (e.g., "shortest route from IIT KGP to Kolkata with POIs and congestion", or "population and farmland within a 2-km buffer of a new canal").
- Need **homogeneity (syntactic and semantic)** — same reference system (e.g., lat–long), scale, data description, **standard encoding and sharing** — before querying across organizations.
- Trend: from **project mode to societal mode** — data (taxpayers' money) should be available ubiquitously, subject to policy and security.
- **Spatial Data Infrastructure (SDI):** treat spatial data **as infrastructure** (like the telecom infrastructure — hook in and use) for development planning and disaster management.
- Organizations benefit: **less infrastructure and spatial-service expertise needed**, avoid the **3–5-year hardware refresh**, share data with security controls, **pay as you go**, economy of scale, **green computing**.

Geospatial cloud — features

Shared resource pooling for organizations with common goals · **choice of deployment, service and business models** · **managed services** prevent data/work loss from outages · **controls for secure data sharing** · organizations acquire **web-service space as needed** (harvest only the services/data needed).

Architecture (cloud actors applied)

`Customer → Negotiator (optional) → Broker(s) (agents for data sets/services) → Data centers (VMs hosting organizations' spatial data and spatial web services)`, with an **SLA manager** (checks SLAs aren't violated) and **(security) auditor**.

- **Interface/"Interoperable" GIS:** holds **no data itself**; a **middle tier** that connects consumers to different spatial service providers (resource allocation/manipulation, data services, interface services).
- **Stack:** hardware → **hypervisor** → VMs (guest OS) → **instances of spatial web services** (per organization/state/district) — analogous to instantiating your own Google Docs domain on shared infrastructure.
- **Model: web services are the key technology**; integrate data from **heterogeneous back-end data services** (inside or outside the cloud); can run through the **PaaS model** (makes load balancing and scaling transparent); a **coordinating cloud with a central repository/registry** knows where each organization's data is; integrate into a **unified format**.

IIT KGP experiment (on Meghamala)

- **Bankura district (West Bengal): national highway** network (from the highway authority) and **local road network** (state/district) in **two separate VMs** → a processing service **merges them on the fly** → **shortest-path query** over the combined network.
- **Canal + river data** in separate VMs → **merged water network** → **500 m buffer** to find **flood-vulnerable areas**.
- Point: **separate repositories pulled in service mode** to answer a business rule, without moving whole datasets.

Challenges

- **Implementing spatial databases** in the cloud (data semantics), **auto-scaling** spatial DBs, **multi-tenancy** and **policy management** among tenants, **geographically situated backups**, data security.
- **Interoperability at three levels: data level, service level, security level** (different security policies on different data sets).
- Security: **multi-tenancy** and **lack of complete control** over data/apps; which assets to deploy, their value, how they can be compromised.

MCQ traps (7.5)

- Geospatial data can be **static or dynamic**; scale ranges from **metres to globe**.
- Interoperability levels: **data, service, security**.
- **SDI** treats spatial data as **infrastructure**.
- Users want **services** (answers), not raw data; web services are the key technology.
- Interface GIS holds **no data** itself.

Week 7 — High-yield takeaways

1. MCC = cloud + mobile + wireless; moves compute/storage off the device; MBaaS; examples Silk, Siri, iCloud, AR.
2. Profiler (time, power, traffic) → solver (what runs where) → synchronizer (merge results).
3. MAUI = method annotation, remoteable, native local, RPC; COMET = binaries only, thread migration via DSM, sends modified heap.
4. $E_{saved} = \frac{C}{M}(P_c - P_i/F) - P_{tr}\frac{D}{B}$ — offload when compute-heavy and data-light.
5. Cloudlet = trusted, resource-rich, nearby, single-hop, middle tier.
6. Fog = Cisco's term; extends cloud to the edge; enablers: virtualization, containers (Docker), SOA, SDN (OpenFlow).
7. Cloud vs fog table: latency, jitter, hops, location awareness, distribution, node count, mobility, last mile.
8. Fog use cases: evacuation, disasters, sensor aggregation, connected vehicles (V2V/V2I/I2I, RSU), smart grid; security issues: dispersed devices, MITM, privacy, smart-meter tampering.
9. Geospatial cloud: data/computation/network-intensive; SDI; broker + SLA manager + auditor; Bankura road and water-network merge demos; interoperability (data, service, security).

Week 8 — Docker Introduction, Green Cloud, Sensor Cloud, IoT Cloud, Course Summary (Lectures 36–40)

> **Source:** L36 Introduction to Docker Container · L37 Green Cloud · L38 Sensor Cloud Computing · L39 IoT Cloud · L40 Course Summary and Research Areas
>
> High-yield: **Docker philosophy (develop–ship–run anywhere), shipping-container analogy, image vs container, VM vs container, Docker components and commands; green-cloud energy figures, DC architectures, server/switch energy models, PUE; sensor-cloud definition, virtual sensors and their 4 configurations, proxies; IoT definition, IoT architecture, MQTT/CoAP/AMQP, IoT-cloud components, vehicular cloud services.**
>
> Containers, Kubernetes and Docker architecture are covered in more depth in **Week 10 (L47–L50)**.

8.1 Introduction to Docker containers (L36)

> 🎯 **Your practice set for L36** (your file *w5*): 50 questions · 13 easy · 25 medium · 12 hard.
> Answers checked against this lecture.
> Full set and answer key: chapter "Your practice sets", Week 8.

Motivation

- Software must be **redeployed, reconfigured or recompiled** every time it moves from one environment to another (laptop → server → phone → another cloud). With many apps (static websites, Python workers, user DBs, analytics DBs, queues, API endpoints, web front ends) and many targets, the **"app × environment" matrix** explodes. Goal: **tick every cell** with minimal effort — anything runs anywhere.
- **Container technology** bundles the **application with all its dependencies** into one unit that runs wherever the container fits.
- **Cloud is a platform to host containers** — containers **enhance** cloud; they are **not a competitor** to it.

Shipping-container analogy

- Goods of every kind (boxes, cars, barrels, pianos, servers) and every transport (trucks, forklifts, cranes, trains, ships) → chaos until a **standard shipping container** is introduced.
- **Inside the container, the product dictates** (a piano needs a different environment than a car); **outside, the infrastructure dictates** (standard size and handling for any carrier).

- Docker does the same for software: inside — the app with its **libraries, binaries and dependencies**; outside — any host that can run the container.

Docker basics

- **Docker:** an **open-source container management service**; **initial release March 2013**.
- Motto: **"Develop, Ship, Run Anywhere"** — build once, run on desktops, Android/iOS, any cloud (OpenStack, Azure, Bluemix, Amazon…).
- A buzzword in modern development, especially **agile projects**.
- **Features: smaller OS footprint** via containers (reduces the size of development) · teams (development, QA, operations) work seamlessly across applications · deploy **anywhere — physical or virtual machines and the cloud** · **lightweight ⇒ easily scalable**.

Docker components (lecture list)

Component	Purpose
Docker for Mac / Linux / Windows	Run Docker containers on that OS
Docker Engine	**Builds Docker images and creates Docker containers** (sits on the host OS — loosely analogous to a hypervisor)
Docker Hub	**Registry used to host various Docker images** (developers submit images for others)
Docker Compose	**Defines an application using multiple Docker containers**

Traditional virtualization vs Docker

Traditional virtualization (bottom → top)	Docker (bottom → top)
Server (physical) → **Host OS** (Linux/Windows) → **Hypervisor** (VMware, Xen, KVM, Hyper-V) → **VMs each with a guest OS** → apps	Server → **Host OS** → **Docker Engine** → **containers** (apps + dependencies)

- **Container = an abstraction at the application layer that packages code and dependencies together.**
- **Multiple containers run on the same machine and share the OS kernel**, each as an **isolated process in user space**.
- Containers take **less space than VMs**: images typically **tens of MB**, start **almost instantaneously**.
- **VMs** run a full guest OS in each box → **resource-intensive**; disk image and app state entangled with OS settings, installed dependencies and security patches. Any change needs a new virtualized environment.
- **Containers** share a single kernel; the image holds only the **executable and its package dependencies** (never installed on the host) → **no configuration entanglement** → runs anywhere; a small change saves **only the difference** between container A and A′.

- Containers run **natively on the host kernel** → **better performance** than VMs, which get **virtual access to host resources through a hypervisor**; each container is a discrete process using no more memory than any executable.

Image vs container (definitions)

Term	Definition
Image	**A lightweight, stand-alone, executable package that includes everything needed to run a piece of software — code, runtime, libraries, environment variables, config files**
Container	**A runtime instance of an image** — what the image becomes in memory when executed; runs **completely isolated from the host environment** by default, accessing host files and ports only if configured to

Workflow and commands

- **Source repository → build on the Docker engine → container image → push to a container image registry → users search, pull and run** (Docker works in a **client–server** mode).
- On a cloud: infrastructure → hypervisor → guest OS (VM) → Docker → containers, i.e., IaaS VMs can host containers.
- **Image commands:** `images` (list local images) · `run` (create a container from an image and execute a command) · `tag` · `pull` (download an image from a repository) · `rmi` (delete a local image; also removes unused intermediate images).
- **Container commands:** `ps` (list running containers) · `ps –a` (all containers including stopped) · `top` (processes in a container) · `start`, `stop`, `pause` · `rm` (delete a container) · `commit` (create an image from a container).
- **Dockerfile** — a **build script** that creates an image automatically; can be versioned in **Git/SVN** with its dependencies. **Docker Hub** (hub.docker.com) is a public repository of images and can **automatically build images from Dockerfiles on GitHub**.
- **Applicability:** application-level virtualization; one host runs several (e.g., spatial) applications for better utilization; **build once, deploy/run anywhere**; better collaboration.
- **Who uses Docker for what: developers** — eliminate "**works on my machine**" problems; **operators** — run and manage apps **side-by-side in isolated containers** for better **compute density**; **enterprises** — build **agile software-delivery pipelines** to ship features faster and more securely.

MCQ traps (8.1)

- **Image = static package; container = running instance** of an image.
- Containers **share the host kernel**; VMs each have a **guest OS**.
- **Docker Hub = registry; Docker Compose = multi-container apps; Docker Engine = builds images/creates containers.**
- Containers are **lighter and start faster** than VMs and give **better performance** (native kernel access).

- `rmi` removes **images**; `rm` removes **containers**; `commit` makes an image from a container; `ps -a` includes stopped containers.
- Docker's first release: **March 2013**.

8.2 Green Cloud (L37)

Why green cloud — the "dark side" of cloud

- Data centers consume **huge energy**; the **cost of computing equipment ≈ cost of environmental infrastructure** (space, AC, power) — roughly "x and x".
- **Scaling may hit a hard limit:** money may be available but **power supply** may not.
- **Carbon footprint** and environmental harm grow with consumption.
- Inefficient placement wastes energy: spreading VMs 1 per server looks "load balanced" but packing them and putting idle servers in **sleep/off mode** saves energy (see Week 5 consolidation example).
- Every layer matters: an inefficient **algorithm** at SaaS level (extra loops, CPU time) also wastes energy in the IaaS below.

Definitions

- **Green computing:** the **environmentally responsible and eco-friendly use of computers and their resources**; more broadly, the **study of designing, manufacturing/engineering, using and disposing of computing devices in a way that reduces their environmental impact**.
- **Green cloud computing:** aims to achieve **not only efficient processing and utilization of computing infrastructure but also minimize energy consumption**.

Facts and figures quoted (recognition)

Figure	Value
Gartner (2007): IT industry's share of world CO_2 emissions	**2 %**
US data centers' share of US power (≈2006–07)	**1.5 %**, **more than doubled since 2000**, cost **$4.5 billion**
Data-center energy per square foot vs typical office building	**10–100 times** more
Energy share of data-center **operational cost (OPEX)**	**≈10 %**, could rise to **50 %**
Cooling system cost	**$2–5 million per year**
Energy breakdown in a data center	**IT equipment 40 %, power distribution 15 %, cooling 45 %**
Data-center carbon emissions worldwide	**more than Argentina's and the Netherlands' emissions**
Amazon: energy-related costs	**42 %** of total budget (direct power + cooling amortized over 15 years)
Google, Microsoft, Yahoo	Build data centers near the **Columbia River** for **cheap hydroelectric power** (less transmission loss)

Data-center network architectures

Architecture	Description
Two-tier	**Access** and **core** layers; full-mesh core network; load balancing using **ECMP** (equal-cost multi-path); layer-3 switches at core, rack (layer-2) switches to servers
Three-tier (most widely used today)	**Access, aggregation and core** layers; hierarchical; **scales to over 10,000 servers**
Three-tier high-speed	Increased core bandwidth, **2-way ECMP**, **100 Gigabit Ethernet** links (vs 1–10 GE)

> ⚠️ *Transcript slip:* it says "load balancing using ICMP protocol" — the slide's protocol is **ECMP** (equal-cost multi-path routing).

- Faster architectures also **consume more energy** → **performance vs energy trade-off**.

Energy models

- **Server energy model:** an **idle server consumes about 66 % of its peak power** (at peak CPU frequency) — fixed power for memory modules, disks, I/O, plus a CPU part that scales with frequency/load. ⇒ Idle servers waste energy; consolidate and switch off.
- **Switch energy model:**

$$P_{switch} = P_{chassis} + n_{linecards} \cdot P_{linecard} + \sum_{r} n_{ports,r} \cdot P_r$$

Typical share: **chassis ≈ 36 %**, **line cards ≈ 43 %**, **port transceivers ≈ 11 %**. Load-aware traffic distribution can cut switch energy.

Power Usage Effectiveness (PUE)

PUE / DCiE calculator

Total facility power (kW)
1500

IT equipment power (kW)
600

PUE = total / IT	**2.5**
DCiE = 1/PUE	**40 %**
Non-IT overhead (cooling, power distribution…)	**900 kW**

Room to improve (ideal PUE = 1.0)

Worked example with the default values (interactive in the HTML version).

(The lecture mentions PUE as a measure without a formula; the standard definition is given here.)

$$\text{PUE} = \frac{\text{Total facility power}}{\text{IT equipment power}} \qquad \text{DCiE} = \frac{1}{\text{PUE}} \times 100\%$$

- **Ideal PUE = 1.0** (all power goes to IT); **higher = worse**.
- **Example using the lecture's breakdown:** IT = 40 % of total ⇒ **PUE = 100/40 = 2.5**, DCiE = 40 %.
- **Trap:** PUE is **total ÷ IT**, not IT ÷ total (that is DCiE).

Performance vs energy; provider initiatives

- Shift the focus from optimizing resource management **purely for performance** to **optimizing energy efficiency while maintaining high service-level performance** — reduce power cost, maximize revenue.
- Providers must keep **profit margins** from being eroded by energy costs (e.g., Amazon 42 %; siting near hydro power).

Green cloud architecture

- **Green Cloud Broker / allocator** (acts for consumers): **lease cloud services, schedule applications**; **analyze user requirements**, **calculate cost and carbon footprint** of services, **carbon-aware scheduling**, monitor CO_2; works with green negotiator, service analyzer, consumer profiler, pricing, **energy monitor**, service scheduler, VM manager, accounting → **QoS- and energy-aware provisioning**.
- **Green middleware per layer:**
  - **IaaS:** virtualization + **energy and temperature sensors**, **demand predictor**.

    - **PaaS: green profiler**, **power capping**, **green compiler**, green resource allocation.
    - **SaaS: power capping**, green software services.
- **Summary:** clouds are data centers offering subscription services with **high energy use → high operating cost + environmental impact** → need a **carbon-aware green computing framework**; many open research issues.

MCQ traps (8.2)

- **Cooling (45 %) > IT equipment (40 %) > power distribution (15 %).**
- Idle server ≈ **66 % of peak power** — idle is far from free.
- **Three-tier** (access–aggregation–core) is the most widely used DC architecture.
- PUE = **total facility / IT** power; ideal 1.0.
- Green computing covers the **whole life-cycle** (design, manufacture, use, **disposal**).

8.3 Sensor Cloud (L38)

Motivation

- Sensors are everywhere (temperature, pressure, atmospheric, cameras, RFID, mobile phones, biological/physiological). Sensors → **decision support/intelligence** → **actuators** (a modern car may soon have more sensors than mechanical parts).
- The Internet is a **real-time information source** (blogs, social networks); **cloud** handles the **big-data revolution**. Combining sensor data + Internet data creates demand that cloud can meet.
- **Wireless Sensor Network (WSN):** seamlessly couples the **physical environment with the digital world**; sensor nodes are **small, low-power, low-cost** with **sensing, processing, memory, communication bandwidth and battery** constraints; in aggregate they have substantial data acquisition and processing capability. Nodes → **sink/gateway** → rest of the world (paths or clusters with cluster heads). Applications: environment, healthcare, education, defence, manufacturing, smart homes/phones.

Limitations of sensor networks (list question)

1. **Very challenging to scale** sensor networks to large sizes (or change granularity).
2. **Proprietary, vendor-specific designs** → hard to **interconnect** different sensor networks.
3. **Sensor data cannot be easily shared** between groups of users (proprietary formats, no sharing mechanisms).
4. **Insufficient computational and storage resources** for large-scale applications.
5. Used for **fixed and specific applications that cannot easily be changed once deployed**.
6. **Slow adaptation** of large-scale sensor network applications.

Limitation on the cloud side: the **immense power of the cloud can only be fully exploited if it is seamlessly integrated into our physical lives** — i.e., provide **real-world information to the cloud in real time** and let the cloud **act and serve us instantly**; **add sensing capability to the cloud**. **Tourist scenario:** a phone records gestures/activities (camera, mic), geographic data, wearable physiological sensors (cross-checked with medical records), speech recognition, image processing of restaurant logos, social-network friends — **the phone alone cannot do all of this** ⇒ need **real-time processing of heterogeneous data sources** (body-area networks to wide-area water-quality monitoring), **automatic workflow formation and service invocation** in the cloud, and **swift processing** using cloud power.

Definition of sensor cloud

> **Sensor cloud: an infrastructure that allows truly pervasive computation using sensors as an interface between the physical and cyber worlds, the data-computing clusters as the cyber backbone, and the Internet as the communication medium.**

- It **integrates large-scale sensor networks with sensing applications and cloud computing infrastructure**; **collects and processes data from various sensor networks**; **enables large-scale data sharing and collaboration** among users and applications in the cloud; **delivers cloud services via sensor-rich devices**; allows **cross-disciplinary applications that span organizational boundaries**.
- Users can **collect, access, process, visualize, archive, share and search** large amounts of sensor data; supports the **complete sensor-data life cycle** (collection → decision support); sensor networks spread over huge areas can be **connected together and used simultaneously by multiple users on demand**.

Proxies

Proxy	Role
Sensor-cloud proxy	**Interface between sensor resources and the cloud fabric**; manages sensor resources and network connectivity; **data format conversion (e.g., XML)**, **interoperability**, **data cleaning and aggregation**, **data transfer to cloud storage**
Sensor-network proxy	For **sensor resources that do not have a direct connection to the cloud**

Use case — traffic flow sensors on traffic lights give real-time flow for **drivers' route planning**; augmented with cheap **humidity and temperature** sensors they give a **local heat index** that supplements the meteorological department's precise but sparse sensors — one physical infrastructure, several **virtualized** views (met dept, traffic dept, drivers).

Virtual sensors (core idea)

- **A virtual sensor is an emulation of a physical sensor that obtains its data from underlying physical sensors.**
- Provides **customized views** to users using **distribution and location transparency**.
- WSN hardware can barely run multiple tasks and **cannot host multiple VMs** → virtual sensors act as **images in software** of the physical sensors.
- A virtual sensor holds **metadata about the physical sensors and the users currently holding them**.
- Layers: physical sensors (data processing, routing, power management, clock sync, local fault detection) → **virtual sensors** → **virtual sensor groups** (per application: met, traffic, disaster…) → end users. The **sensor-cloud admin** registers physical sensors and manages the infrastructure (sensors may go up/down).
- Request flow: **client → portal server** (provisioning, resource management, monitoring) → **virtual sensor groups → virtual sensors → physical sensors**.

Four virtual-sensor configurations (very examinable) | Configuration | Meaning | |—|—| | **One-to-many** | **One physical sensor ↔ many virtual sensors.** Users each see their own virtual image, but the physical sensor is **shared**; the **middleware computes the physical sensor's sampling duration and frequency** considering all users | | **Many-to-one** | **Geographical area divided into regions, each with one or more physical sensors/WSNs**; when a user needs **aggregate data** of a phenomenon for a region, all underlying WSNs for it are switched on and the user gets the **aggregated** value | | **Many-to-many** | **Combination** of one-to-many and many-to-one | | **Derived** | A **versatile virtual sensor derived from a combination of multiple (different types of) physical sensors** — e.g., proximity, light, temperature, humidity → answer "**Are environmental conditions safe in a wildlife habitat?**"; a **generalization of the other three** (lies in the *type* of physical sensors it communicates with) |

Layered sensor-cloud architecture

WSN providers → sensor-centric middleware (WSN registration, maintenance, data collection) **→ virtualization middleware** (virtual sensor provisioning, management, **image life cycle**) **→ user interfaces** (user repository, data repository, session and membership management). **Summary:** sensor-cloud infrastructure **virtualizes sensors** and provides management for them; users **create virtual sensor groups dynamically by selecting templates**; focuses on **sensor system management and sensor data management**; takes the **burden of deploying and managing the network away from the user** (mediator).

MCQ traps (8.3)

- **One-to-many** = one physical → many virtual (middleware sets sampling); **many-to-one** = many physical (regions) → aggregate; **derived** = combine **different types** of sensors, generalizes others.
- **Sensor-network proxy** is for sensors **without direct cloud connectivity**; **sensor-cloud proxy** does format conversion/aggregation.

- WSN nodes **cannot run multiple VMs** → hence virtual sensors in software.
- In the definition: sensors = interface, **data-computing clusters = cyber backbone**, **Internet = communication medium**.

8.4 IoT Cloud (L39)

Internet of Things

- **IoT = extending the current Internet and providing connection, communication and inter-networking between devices (physical objects, "things").**
- **Objects or people are given unique identifiers and the ability to transfer data over a network without requiring human-to-human or human-to-computer interaction.**
- A "thing": a person with a **heart-monitor implant**, a farm animal with a **biochip transponder**, an automobile with sensors that **alert the driver when tyre pressure is low**, or any natural/man-made object that can be **assigned an IP address**.
- Difference from a plain sensor deployment: in IoT the **thing is uniquely identifiable**; in a sensor deployment we mostly care about the data.
- IoT includes **actuation** too (switch on a light/fan from a phone app).
- Slogan: **"any time, any place connectivity for anyone and anything."** Example: low fuel → alert driver + locate nearest station/workshop.
- Wireless sensor technology bridges the **physical and virtual worlds**; things respond to changes in their physical environment (e.g., an electronic jacket heating up when it gets cold).

Basic IoT architecture

`Things → Gateway → Network/Cloud` Layered: **devices → communication (MQTT, HTTP) → aggregation/bus layer (Enterprise Service Bus — ESB, message broker) → event processing and analytics → web portal, dashboard, API management**; vertical: **device manager**, **identity and access management**.

Characteristics/issues of IoT systems

Issue	Meaning
Scalability	Growing numbers of sensors/devices
Big data	Huge volumes to process/analyse
Role of cloud	IoT often uses cloud for large storage/processing and flexible scaling
Real time	Data flows continually; timely responses to event streams
Highly distributed	From a room to a campus or city
Heterogeneity	Diverse devices, data formats and networks (**Wi-Fi, ZigBee, Bluetooth**) → **interoperability**; handled locally by **IoT gateways** (the gateway hides the room's heterogeneity from the outside world)

Why IoT + cloud ("IoT cloud")

- Cloud makes applications **infrastructure-free**, **pay-as-used and on-demand**; clients upload data and applications; services used **without understanding the infrastructure**; **economy of scale**; data/services stored remotely but **accessible from anywhere**.
- Wide IoT deployment (logistics, smart city, healthcare, parking) creates high demand for **data storage, processing and management in cloud data centers**; cloud services are **mature**, with **elastic computation and data-management scalability**.
- **Cloud acts as both a computational/data-processing platform and a management platform for IoT.**
- **Two-way coordination:** a **cloud service can request an IoT service** (e.g., to **reduce the amount of sensing data**), and an **IoT service can request cloud services to provision more resources**.

Cloud components for IoT (reference architecture)

User layer (end-user apps) → **proximity network** (sensors/actuators, agents, firmware, network connectivity, user interface, devices) → **IoT gateway** (application logic, analytics agent, device data store) → **public network** (peer cloud, DNS, **CDN**, firewall, load balancer) → **provider cloud** (**device registry, device management, API management**, application logic, **analytics**, data repository, cognitive/actionable insight, **streaming computation**) → **enterprise network** (user directory, enterprise apps — e.g., car anomaly → cloud analytics → car maker's workshop → corrective action).

iCOMOT (an IoT cloud system framework)

- **Top layer:** a typical IoT application across IoT and cloud — **light-weight analytics and control** (near the edge) + **large-scale data analytics** (cloud).
- **Middle layer:** the **software layer** built on cloud services and IoT elements — sensors, **gateways**, load balancer, event handler, **message-oriented middleware**, **NoSQL databases**, near-real-time data processing.

- **Bottom layer:** iCOMOT **tools/services to deploy, configure, govern, control and monitor** the software layer.

Infrastructure, protocols and platforms

Category	IoT side	Cloud side	Purpose
Infrastructure	Industrial/common **IoT gateways** (e.g., Intel IoT gateway), **OS containers (Docker)**	**VMs and OS containers**	Execute software components
Connectivity protocols	**MQTT** (Message Queue Telemetry Transport), **CoAP** (Constrained Application Protocol), **HTTP**	**MQTT, AMQP, HTTP**	Connect IoT elements with each other and with cloud services
Platform services	Light-weight data services	Load balancers (e.g., **HAProxy**), databases	Core platform services

Case: IoT-based vehicular data cloud

- A **multi-layered vehicular cloud** using IoT + cloud to address transport issues; two innovative services: **intelligent parking** and **vehicular data mining (e.g., vehicle warranty analysis)**.
- Architecture: **IoT devices** (sensors, actuators, GPS, smartphones; street lights, utilities, smart metering, road-side infrastructure) ⇄ **communication** (satellite, **VANET**, 3G/4G, Internet) ⇄ **cloud** (remote diagnosis, traffic, **GIS**, SaaS/PaaS/IaaS).
- **Services:**
  - **Network and data processing as a service** — vehicles **provide their networking and data-processing capability to other vehicles** through the cloud.
  - **Storage as a service** — vehicles with **unused storage** (in the **OBU — on-board unit**) share it with vehicles needing more.
  - **Cooperative information services (platform/community)** — **traffic information, hazard-location warning, lane-change warning** (safety services).
- **Intelligent parking:** modules — **driver module, communication module (V2I, I2I), functional module (parking management, monitoring, checking, reservation, advertisement), application module**; cars (with sensors/timestamps) find vacant lots via the gateway and cloud before reaching the parking lot.

Summary

IoT is dynamic and growing (domestic, commercial, industrial, health), needing large processing/storage and facing **scaling, speed, safety, security and privacy** challenges; cloud offers large, flexible, scalable resources ⇒ **IoT cloud is an enabling technology** for future applications.

MCQ traps (8.4)

- **MQTT = Message Queue Telemetry Transport; CoAP = Constrained Application Protocol** (IoT side); **AMQP** listed on the cloud side.
- IoT things have **unique identifiers** and communicate **without human interaction**.
- **Gateways** handle heterogeneity/interoperability **locally**.
- Coordination is **two-way**: cloud ↔ IoT services request each other.
- Vehicular cloud: **storage as a service via unused OBU storage**; **network & data processing as a service** by vehicles.

8.5 Course summary and research areas (L40)

What the course covered (recap list)

NIST definition and XaaS; characteristics, advantages, disadvantages; **evolution** (cluster, grid, distributed → cloud as **utility computing**: pay-as-you-go, metered, scalable, no infrastructure at user end); architecture and service models; **foundation technologies — SOA, web services, XML**; **SLA** (uptime like 99 %/99.9 %; critical for banks, defence, disaster management, healthcare); **cloudonomics** (when to go to cloud — also security/privacy policy limits, e.g., labs on cloud but not student/employee data); **resource management** (finite resources, energy); **data management** (GFS, HDFS, big data, MapReduce); **security** (IAM, RBAC vs risk-based access control, trust/reputation/risk/competence, authentication, key management for encrypted data — e.g., competitive exams conducted on cloud); **open-source and commercial clouds** (OpenStack/Meghamala, Amazon, Azure, GCP, IBM Bluemix); **companion technologies** (fog, sensor cloud, containers/Docker, green cloud).

Research areas (recognition list)

Area	Topics
Infrastructure and services	Cloud architecture, storage/data architecture, distributed & cloud networking, IaaS/PaaS/SaaS, **storage-as-a-service, network-as-a-service, information-as-a-service, science-as-a-service**
Management, operations, monitoring	Cloud composition, **service orchestration**, **cloud federation**, bridging, **cloud bursting**, cloud migration, hybrid cloud integration, **green and energy management** (stressed as a key challenge), configuration and **capacity management**, workload profiling and deployment control, **metering, monitoring and auditing**, service management
Security and privacy	Data privacy, access control, identity management, **side-channel attacks**, **security as a service**, legal issues (data location/jurisdiction)
Performance, scalability, reliability	Performance of cloud systems/apps, availability, **micro-service-based architectures**
Systems software and hardware	Virtualization technologies, service composition, provisioning, orchestration, **hardware architecture support for cloud**
Data analytics in cloud	Analytics applications, scientific computing and data management, **big-data management and analytics**, storage-data-analytics clouds
Service management	**Service discovery and recommendation**, composition, **QoS management**, security/privacy of services, **semantic services**, service-oriented software engineering
Related technologies	Fog, IoT cloud, sensor cloud, containers (Docker)

- Use **cloud simulators** when real infrastructure isn't available; follow top journals/conferences for current directions.

Week 8 — High-yield takeaways

1. Docker (Mar 2013): develop–ship–run anywhere; shipping-container analogy; Engine/Hub/Compose; image (package) vs container (running instance); containers share host kernel, tens of MB, near-instant start; Dockerfile; commands (`ps -a` , `rmi` , `commit` …).
2. Green cloud: DC energy = IT 40 %, cooling 45 %, power distribution 15 %; idle server ≈ 66 % of peak; switch = chassis 36 %, line cards 43 %, ports 11 %; Gartner 2007: IT = 2 % of CO_2; Amazon energy = 42 % of budget; three-tier DC most common; PUE = total/IT (ideal 1).
3. Green broker = carbon-aware scheduling; green middleware per layer (sensors & demand predictor; green profiler/compiler & power capping).
4. Sensor cloud definition (sensors = interface; clusters = backbone; Internet = medium); 6 WSN limitations; proxies; virtual sensors (one-to-many, many-to-one, many-to-many, derived).
5. IoT: unique IDs, no human interaction; things → gateway → cloud; MQTT/CoAP/HTTP (IoT), MQTT/AMQP/HTTP (cloud); cloud as processing + management platform; vehicular cloud services

(NaaS/data processing, storage via OBU, cooperative safety info), intelligent parking.

6. Research areas: infrastructure & services, management (green energy!), security, performance, systems, analytics, service management, related tech.

Week 9 — Cloud–Fog–Edge Paradigm, Resource Management, Cloud Federation, VM Migration Basics (Lectures 41–45)

> **Source:** L41 Cloud–Fog Computing – Overview · L42 Resource Management – I · L43 Resource Management – II · L44 Cloud Federation · L45 VM Migration – Basics, Migration Strategies
>
> High-yield: **health cloud–fog case study (levels, module placement, iFogSim results); resource-management taxonomy (architecture / infrastructure / algorithms); service placement (constraints, offloading, control); federation architectures and coupling levels; reasons for VM migration; cold vs live; pre-copy vs post-copy; downtime vs migration time.**
>
> The **numerical analysis of VM migration (formulas)** is in **Week 10, L46**.

9.1 Cloud–Fog computing overview and health case study (L41)

Recap and motivation

- Cloud = XaaS (IaaS/PaaS/SaaS, storage-aaS, DBaaS…), pay-as-you-go; NIST characteristics. It *feels* centralized: data goes to the cloud, results come back → **network delay** is a problem for **mission-critical** applications.
- **Fog** = processing and applications concentrated in **devices at the network edge** instead of almost entirely in the cloud; **term coined by Cisco** to ease wireless data transfer to distributed IoT devices (Cisco's routers etc. have spare compute).
- **Example:** 4 studios × 10 temperature sensors = 40 sensors. Instead of sending all readings to the cloud, a local node checks whether each room is within range (e.g., 20–22 °C ± 2) and transmits only "within range" or anomalies → **less load**. The cloud still does **global analytics** across rooms.
- **Vision of fog:** enable applications on **millions/billions of connected devices to run directly at the network edge**, reducing overall network traffic → saves **delay, energy and cost**.
- Fog is **not a replacement** for cloud — it is **complementary/supportive**. The cloud takes **global** decisions and provisions services fog can't; fog has **low resources**, is **near the things**, has **no global view**, may give **better security**, supports **mobility** and **low latency**.

Cloud–fog–edge layers

- **Edge** = the sensing devices (IoTs, body sensors) — may do trivial processing (e.g., **noise reduction**) so cleaner data is sent. (Fog and edge are often used **interchangeably**; the line is thin.)
- **Fog** = cellular base stations, network routers, **Wi-Fi gateways**, dedicated fog nodes — run applications on spare resources.
- **Cloud** = global view and heavy analytics.

- **Communication is two-way:** e.g., in health, edge (body-area network: pulse, heart rate, temperature, BP) → fog (pre-processing; can **alert the patient immediately**) → cloud (processed data/health metrics, sampled data) and back.
- **Gains:** better **response time** (no round trip to cloud) and **less data volume to cloud** → **lower energy and cost**; fog can also pull **other data** (e.g., nearby meteorological data for agriculture).
- **Cost:** you need fog-capable devices; new challenges — **interoperability** among heterogeneous fog nodes (routers vs phones), **timing relationships**, overall management of data flow; must check **whether it actually gains** for a given application.

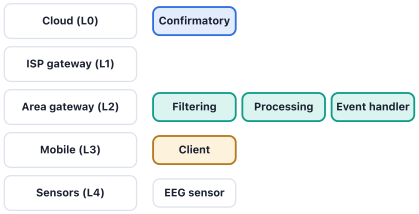
Limitations motivating cloud–fog–edge

Layer	Limitations
Cloud	**High latency**; **not an HPC platform**; large data volumes need **high bandwidth**; not designed for the **3 Vs (volume, variety, velocity)** of IoT big data; interoperability across IoT sources
Edge/IoT devices	Limited **processing**, **storage** (e.g., a pulse oximeter doesn't store data), **power** (especially when mobile)
Fog (benefits)	**Much lower latency** (real-time use) · **less network congestion** · **reduced cost of execution in the cloud** · **location awareness** (fog knows its own coordinates → spatio-temporal data without sending GPS each time) · better handling of the colossal data generated by sensors · can help **interoperability** (fog nodes follow an agreed protocol towards the cloud)

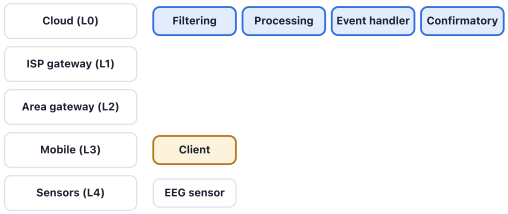
Case study — health cloud–fog framework (IIT KGP lab)

Module placement: fog-based vs cloud-based

diagram sequence



1. Fog–edge placement: client at mobile; filtering, processing, event handling at the **area gateway**; **confirmatory in cloud** (global view). ⇒ low network usage, low cloud cost, ~constant latency.



2. Cloud-only placement: everything except the client runs in the cloud ⇒ network usage, cloud execution cost and latency grow with configuration; mobile energy is the same in both.

Hierarchy: Level 0 = Cloud → Level 1 = ISP → Level 2 = Area gateway → Level 3 = Mobile devices → Level 4 = Sensors (e.g., **EEG**). The fog may be a **network of devices**, not one box. Upward traffic carries more data; downward carries analysed (smaller) data. **Simulation tools: CloudSim** (from the **University of Melbourne**) and **iFogSim** (developed jointly by **IIT Kharagpur and the University of Melbourne**). **Application modules (flow):** EEG sensor → **client module** → **data filtering module** → **data processing module** → **event handling module** → **confirmatory module** (confirms the event/alert) → client module → display/actuator (inform patient).

Module placement compared | Module | Fog-based placement | Cloud-based placement | |—|—|—| | Client module | Mobile | Mobile | | Data filtering | **Area gateway** | Cloud | | Data processing | **Area gateway** | Cloud | | Event handling | **Area gateway** | Cloud | | **Confirmatory module** | **Cloud** (needs the **global view**) | Cloud |

Results (synthetic lab setup — not to be over-generalized):

- **Network usage: very low for fog** (only positive/candidate cases go to the cloud's confirmatory module); **high for cloud** (every module's data travels to the cloud).
- **Cost of execution in the cloud: much higher** with cloud-based placement (fog uses surplus local resources at practically no extra cost).
- **Latency: roughly stable for fog**; **increases** for cloud as configurations scale.
- **Energy:** data-center, mobile and edge energy patterns differ by placement. **Prototype:** wearable health bands → **Raspberry Pi as fog devices** → **AWS** or **in-house OpenStack** as cloud (in-house has little network delay inside the IIT KGP network); medical-domain logic decides alerts ("see a doctor"), part on the fog and part in the cloud.

MCQ traps (9.1)

- In the fog placement, the **confirmatory module stays in the cloud** (global view); filtering/processing/event handling go to the **area gateway**.
- iFogSim = **IIT KGP + University of Melbourne**; CloudSim = **University of Melbourne**.
- Cloud is **not** an HPC platform.
- Fog **complements** the cloud; communication is **bidirectional**.

9.2 Resource management in cloud–fog–edge — I (L42)

Why resource management at fog/edge

- Processing IoT data directly in the cloud is **inefficient for time-sensitive** applications and uses a lot of network → fog/edge processing close to data sources.
- But adding IoT devices is not automatically a win: some fog/edge resources become **over-constrained** while others have spare capacity → **resource management is necessary** (and itself **costs** resources and time).
- Issues: **resource allocation, workload balancing, resource provisioning, task scheduling, QoS**.
- Fog/edge benefits recap: **latency** (aggregate before sending; some decisions — e.g., temperature alarm — at the edge; aggregation of several sensors at fog avoids one misbehaving sensor's false alarm), **reliability**, **faster response**, **interoperability** (convert °C vs °F from different sensor makes at edge/fog), and even **privacy/trust** (strip patient identity before sending to the cloud, re-attach on return; fog may be within your premises).

Fog environment model

- **Three layers: client (edge) layer, fog layer, cloud layer.**
- The **fog layer accomplishes the clients' resource requirements**; **if resources in the fog are unavailable or limited, the request is passed to the cloud layer** (or to collaborating fog nodes). The cloud also handles **global computations** (e.g., campus-wide AC health statistics).
- **Fog server manager** employs the available processors for clients; **VMs on fog data servers** process the data and deliver results to the fog server manager; fog servers use **server virtualization**. A fog layer may form a **fog data center/fog network** needing management. Must handle **mobility** (device moves from one fog node's area to another → hand-off without losing aggregated data).
- **Scenarios: VANET** — vehicles' sensors and **OBUs** communicate **V2V** or **V2I** with **road-side infrastructure**, which connects to cloud data centers (e.g., certificate revocation, correlating slowdowns with accidents); highly mobile, frequent make-and-break. **Healthcare** — body-area network → nearest hop/mobile device (fog) → health cloud. Static cases — room temperature/humidity sensors.
- Trend: **dedicated micro data centers** and **Internet nodes (routers, gateways, switches) augmented with computing**; bring computing down from the cloud towards the sensor end.

Taxonomy of resource management (survey: Hong & Varghese, 2019)

Three pillars: Architecture, Infrastructure, Algorithms.

Pillar	Component	Meaning
Architecture	**Data flow**	Direction of workload movement: **aggregation**, **sharing**, **offloading** (user → edge, or cloud → edge)
	Control	How resources are controlled: **centralized** (one controller for many edge nodes), **distributed**, (hierarchical)
	Tenancy	Whether **a single application or multiple applications** are hosted on an edge node
Infrastructure	**Hardware**	**Small-form-factor devices**: network gateways, **Wi-Fi routers, set-top boxes**, small home servers, **edge ISP servers**, cars/vehicles, **drones** used as compute servers
	System software	Runs directly on fog/edge hardware (CPU, memory, devices); manages and distributes resources — **OS, virtualization software**
	Middleware	Runs on the OS and provides **complementary services not supported by system software — coordinates distributed nodes, deploys VMs and containers**
Algorithms	**Discovery**	Identify edge resources onto which workloads (from cloud or user devices) can be deployed
	Benchmarking	Capture **performance** of resources (CPU, storage, network) — guides decisions
	Load balancing	Distribute tasks across edge data centers efficiently (optimization, search) — avoid skew
	Placement	Place incoming tasks on suitable resources considering availability and **environmental change** (dynamic-condition-aware or iterative techniques)

- Architecture choice depends on the application: static sensors → centralized control may suffice; vehicles over a large area → distributed control.

MCQ traps (9.2)

- Fog environment = **client/edge, fog, cloud** layers; fog serves requests and **forwards to cloud when resources are limited**.
- Taxonomy pillars: **Architecture (data flow, control, tenancy) · Infrastructure (hardware, system software, middleware) · Algorithms (discovery, benchmarking, load balancing, placement)** — don't mix components across pillars.
- **Middleware** (not system software) deploys VMs/containers and coordinates nodes.

9.3 Resource management in cloud–fog–edge — II: service placement (L43)

Service placement problem

- A **fog orchestration engine/orchestrator** decides placement so that load isn't skewed and **time and energy are optimized**. Fog is **highly virtualized**, providing compute, storage and control between end users and cloud servers; it acts as an **interoperability** layer.
- **Key considerations: location awareness** (vehicle → nearest RSU; ambulance sending patient data along its route), **low latency**, **better bandwidth utilization**, **scalability**, **mobility support**.

Survey structure (service placement — four groups)

Group	Contents
Problem statement	**Infrastructure model, application model, deployment pattern**
Service placement taxonomy	**Control plane design, placement characteristics, system dynamicity, mobility support**
Optimization strategies	**Objectives (latency, resource utilization, cost, energy consumption)**, metrics, problem formulation, resolution strategies
Evaluation environment	**Analytical tools, simulators, experimental test-beds**

Worked placement example — video capture & recognition (Google Glass)

Pipeline: **vision capture (A) → coarse feature extraction (B) → face recognition (C) / object recognition / OCR → learning-based activity inference → rendering → display effect (H)**. | Component | Placed on | |—|—| | Vision capture, coarse feature extraction, display | **The device (Google Glass)** | | Face recognition | **Local laptop/edge server via a smart Wi-Fi gateway** (too heavy for the Glass) | | Object recognition | **LAN router** with spare compute | | OCR | **Cloudlet** | | **Learning-based activity inference** | **Cloud** (heavy compute; may need **external data/models**) | | Rendering | **Smart Wi-Fi gateway** | → Distributing across edge/fog/cloud gives **better efficiency, lower latency, better scalability, mobility** (another gateway takes over as the user moves).

Application placement — definition and constraints

- **Application placement = the mapping pattern by which application components and links are mapped onto the infrastructure graph** (computing devices and physical edges/links). Requires **finding available resources** and their free capacity, and matching **application requirements** (e.g., face recognition needs ≥ 500 MB RAM).
- A **centralized or distributed/collaborative orchestration manager** handles it; providers aim for **optimal or near-optimal** placement.

Constraint type	Examples
Resource-level	Finite **CPU, RAM, storage, bandwidth** of each node
Network-level	**Latency, bandwidth**
Application-level	**Locality** (restrict services to a specific location — e.g., "must not leave the IIT KGP network"), **delay sensitivity/deadlines**

Offloading (the core mechanism)

- **Offloading = moving a server/application and its data towards the network edge**; augments computing of user devices; then **results must be aggregated/assembled**. | Direction | Techniques | |—|—| | **User device → edge** (typically **single hop**) | **Application partitioning**, **caching mechanisms** | | **Cloud → edge** | **Server offloading** (**replication** of servers at the edge, or **partitioning** a server across edge nodes), **caching** (**content-popularity-based**, **multi-layer**) |

Control of placement

Control	Approaches
Centralized	**Solver-based**, **graph-matching-based**
Distributed	**Blockchain-based**, **game-theoretic**, **genetic-algorithm-based**

Infrastructure, system software, middleware, algorithms (detail)

- **Hardware:** fog/edge uses **low-power, small-form-factor devices** that already exist with spare capacity — **compute devices** (single-board computers, commodity products) and **network devices** (gateways, routers, **Wi-Fi access points, edge racks**) — together enabling processing of **IoT and CPS data**.
- **System software must support multi-tenancy and isolation**:
  - **System virtualization: VMs, containers**, **VM and container migration**.
  - **Network virtualization: SDN** (separates **control plane and data plane**; modern switches are mostly SDN-enabled), **NFV**, **overlay networks**.
- **Middleware** provides **performance monitoring, coordination, orchestration, communication facilities and protocols** — e.g., **volunteer edge computing, hierarchical fog computing, mobile fog computing, cloud orchestration management**.
- **Algorithms — discovery** (identify edge resources; programming infrastructure, **handshaking protocols, message passing**), **benchmarking** (capture resource performance to maximize placement quality), **load balancing** (by **priority, fairness**...), **placement** (identify resources appropriate for a workload).

MCQ traps (9.3)

- Optimization objectives listed: **latency, resource utilization, cost, energy consumption**.
- Constraints: **resource-level, network-level, application-level** (locality, delay sensitivity).

- Device→edge offloading uses **application partitioning and caching**; cloud→edge uses **server offloading (replication/partitioning) and caching**.
- Centralized control: **solver, graph matching**; distributed: **blockchain, game theory, genetic algorithms**.
- SDN separates **control and data planes**.

9.4 Cloud federation (L44)

Definition

- **Federated cloud (cloud federation): the deployment and management of multiple external and internal cloud services to match a business need.** A federation is a **union of several smaller parts that perform a common action**.
- Example: IIT KGP's Meghamala (internal only) runs an exam for 2000 first-year students; if capacity is short, **borrow capacity from external providers** — seamlessly, with timing constraints met.
- **Collaboration between CSPs to achieve: capacity utilization** (lend surplus at a price) · **enhanced interoperability** · **cataloguing of services** (a global catalogue of the federation's services) · **insight about providers and SLAs** (how much each exposes depends on coupling).

Motivation

Maximize resource utilization · minimize power consumption · load balancing · global utility · expand CSPs' footprint (each CSP effectively shares the others' footprints → marketability, profitability).

Characteristics

- Overcome current cloud limitations — **service interruption, lack of interoperability, degradation of services** (other members absorb overloads/faults).
- Cloud federation is **one of several inter-cloud organizations**.
- **Voluntary** participation (not regulated/forced).
- **Maximum geographical separation** — ideally in different **power zones, network zones, seismic zones** so one disaster doesn't hit all members (high reliability).
- **Well-defined marketing system and regulated federation agreement.**
- An environment where **multiple service providers share their resources**.
- Issues to solve: **portability** between providers; **interoperability** of services and data.

Coupling levels

Coupling	Interoperation	Control over remote resources	Monitoring info	Advanced features
Loosely coupled	**Limited**	**Little or none** (VM placement unknown)	**Limited** — e.g., CPU, memory, disk consumption of VMs	**Usually none** — no cross-site networks, **no VM migration**
Partially coupled	Via a **contract/framework agreement** stating terms under which one partner uses another's resources	**Some** — e.g., **affinity rules** forcing VMs onto the same physical cluster	**More** — e.g., host of each VM, **energy consumption**	**Some advanced networking** — e.g., **virtual networks across site boundaries**
Tightly coupled	Clouds **normally governed by the same administrator** (same corporation)	**Advanced** — e.g., decide **exact placement** of remote VMs	**All** available monitoring info	**Cross-site networks, cross-site VM migration, high-availability techniques, virtual storage across sites**

Federation architectures (classic matching question)

Architecture	Description	Coupling
Cloud bursting / hybrid	**Combines existing on-premise infrastructure (usually a private cloud) with remote resources from one or more public clouds to provide extra capacity for peak demand** (don't size the private cloud for the peak); local cloud OS has **no advanced control** over remote virtual resources beyond basic operations; **supported by most existing cloud managers**	**Loosely coupled**
Broker	A **broker serving various users has access to several public clouds**; a simple broker deploys resources in the cloud **selected by the user**; an **advanced broker** offers **service management — scheduling decisions based on cost, performance, energy** — and may **distribute service components across clouds**; **the most common federation scenario** (easy to implement/maintain, preserves providers' privacy)	**Loosely coupled** (public clouds don't allow advanced control)
Aggregated	**Two or more partner clouds interoperate to aggregate their resources and present users with a larger virtual infrastructure**; partners get some advanced control over remote resources; coupling is higher when partners belong to the **same corporation** than when **different companies agree to cooperate**	**Partially coupled**
Multi-tier	**Two or more cloud sites, each running its own cloud OS, usually belonging to the same corporation, managed by a third (root) cloud OS instance in a hierarchical arrangement**; the root has **full control** over the sites' resources and **exposes them as if located in a single cloud**; good for corporations with **geographically distributed** infrastructure (uniform access) and for **high availability, load balancing, fault tolerance**	**Tightly coupled**

MCQ traps (9.4)

- **Bursting & broker = loosely coupled; aggregated = partially coupled; multi-tier = tightly coupled.**
- **Cross-site VM migration** needs **tight** coupling; loosely coupled federations usually can't migrate VMs.
- Partially coupled ⇒ **contract/framework agreement**, affinity rules, energy info.
- Broker is the **most common** federation scenario; multi-tier is **hierarchical** under a root cloud OS.
- Federation members should be **geographically separated** (power/network/seismic zones).

9.5 VM migration — basics and strategies (L45)

What and why

- **VM migration: the process of moving running applications or VMs from one physical host/server to another.** What moves: **process (CPU) state, memory, storage, network connections** (and active I/O).
- **Why migrate:** | Reason | Explanation | |—|—| | **Load balancing** | Allocation/de-allocation leaves some servers overloaded; distribute VM load fairly | | **Maintenance** | Server needs maintenance → move its VMs **transparently** (customer only cares about SLA) | | **Power management / operational parameters** | **Consolidate VMs on fewer servers**; put underutilized servers into **low-power mode** | | **QoS violation** | If the provider fails to meet QoS, a **user can migrate the VM to another provider** (inter-provider) | | **Fault tolerance** | On (anticipated) failure, migrate VMs to another data center (proactive fault tolerance) | | **Resource sharing** (L46) | Relocate VMs from overloaded to under-loaded servers to relieve memory/cache/CPU contention |

Cold vs live migration

	Cold / non-live migration	**Hot / live migration**
During migration	VM on the source is **turned off or suspended**	VM **continues to provide service — not suspended**
User impact	**Downtime** (may incur penalties/credits); for large VMs downtime can be **very high**; **severe degradation** for real-time apps	Nearly **seamless** — user practically unaware
Complexity	Simple, easy to implement	**Tricky**; needs a defined process

- Live migration moves the **entire VM — all user processes and kernel state — without shutting it down**.
- **Two main live-migration approaches: pre-copy and post-copy.**

Costs and goals

- **Costs: communication overhead**, **migration time**, **downtime** → may **violate SLAs** (penalty) and hurt **reputation**.
- **Downtime = total time during which services remain unavailable to the user.** → **minimize.**
- **(Total) migration time = total time taken to move a VM from source to destination host** (whole process). → **minimize** (migration is overhead).
- Migration should **not disrupt active services through resource contention** (CPU, network bandwidth) at source/destination.

What must be migrated

Item	Notes
CPU context of the VM	Small; transferred during stop phase
Main memory contents	**The big one** — dominates time
Disk	If on **NAS** accessible from both hosts → just reconnect; local disk can be mirrored; disk data migration may be non-critical
Network	If both hosts on the **same LAN**: migrate the IP, **advertise the new MAC↔IP mapping via ARP reply**, switches learn the new MAC location; packets redirected with **transient losses**; otherwise proxying etc.
I/O devices	**Virtual I/O devices are easier** to migrate; **direct assignment of physical devices** to VMs is **hard** to migrate

Memory migration phases

- **Push:** source VM keeps running while pages are pushed to the destination; **pages modified during the process are re-sent** (consistency).
- **Stop-and-copy:** source VM **stopped**, pages copied, new VM started.
- **Pull:** new VM runs; if it accesses a page not yet copied → **page fault** → page **pulled** across the network from the source.
- **Pure stop-and-copy:** simple, but **both downtime and total migration time ∝ VM memory size** → unacceptable outage for live services.

Pre-copy live migration (iterative)

1. **(Iterative) pre-copy phase:** VM keeps running at the source; **all pages are copied in round 1**; in later rounds, pages **dirtied** during the previous round are **re-sent**.
2. **Pre-copy termination** — stop iterating when **any** threshold is hit:
   - **number of rounds exceeds a threshold** (N_{max});
   - **total memory transmitted exceeds a threshold**;

    - **number of dirty pages in the previous round drops below a threshold**.
3. **Stop-and-copy phase:** VM **suspended at the source**; **remaining dirty pages and CPU state** copied to the destination.
4. **Restart/resume phase:** VM restarted at the destination.

- **Downtime ≈ stop-and-copy + restart** (small if few dirty pages remain).

Post-copy live migration

1. **Stop phase: stop the source VM and copy the CPU state** to the destination.
2. **Restart** the destination VM (its memory is **empty**).
3. **On-demand copying:** when the VM touches a page not yet copied, it is **fetched from the source** (demand paging; also **active push** and **pre-paging**). Many pages may never be used, so only what's needed is copied.

- Each page is transferred **at most once** (contrast with pre-copy where hot pages are re-sent).

MCQ traps (9.5)

- **Cold** = VM off/suspended during migration; **live** = keeps running.
- **Pre-copy**: memory first (iteratively), CPU state last; **post-copy**: **CPU state first**, memory on demand.
- Pre-copy stops on **rounds / total data / dirty pages below threshold**.
- Pure stop-and-copy: downtime **and** migration time both $\propto$ memory size.
- Moving the IP on the same LAN uses an **ARP reply** to announce the new MAC mapping.
- Downtime $\neq$ migration time (in live migration downtime $\ll$ migration time).

Week 9 — High-yield takeaways

1. Fog (Cisco) complements cloud; edge = sensors; two-way flow; gains in latency, bandwidth, cost; challenges: interoperability, timing, management.
2. Health case: Levels 0 cloud – 1 ISP – 2 area gateway – 3 mobile – 4 sensors; fog placement puts filtering/processing/event handling at the gateway, **confirmatory module in cloud**; fog gives lower network usage, lower cloud cost, stable latency; tools CloudSim & iFogSim (IIT KGP + Melbourne); prototype: Raspberry Pi fog + AWS/OpenStack.
3. Fog environment model: client, fog, cloud; fog forwards to cloud when short of resources.
4. RM taxonomy: Architecture (data flow, control, tenancy), Infrastructure (hardware, system software, middleware), Algorithms (discovery, benchmarking, load balancing, placement).
5. Service placement: considerations (location, latency, bandwidth, scalability, mobility); constraints (resource, network, application); offloading (device→edge: partitioning, caching; cloud→edge:

replication/partitioning, caching); control (centralized: solver/graph matching; distributed: blockchain/game theory/GA).

6. Federation: motivations; characteristics (voluntary, geographic separation, agreements); coupling (loose/partial/tight); architectures bursting (loose), broker (loose), aggregated (partial), multi-tier (tight).
7. VM migration: reasons (load balancing, maintenance, power, QoS, fault tolerance); cold vs live; pre-copy (iterative, thresholds, stop-and-copy) vs post-copy (CPU first, demand paging); downtime vs migration time.

Week 10 — VM Migration Analysis, Containers & Kubernetes, Docker Architecture and Demo (Lectures 46–50)

> **Source:** L46 VM Migration – analysis and strategies · L47 Containers, Container-based Virtualization, Kubernetes, Docker Container · L48 Docker Container – Overview, Components, Architecture · L49 Docker Container – Demo (install) · L50 Docker Container – Demo (MySQL + phpMyAdmin, ship to Windows)
>
> **Numerical alert:** the **pre-copy migration-time and downtime formulas** (§10.1) are prime numerical-MCQ material. Basics of migration are in Week 9 (§9.5); the first Docker introduction is in Week 8 (§8.1).

10.1 VM migration — analysis and multi-VM strategies (L46)

Why migrate (recap, L46 list)

Load balancing (unbalanced hosts, impending downtime) · **proactive fault tolerance** (anticipate failure, guarantee critical availability/reliability) · **power management** (move VMs off lightly loaded servers, put those servers in **sleep/off** mode) · **resource sharing** (relieve memory/cache/CPU contention by moving VMs from overloaded to under-loaded servers) · **system maintenance** (upgrades without long provider downtime, which hurts credibility and revenue).

Pre-copy recap and who uses it

1. **Iterative push:** all pages sent while the VM runs; pages **dirtied** (modified) during a round are re-sent in later rounds → frequently written pages are sent many times → **long migration time**.
2. **Termination** when any holds: iterations exceed a limit (N_{max}) · total memory sent exceeds a threshold · dirty pages in the last round fall below a threshold.
3. **Stop-and-copy:** suspend the VM at the source; send **CPU state + remaining dirty pages**.
4. **Commit/resume:** the **hypervisor resumes the VM at the destination**.

- **KVM, Xen and VMware use the pre-copy technique for live migration.**
- **Flow:** start → **select destination server** → **reserve resources** at destination → treat whole memory as dirty and copy → iteratively copy dirty pages → (threshold met) **suspend VM, transfer VM state (CPU registers, remaining memory)** → **resume at destination** → commit.
- **Post-copy flow:** start → select destination → reserve resources → **capture VM state (CPU registers, I/O state)** → transfer **minimal state** → **resume VM at destination** → **active push** of memory pages; on a **page fault**, copy the faulted page from the source → done when all pages are transferred. Techniques: **demand paging, active push, pre-paging** (prefetching).

Notation (lecture)

Symbol	Meaning
T_{mig}	Total migration time
T_{down}	Total downtime
V_m	**Memory size of the VM**
R	**Transmission rate** (assumed **constant** during migration)
P	Page size
D	**Dirty page rate** (pages dirtied per second)
$r = \frac{P \cdot D}{R}$	**Dirtying ratio** — memory-dirtying rate ÷ transmission rate
n	Number of pre-copy iterations (rounds after round 0)
V_{th}	Threshold of remaining (dirty) memory at which iterations stop
N_{max}	Maximum allowed number of iterations
T_{res}	**Time to restart (resume) the VM at the destination**

Non-live (cold) migration

$$T_{mig} = \frac{V_m}{R}, \qquad T_{down} \approx T_{mig}$$

(Memory transfer dominates; CPU-state transfer is small. The VM is down for the whole transfer.)

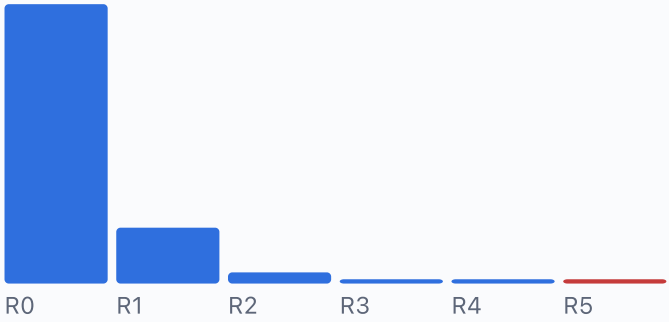
Pre-copy live migration — derivation result

Pre-copy live migration: rounds, migration time, downtime calculator + animation

VM memory V_m (MB)	Transmission rate R (MB/s)	Dirtying rate P·D (MB/s)	Stop threshold V_{th} (MB)
1024	**100**	**20**	**1**

N_{max} rounds	Resume time T_{res} (s)
30	**0.1**

data sent per round (MB) — last (red) = stop-and-copy → downtime



Dirtying ratio r = P·D / R	**0.2**
Rounds n (after round 0)	**5**
Total migration time T_{mig}	**12.899 s**
Downtime $T_{down} = r^n \cdot V_m/R + T_{res}$	**0.103 s**
Cold migration ($T_{mig} = T_{down} = V_m/R$)	**10.24 s**

Worked example with the default values (interactive in the HTML version).

- Round 0 sends the whole memory: $T_0 = \frac{V_m}{R}$.
- During round 0, PDT_0 bytes get dirty → round 1 takes $T_1 = \frac{PD}{R} T_0 = r\,T_0$; in general $T_j = r^j \frac{V_m}{R}$.
- Round n (the stop-and-copy round) takes $T_n = r^n \frac{V_m}{R}$.

$$\boxed{T_{mig} = \sum_{j=0}^{n} T_j + T_{res} = \frac{V_m}{R} \cdot \frac{1 - r^{n+1}}{1 - r} + T_{res}}$$

$$\boxed{T_{down} = r^n \frac{V_m}{R} + T_{res}}$$

Number of rounds: iterate until the remaining dirty memory $r^n V_m \le V_{th}$, capped at N_{max}:

$$n = \min\left(\left\lceil \log_r \frac{V_{th}}{V_m} \right\rceil ,\ N_{max} \right)$$

Interpretation and parameter effects

- Needs $r < 1$ (dirtying slower than the network) for the geometric series to shrink; if $r \geq 1$ the rounds never shrink → stop at N_{max} (long downtime) — post-copy or throttling helps.
- **↑ R (bandwidth)** → ↓ both T_{mig} and T_{down}. **↑ dirty rate** → ↑ r → more rounds, longer T_{mig}, possibly larger T_{down}. **↑** V_m → both scale up proportionally.
- **Downtime ≪ migration time** in live migration (only the last small round + restart); in cold migration they are equal.
- Assumptions: constant R, constant dirtying rate, thresholds as given; I/O and network re-mapping ignored (they make it more complex). Why compute it: to **plan maintenance windows** and estimate how long fault-triggered migration takes.

Worked example: $V_m = 1024$ MB, $R = 100$ MB/s, dirtying rate $P \cdot D = 20$ MB/s ⇒ $r = 0.2$; $T_{res} = 0.1$ s.

- **Cold:** $T_{mig} = T_{down} = 1024/100 = 10.24$ s.
- **Pre-copy with** $n = 3$**:** $T_{mig} = 10.24 \times \dfrac{1 - 0.2^4}{0.8} + 0.1 = 10.24 \times 1.248 + 0.1 \approx 12.88$ s; $T_{down} = 0.2^3 \times 10.24 + 0.1 = 0.082 + 0.1 \approx 0.18$ s.
- **Rounds for** $V_{th} = 1$ **MB:** need $0.2^n \leq 1/1024 \Rightarrow n \geq \ln(1024)/\ln(5) = 6.93/1.61 \approx 4.31 \Rightarrow n = 5$ (if $N_{max} \geq 5$).
- **Trade-off:** live migration takes **longer in total** (12.9 s vs 10.2 s) but cuts **downtime** from 10.2 s to ~0.2 s.

Migrating multiple VMs (m VMs of equal size, one source → one destination)

	Serial migration	**Parallel migration**
How	**First VM** runs its **pre-copy cycle at the full rate** R while the other $m - 1$ VMs keep serving	**All** m **VMs start pre-copy simultaneously**; each gets about R/m of the bandwidth
Stop phase	When the first VM enters stop-and-copy, the **remaining** $m - 1$ **VMs are suspended** (so they stop dirtying memory) and **copied** after it	Equal sizes and rates ⇒ all VMs **enter and finish stop-and-copy at about the same time**
Downtime	**Stop-and-copy of the first VM + migration time of the other** $m - 1$ **VMs + time to resume the VMs**	**Stop-and-copy time of any one VM (at rate** R/m**) + resume time**

- Multiple-VM migration is typical for **maintenance** or **fault recovery** (a whole host is evacuated); single-VM migration is typical when one VM needs more resources than its host has.

MCQ traps (10.1)

- $r = PD/R$; **migration time is a geometric sum**, **downtime is only the last term** (+ restart).
- **Cold:** downtime = migration time = V_m/R.
- Pre-copy converges only if $r < 1$.
- **KVM, Xen, VMware** use **pre-copy**.
- Serial: others are **suspended** when the first enters stop-and-copy (to stop dirtying). Parallel: each VM gets R/m.

- Live migration **increases total migration time** but **reduces downtime**.

10.2 Containers and Kubernetes (L47)

Container — core concept

- Shipping-container analogy: a **self-contained box** of standard size/shape; carriers don't care what's inside → **large-scale interoperability**.
- **A container is a lightweight virtualization technique.** It **packages the code and all its dependencies so the application runs quickly and reliably from one computing environment to another** (Linux → Windows → Mac → cloud).
- **Containers are lightweight packages of application code together with dependencies such as specific versions of programming-language runtimes and libraries required to run the software services.**
- They **share CPU, memory, storage and network resources at the OS level** and offer a **logical packaging mechanism in which applications are abstracted from the environment in which they actually run** → **decoupling** → deploy **easily and consistently** on laptop, data center, or public cloud.
- Supports **agile development** (no worry about environment dependencies), **efficient operations** (use just the resources needed), and **run anywhere**.

Three benefits (list question)

Benefit	Meaning
Separation of responsibilities	**Developers focus on application logic and dependencies; IT operations teams focus on deployment and management**, not application details
Workload portability	Run **virtually anywhere** — Linux, Windows, Mac; VMs or physical machines; developer machines, on-premise data centers, public clouds
Application isolation	Containers **virtualize CPU, memory, storage and network at the OS level**, giving developers a view of the OS **logically isolated** from other applications

Evolution of deployment (classic three-picture question)

Era	Stack	Properties
Traditional deployment	Hardware → OS → apps	**No way to define resource boundaries** for apps on a physical server → **resource-allocation issues** (one app starves others)
Virtualized deployment	Hardware → OS → **hypervisor** → VMs (each with **guest OS**) → apps	Apps **isolated between VMs**; **better utilization and scalability**; **each VM is a full machine** with its own OS
Container deployment	Hardware → OS → **container runtime** → containers	Like VMs but with **relaxed isolation properties to share the OS among applications ⇒ lightweight**; each container has its **own file system, share of CPU, memory, process space**; **decoupled from infrastructure ⇒ portable across clouds and OS distributions**

Containers vs VMs

- Both package apps with libraries/dependencies in isolated environments ("containers **and** VMs", not versus).
- **Containers are far more lightweight.**
- **Containers virtualize at the OS level; VMs virtualize at the hardware level.**
- **Containers share the host OS kernel and use a fraction of the memory VMs require.**

Popular open-source implementations

- **Docker:** an **open platform for developing, shipping and running applications**.
- **Kubernetes:** an **open-source system for automating deployment, scaling and management of containerized applications**; **originated at Google**, open-sourced around **2014**.

Kubernetes (K8s)

- **Definition:** *a **portable, extensible, open-source platform for managing containerized workloads and services**, that facilitates both **declarative configuration and automation**.* Large, rapidly growing ecosystem; services, support and tools widely available.
- Name from **Greek: "helmsman" or "pilot"**.
- **Operates at the container level rather than the hardware level**; provides **PaaS-like features** — **deployment, scaling, load balancing** — and lets users integrate **logging, monitoring and alerting**.
- **Not monolithic:** default solutions are **optional and pluggable**; provides **building blocks for developer platforms** while **preserving user choice and flexibility**.

Kubernetes components | Component | Role | |—|—| | **Cluster** | A set of **worker machines called nodes** that run containerized applications | | **Worker node(s)** | **Every cluster has at least one worker node**; nodes **host the Pods** | | **Pod** | The **components of the application workload** (run on worker nodes) | | **Control plane** | **Manages the worker nodes and the Pods** in the cluster; in production it

runs across multiple computers and the cluster runs **multiple nodes** → **fault tolerance and high availability** | | Other parts named | **Scheduler** (control plane), **kubelet**, **kube-proxy** (on nodes) |

Docker container (L47 summary)

- Docker gives a **sandboxed process isolated from all other processes** on the host.
- **A container is a runtime instance of an image**; create/start/stop/move/delete via the **Docker API or CLI**; runs on local machines, VMs, clouds.
- **A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.**
- **Images become containers at runtime — Docker images become containers when they run on the Docker Engine.**
- Available for **Linux and Windows**; containerized software **always runs the same regardless of infrastructure**; containers **isolate software from its environment** so it works uniformly despite differences (e.g., dev vs staging).
- Docker claims: **standard** (industry standard, portable), **lightweight** (share the machine's OS kernel), **secure** (strong default isolation).

MCQ traps (10.2)

- **Containers virtualize the OS; VMs virtualize hardware.**
- Containers share the **host kernel** (relaxed isolation); VMs each have a guest OS.
- **Kubernetes = orchestration** (deployment, scaling, management of containers); **Docker = build/ship/run containers**.
- A K8s cluster has **at least one worker node**; **Pods run on worker nodes**; the **control plane manages nodes and Pods**.
- Kubernetes is **not monolithic**; components are **pluggable**.
- Kubernetes originated at **Google**.

10.3 Docker — overview, benefits, architecture (L48)

Docker as a platform

- **A platform that allows you to build, ship and run any app, anywhere** — a standard way to solve one of the most challenging aspects of software: **deployment** (also development and testing).
- **Traditional pipeline:** VMs, **configuration-management tools**, **package management**, **complex webs of libraries and dependencies**, each maintained by **specialists** with their own approaches → poor interoperability (moving from Ubuntu to Red Hat/CentOS needs re-engineering) — a big hindrance for **large-scale deployment** where the target site lacks expertise.

- **With Docker: everything goes through a common pipeline to a single output that can be used on any target** — no need to maintain an array of configuration mechanisms. **No need to throw away an existing (tested/certified) software stack** — package it in a container as-is for others; you can still **inspect how containers are built**.
- Name analogy: a **docker** is a **laborer who moves commercial goods in and out of ships at ports** — fitting odd-shaped items efficiently; software teams spend similar effort fitting "odd-shaped" software into different environments.
- Reference book: ***Docker in Practice*** — **Ian Miell and Aidan Hobson Sayers**.

Before vs with Docker

Before Docker	With Docker
Install, configure and maintain the complex application separately on the **development machine, test server and live server** (three efforts); even VMs required time to deploy, boot and manage overhead resources	**Install, configure and maintain the application once in a Docker image**, then `docker run` it on the dev laptop, test server and live server

- **Configuration effort is separated from resource management; deployment becomes trivial** — the environment image is pulled and run, consuming fewer resources and **contained** so it doesn't interfere with other environments. It **doesn't matter whether the target runs Red Hat, Ubuntu or CentOS, as long as Docker runs on it**.

Key uses/features of Docker (list question)

Feature	Meaning
Replacing virtual machines	When you **care only about the application, not the OS**; **quicker to start**, **lightweight to move**; **layered file system** makes sharing changes easy; command-line driven and **scriptable**
Prototyping software	A **sandbox environment almost instantly**, without disturbing your setup or provisioning a VM
Packaging software	A Docker image has **effectively no dependencies** → a great way to package software that runs on any modern Linux machine
Enabling a microservices architecture	**Decompose a complex system into a series of composable parts** — manageable, pluggable
Modelling networks	Spin up **hundreds (even thousands) of isolated containers on one machine**
Full-stack productivity when offline	Bundle all parts of the system into containers and run them on a laptop on the move
Reducing debugging overhead	State the **steps to reproduce a bug** clearly (even as a script) on a system with known properties
Documenting software dependencies and touchpoints	Building images in a structured way **forces you to document dependencies explicitly from the base**
Enabling continuous delivery (CD)	**CD = a paradigm of software delivery based on a pipeline that rebuilds the system on every change and delivers it to production (live) through an automated or partly automated process**; Docker builds are **more reproducible and replicable** than traditional builds

Key concepts: image, layer, container

- **Image:** a **collection of filesystem layers plus some metadata**; taken together they can be spun up as containers. (Stored on disk.)
- **Layer: a collection of changes to files** — e.g., the differences between **v1 and v2** of an application are stored in a layer (on top of, e.g., a Debian base layer).
- **Container: a running instance of an image**; **multiple containers can run from the same image**.
- Images are found via a **registry/namespace**.

Key commands

Command	Purpose
`docker build`	**Build a Docker image** (from a Dockerfile)
`docker run`	**Run a Docker image as a container**
`docker commit`	**Commit a Docker container as an image**
`docker tag`	**Tag a Docker image**

Docker architecture

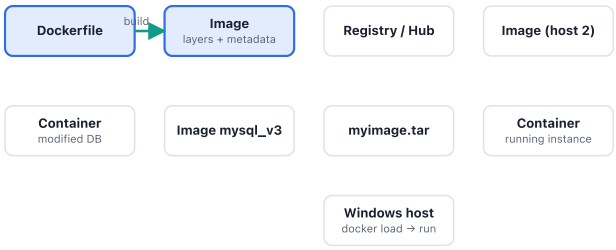
- Docker on a host is split into **two parts: a daemon with a RESTful API, and a client that talks to the daemon**.
- You invoke the **Docker client** to get information from or give instructions to the daemon; the **daemon is a server** that receives requests and returns responses **over HTTP**; it in turn **requests other services (registries) over HTTP** to send and receive images.
- The **daemon takes care of images and containers** behind the scenes; the **client is the intermediary** between you and the RESTful API.
- **Private Docker registry:** a service that **stores Docker images**, **on the internal network** and **not publicly accessible**; any Docker daemon with access can request images from it.
- **Docker Hub:** the **public registry run by Docker Inc.**; **other public registries** can also exist.
- The host machine typically sits on a **private network**.

MCQ traps (10.3)

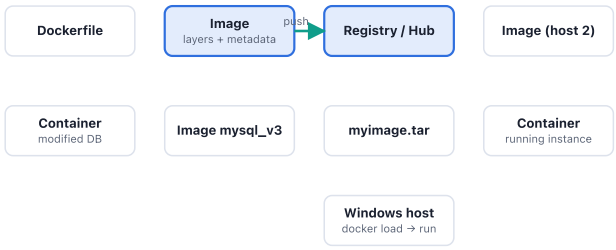
- **Docker architecture = client + daemon (RESTful API, HTTP)**; the **daemon** (not the client) manages images and containers.
- **Docker Hub = public** (Docker Inc.); **private registry = internal network, not public**.
- `docker commit` turns a **container into an image**; `docker build` builds an image from a **Dockerfile**.
- Image = **layers + metadata**; layer = **file changes** (e.g., v1→v2 diff).
- CD = rebuild on **every change** and deliver to production automatically/partly automatically.

10.4 Docker demo — install, build, ship, run (L49–L50)

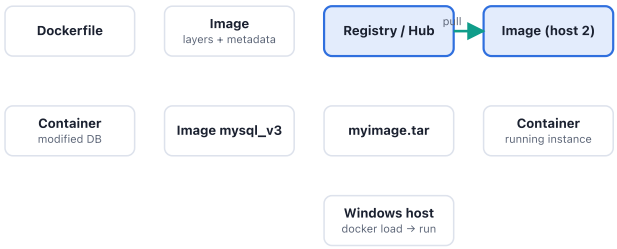
Docker: build → ship → run (and the MySQL demo) diagram sequence



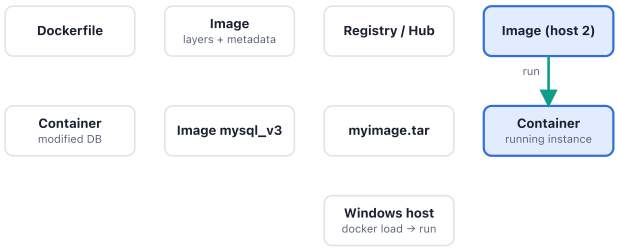
1. `docker build` : a **Dockerfile** (build script, versioned in Git) produces an **image** = file-system layers + metadata.



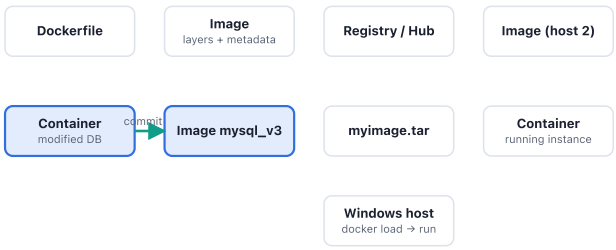
2. Push the image to a **registry** — **Docker Hub** (public, Docker Inc.) or a **private registry** on the internal network.



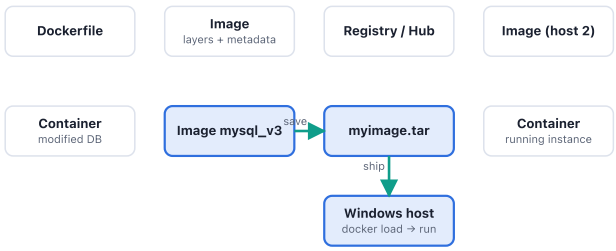
3. Any host with the Docker Engine **pulls** it (`docker run` auto-pulls if missing).



4. `docker run` creates a **container** — a runtime instance sharing the host kernel.



5. Demo (L50): after adding DB records, `docker commit` saves the running container as image **mysql_v3**.



6. `docker save` → tar, ship it (e.g., Google Drive), then `docker load` + `docker run` on Windows: data intact, no MySQL/PHP/Apache install.

Goal

Show “build once, run anywhere”: pull **MySQL** (relational DB) and **phpMyAdmin** (web-based GUI front end for MySQL) images from **Docker Hub** on **Ubuntu**, modify the database, package it, **ship it to a Windows machine** and run it **without installing MySQL, Apache, PHP or phpMyAdmin there**.

Traditional way: install MySQL, a web server (Apache), PHP and phpMyAdmin **separately on every machine**; to move data, **back up the old MySQL, transfer, restore** on the new server — needs expertise. **Docker way:** import MySQL and phpMyAdmin images from Docker Hub → bundle → **build a Docker image at the source that includes everything** (app + dependencies) → **load the image at the**

target and run — no separate installation at the destination; only the **Docker Engine** must be installed there.

Steps (sequence)

1. **Install Docker Engine on Ubuntu** (per docs.docker.com): `apt-get update` → install packages so **apt can use a repository over HTTPS** → **add Docker's official GPG key** → set up the repository → `apt-get update` → install Docker Engine.
2. **Verify:** `sudo docker run hello-world` — **`docker run` pulls the image from Docker Hub if it is not available locally**, then runs it; prints **"Hello from Docker!"** ⇒ installation OK.
3. **Pull images:** `docker pull mysql`, `docker pull phpmyadmin`; check with `docker images`.
4. **Write a Dockerfile** on top of the MySQL image that **copies `/var/lib/mysql` to a custom directory and sets it as the data directory** — so the **data persists** in images made from the container; **`docker build`** the new image.
5. **Run containers:** `docker run --name <mysql-name> -e <root-password env var> -d <image>` (first attempt failed until the **environment variable** was supplied); `docker ps` shows running containers. Run phpMyAdmin with **`--link <mysql-container>:db`** and **port forwarding `-p 8080:80`**; open **localhost:8080**.
6. In phpMyAdmin: create database → table (course id, course name, credit) → insert records (1001 Cloud Computing 3; 1002 Networking 4).
7. **`docker commit <container-id> mysql_v3`** — saves the running container's state (with the new DB/table/records) as a **new image**.
8. **`docker save mysql_v3 > myimage.tar`** — export the image as a **tar archive**.
9. **Ship** the tar (Google Drive in the demo; scp/FTP also fine).
10. On **Windows** (with **Docker Desktop** GUI showing images and containers): **`docker load`** the tar → image `mysql_v3` appears → `docker run --name mysql1 -d mysql_v3`.
11. Run phpMyAdmin **pulled fresh from Docker Hub** (unchanged, so not shipped) linked to `mysql1:db`, port 8080 → the **same database, table and records are present**. State preserved; **no separate MySQL/PHP/Apache installed**.

MCQ traps (10.4)

- **`docker save`** → **tar file;** **`docker load`** → **image from tar** (not `export` / `import` in this demo).
- **`docker commit`** captured the modified database into a new image.
- Only the **modified** MySQL image was shipped; phpMyAdmin was **pulled from Docker Hub** at the destination.
- `docker run` **auto-pulls** missing images from Docker Hub.
- Destination needs only the **Docker Engine** (Docker Desktop on Windows).

Week 10 — High-yield takeaways

1. $r = PD/R$**;** $T_{mig} = \frac{V_m}{R}\frac{1-r^{n+1}}{1-r} + T_{res}$**;** $T_{down} = r^n \frac{V_m}{R} + T_{res}$**;** $n = \min(\lceil \log_r(V_{th}/V_m) \rceil, N_{max})$**; cold:** $T_{down} = T_{mig} = V_m/R$**.**
2. Pre-copy used by KVM, Xen, VMware; post-copy = CPU state first + demand paging/active push/pre-paging.
3. Multi-VM: serial (first VM at full R, others suspended at its stop-and-copy) vs parallel (each R/m, simultaneous stop-and-copy).
4. Containers: lightweight, OS-level virtualization, share kernel; benefits = separation of responsibilities, workload portability, application isolation; traditional → virtualized → container deployment.
5. Kubernetes (Google, Greek "helmsman"): portable, extensible, open-source, declarative config + automation; cluster → worker nodes → Pods; control plane; not monolithic.
6. Docker: build/ship/run anywhere; uses (replace VMs, prototyping, packaging, microservices, network modelling, offline full stack, debugging, documenting dependencies, CD); image = layers + metadata; container = running instance; build/run/commit/tag; architecture = client + daemon (REST over HTTP) + private registry + Docker Hub.
7. Demo: install engine → hello-world → pull MySQL/phpMyAdmin → Dockerfile (persistent datadir) → run with `--link` and `-p 8080:80` → commit → save (tar) → ship → load → run on Windows; data intact.

Week 11 — Dew Computing, Serverless Computing I–II, Sustainable Cloud Computing I–II (Lectures 51–55)

> **Source:** L51 Dew Computing · L52 Serverless Computing – I · L53 Serverless Computing – II · L54 Sustainable Cloud Computing – I · L55 Sustainable Cloud Computing – II
>
> High-yield: **dew's two features (independence, collaboration) and its service categories (IaD, SiD, PiD, storage-in-dew, WiD, DBiD), DVM; serverless definition, FaaS vs BaaS, drawbacks, AWS Lambda / Google Cloud Functions / Azure Functions terms; sustainable cloud conceptual model (cloud architecture, cooling manager, power manager, remote CDCs), energy-vs-reliability trade-off, DVFS, taxonomy components and energy breakdown.**

11.1 Dew computing (L51)

Motivation

- Cloud (and fog/edge, cloudlet, MCC, MEC…) all depend on **reliable, constant Internet connectivity**; if it's lost, work stops.
- **Dropbox** shows the alternative: a **local copy** of your folder is usable **offline** and **syncs with the cloud** when the connection returns. Dropbox is the classic **example of dew computing**.

Definitions

- **Dew computing: a computing paradigm that combines the core concept of cloud computing with the capabilities of end devices (personal computers, mobile phones…), used to enhance the end-user experience compared with using only cloud computing.**
- It **attempts to solve cloud's major problem: reliance on (constant) network access** — services keep working when connectivity is absent.
- Alternative definition: *a computing model enabling a ubiquitous, pervasive and convenient, ready-to-go, plug-in-facility-empowered personal network that includes a single-hybrid peer-to-peer communication link*; primary goal: **access a pool of raw data equipped with metadata that can be rapidly created, edited, stored and deleted with minimal Internet management effort** (i.e., also offline).
- Characteristics: **user-centric, highly flexible, personalized**; located **very close to end devices** — the **first layer of the IoT–cloud continuum** that the user interacts with; **micro-service-based**, in a **vertically distributed hierarchy**; made of smart devices (smartphones, smart watches, tablets, laptops) at the network edge that collect/process IoT data; services are **relatively always available** and **permanent interaction (connectivity) is not mandatory**; **does not rely on centralized servers/cloud data centers or permanent network connectivity** (but syncs with the cloud).

- Bonus: you can **save energy/cost** by syncing only when a favourable (high-bandwidth) connection is available.

Two key features (very examinable)

Feature	Meaning
Independence	**The local device must be able to provide service without a continuous connection to the Internet**
Collaboration	**The application must be able to connect to the cloud service and synchronize data when appropriate** (automatically; role/rule-based sharing possible)

Analogy: cloud is far above the ground (remote service), **fog** is closer to the ground (beside the user), **dew** is **on the ground** (at the user's own system/premises).

Dew service categories (matching question)

Category	Meaning	Example
Infrastructure-as-Dew (IaD)	**Local device dynamically supported by cloud services** — either an **exact duplicate DVM instance in the cloud kept in the same state**, or all local **settings and data (system + each app) saved in the cloud**	iCloud
Software-in-Dew	**Configuration and ownership of software are saved in the cloud** — apps a user installs are saved to the account and can be installed on any linked device	**Google Play, Apple App Store**
Platform-in-Dew (PiD)	**A software-development suite installed on the local device, with settings and application data synchronized to the cloud** — must sync **development data, system deployment data and online backups**	**GitHub**
Storage-in-Dew	**Local device's storage is partially or fully copied to the cloud**; satisfies **both independence and collaboration** (offline access + automatic folder sync)	**Dropbox, Google Drive**
Web-in-Dew (WiD)	**Local device possesses a duplicated fraction of the WWW** (or a modified copy) → independence; that fraction **syncs with the web** → collaboration	—
Database-in-Dew (DBiD)	**Local device and cloud both store copies of the same database**; one is designated the **main version** by the DBA; **increases reliability** (one acts as the other's backup)	—

> The lecture calls storage-in-dew "SID"; in the literature, *SiD* usually denotes software-in-dew and *STiD* storage-in-dew. Match by the description.

Dew architecture

- **Cloud ⇄ (single-hybrid P2P link, when connected) ⇄ Dew Virtual Machine (DVM) on the local host ⇄ IoT devices/sensors.**

- **DVM = an isolated environment for executing the dew server on the local system** (must be installed, like installing the Dropbox client). It contains a **dew server** (core), a **dew client program**, a **dew DBMS/database**, **dew client services** (and possibly AI modules that learn access patterns).
- **Dew server's basic functions: serve the user with requested services** (through client services) and **perform synchronization and correlation between local and remote data**. It aims at **three goals: data replication, data distribution, synchronization** (plus housekeeping of the local data organization).

Challenges

Power management · processor utility · data storage (local vs subscribed cloud space; hard on low-resource devices like smartwatches) · viability of the operating system (which OSs the DVM runs on) · programming principles · information/database security. Placement in the hierarchy: cloud servers → fog/edge servers → edge devices → smart devices → smart IoT modules; **dew-enabled computing is realized at the lower levels (edge devices, smart devices, IoT modules)**, working from the last synchronized state and syncing upward when connected.

MCQ traps (11.1)

- Dew's two features: **independence + collaboration** (not "scalability + elasticity").
- Dropbox/Google Drive = **storage-in-dew**; App Store/Play = **software-in-dew**; GitHub = **platform-in-dew**; iCloud = **infrastructure-as-dew**.
- DBiD **increases reliability** (backup); one copy is the **main version**.
- Dew does **not** require permanent connectivity or a central server, but it **does sync** with the cloud.
- DVM goals: **replication, distribution, synchronization**.

11.2 Serverless computing — concepts (L52)

Definitions

- **Serverless computing: a method of providing back-end services on an as-used basis**; the provider lets users **write and deploy code without worrying about the underlying infrastructure**.
- It **simplifies code development and deployment and eliminates the need for system administration** — developers focus on **core business logic** without overhead such as **instantiating VMs or containers** or **monitoring infrastructure** (needed even in IaaS/PaaS).
- **Developers express logic as functions** submitted to the provider to run in a **shared runtime environment**; the **provider manages scalability by running multiple functions in parallel**.
- After the container boom, providers offered **better-fitting containers that need less time to boot** and **more automation in orchestration**; **serverless promises full automation in managing fine-grained containers**.

- **Serverless computing is a form of cloud computing that allows users to run event-driven and granular applications without having to address the operational logic.**
- Developers work at **high-level abstractions — functions, queries, events** — and interconnect elements of business logic into workflows; the provider maps them to concrete resources and ensures applications are **orchestrated (containerized), deployed, provisioned and available on demand**, **billing only for resources used**.
- The provider **still has servers** — it **dynamically allocates and provisions** them; code runs in **almost stateless containers** that are **event-triggered** and **may be ephemeral** (may last for one invocation).

Evolution leading to serverless

1960s mainframes (IBM) → 1970s virtualization → 1980s **RPC** → 1990s virtual private servers, CGI → 2000s AWS, Google App Engine, containers → 2010s **Docker (2013), Kubernetes**, event-driven architectures. **Container orchestration + microservices → FaaS**; **FaaS + event-driven workflows → serverless computing.**

FaaS vs BaaS

	Function as a Service (FaaS)	**Backend as a Service (BaaS)**
Definition	**A form of serverless computing in which the cloud provider manages the resource life cycle and event-driven execution of user-provided functions**; users supply **small, stateless functions**	**Replaces server-side components with off-the-shelf services**; developers **outsource all behind-the-scenes aspects** and keep app logic in the **front end**
Examples	AWS Lambda, Google Cloud Functions, Azure Functions; **ExCamera** (edits/transforms/encodes video with low latency and cost — many tasks run **concurrently**)	**Remote authentication, database management, cloud storage, hosting**; **Google Firebase** (fully managed database used directly from apps)

How FaaS works

- Serverless apps are **event-driven, cloud-based**, relying on **third-party services, client-side logic and cloud-hosted RPCs**.
- Deployed code **executes in an isolated environment when triggered**; **each function is a small part of the application**.
- **Execution time is limited** (e.g., **AWS Lambda ≈ 15 minutes**).
- **Functions are not constantly active** — the platform **listens for events** that instantiate them: **client requests, events from external systems, data streams**.
- The FaaS provider **horizontally scales** function execution with the **number of incoming events**.

Drawbacks/challenges (list question)

Issue	Why it hurts
Asynchronous calls	Async calls to/between functions increase complexity (request–response sync calls are easier)
Functions calling other functions	Complex debugging, loss of feature isolation, **extra cost** (pay for two functions running at once)
Shared code between functions	Changing shared code may break dependent functions
Image size limit / warm-up	Risk of hitting the image-size limit (**AWS Lambda ≈ 50 MB**); **bigger image ⇒ longer warm-up (start) time**
Too many libraries	More space, higher risk of hitting size limits
Too many technologies	Many frameworks/languages ⇒ maintenance complexity and higher skill needs
Too many functions	Since **inactive functions cost nothing**, people create new ones instead of updating/reusing ⇒ **lower maintainability and understandability**

Popular platforms: AWS Lambda (Amazon), Google Cloud Functions, Microsoft Azure Functions.

MCQ traps (11.2)

- "Serverless" does **not** mean no servers — servers are hidden and managed by the provider.
- **FaaS = provider runs your event-triggered functions; BaaS = provider supplies ready back-end services** (auth, DB, storage).
- Functions are **stateless, short-lived, event-triggered**, billed **only when executing**.
- Lambda limits quoted: **~15 min** execution, **~50 MB** image.

11.3 Serverless computing — platforms (L53)

Serverless vs cloud focus

- **Serverless hides servers by providing a programming abstraction for application builders** → cloud software easier to write.
- **Cloud computing targeted system administrators; serverless targets programmers.** The provider takes over many **operational responsibilities** needed to run applications (overlaps with PaaS but goes further).
- **Focus shifts from servers to applications** (some literature calls traditional cloud "serverful"). Serverless will change how programmers work **as dramatically as cloud changed how operators work**.
- **Serverless applications need no server provisioning and no server management.**

Provider comparison

	AWS	Google	Microsoft Azure
Virtual servers	AWS (EC2) instances	VM instances	Virtual machines
Functions (FaaS)	**AWS Lambda**	**Google Cloud Functions**	**Azure Functions**
Database	**Amazon DynamoDB**	**Google Cloud Datastore**	**Azure Cosmos DB**
Storage	**Amazon S3**	**Google Cloud Storage**	Azure storage

AWS Lambda

- **An event-driven, serverless computing platform provided by Amazon as part of AWS.** You don't worry about which AWS resources to launch or how to manage them — **just put code on Lambda and it runs**.
- **Code executes in response to events in AWS services**, e.g., **adding/deleting files in an S3 bucket**, an **HTTP request from Amazon API Gateway** (also DynamoDB, SNS, Kinesis…).
- Lambda is used to execute **background tasks**.
- Frees you from managing OS access, patching, right-sizing, provisioning, scaling.
- **Flow:** upload code (Java, Python, Go, C#, Node.js…) → AWS services **trigger** Lambda → Lambda **executes** → **AWS charges only when Lambda executes code**. | Lambda concept | Meaning | |—|—| | **Function** | A program/script that runs in Lambda; Lambda passes **invocation events**, the function processes them and returns a response | | **Runtime** | Lets functions in various languages run on the **same base execution environment** | | **Event source** | An **AWS service (e.g., SNS) or custom service that triggers the function** | | **Lambda layers** | **Distribution mechanism for libraries, custom runtimes and other dependencies** | | **Log streams** | Annotate code with **custom logging statements** to analyse execution flow and performance |

Google Cloud Functions

- **A serverless execution environment for building and connecting cloud services**; write **simple, single-purpose functions attached to events emitted from cloud infrastructure and services**; a function is **triggered when an event being watched is fired**; runs in a **fully managed environment**.
- **Flow:** GCP services (Cloud Storage, Pub/Sub, Stackdriver, Datastore…) **emit events** (e.g., object uploaded/deleted/metadata updated in a bucket) → **event data** (metadata) → if a function is configured with that **trigger**, it is **invoked** with the event data → it may **call other APIs (Google or external)** and **write back to cloud services** → it signals completion. **Multiple events ⇒ multiple invocations** (handled by the infrastructure).
- Keep functions **single-purpose**, with **minimal execution time**; functions are **stateless** — keep state outside (e.g., memcache). | Term | Meaning | |—|—| | **Events** | Things that happen in GCP services — file uploaded, message published to a queue, direct HTTP invocation | | **Triggers** | Choosing to respond to an event — **event + function** | | **Event data** | Data passed to the function

when the trigger fires | | **Event providers** | **HTTP**, **Cloud Storage**, **Cloud Pub/Sub**, **Firebase** (DB, analytics, auth), **Stackdriver Logging**, **Cloud Firestore**, **Compute Engine**, **BigQuery** |

Azure Functions

- **A serverless solution that lets you write less code, maintain less infrastructure and save on costs**; the provider supplies all up-to-date resources.
- **A function is the primary concept; it has two pieces: your code (many languages) and configuration (`function.json`)**. For **compiled languages** the config is **generated automatically from annotations**; for **scripting languages** you **provide it yourself**.
- Languages: C#, Java, JavaScript, Python, PowerShell…; **custom handlers** for virtually any language. Automated deployment options; monitoring/testing for troubleshooting.
- **Pricing: Consumption plan — pay only while functions run**; Premium and App Service plans for specialized needs.
- **Common serverless patterns:** serverless APIs and mobile/web back ends · event and stream processing · IoT data processing · big-data and ML pipelines · integration and enterprise service bus · automation and digital transformation.
- **Scenarios: web-app back end** — request in web app → **queued in Service Bus** → function processes → output to **Cosmos DB**; **mobile-app back end** (financial service) — HTTP API call from mobile app → processed by function → output to Cosmos DB → data transfer **triggers a second function** → notification via **Notification Hubs**.

MCQ traps (11.3)

- Lambda / Cloud Functions / Azure Functions are the **FaaS** offerings; DynamoDB / Cloud Datastore / Cosmos DB the databases.
- **Lambda layers** = libraries/runtimes/dependencies; **event source** = trigger origin; **log streams** = custom logging.
- Google: **trigger = event + function**; functions are **stateless, single-purpose**.
- Azure: function = **code +** **`function.json`**; config auto-generated for **compiled** languages only.
- Billing is per **execution** (Consumption plan).

11.4 Sustainable cloud computing — I (L54)

Problem

- CSPs (Amazon, Google, IBM, Microsoft…) run large, geographically spread **cloud data centers (CDCs)** for ever-increasing, peaky user demand.
- They must **honour SLAs** (penalties otherwise) **and** keep **energy, maintenance cost and carbon footprint** down to earn a return on investment.

- **The financial and carbon-footprint costs of running such large infrastructure negatively impact the sustainability of cloud services.**
- **Focus of sustainable cloud computing: minimize energy consumption and carbon footprint while ensuring the reliability of CDCs.**
- Putting lightly loaded servers to **sleep** saves energy, but wake-up **delay** can **violate SLAs**, especially with simultaneous peaks.
- Network, storage and servers must be available **round the clock**; **idle components still consume energy**; creating, processing and storing every bit adds energy cost and **heat**, which needs more energy for **cooling**.
- **Projected CDC energy consumption ≈ 8000 TWh by 2030** (lecture graph).
- Components other than servers (**network, storage, memory, cooling**) consume huge energy → need **energy-aware resource management of all resources holistically**; **under-loading and over-loading** make CDC energy use inefficient.
- CSPs turn to **renewable energy** (often **hybrid** with grid); future CDCs must cut **carbon emissions and heat release (greenhouse gases)**.
- **Data-center (re)location** is driven by **waste-heat recovery opportunities, access to green (renewable) resources, and proximity to cooling resources** (short cooling ducts; near power stations to cut transmission losses).

Conceptual model of sustainable cloud computing (layered)

Component	Role
Cloud architecture	**IaaS, PaaS, SaaS** (with PaaS-level managers: remote-CDC manager, VM/resource manager, green resource manager, thermal profiling, cooling manager)
Cooling manager	Issues **thermal alerts** when temperature exceeds a **threshold**; a **heat controller** acts to control temperature with **minimal impact on CDC performance** (temperature control, heat exchange)
Power manager	Controls power from **renewable sources** (solar, wind) and **grid/fossil fuel**; **grid energy used for deadline-oriented workloads** to maintain reliability; **ATS (Automatic Transfer Switch)** manages energy from both sources; **PDU (Power Distribution Unit)** delivers electricity to servers and cooling devices
Remote CDCs	**VMs/workloads migrated to remote CDCs to balance load** (e.g., where renewable energy is available)

Energy vs reliability (trade-off)

Energy goals	Reliability goals
Reduce energy consumption · reduce under/over-loading (better load balancing) · minimize heat concentration and dissipation · reduce carbon footprint · improve bandwidth and computing capacity · improve storage management	**Avoid SLA violations** · identify system failures and their reasons to manage risk · protect critical information from security attacks · **secure VM migration** · reduce **turn-on (switching) time**

- Improving energy use **reduces electricity bills and operational cost** → sustainability. But **reliability via service replication needs extra resources and energy** → **trade-off between**

energy consumption and reliability for cost-efficient services.

- **DVFS (Dynamic Voltage and Frequency Scaling)** reduces energy, but **switching between high and low scaling modes increases response time and service delay** → may violate SLAs.
- **Excessive turning off/on** reduces reliability of components; **power modulation decreases reliability** of server components such as storage devices.
- ⇒ Need **energy-aware resource management that reduces power without affecting reliability**.
- Aspects to address: **application model, virtualization, waste-heat utilization, thermal-aware scheduling, renewable energy, resource-targeted energy management, capacity planning** (detailed in L55).

11.5 Sustainable cloud computing — II: taxonomy (L55)

Why it's getting harder

- IoT and streaming drive exponential demand → more data centers, denser servers → **more heat, more power, more cooling**. Chips shrink while compute grows, but **heat dissipation limits** density. In modern data centers, **IT cost ≈ environment cost** (cooling, power, controls) — "x and x".
- Must treat **computing, cooling, heat, power together** (integrated), including what to do with **waste heat**.

Taxonomy components (survey: Gill & Buyya, "Sustainable Cloud Computing: 360°

view")

Component	Key points
Application model/design	**Data-parallel, function-parallel, message-passing**; QoS, workload type (critical/interactive vs batch), architecture; an efficient application structure improves CDC energy efficiency; need **green-ICT-based** applications
Energy management	Energy share of CDC components (lecture): **processor ≈ 45 %, cooling ≈ 20 %, memory ≈ 15 %, storage ≈ 10 %, network ≈ 10 %**; static (system/CPU level) and dynamic (resource management, consolidation, configuration) techniques; **DVFS** helps utilization but switching adds response time/delay; sleep/wake is risky
Virtualization	**VM migration, VM elasticity, load balancing, consolidation, fault tolerance, scheduling** — move workloads where renewable energy is available
Thermal-aware scheduling	Architecture (single-/multi-core), heat modelling, thermometers, scheduling, monitoring, simulation; tackles heating during workload execution; current approaches mostly aim to **reduce PUE**, but **PUE reduction may not always reduce total cost of ownership**
Cooling management	Efficient cooling to keep CDC temperature in range as size and energy density grow
Renewable energy	Solar, wind, hydro (near-zero CO_2); energy source, **storage device**, **location (on-site/off-site)**; **carbon usage effectiveness (CUE) can be reduced by adding renewable energy**, but can renewables meet **peak demand**? → switching with grid; **place CDCs near renewable sources** (which need large areas)
Waste-heat utilization	Reuse heat generated by the systems; **vapour-absorption-based free cooling** uses waste heat and **can make PUE (near) ideal by neutralizing cooling expenses**
Capacity planning	Plan **power infrastructure, IT resources, cooling, workloads**; **predictive models**; investment is upfront while utilization grows slowly → plan for **ROI**; auto-scaling, utility functions; storage and processing capacity
Sustainability metrics	Performance metrics for **providers** (throughput, application, security) and **users** (satisfaction, response time, correctness, reliability) plus metrics for **waste heat, cooling, thermal-aware scheduling, energy management, virtualization, capacity planning, renewable energy**

Research categories: application design · sustainability metrics · capacity planning · energy management · virtualization · thermal-aware scheduling · cooling management · renewable energy · waste-heat utilization. **Conclusion:** next-generation clouds must be **energy-efficient and sustainable** — powered by **renewables**, placed in **cool** locations near power generation, with lower carbon footprint — while keeping **high performance and reliability**.

MCQ traps (11.4–11.5)

- Sustainable cloud = **minimize energy + carbon footprint while ensuring reliability** (not just cheaper hardware).
- **Grid power** for **deadline-oriented** workloads; **ATS** switches sources; **PDU** distributes power.
- **DVFS** saves energy but **switching increases response time**.

- Two different breakdowns: L37 green cloud (whole data center: **cooling 45 %, IT 40 %, power distribution 15 %**) vs L55 (by component: **processor 45 %, cooling 20 %, memory 15 %, storage 10 %, network 10 %**). Read the question carefully.
- **CUE** is reduced by adding renewables; **vapour-absorption free cooling** reuses waste heat.
- **PUE reduction ≠ guaranteed TCO reduction.**

Week 11 — High-yield takeaways

1. Dew = cloud concept + end-device capability; works offline; **independence + collaboration**; Dropbox example; categories IaD (iCloud), software-in-dew (Play/App Store), PiD (GitHub), storage-in-dew (Dropbox/Drive), WiD, DBiD; DVM with dew server (replication, distribution, synchronization).
2. Serverless = back-end services as-used; event-driven, granular apps; developers write functions; provider handles provisioning/scaling; billing per use; still uses servers.
3. FaaS (provider runs event-triggered stateless functions; Lambda ~15 min) vs BaaS (off-the-shelf back ends: auth, DB, storage — Firebase).
4. Drawbacks: async calls, function chaining cost, shared code, image size (~50 MB) and warm-up, too many libraries/technologies/functions.
5. Lambda (event source, runtime, layers, log streams; S3/API Gateway triggers; pay per execution); Google Cloud Functions (event, trigger = event + function, event data, providers HTTP/Storage/PubSub/Firebase…); Azure Functions (code + function.json; Consumption plan; Service Bus/Cosmos DB scenarios).
6. Sustainable cloud: minimize energy & carbon, keep reliability; model = cloud architecture + cooling manager + power manager (renewable/grid, ATS, PDU) + remote CDCs; energy vs reliability trade-off; DVFS; ~8000 TWh by 2030.
7. Taxonomy: application design, sustainability metrics, capacity planning, energy management (processor 45 %…), virtualization, thermal-aware scheduling, cooling, renewables (CUE), waste-heat utilization (vapour absorption).

Week 12 — Cloud in the 5G Era, CPS & Cloud, Case Studies: Spatial Cloud and Internet of Health Things (Lectures 56–60)

> **Source:** L56 Cloud Computing in 5G Era · L57 CPS and Cloud Computing · L58 Case Study I (Spatial Cloud Computing) · L59 Case Study II (Internet of Health Things) Part A · L60 Case Study II Part B
>
> High-yield: **5G features and its three service classes (eMBB, URLLC, mMTC), two aspects of the 5G–cloud relationship; CPS definition, origin, characteristics, applications, cyber-physical cloud computing (CPCC) definition and benefits; spatial data, Tobler's law, spatiotemporal cloud computing definition, OGC services, trajectory cloud; IoHT case-study topology, latencies, module placement, results, accelerometer pipeline (5-point smoothing, kNN), dew comparison.**

12.1 Cloud computing in the 5G era (L56)

Why 5G matters for cloud

- Cloud→fog→edge→IoT continuum tried to move computing towards the edge because of **latency (round-trip delay)** and **data load** from billions of IoT devices, mostly over **mobile/wireless** links.
- Further cloud challenges: **mobility** (devices/edge, even fog, may move or switch), **real-time applications** (gaming, healthcare, traffic) needing quick responses, **QoS** for different traffic types, **security**.
- Cloud's success depends on **ubiquitous connectivity**; the mobile network is now the de-facto backbone → the **mobile generation matters**.

What 5G is

- **5G = fifth-generation mobile network — a new global wireless standard after 1G–4G**, designed to **connect virtually everyone and everything together** (machines, objects, devices).
- Delivers **multi-Gbps peak data speeds, ultra-low latency, more reliability, massive network capacity, increased availability, and a more uniform user experience to more users**; improved efficiency, new industries.
- **Unified, more capable air interface** with extended capacity for next-generation experiences, new deployment models and services; **designed for forward compatibility**.

Generations (lecture) | Gen | Era | Delivered | Speed (lecture figures) | |—|—|—|—| | **1G** | 1980s | **Analog voice** | ~2 kbps | | **2G** | Early 1990s | **Digital voice** (e.g., CDMA) | ~384 kbps | | **3G** | Early 2000s |

Mobile data | 56 Mbps | | **4G LTE** | 2010s | **Mobile broadband** | 1 Gbps | | **5G** | Now | Unified air interface; connect everything | **10 Gbps**, latency **1 ms** |

Three main classes of 5G connected services

Class	Meaning	Examples
Enhanced Mobile Broadband (eMBB)	Better smartphones plus **new immersive experiences — faster, more uniform data rates, lower latency, lower cost per bit**	**VR, AR**, enhanced indoor/outdoor broadband, enterprise collaboration
Mission-critical communications — URLLC (Ultra-Reliable Low-Latency Communication)	**Ultra-reliable, available, low-latency links** that transform industries	**Remote control of critical infrastructure, vehicles and medical procedures**; **autonomous vehicles, smart grid, remote patient monitoring, telehealth, industrial automation**
Massive IoT — mMTC (massive Machine-Type Communication)	**Seamlessly connect a massive number of embedded sensors**, by **scaling down data rates, power and mobility** — extremely lean, low-cost connectivity	**Asset tracking, smart agriculture, smart cities, energy monitoring, smart home**

- Literature calls this **"network as a platform"**.

5G and cloud computing

- **5G is a perfect companion to cloud** — distribution and diversity of compute and storage. **On-premise and edge data centers close the gap between resource-constrained, low-latency devices and distant cloud centers → heterogeneous, distributed computing**.
- Providers must offer **full end-to-end orchestration**; **service orchestration** lets applications **interact with network resources — select location, QoS, influence traffic routing** — to deliver apps on demand (e.g., classify streaming traffic by QoS).
- **Two key aspects of the 5G–cloud relationship:**
  1. **Further development of cloud computing has to meet 5G needs** — reflected in the **growing role of edge, mobile edge and fog computing** in the cloud paradigm.
  2. **5G is undergoing "cloudification" through softwarization — NFV (network function virtualization), SDN (software-defined networking)** (and serverless/FaaS-style functions).
- Both influence each other; 5G brings convergence of **computing, IoT and cloud** → the **era of hyper-connectivity**.

Edge computing in 5G

- **(Mobile) edge computing enables the evolution of 5G by bringing cloud capabilities near end users (user equipment)**, overcoming cloud's high latency.

- Preferred for **interactive, computation-intensive** next-gen apps like **AR/VR**, which have stringent QoS (**low latency, high throughput**) and generate huge data processed at different hierarchy levels.
- **5G is expected to:** handle **massive data** from mobile devices and IoT · meet **stringent QoS** for highly interactive apps (**ultra-low latency, high throughput**) · support a **heterogeneous environment** (interoperability of diverse equipment, QoS needs, network types).
- **Edge-computing applications:** healthcare · entertainment and multimedia · **VR/AR/MR** · **tactile Internet** · IoT · **factories of the future** · **emergency response** · **Intelligent Transportation Systems (ITS)**.

Mobile cloud computing in 5G

Key MCC features for 5G: **sharing resources for mobile apps**; **improved reliability** (data backed up in the cloud); **fewer device resources consumed** (processing offloaded); compute-intensive processing of mobile requests **offloaded** to the cloud; devices connect via **base stations**. 5G backbone gives **higher data rates, low latency, reliability, QoS handling** → a **win-win** for cloud.

MCQ traps (12.1)

- **eMBB = AR/VR & broadband; URLLC = autonomous vehicles, telesurgery, smart grid, industrial automation; mMTC = massive IoT sensors (smart agriculture, asset tracking).**
- 1G analog voice; 2G digital voice; 3G mobile data; 4G mobile broadband.
- "Cloudification of 5G" = **softwarization via NFV and SDN**.
- Edge computing brings cloud capability **near the user equipment** to cut latency.

12.2 Cyber-Physical Systems and cloud computing (L57)

CPS — definition and origin

- **CPS = an orchestration of computing and physical systems. Embedded computers monitor and control physical processes, usually with feedback loops where physical processes affect computations and vice versa.**
- Term **coined around 2006 by Helen Gill at the US National Science Foundation (NSF)**.
- **CPS is about the intersection (interaction), not the union, of the physical and the cyber.** It combines engineering models/methods from mechanical, environmental, civil, electrical, biomedical, chemical, aeronautical and industrial engineering with **computer-science** methods.
- **Examples: modern cars** (brake lights on braking, airbags on collision, **slow-vehicle alerts (SVA) in VANETs** that react faster than humans), **air-traffic systems** (thousands of programs + humans in the loop), traffic control, medical devices/telehealth.
- **Applications:** automotive systems, **automated manufacturing**, military systems, **assisted living**, medical systems, **traffic control and safety**, process control, energy, robotics…

Characteristics

- CPS describes a broad range of **complex, multidisciplinary, physically aware next-generation engineering systems that integrate embedded computing (cyber) into the physical world**.
- **Physical and software components are deeply intertwined**; they **operate on different spatial and temporal scales**, **exhibit multiple distinct behavioural modalities**, and **interact in ways that change with context**.
- **Transdisciplinary** — merges **cybernetics, mechatronics, design and process science**. CPS = cyber + physical + computation + dynamics + communication + security + safety (+ human in the loop).
- **Modelling** a CPS with multiple interacting computational units is a **serious challenge**.
- Some literature treats the **IoT-driven ecosystem (IoT + edge + fog + cloud) as a CPS**.

Potential application areas (list question)

Area	Examples
Intervention	**Collision avoidance**
Precision	**Robotic surgery, nano-level manufacturing**
Operation in dangerous/inaccessible environments	**Search and rescue, fire-fighting, deep-sea exploration**
Coordination	**Air-traffic control, war-fighting**
Efficiency	**Zero-net-energy buildings** (lights/AC by occupancy and daylight)
Augmentation of human capabilities	**Healthcare monitoring and delivery**

Cloud + CPS

- Sensors–actuators–processors form a vast network; integrating computation with physical processes is **"computing as a physical act"**: the real world is monitored by sensors → data go to **cyberspace** where applications/services use them → **affect the physical environment**.
- **Cloud computing provides a flexible platform for realizing the goals of CPS** (with the edge–fog–cloud layers; e.g., vehicle **OBU = edge**, **RSU = fog**, back-end **cloud**; location-driven services: safety, health, infotainment).
- **Cyber-Physical Cloud Computing (CPCC) architectural framework: *a system environment that can rapidly build, modify and provision cyber-physical systems composed of a set of cloud-computing-based sensor, processing, control and data services*.**
- **Benefits: efficient use of resources · modular composition · rapid development and scalability · smart adaptation to the environment at every scale · reliable and resilient architecture** (and stored data helps learn better CPS models).
- **Cyber-physical cloud services: sensor services, actuator services, processor services, data services** (+ semantic engines), used by domains such as schools, hospitals, traffic control,

ambulances, medical devices.

- **High-level CPCC: physical scenario** (sensors, users, mobile objects such as an ambulance needing priority) ⇄ **cyber-physical cloud platform** (**service catalog, cloud service API, runtime services**) ⇄ **CPS models** (functional and non-functional requirements; system modeller/builder).
- **Example — intelligent machining monitoring:** physical resources (**CNC controllers, machine tools**, local terminals/sensors) → **local servers** (pre-processing, diagnosis, decisions; actuate CNC controller) → **cloud** (advanced signal processing, **feature extraction and selection**, **cognitive decision-making**) → feedback.
- **Cloud–edge framework for cyber-physical-social systems (CPSS):** many **local CPSSs (cyber, social, physical worlds)** solve local issues at the edge; the **cloud plane** takes **global** decisions; the **application plane** hosts smart city, home, factory, health, vehicles, grid.

MCQ traps (12.2)

- CPS term: **Helen Gill, NSF, ~2006**.
- CPS = **intersection**, not union, of cyber and physical.
- CPS feedback loop is **two-way** (physical ↔ computation).
- CPCC = rapidly **build, modify, provision** CPS from cloud **sensor, processing, control, data** services.
- Zero-net-energy building = **efficiency**; collision avoidance = **intervention**; robotic surgery = **precision**; air-traffic control = **coordination**.

12.3 Case Study I — Spatial cloud computing (L58)

Spatial / geospatial data

- **Spatial data = a feature/attribute together with a location (coordinates)**, often with **time** (spatio-temporal: forest cover, river course, crops, weather over time).
- **Spatial objects** (visible from above: building footprints, roads as **polylines**) vs **non-spatial/attribute data** (building name, departments, occupancy; road name, type, width).
- **Geospatial data = information describing objects, events or other features with a location on or near the Earth's surface**; it typically combines **location information (coordinates, e.g., lat–long)**, **attribute information** and **temporal information**. (Spatial data can also be non-geographic, e.g., medical images — not in scope here.)
- **> 90 % of the world's data has some spatial context.**
- Maps are turned into **information** — patterns, trends, decisions: **crime-hotspot mapping** (route limited police patrols by time/events), **flood/drought analysis and prediction**, **optimal paths**, **congestion prediction**, cyclone tracking and vehicle detours.
- **Spatial analysis** solves **location-oriented problems** ("what happens around a place and how places relate") — **beyond mapping**. Three steps: **identify datasets (bring them onto the same**

base map/location reference) → analyse → draw inferences. Users: local/state/central government, businesses, utilities (telephone, water, electricity), e-commerce, researchers, NGOs.

Challenges that make cloud a good fit

- **Voluminous, spatio-temporal** data; moving terabytes is a serious constraint.
- **Integration of heterogeneous, multimodal sources** (e.g., crop-production estimation needs crop data, rainfall, drought, temperature, economic demand, fertilizer, water supply).
- Better **discovery, access and utilization** of data and processing.
- **Real-time/near-real-time** resources for **emergency response** (fire, cyclone).
- **Access spikes** and the need for **reliable, scalable service for massive numbers of concurrent users**. ⇒ **Spatial (spatio-temporal) analytics + cloud computing is a good match** (elastic, on-demand platform; integrate observation systems; configure compute for intensive simulations, e.g., climate models).

Spatiotemporal cloud computing (definition)

> **A cloud computing paradigm that is driven by geospatial sciences and optimized by spatiotemporal principles for enabling geospatial science discoveries and cloud computing within a distributed computing environment.**

- **Tobler's First Law of Geography:** *everything is related to everything else, but near things are more related than distant things* — e.g., Kharagpur's temperature is closer to Midnapore's (~10–15 km away) than to Darjeeling's or Shimla's. Domain science must be **embedded** in the analysis.
- **Spatial cloud** supports shared resource pooling for organizations with common goals; choice of deployment/service/business models; managed services preventing data loss from outages; lower financial risk; easy to use, scalable, cost-effective, reliable.

Spatial cloud architecture

- Cloud layers (SaaS, storage-aaS, PaaS, IaaS) host **spatial web services**: **data provisioning** (e.g., **Web Feature Service, catalog service**), **processing services**, **map services**, specific catalogues → used by organizations and individuals; organizations want to **share** spatial data; easier GIS decision-making (integrate latest databases, merge separate systems, exchange information).
- **OGC = Open Geospatial Consortium** (makes standards for spatial data); any **OGC client** can find services in a **registry** and use **feature, map and processing services** over spatial databases (e.g., PostgreSQL/PostGIS, Oracle). These services can be **instantiated as cloud services**.

Mobility analytics and the trajectory cloud

- **Movement has intent** (humans, animals, hurricanes, migratory birds). Urban **travel time** matters; knowing **mobility patterns** helps reduce congestion, predict travel time, plan buses, waiting places, signals; smart-home sensors track movement (e.g., of the elderly).

- Data: **daily GPS footprints, travel logs, check-ins, mobile-phone activity logs**, vehicle/transport data, IoT home sensors.
- A **generic framework for trajectory-trace mining for smart-city applications** (ex-PhD work at IIT KGP): model city dynamics, effective storage and computation, **learning techniques to extract mobility patterns**, **recommendation systems**. **SDI (Spatial Data Infrastructure)** built on web services.
- **Spatial trajectory** = a trace generated by a moving object in geographic space (series of (x, y, t)); **semantic trajectory** adds **intent/meaning** (students stopping at food joints in the evening).
- **Trajectory cloud ("TrasCloud" in the transcript):** an **end-to-end cloud framework for analysing urban dynamics** that enables **trip planning, travel-time prediction, ride sharing**, and **minimizes service waiting and provisioning time** for location-based services (**food delivery, medical emergency**); key to **ITS**.
  - **Pipeline:** urban sensing/data acquisition (**GPS, road network, traffic info**) → **trajectory pre-processing** → **trajectory data modelling** (**map matching, geo-tagging**) → **trajectory trace processing** (information retrieval, **spatio-temporal range queries**, trajectory-based query processing).
  - **Services: trajectory trace indexing**, **map matching**, **query processing**.
  - **Implemented on Google Cloud Platform:** indexing → **BigQuery + Cloud SQL**; map matching → **Compute Engine**; query services → **Compute Engine + Cloud SQL** (also tried on Meghamala).

MCQ traps (12.3)

- Geospatial data = **location + attributes + time**.
- **Tobler's first law**: nearer things are **more** related.
- Spatiotemporal cloud computing is **driven by geospatial sciences** and **optimized by spatiotemporal principles**.
- **OGC** = Open Geospatial Consortium (standards); WFS/WMS-type services.
- Trajectory cloud on GCP used **BigQuery, Cloud SQL, Compute Engine**.

12.4 Case Study II — Internet of Health Things (IoHT) (L59–L60)

Context

- A **cloud-to-things continuum** — **cloud, fog, edge, IoT** must be viewed as **one integrated system**, not separate layers (orchestration across layers; aggregation at lower layers also protects **privacy** of raw data).
- Cloud: subscription-based, (theoretically) infinitely scalable, ubiquitous — but needs **good connectivity** and **everything travels to the cloud and back** (e.g., room-temperature control could be decided by a local switch/router or an aggregate sent upward).

- **Layer roles: cloud** — business analytics and intelligence; **fog** — local network data analysis, reduction, control response, virtualization/standardization; **edge** — large-volume **real-time processing at source**, on-premise visualization (industrial PCs, embedded systems, gateways, micro data storage); **sensors/controllers** — data origin.
- **Cloud limitations: high latency**, large data volume, **high bandwidth** needs (for real-time responses). **IoT limitations: processing, storage, power**, and **no global view** (one sensor sees only its corner of a room; a switch/server aggregating 10–20 sensors can compute mean/variance and decide).
- **Benefits of fog/edge:** reduced latency (real-time support), **less network congestion**, **reduced cost of execution at the cloud** (subscription), better handling of data near sensors (cleaning), **location awareness**, data/information security.
- **Downsides:** edge devices must be **smart enough**; configuring an edge–fog–cloud infrastructure is **much more complicated** and the **organization must manage it**; proper orchestration needed.

Objectives of the case study (IIT KGP M.Tech project)

1. **Design a fog–edge-computing-based health model to reduce latency, network usage and cost incurred at the cloud.**
2. **Test the model with the iFogSim simulator** (initially developed by **IIT Kharagpur and the University of Melbourne**, now maintained by Melbourne's **CLOUDS lab**; linked with **CloudSim**).
3. **Develop a customized wearable device** to collect health parameters, and **implement the model on hardware**.
4. **Study dew-based computing** and its efficacy. **Workflow:** conceptualization and modelling → design/implement the fog model → **performance evaluation in iFogSim** → results → **hardware implementation** (fabricate a customized body sensor; **activity detection with an accelerometer**; **heart-attack alert** — *explicitly no medical/clinical implication; demonstration only*) → propose a **dew-based framework** → inferences.

Topology and latency settings (iFogSim)

Level 0 = Cloud → Level 1 = Fog (ISP gateway / area gateway) → Level 2 = Edge (mobile) → Level 3 = IoHT (body sensors). The cloud is far more resourceful (MIPS, RAM, bandwidth); mobiles/edge are less. | Link | Latency (units used in the study) | |—|—| | Body sensor → mobile | **1** | | Mobile → area gateway | **2** | | Area gateway → ISP gateway | **2** | | ISP gateway → **cloud** | **100** | | Mobile → display | **1** | → the cloud hop dominates latency.

Application modules and placement

Flow: **body sensors → client module → data filtering module → data processing module → event handler module → client module → display** (alert: normal / visit a doctor), with a **confirmatory module** (confirms events against a knowledge base; logs data). | Module | Fog–edge-based placement | Cloud-based placement | |—|—|—| | Client module | **Mobile (edge)** | Mobile | | Data filtering | **Area**

gateway (fog) | Cloud | | Data processing | **Area gateway (fog)** | Cloud | | Event handler | **Area gateway (fog)** | Cloud | | Confirmatory module | **Cloud** | Cloud |

Simulation results (configurations 1–5: increasing users and area gateways)

Metric	Cloud-only	Fog–edge
Average latency	**Increases drastically** with configuration size	**≈ constant, near zero** (control-loop modules sit at the area gateway)
Network usage	**Much higher**	Much lower
Cost of execution (in cloud)	**Much higher** — only cloud resources incur (subscription) cost; other resources are owned by the organization	Much lower
Energy	**Data-center energy increases** as modules shift to the cloud	Fog devices take part of the load
Mobile energy	**Same** in both (mobile load unchanged)	Same

Caveat: with fog/edge, **you manage** those layers; with cloud-only, the provider manages everything.

Hardware implementation

- **Customized BP and pulse meter, NodeMCU board, accelerometer**, data sent over **Wi-Fi/Bluetooth** to the **mobile (edge)** → **Raspberry Pi as fog devices** → **AWS** (also in-house **Meghamala**) as cloud.
- **Activity detection pipeline (accelerometer):**
  1. **Data extraction:** three axes x, y, z → magnitude $a = \sqrt{x^2 + y^2 + z^2}$.
  2. **5-point smoothing** to reduce induced noise: each value = **average of 5 signals — 2 preceding, the signal itself, 2 succeeding**: $\tilde{a}_i = \frac{a_{i-2}+a_{i-1}+a_i+a_{i+1}+a_{i+2}}{5}$.
  3. **Feature extraction** from the filtered signal: **minimum amplitude, maximum amplitude, mean amplitude, standard deviation, energy in time domain, energy in frequency domain**.
  4. **Classifier: k-nearest neighbours (kNN).**
     - *Worked example:* readings 2, 4, 6, 8, 10 → smoothed value at the centre = (2+4+6+8+10)/5 = **6**. Magnitude of (3, 4, 0) = **5**.
- **Heart-attack alarm:** a simple **if–then rule** on BP/pulse thresholds (e.g., *if* P > 170 *and* S ≥ 180 *then* alarm = true, else false) — **demonstration only, no clinical validity**.

Dew-based extension

- **Dew computing = on-premise computing whose on-premise computers provide functionality independent of cloud services and also collaborative with them** — fully realizing on-premise resources.

- If the Internet connection breaks, the **dew server** keeps serving (sensors → dew server → display); the **database is synced** with the cloud (AWS) when the connection returns — useful when outages are short.

Comparison (lecture table) | Aspect | Cloud | Cloud–fog–edge | With dew | |—|—|—|—| | On-premise resource utilization | **Low** | **Sub-optimal** | **Optimal** | | Connectivity / uptime / bandwidth required | **High** | Low | Low | | Latency | **High** | Low | Low | | Processing power & storage | **High** (≈ infinitely scalable) | **Limited** | **Moderate** |

MCQ traps (12.4)

- In the iFogSim topology, **ISP gateway → cloud latency (100)** dwarfs the other hops (1–2).
- Fog placement: **filtering, processing, event handling at the area gateway**; **confirmatory in cloud**; **client at mobile**.
- **Mobile energy stays the same**; **cloud energy and cost** rise in cloud-only; fog latency stays ~constant.
- 5-point smoothing averages **2 before + itself + 2 after**; classifier = **kNN**.
- Raspberry Pi = **fog**; mobile = **edge**; AWS/Meghamala = **cloud**.
- Dew gives **optimal** on-premise utilization and works **during outages**.

Week 12 — High-yield takeaways

1. 5G: multi-Gbps, ~1 ms latency, massive capacity, forward compatible; **eMBB (AR/VR) · URLLC (autonomous vehicles, telesurgery, smart grid) · mMTC (massive IoT)**; 5G–cloud: cloud must meet 5G needs (edge/MEC/fog) and 5G is cloudified via NFV/SDN; edge apps: AR/VR, tactile Internet, ITS, factories, emergency response.
2. CPS: computing + physical with feedback; Helen Gill (NSF, 2006); intersection not union; intertwined, multi-scale, multi-modal, context-dependent; applications (intervention, precision, dangerous ops, coordination, efficiency, augmentation); CPCC = rapidly build/modify/provision CPS from cloud sensor/processing/control/data services.
3. Spatial cloud: geospatial = location + attributes + time; >90 % data spatial; Tobler's law; spatiotemporal cloud computing definition; OGC services; trajectory cloud (indexing, map matching, query) on GCP (BigQuery, Cloud SQL, Compute Engine).
4. IoHT: levels cloud–fog–edge–IoHT; latencies 1/2/2/100/1; module placement; fog keeps latency ~constant and lowers network use and cloud cost; mobile energy unchanged; hardware (NodeMCU, BP/pulse, accelerometer, Raspberry Pi fog, AWS); magnitude → 5-point smoothing → features → kNN; dew for outages.

Formula Sheet — all numerical tools in the course

> Format per formula: **Purpose · Expression · Variables · Conditions · Meaning · ⚠ Likely mistake.** Week references point to the detailed notes.

A. SLA and availability (Weeks 3, 5)

A1. Availability (standard / Azure "monthly uptime %")

$$A(\%) = \frac{T - d}{T} \times 100 = \left(1 - \frac{d}{T}\right) \times 100$$

- T = expected (scheduled) service time in the period (days × service hours/day, or "maximum available minutes"); d = total downtime (sum of outages).
- Meaning: fraction of scheduled time the service was up.
- ⚠ Use **scheduled** hours (e.g., 12 h × 30 d = 360 h), not 24 h × 30 d, when the SLA says 12 h/day. Convert minutes ↔ hours carefully.

A2. Lecture's KPI form (L11, L21 Problem 1)

$$A = 1 - \frac{\text{downtime}}{\text{uptime}}$$

- ⚠ Slide form uses **uptime** in the denominator (L21 P1: 1 – 10.75/349.25 = 96.92 %). The standard form above gives 97.01 %. The verdict is usually the same; if options are close, check both.

A3. Allowed downtime for a target availability

$$d_{max} = (1 - A)\,T$$

- 99.9 % over 30 × 24 h → 0.72 h = **43.2 min**; 99.95 % over 360 h → **10.8 min**; 99 % over 30 × 24 h → 7.2 h.

A4. Service credit and payable amount

$$\text{Bill} = \text{rate} \times \text{days}, \quad \text{Credit} = c\% \times \text{Bill}, \quad \text{Payable} = \text{Bill} - \text{Credit}$$

- c is chosen from the **band containing the measured availability** (e.g., < 99.95 % but ≥ 99 % → 10 %; < 99 % → 25 %).
- L21 P2: \$50 × 30 = \$1500; A = 97.385 % → 25 % → credit \$375 → **payable \$1125**.
- ⚠ A lower availability falls in the **bigger** credit band.

B. Cloud economics (Weeks 3, 5)

B1. Coefficient of variation

$$C_v = \frac{\sigma}{|\mu|}$$

- σ = std. deviation of demand, μ = mean demand. Lower C_v = smoother demand = better utilization/risk–return.
- ⚠️ Not variance, not correlation coefficient.

B2. Aggregating n independent identical demands

$$\mu_{agg} = n\mu, \quad \sigma_{agg} = \sqrt{n}\,\sigma \;\Rightarrow\; C_v^{agg} = \frac{C_v}{\sqrt{n}}$$

- 100 workloads → C_v falls to **1/10**. Perfectly correlated: C_v **unchanged** (σ grows like n). Negatively correlated (X, $1-X$): $C_v = 0$.
- ⚠️ Factor is $1/\sqrt{n}$, not $1/n$.

B3. Utility pricing — owning vs cloud

$$U = \frac{C}{B}, \qquad B_T = P \cdot B \cdot T, \qquad C_T = \int_0^T U B\, D(t)\, dt = U\, B\, A\, T$$

$$\boxed{\text{Cloud cheaper} \iff U < \frac{P}{A}}$$

- P = peak demand, A = average demand, B = owned unit cost, C = cloud unit cost, T = period, $D(t)$ = demand.
- Spiky demand (large P/A) favours cloud; cloud can be pricier per unit ($U > 1$) and still win.
- ⚠️ $U = C/B$ (cloud ÷ owned), compared with **peak ÷ average**. Owned cost scales with **peak**.

B4. Penalty of mis-provisioning

$$\text{Penalty} \propto \int_0^T |D(t) - R(t)|\, dt$$

- Flat demand → 0; linear → periodic provisioning OK; exponential $D = e^t$ with delay t_p → $D - R = (1 - e^{-t_p})e^t$ → penalty grows **exponentially**.
- L22 P1 ($R = D + \delta D'$, $\delta = \pi/12$): lecture answer $30\pi/12 \approx$ **7.86** (strict $\int |D - R| = 70\pi/12 \approx 18.33$).

B5. Direct cost comparison (L22 P2)

$$B_T = PBT, \qquad C_T = C \int_0^T D(t)\, dt$$

- $D = 50(1 + e^{-t})$, $P = 100$, $B = 120$, $C = 200$, $T = 100$: $B_T = 1.2\text{M}$, $C_T \approx 1.01\text{M}$ → ratio 0.84 → cloud cheaper.

B6. Core-hour economics (L22 P3)

$$\text{cost/h} = \frac{\text{Capex}}{\text{years} \times 8760}, \quad \text{per core-h} = \frac{\text{cost/h}}{\text{cores}}, \quad \text{per effective h} = \frac{\text{cost/h}}{\text{efficiency}}$$

$$\text{Total per effective h (in-house)} = \frac{\text{cost/h}}{e} + \text{power\&cooling/h} + \text{mgmt/h}$$

- ₹6,00,000 over 3 y → ₹22.83/h → ₹1.90 per core-h (12 cores); cloud ₹42/h, 8 cores → ₹5.25 per core-h; effective: 57.08 vs 52.5; totals **85.075 vs 53.5 (ratio 1.59)**; in-house wins at ≈ 90 % efficiency (break-even ≈ 89.5 %).
- ⚠ 3 years = **26,280 h**; divide by efficiency (40 % → × 2.5); add power/cooling and management.

C. Reliability and distributed storage (Week 3, 5)

C1. System failure with N components

$$P(\text{system failure}) = 1 - (1 - p)^N$$

- p = per-component failure probability. Approaches 1 as N grows → **fault tolerance is necessary**.
- $p = 0.001, N = 1000$ → ≈ 0.63. ⚠ Transcript wording suggests the opposite; trust the formula.

C2. HDFS/GFS blocks and storage

$$\text{blocks per file} = \left\lceil \frac{\text{file size}}{\text{block size}} \right\rceil, \qquad \text{stored blocks} = \sum \text{blocks} \times r$$

- Lecture: block 64 MB, $r = 3$; files 64 KB, 65 MB, 127 MB → 1 + 2 + 2 = 5 → **15**.
- ⚠ Lecture uses **64 MB** (Hadoop 1 / GFS); Hadoop 2+ default 128 MB — use the question's value. A small file still counts as one block.

C3. Dynamo quorum consistency

$$R + W > N$$

- N replicas, R read quorum, W write quorum; overlap guarantees the latest write is read. Small W (fast writes) needs large R, and vice versa.
- ⚠ Strictly greater: $R + W = N$ is **not** enough.

D. Parallel computing and MapReduce (Weeks 3, 5)

D1. Parallel efficiency

$$\varepsilon = \frac{T_1}{p\,T_p} \qquad (\text{ideal } T_p = T_1/p \Rightarrow \varepsilon = 1)$$

- Scalable ⇔ ε stays constant as data and processors grow proportionally.

D2. Word counting (n docs, m words, f occurrences/word/doc, c = time per word read/sent; $T_1 = nmfc$)

$$\text{Split words: } \varepsilon = \frac{1}{p} \qquad \text{Split documents: } \varepsilon = \frac{1}{1 + \dfrac{2p}{nf}}$$

- Split-documents time: $T_p = \frac{n}{p} mfc + 2mc$. → ≈ 1 when $p \ll nf$; scalable. Split-words is **not** scalable.

D3. MapReduce efficiency (uniprocessor time wD, map output σD, c = I/O per word)

$$\varepsilon_{MR} = \frac{1}{1 + \dfrac{2c\,\sigma}{w}}$$

- Independent of P (scalable); worse with larger map output σ or I/O cost c.

D4. Combining partial averages (L23)

$$\bar{x} = \frac{\sum_i \bar{x}_i\, n_i}{\sum_i n_i}$$

- Mappers emit (avg, count) or (sum, count). {10,20},{30,40},{50} → (15·2 + 35·2 + 50·1)/5 = **30**. ⚠️ Average of averages (33.3) is wrong.

E. VM migration (Weeks 9–10)

E1. Cold (non-live) migration

$$T_{mig} = \frac{V_m}{R}, \qquad T_{down} \approx T_{mig}$$

E2. Pre-copy live migration

$$r = \frac{P \cdot D}{R}, \qquad T_j = r^j \frac{V_m}{R}$$

$$T_{mig} = \frac{V_m}{R} \cdot \frac{1 - r^{n+1}}{1 - r} + T_{res}, \qquad T_{down} = r^n \frac{V_m}{R} + T_{res}$$

$$n = \min\left(\left\lceil \log_r \frac{V_{th}}{V_m} \right\rceil,\ N_{max}\right)$$

- V_m = VM memory, R = transmission rate, P = page size, D = pages dirtied/s, V_{th} = stop threshold, T_{res} = resume time.
- Needs $r < 1$ to converge. Example: 1024 MB, 100 MB/s, $r = 0.2$, $n = 3$, $T_{res} = 0.1$ s → $T_{mig} \approx 12.88$ s, $T_{down} \approx 0.18$ s (cold: 10.24 s each).
- ⚠️ Downtime is only the **last** term; live migration **increases** total time but **reduces** downtime.

E3. Multiple VMs

- **Parallel:** each VM gets ≈ R/m; downtime ≈ stop-and-copy time of one VM (at R/m) + resume.
- **Serial:** first VM at full R; others suspended when it enters stop-and-copy; downtime = its stop-copy + migration of the other $m - 1$ + resume.

F. Energy and green cloud (Weeks 5, 8, 11)

F1. Consolidation power (L24 example)

$$P_{total} = \sum_{\text{on nodes}} P(\text{load})$$

- Idle 105 W, full (8 VMs) 170 W, 2 VMs ≈ 138 W: 8 VMs on 1 node + 3 idle = 485 W; spread 2/node = 552 W; 1 node with 3 off = **170 W**.

F2. Linear server power model *(standard clarification; real curve is non-linear/concave)*

$$P(u) = P_{idle} + (P_{max} - P_{idle})\,u$$

- Idle server ≈ **66 %** of peak (L37) → consolidation pays.

F3. Switch power (L37)

$$P_{switch} = P_{chassis} + n_{lc}P_{linecard} + \sum_r n_{ports,r}P_r$$

- Shares: chassis ≈ 36 %, line cards ≈ 43 %, port transceivers ≈ 11 %.

F4. PUE and DCiE *(standard definitions)*

$$\text{PUE} = \frac{\text{Total facility power}}{\text{IT equipment power}}\ (\geq 1), \qquad \text{DCiE} = \frac{1}{\text{PUE}} \times 100\%$$

- IT = 40 % of total → PUE = 2.5. ⚠️ Not IT ÷ total.

F5. Energy breakdowns (know which one is asked)

- Whole data center (L37): **cooling 45 %, IT 40 %, power distribution 15 %.**
- By component (L55): **processor 45 %, cooling 20 %, memory 15 %, storage 10 %, network 10 %.**

G. Mobile cloud computing — offloading (Week 7)

$$E_{saved} = P_c\frac{C}{M} - P_i\frac{C}{S} - P_{tr}\frac{D}{B} = \frac{C}{M}\left(P_c - \frac{P_i}{F}\right) - P_{tr}\frac{D}{B}, \quad S = FM$$

- P_c, P_i, P_{tr} = mobile power when computing/idle/transmitting; C = instructions; M, S = mobile/cloud speeds; F = speed-up; D = data; B = bandwidth.
- Offload iff $E_{saved} > 0$ → **large C, small D, large F, high B**.
- Example: $C/M = 2$ s, $P_c = 1$, $P_i = 0.2$, $F = 20$, $P_{tr} = 1.5$, $D/B = 1$ s → 0.48 J saved.
- ⚠ The idle term uses **cloud** time C/S; the transmit term uses D/B.

H. Sensor/IoHT signal processing (Week 12)

$$a = \sqrt{x^2 + y^2 + z^2}, \qquad \tilde{a}_i = \frac{a_{i-2} + a_{i-1} + a_i + a_{i+1} + a_{i+2}}{5}$$

- Accelerometer magnitude, then 5-point smoothing → features (min/max/mean amplitude, standard deviation, time- and frequency-domain energy) → kNN.

I. Quick unit conversions

- 1 day = 1440 min; 30-day month = 720 h = 43,200 min; 1 year = 8760 h.
- 1 GB = 1024 MB (HDFS/memory convention); 1 Gbps ≈ 125 MB/s (bits vs bytes!).
- Percent ↔ fraction: 99.95 % = 0.9995 → unavailability 0.0005.

Classification & Comparison Sheet

> Compact tables of every categorization the lectures emphasize. Rows reflect the **lectures' own classifications** (⚠️ marks where today's reality differs).

---

1. NIST model and cloud basics

Item	Members
5 essential characteristics	On-demand self-service · Broad network access · Resource pooling (multi-tenant) · Rapid elasticity · Measured service
3 service models	SaaS · PaaS · IaaS (+ XaaS/BPaaS in lectures)
4 deployment models	Private · Public · Community · Hybrid
5 NIST cloud actors	Consumer · Provider · Carrier · Broker · Auditor
Economic properties (L12)	Common infrastructure · Location independence · Online connectivity · Utility pricing · On-demand resources
Evolution	Distributed → Grid & Cluster → Utility (concept) → Cloud (realization)

---

2. Distributed paradigms

Paradigm	Key definition	Types/notes
Distributed	Autonomous entities, own local memory, message passing	Fault tolerance, resource & load sharing, easy expansion
Grid	Harness unused cycles; single system image; dependable, consistent, pervasive, inexpensive access	Computational, data, collaborative, network, utility grids; analogy: power grid
Cluster	Interconnected stand-alone computers as one integrated resource	HA/failover, load-balancing, parallel processing; more loosely coupled than SMP
Utility	Charge per use, not flat rate (metered)	Risks: backup, security, competence, SLA definition, charge-back value

3. Service models

	IaaS	PaaS	SaaS
Consumer gets	Raw VMs, storage, network	Dev/runtime platform	Ready application
Provider manages up to	Virtualization (VMs)	Runtime/middleware	Application
Network (OSI) responsibility	Up to data link	Up to presentation/session	Whole stack
Useful when	Volatile demand, no capital, limit capex, trials/PoC	Many/remote developers, automate test & deploy, agile	Outside interplay (email), web+mobile, short-term use, demand spikes (tax/billing)
Not ideal when	Regulation forbids outsourcing; highest dedicated performance needed	High portability needed; proprietary languages → **vendor lock-in**; custom HW/SW for performance	Real-time/fast processing; data can't be hosted externally; on-premise already suffices
Lecture examples	Amazon EC2, Compute Engine, Meghamala	**Azure**, Google App Engine	Salesforce CRM, Google Docs
Stakeholder	IT architects/operations	Software architects/developers	End users/app admins

4. Deployment models

Model	Up-front cost	Resources	Notes
Public	**Low**	Illusion of unlimited	Multi-tenancy risk (co-residence with adversary), restrictive default SLA, provider premises
On-site private	Significant–high	**Limited** (fixed)	Strong external security; needs IT skills; perimeter around subscriber + private cloud
Outsourced private	Modest–significant	**Extensive**	**Two** security perimeters + protected link; negotiated SLA
On-site community	**Highly variable**	Limited	RBAC/ABAC policies; boundary controllers
Outsourced community	Modest–significant	Extensive	—
Hybrid	Varies	Mix	Constituents remain unique; portability; complex; members join/leave

5. Virtualization

	Full (1st gen)	Para (2nd gen)	Hardware-assisted (3rd gen)
Technique	Binary translation; all HW emulated	Cooperative; **modified guest** (ring 1); special API	CPU extensions **Intel VT / AMD-V**; guest at ring 0
Examples	VMware, Microsoft; QEMU, Bochs	VMware, **Xen**	VMware, Xen on aware HW
Pro / Con	Portable / slow	Fast / kernel recompiled	Unmodified OS / less portable, speed–flexibility issues

- **Popek–Goldberg:** equivalence, resource control, efficiency; possible if **sensitive ⊆ privileged**.
- **IaaS hierarchy:** Cloud manager (+**DOS** metadata) → Cluster manager (+**PLS** persistent storage) → Computer manager (per host; start/suspend/reconfigure VMs).
- **IaaS control:** provider — total over hardware, administrative over hypervisor; subscriber — total over guest OS, middleware, apps.

6. XML and web services

Pair	Distinction
Well-formed vs valid	Syntax-correct vs also conforms to DTD/XSD (valid ⇒ well-formed)
DTD vs XSD	Non-XML, can declare entities vs XML (2001), data types, **no entities**
Validating vs non-validating parser	Must fetch all entities & fail if not vs stop processing DTD at first unresolved entity
SAX vs DOM vs JDOM vs DOM4J	Event-based fast low-memory vs in-memory tree slow vs Java OO tree vs Java mixed SAX/DOM
SOAP / WSDL / UDDI	Messaging (stateless, one-way, over HTTP) / description / registry
SOA roles & ops	Provider–requestor–registry; publish–find–bind (IBM model)
WS vs CORBA/DCOM	HTTP port 80 (firewall-friendly) vs non-standard ports

7. SLA

Item	Content
Hierarchy	KPI (low-level metric) → SLO (measurable objective) → SLA
Types	Off-the-shelf/non-negotiable/direct (public clouds; not for mission-critical) vs negotiable (via one or more external agents)
WS vs cloud SLA	QoS: response time/availability/cost vs security/privacy/trust · Automation: not automated vs required · Resource allocation: UDDI vs no central directory
WS-Agreement vs WSLA	XML language+protocol for runtime agreements vs XML-schema language + runtime interpreter (lacks metric semantics)
CSCC 7 steps	Actors · business policies · service/deployment models · performance objectives · security/privacy · service management · failure management (exit clause)
Azure terms	Fault domain (shared power/network) · update domain (updated together) · availability set (VMs across fault domains) · error 5xx / success 2xx

8. Data management systems

System	Key facts
GFS / HDFS	Master / **NameNode**; chunk server / **DataNode**; chunk / **block** (64 MB in lecture); **3 replicas** on different racks; read → metadata of **one** replica; append → **all** replicas, **primary** picks offset; heartbeats
BigTable	Built on GFS; (row, column, timestamp); column-family:label (families fixed, labels dynamic); versions by decreasing timestamp; tablets → tablet servers → SSTables; root metadata tablet
Dynamo (Amazon)	Key–value; many small concurrent updates; MD5 128-bit ring of virtual nodes; $N-1$ extra replicas; quorum $R+W>N$; eventual consistency; versioning; no DFS dependence; Berkeley DB
Datastore (GAE) / SimpleDB	One big table; column-family storage; sharded; lexicographic keys (prefix/range queries); single-property & kind indexes; highest-selectivity index
Row vs column store	OLTP writes (B+-tree) vs warehouse aggregation (join indexes)
Parallel DB architectures	Shared memory (SMP) · shared disk (NAS/SAN) · shared nothing

9. Platforms and their components

Platform	Component → role
OpenStack	Nova compute · Neutron networking · **Swift object** · **Cinder block** · Keystone identity · Glance images · Ceilometer telemetry · Horizon dashboard (NASA + Rackspace, 2010)
OpenStack storage	Ephemeral (Nova, dies with VM) · Block (Cinder, persistent device) · Object (Swift, anywhere, VM images)
Azure	Windows Azure (compute/storage/fabric) · SQL Azure · AppFabric; web role vs worker role (via queues); resource group, App Service plan
GCP	App Engine (PaaS) · Compute Engine (IaaS) · Cloud Storage (object) · Cloud SQL (MySQL) · Datastore (NoSQL) · BigQuery · Endpoints
Serverless	AWS Lambda / Google Cloud Functions / Azure Functions; DBs DynamoDB / Cloud Datastore / Cosmos DB
Docker	Engine (build/run) · Hub (public registry) · Compose (multi-container) · daemon + client (REST/HTTP) · private registry
Kubernetes	Cluster → worker nodes → Pods; control plane (scheduler); kubelet, kube-proxy

10. Security

Item	Members / mapping
Attack → property	Interruption→availability · Interception→confidentiality · Modification→integrity · Fabrication→authenticity
Threat classes	Disclosure (snooping) · Deception (spoofing, repudiation of origin, denial of receipt) · Disruption · Usurpation (delay, DoS)
Passive vs active	Release of contents, traffic analysis (hard to detect) vs masquerade, replay, modification, DoS
Mechanism types	Secure $R \subseteq Q$ · precise $R = Q$ · broad otherwise
Gartner's 7	Privileged user access · regulatory compliance & audit · data location · data segregation · recovery · investigative support · long-term viability
RPO vs RTO	Max data loss vs max recovery time
Cloud-specific	Co-tenancy · lack of control · reputation fate-sharing
EC2 co-residency	Placement → extraction; checks: same Dom0, small RTT, internal IPs within 7; countermeasures: random IPs, hide Dom0, no co-residence
Collaborative SaaS	SelCSP (trust/reputation/competence) · risk-based access (fuzzy) · IDRM (NP-hard, greedy) · conflicts (cyclic inheritance, SoD)

11. Cloud vs fog vs edge vs dew

Aspect	Cloud	Fog	Edge	Dew
Location	Internet, central DCs	Edge of local network (routers, gateways, base stations)	Devices/sensors	User's own device (on the ground)
Latency / hops	High / multi-hop	Low / one hop	Lowest	Works offline
Mobility / location awareness	Limited / no	Supported / yes	—	—
Last mile	Leased line	Wireless	—	Sync when connected
Security	"Undefined"	"Can be defined"	—	—
Key idea	Global analytics	Aggregate, real-time local decisions (Cisco term)	Pre-processing, noise reduction	**Independence + collaboration**

- **Fog enablers:** virtualization · containers (Docker) · SOA · SDN (OpenFlow).
- **Dew categories:** IaD (iCloud) · software-in-dew (Play/App Store) · PiD (GitHub) · storage-in-dew (Dropbox/Drive) · WiD · DBiD.
- **Cloudlet:** trusted, resource-rich, nearby, single-hop middle tier.

12. MCC

Item	Content
Components	Profiler (time, power, traffic) · Solver (placement) · Synchronizer (merge results)
MAUI vs COMET	Method-level, annotated "remoteable", native local, RPC vs binaries only, thread migration via DSM, modified heap only
Partitioning	Static (ILP at launch) vs dynamic/adaptive (long-running)

13. Resource management

Aspect	Definition
Provisioning	Allocate provider's resources to a customer
Allocation	Distribute economically among competing users
Adaptation	Adjust dynamically to requirements
Mapping	Match required ↔ available
Modelling	States/transitions/outputs of RM
Estimation	Close guess of requirements
Discovery & selection	Find authenticated resources, choose best
Brokering	Negotiate via agent
Scheduling	Timetable of events/resources

- **Fog RM taxonomy:** Architecture (data flow, control, tenancy) · Infrastructure (hardware, system software, middleware) · Algorithms (discovery, benchmarking, load balancing, placement).
- **Placement constraints:** resource · network · application (locality, deadlines). **Offloading:** device→edge (partitioning, caching); cloud→edge (server replication/partitioning, caching). **Control:** centralized (solver, graph matching) vs distributed (blockchain, game theory, GA).

14. Federation

Architecture	Coupling	Description
Cloud bursting / hybrid	Loose	Private + public for peaks
Broker	Loose	Broker over several public clouds; most common
Aggregated	Partial	Partners pool resources into a larger virtual infrastructure
Multi-tier	Tight	Root cloud OS over sites of one corporation (hierarchical)

- **Coupling:** loose (limited monitoring, no VM migration) · partial (contract, affinity rules, energy info, cross-site virtual networks) · tight (same admin, exact placement, cross-site migration, HA, virtual storage).

15. VM migration, containers

Pair	Distinction
Cold vs live	VM suspended (downtime = migration time) vs keeps running
Pre-copy vs post-copy	Memory iteratively first, CPU last (KVM, Xen, VMware) vs CPU first, memory on demand
Serial vs parallel (multi-VM)	First at R, others suspended at its stop-copy vs all at R/m together
VM vs container	HW-level, guest OS, heavy vs OS-level, shared kernel, light (tens of MB), fast start
Image vs container	Static package (layers + metadata) vs running instance
Deployment eras	Traditional (no resource boundaries) → virtualized (VM isolation) → container (relaxed isolation, portable)
Container benefits	Separation of responsibilities · workload portability · application isolation
Docker commands	build · run · commit · tag · save/load (tar) · ps / ps -a · images · pull · rmi (images) vs rm (containers)

16. Serverless

	FaaS	BaaS
What	Provider runs event-triggered, stateless user functions (limited time, e.g., Lambda ~15 min)	Off-the-shelf back-end services (auth, DB, storage, hosting)
Example	Lambda, Cloud Functions, Azure Functions; ExCamera	Google Firebase

- Lambda: function · runtime · event source · layers · log streams. Google: event · trigger (= event + function) · event data · providers. Azure: code + function.json; Consumption plan.

17. Sensor cloud, IoT, 5G, CPS

Topic	Classification
Virtual sensors	One-to-many (middleware sets sampling) · many-to-one (regional aggregate) · many-to-many · derived (different sensor types)
Proxies	Sensor-cloud proxy (format conversion, aggregation) · sensor-network proxy (no direct cloud link)
IoT protocols	IoT side: MQTT, CoAP, HTTP · cloud side: MQTT, AMQP, HTTP
Vehicular cloud services	Network & data processing aaS · storage aaS (OBU) · cooperative safety info
5G classes	eMBB (AR/VR) · URLLC (autonomous vehicles, telesurgery, smart grid) · mMTC (massive IoT)
CPS applications	Intervention · precision · dangerous ops · coordination · efficiency · human augmentation

18. Green / sustainable cloud

Item	Content
DC network	Two-tier (access+core) · three-tier (access, aggregation, core; most used, >10k servers) · three-tier high-speed (100 GE)
Energy split (L37)	Cooling 45 % · IT 40 % · power distribution 15 %
Energy split (L55)	Processor 45 % · cooling 20 % · memory 15 % · storage 10 % · network 10 %
Green middleware	IaaS: energy/temperature sensors, demand predictor · PaaS: green profiler, power capping, green compiler · SaaS: power capping, green services
Sustainable model	Cloud architecture · cooling manager · power manager (renewable + grid, ATS, PDU) · remote CDCs
Taxonomy	Application design · metrics · capacity planning · energy management · virtualization · thermal-aware scheduling · cooling · renewables · waste heat

High-Yield Revision List

> The most testable facts in one place: **numbers & counts**, **one-line distinctions**, and **"if the question mentions X → think Y" cues**.

1. Numbers and counts to remember

Fact	Value
NIST essential characteristics / service models / deployment models / actors	**5 / 3 / 4 / 5**
Gartner cloud security risks	**7**
CSCC steps to evaluate SLAs	**7** (April 2012)
GFS/HDFS replication factor; lecture block/chunk size	**3**; **64 MB**
Popek–Goldberg properties	**3** (equivalence, resource control, efficiency)
Virtualization generations	**3** (full, para, hardware-assisted)
IaaS manager levels	**3** (cloud, cluster, computer manager)
Meghamala VM flavours / OS images	**3 / 3** (Ubuntu, CentOS, Fedora)
C_v reduction when aggregating n independent loads	$1/\sqrt{n}$ (100 loads → 1/10)
Cloud cheaper condition	$U = C/B < P/A$
Quorum consistency	$R + W > N$
Word-count efficiencies	split words $1/p$; split docs $1/(1 + 2p/nf)$
MapReduce efficiency	$1/(1 + 2c\sigma/w)$
Idle server power	≈ **66 %** of peak
DC energy (L37)	cooling **45 %**, IT **40 %**, power distribution **15 %**
Component energy (L55)	processor **45 %**, cooling **20 %**, memory 15 %, storage 10 %, network 10 %
Gartner 2007: IT share of world CO_2	**2 %**
US DC share of power (≈2007)	**1.5 %**, cost $4.5 B
Amazon energy-related cost share	**42 %**
Servers' share of world electricity (L24)	**≈ 0.5 %** (≈ 1 % with cooling)
Projected CDC energy by 2030	≈ **8000 TWh**
Docker first release	**March 2013**
CPS term coined	**~2006, Helen Gill, NSF**
EC2 co-residency success	≈ **40 %**; IPs within **7**
Lambda limits (lecture)	~**15 min** execution, ~**50 MB** image
SLA examples	EC2 **99.5 %**, Azure **99.9 %** (historical)
5G	~**10 Gbps**, ~**1 ms** latency
IoHT latencies	sensor→mobile 1, mobile→gateway 2, gateway→ISP 2, **ISP→cloud 100**
Human response / VoIP limit / light in fibre	10–100 ms / ~200 ms / ~124 miles per ms

2. One-line distinctions

- **Grid** harnesses unused cycles across organizations; **cluster** = interconnected stand-alone computers acting as one resource.
- **Utility computing** = concept (metered); **cloud** = its realization.
- **Elasticity** = up *and* down; **scale-up** (bigger box, cheaper small) vs **scale-out** (more boxes, only option large).
- **Client–server**: no virtualization; **cloud**: virtualization is core.
- **On-site private**: limited resources, high up-front cost; **outsourced private**: two perimeters, extensive resources.
- **Para-virtualization** modifies the guest; **full** translates binaries; **hardware-assisted** uses Intel VT/AMD-V.
- **DOS** (metadata, cloud manager) vs **PLS** (persistent VM storage, cluster manager).
- **Well-formed** (syntax) vs **valid** (+ schema); **DTD** (entities, non-XML) vs **XSD** (XML, types, no entities).
- **SAX** (events, fast, low memory) vs **DOM** (tree, slow, modifiable).
- **SOAP** messaging · **WSDL** description · **UDDI** registry.
- **KPI → SLO → SLA**; **off-the-shelf** SLA unsuitable for mission-critical.
- **GFS read**: one replica's metadata; **append**: all replicas, primary chooses offset.
- **BigTable** bulk, GFS-based; **Dynamo** key–value, small concurrent updates, quorum.
- **Mapper failure** → redo its whole task; **reducer failure** → only unfinished tasks.
- **Swift** object vs **Cinder** block; **Glance** images; **Keystone** identity; **Ceilometer** metering.
- **Web role** (front end) vs **worker role** (background), linked by **queues**.
- **App Engine** PaaS vs **Compute Engine** IaaS.
- **RPO** (data loss) vs **RTO** (recovery time).
- **Interception** = confidentiality; **modification** = integrity; **fabrication** = authenticity; **interruption** = availability.
- **Traffic analysis** passive; **replay** active.
- **MAUI** (annotated methods) vs **COMET** (binaries, DSM threads).
- **Cloudlet** = single-hop middle tier; **fog** = Cisco's term; **dew** = works offline (independence + collaboration).
- **Bursting/broker** loose; **aggregated** partial; **multi-tier** tight.
- **Cold** migration: downtime = migration time; **pre-copy**: memory first; **post-copy**: CPU first.
- **Container** (OS-level, shared kernel) vs **VM** (hardware-level, guest OS); **image** vs **container**.
- **FaaS** (your functions) vs **BaaS** (provider's back-end services).
- **eMBB** (AR/VR) · **URLLC** (autonomous/telesurgery) · **mMTC** (massive IoT).
- **One-to-many** virtual sensor (middleware sets sampling) vs **derived** (combine different sensor types).

3. “If the question mentions X → think Y”

X	Y
“peak/average”, “utility premium”	$U = C/B$ vs P/A
“independent workloads”, “smoothing”	$C_v/\sqrt{n}$
“12 hours a day for 30 days”, “outage”	T = 360 h; $A = 1 - d/T$; credit band
“block size”, “replication”	⌈size/block⌉ × replicas
“R, W, N”, “eventual consistency”	Dynamo quorum $R + W > N$
“dirty pages”, “transmission rate”	pre-copy $r = PD/R$, geometric series
“processor speed, bandwidth, idle power”	offloading E_{saved}
“total facility power / IT power”	PUE
“unused cycles”, “single system image”	Grid
“sensitive and privileged instructions”	Popek–Goldberg (sensitive ⊆ privileged)
“binary translation”, “QEMU/Bochs”	Full virtualization
“modified guest OS”, “Xen”	Para-virtualization
“Intel VT / AMD-V”, “ring 0”	Hardware-assisted
“publish, find, bind”	SOA (IBM)
“heartbeat”, “primary chunk server”	GFS fault tolerance / append
“tablet”, “SSTable”	BigTable
“MD5 ring”, “virtual nodes”	Dynamo
“Dom0”, “co-resident”, “side channel”	Hey You, Get Off of My Cloud (EC2)
“IDRM”, “SoD”, “cyclic inheritance”	Collaborative SaaS security
“single hop”, “trusted resource-rich nearby”	Cloudlet
“remoteable”, “annotation”	MAUI
“DSM”, “unmodified binaries”	COMET
“Cisco”, “network edge”	Fog
“works without Internet”, “Dropbox”	Dew
“OpenFlow”	SDN (fog enabler)
“affinity rules”, “contract”	Partially coupled federation
“root cloud OS”, “hierarchical”	Multi-tier federation (tight)
“Pods”, “worker nodes”, “control plane”	Kubernetes
“layers + metadata”	Docker image
“docker save / load”	Ship image as tar
“event source”, “layers”	AWS Lambda

X	Y
“trigger = event + function”	Google Cloud Functions
“function.json”	Azure Functions
“ATS”, “PDU”, “grid for deadline jobs”	Sustainable cloud power manager
“DVFS”	Energy saving but switching delay hurts SLA
“Helen Gill”, “feedback loop”	CPS
“Tobler”	Geospatial cloud — near things more related
“iFogSim”, “area gateway”, “confirmatory module”	Cloud–fog health case (confirmatory in cloud)
“5-point smoothing”, “kNN”	IoHT activity detection

Your practice sets — 400 questions checked against the lectures

These are your practice files after checking each answer against the lecture transcripts: **8 sets · 400 questions**, each built from a **single lecture**. Tap an option to see right away whether it is correct, with the explanation and a link to the matching notes section. Your answers are saved in this browser, so **Show → Wrong** gives you a retry list. ⚠️ marks a question whose key follows a lecture slip or goes beyond the lecture; its note says what to pick in the exam. Options are shuffled because in your files ~90 % of the answers were B or C. Tick *original order* to match the files. These are **your practice questions, not NPTEL assignment questions**.

Your file numbers are not course weeks. Each file is the **first lecture of one week**: **w1** = Week 4 (L16), **w2** = Week 5 (L21), **w3** = Week 6 (L26), **w4** = Week 7 (L31), **w5** = Week 8 (L36), **w10** = Week 1 (L1), **w11** = Week 2 (L6), **w12** = Week 3 (L11).

Left out: 4 off-syllabus files. w6 (titled "Lecture 41: Cloud–Fog Computing Overview") is DFIG wind-generator control and its lab demo; **w7** (titled "Lecture 45: Cloud Migration I") is hybrid AC–DC microgrids (power management and control); **w8** (titled "Lecture 51: Dew Computing") is DC-microgrid energy management with an RTDS simulation (its own Q42 names the Smart Grid course); **w9** (titled "Lecture 56: Cloud Computing in 5G Era") is a power-hardware-in-the-loop test bed for a DC microgrid. Their headers name cloud lectures, but the questions come from the NPTEL *Smart Grid* course. Weeks 9–12 therefore have no practice set of yours; use Notes MCQs for those weeks.

Week 1 · L1 — Cloud Computing — Overview

your file w10 · 50 questions

Lecture 1: Cloud Computing - Overview · 📖 notes: 1.0 Course roadmap — what L1 previews (L1)

Q1. According to the lecture, what is the primary objective of this course on cloud computing? Easy

A. To give an overall picture of what cloud computing is, its components, and recent trends.
B. To provide a detailed guide on building a private cloud from scratch.
C. To train students to become certified cloud service provider administrators.
D. To focus exclusively on the security vulnerabilities within commercial cloud platforms.

Q2. The lecture cites the ACM's 2005 definition of 'computing'. What is the core idea of this definition? Medium

A. The design and construction of microprocessors.
B. The process of storing and retrieving data from a database.
C. Any goal-oriented activity that can be solved through computation.
D. The act of programming high-performance computers.

Q3. What is the primary purpose of a Service Level Agreement (SLA) as described in the lecture? Medium

A. A technical specification of the cloud's hardware.
B. An agreement between the service provider and consumer on the guarantee of services.
C. An internal document for the provider to manage resources.
D. A marketing document detailing the features of a cloud service.

Q4. A university needs 100% service availability for its online exam portal but only 90% for practice sessions. What is the direct implication for the cloud provider, based on the lecture's explanation? Hard

A. The provider will need to provision more redundant resources for the exam portal, leading to a higher cost.
B. The provider will charge the same amount for both services, as the underlying hardware is identical.
C. The provider will manage both with the same resource pool to improve efficiency, with the SLA being just a formality.
D. The provider will refuse the 100% availability request as it's impossible to guarantee.

Q5. What key economic question does the topic of 'Cloudonomics' address for an organization? Medium

A. How to build the most profitable cloud service.
B. How to calculate the electricity cost of running a data center.
C. What is the total market capitalization of the cloud industry?
D. Whether it is more economically beneficial to purchase (build) or hire (use cloud) computing resources.

Q6. From which perspective is 'resource management' primarily a concern, according to the lecture? Easy

A. The network internet service provider.
B. The cloud service provider (CSP).
C. The government regulatory body.
D. The end-user's application.

Q7. The lecture highlights a fundamental security principle by stating, 'you can... reinstall an application but you cannot reinstall your data.' What critical aspect of cloud usage does this emphasize? Hard

A. The high cost of software licenses compared to data storage.
B. The primary value and irreplaceability of the user's data.
C. The need for fast application deployment.
D. The importance of using the latest software versions.

Q8. The lecture mentions 'three nodes of a triangle' that are interlinked in the context of cloud security. What are these three concepts? Medium

A. IaaS, PaaS, and SaaS.
B. Trust, Reputation, and Risk.
C. Confidentiality, Integrity, and Availability.
D. Authentication, Authorization, and Accounting.

Q9. How does the lecture characterize the emergence of cloud computing in the history of technology? Easy

A. As a short-term trend that will be replaced by centralized computing.
B. As a purely economic model that rebrands old technology.
C. As an evolution that amalgamates and builds upon previous developments like distributed computing.
D. As a revolutionary technology with no connection to past paradigms.

Q10. Which of the following is NOT listed as a computing paradigm that contributed to the rise of cloud computing? Easy

A. Distributed Computing
B. Utility Computing
C. Quantum Computing
D. Grid Computing

Q11. According to the lecture, what is a key characteristic of the 'autonomous computational entities' in a distributed system? Medium

A. They all share a single, central memory pool.
B. Each entity has its own local memory.
C. They are all physically located in the same room.
D. They cannot communicate with each other.

Q12. The lecture contrasts distributed computing with the older mainframe-based centralized computing model. What is the main difference highlighted? Medium

A. Centralized systems are cheaper to build.
B. Distributed systems are slower but more secure.
C. Mainframes use terminals, while distributed systems do not have user interfaces.
D. Centralized systems have a single processing unit, while distributed systems have multiple autonomous processors.

Q13. What is the primary benefit of 'fault tolerance' in a distributed system? Medium

A. The system can continue to work even if some of its nodes fail.
B. The system consumes less power.
C. The system runs faster than a single computer.
D. The system is easier to program.

Q14. A bank wants to ensure its ATM network continues to operate even if a regional data center fails. Which property of distributed systems is most crucial for this requirement? Hard

A. Robustness and avoidance of a single point of failure.
B. Load Sharing
C. Performance
D. Ease of Expansion

Q15. Which of the following service models involves providing access to operating systems, storage, and networking resources over the internet? Medium

A. Platform as a Service (PaaS)
B. Infrastructure as a Service (IaaS)
C. Software as a Service (SaaS)
D. Anything as a Service (XaaS)

Q16. What does the lecturer use as an analogy to explain the 'buy vs. hire' decision in Cloudonomics? Medium

A. Subscribing to a magazine vs. buying books.
B. Cooking at home vs. eating at a restaurant.
C. Purchasing a car for daily commutes vs. hiring one for occasional long trips.
D. Renting a house vs. buying one.

Q17. What is described as a 'serious challenge' in resource management for a cloud provider? Hard

A. Optimally managing resources to avoid over-provisioning (waste) or under-provisioning (starvation).
B. Hiring qualified system administrators.
C. Paying for electricity and cooling.
D. Finding enough physical space for data centers.

Q18. The lecture uses Dropbox as a real-world example of what concept? Easy

A. A cloud security protocol
B. A distributed computing research project
C. Storage as a Service
D. Infrastructure as a Service (IaaS)

Q19. What is the stated motivation in the lecture for covering recent trends like 'Fog Computing'? Medium

A. To discourage students from using traditional cloud computing.
B. To fulfill a sponsorship requirement from tech companies.
C. To provide pointers for future research and projects for students.
D. Because they are part of the standard curriculum.

Q20. How do the 'autonomous computational entities' in a distributed system communicate with each other, according to the lecture? Easy

A. By directly accessing a shared hard drive.
B. They do not communicate, they work in isolation.
C. By message passing over a communication network.
D. Through a central coordinating server.

Q21. Which of these is mentioned as an example of a modern, ad hoc distributed network that is highly volatile? Medium

A. A corporate intranet.
B. The global Domain Name System (DNS).
C. A bank's ATM network.
D. Vehicular Ad Hoc Networks (VANETs).

Q22. A startup is launching a mobile game and anticipates a massive, but temporary, spike in users during the first week. Based on the lecture's Cloudonomics principles, what is their most financially sound strategy? Hard

A. Hire resources from a cloud provider to handle the spike and scale down afterward.
B. Limit the number of users who can sign up to match their existing server capacity.
C. Purchase and set up enough servers to handle the peak load, which they can use later.
D. Delay the launch until they can afford to build a large data center.

Q23. When a user moves their data from a personal laptop to a cloud storage service, what key responsibility shifts from the user to the provider? Medium

A. Creating the data in the first place.
B. Managing backups and redundancy to prevent data loss.
C. Organizing the data into folders.
D. Deciding which files are important.

Q24. The lecture's course outline mentions 'deployment models'. What does this term likely refer to in a cloud context? Medium

A. Whether a cloud is public, private, hybrid, or community.
B. The physical layout of servers in a data center.
C. The process of deploying an application to a server.
D. Methods for deploying software patches.

Q25. What is the main vulnerability of a purely centralized computing system like an old mainframe, as implied by the discussion on distributed systems? Medium

A. It is too slow for modern applications.
B. It cannot be connected to a network.
C. It is too expensive to program.
D. It has a single point of failure.

Q26. The course outline mentions that the cloud computing stack is 'moving towards a service oriented architecture (SOA)'. What does this imply about cloud services? Hard

A. The focus is shifting from software services to hardware services.
B. Services are designed as independent, interoperable components that can be combined.
C. Services are becoming less reliable and more complex.
D. All services are being consolidated into a single monolithic application.

Q27. What are the three broad types of computers (nodes) in a distributed system, as categorized in the lecture? Medium

A. Routers, Switches, and Hubs
B. Laptops, Desktops, and Mobile Phones
C. Mainframes, Supercomputers, and Minicomputers
D. Workstations, Server Systems, and Personal Assistance Devices

Q28. According to the lecture, what is a potential negative consequence for a cloud provider of over-provisioning resources? Medium

A. They will incur more cost for underutilized resources.
B. They will violate the terms of their SLAs.
C. Their service performance will degrade significantly.
D. They may run out of resources for new customers.

Q29. What is the significance of the statement that 'each node in the distributed system plays its partial role'? Medium

A. It implies that nodes in a distributed system are less powerful than centralized processors.
B. It indicates that users can only access part of the system at any given time.
C. It means nodes are unreliable and cannot be trusted with a full task.
D. It suggests that a complex problem is broken down and solved collaboratively by multiple nodes.

Q30. Vehicular Ad Hoc Networks (VANETs) are used as an example of a distributed system. Based on the lecture, which of the following is a plausible function for a VANET? Easy

A. Providing infotainment or safety-related information to drivers.
B. Processing a company's financial payroll.
C. Long-term archival of vehicle manufacturing data.
D. Hosting a global e-commerce website.

Q31. How does the lecture relate parallel computing to distributed systems? Medium

A. Parallel computing can be considered a subset of distributed systems, focused on high performance.
B. They are two completely unrelated fields.
C. Parallel computing is an older paradigm that was replaced by distributed systems.
D. Distributed systems are a subset of parallel computing.

Q32. What concern does 'Identity and Access Management' (IAM) address in a cloud security context? Medium

A. Encrypting all data at rest.
B. Managing the reputation of the cloud provider.
C. Ensuring the correct entities (users, services) can access the appropriate resources and nothing more.
D. Protecting data from physical theft.

Q33. The initial lectures of the course are planned to focus primarily on which topic? Easy

A. Hands-on lab sessions with cloud simulators.
B. The economics of building a data center.
C. Advanced security protocols.
D. An introduction to cloud computing, including NIST models and characteristics.

Q34. The concept of 'Anything as a Service' or 'XaaS' implies what about the cloud model? Medium

A. Only hardware can be offered as a service.
B. Cloud computing is limited to only three specific services.
C. The service model is flexible and can be extended to offer potentially any capability as a service.
D. The term 'service' is just a marketing gimmick with no real meaning.

Q35. The lecture describes distributed systems as 'easy to expand'. What does this mean in a practical sense? Easy

A. It is straightforward to add more computers (nodes) to the system to increase its capacity.
B. Expanding the system requires no additional cost.
C. The physical size of the computers can be expanded.
D. The software for distributed systems is simple to write.

Q36. From a consumer's point of view, what is the main purpose of an SLA for a critical application? Medium

A. To get access to the provider's source code.
B. To understand the technical architecture of the cloud.
C. To get the lowest possible price.
D. To have a guarantee of minimal service levels, such as uptime.

Q37. What is the primary difference between resource sharing and load sharing as implied in the lecture? Hard

A. There is no difference; the terms are used interchangeably.
B. Resource sharing is a benefit for the consumer, while load sharing is a benefit for the provider.
C. Resource sharing is about making hardware available, while load sharing is about distributing computational work across that hardware.
D. Resource sharing applies to hardware, while load sharing applies to software.

Q38. According to the lecture, which of the following is a primary motivation for the development of distributed systems? Hard

A. The decreasing cost of single, powerful mainframe computers.
B. The need for robustness and the elimination of single points of failure for critical applications.
C. The lack of networking technology.
D. The desire to centralize all computing resources for easier management.

Q39. The lecture mentions that a cloud provider's resource management must handle several types of resources. Which of the following was NOT mentioned as a typical resource? Medium

A. Processor or CPU
B. Network connectivity
C. Personnel and human resources
D. Working memory or RAM

Q40. Who is the primary intended audience for the lecture module on 'Recent Trends in Cloud Computing'? Easy

A. Students and researchers looking for future project or research topics.
B. Novice users who are new to computing.
C. System administrators focused on current technology.
D. Business managers and executives.

Q41. The lecture cites 'role base access control mechanism' as an example of what broader security concept? Medium

A. Reputation management
B. Access control
C. Network intrusion detection
D. Data encryption

Q42. From a cloud service provider's (CSP) perspective, what is a primary concern related to an SLA they offer? Hard

A. Understanding the resource requirements needed to meet the guaranteed service level.
B. Ensuring the SLA is written in complex legal language.
C. Making sure the customer never reads the SLA.
D. How to market the SLA to the maximum number of customers.

Q43. Why is data management considered 'tricky' in a cloud environment, according to the lecture? Medium

A. Because data transfer speeds are too slow.
B. Because cloud providers prohibit storing large amounts of data.
C. Because data formats are constantly changing.
D. Because traditional database management systems may not be suitable for the scale and service model of the cloud.

Q44. The lecture states that cloud computing 'is not a a suddenly new stuff which came into the play.' What does this imply about its technological origins? Medium

A. Cloud computing is just a new name for the internet.
B. Its core technologies are built upon decades of development in fields like distributed and utility computing.
C. It is a marketing term with no real technology behind it.
D. The concept was invented in the 1960s but was only recently implemented.

Q45. In the context of distributed systems, what does 'load sharing' help to achieve? Medium

A. Ensuring that all nodes are turned on at the same time.
B. Preventing any single node from being overwhelmed by distributing tasks among multiple nodes.
C. Sharing software licenses among different nodes.
D. Reducing the total amount of work a system needs to do.

Q46. The lecture mentions that the course will cover 'case studies'. What kind of clouds will these case studies examine? Easy

A. Only hypothetical and theoretical cloud models.
B. Both open source and commercial cloud platforms.
C. Exclusively clouds used by government and military.
D. Only outdated cloud technologies for historical context.

Q47. What is the fundamental security risk highlighted when you use a cloud service? Medium

A. The cloud provider might go out of business.
B. Your data and computation are located in someone else's domain.
C. Your internet connection might be slow.
D. Your own computer might get a virus.

Q48. What is the primary characteristic of a 'server system' node within a distributed system? Easy

A. It is a handheld device used for mobile access.
B. It is a networking device that only routes traffic.
C. It is used by an end-user to perform their own computations.
D. It is a computer that primarily provides resources and services to other nodes.

Q49. According to the lecture, what is the relationship between the availability guarantee in an SLA and the price of the service? Hard

A. Higher availability guarantees require more resources from the provider, leading to a higher price.
B. Higher availability guarantees generally lead to a lower price.
C. The price is fixed, but the availability changes based on system load.
D. Price is unrelated to the availability guarantee.

Q50. The lecture identifies 'Trust, Reputation, and Risk' as a critical triad in cloud security. What does this triad primarily govern? Medium

A. The technical specifications for data encryption.
B. The physical security measures of data centers.
C. The legal framework for international data transfer.
D. The user's confidence in a cloud provider.

Answer key — Week 1 · L1 (your file w10)

Q1 A · To give an overall picture of what cloud computing is, its components, and recent trends. — The lecturer explicitly states (00:32-00:40) that the goal of the series of lectures is to 'give an overall picture of what cloud computing is and what are its major components and what are the recent trends'.

Q2 C · Any goal-oriented activity that can be solved through computation. — At 01:12, the lecturer defines computing, based on the ACM 2005 curricula, as a 'mean to solve any goal oriented activity', which encompasses a broad range of hardware, software, and systems.

Q3 B · An agreement between the service provider and consumer on the guarantee of services. — The lecture explains at 04:08-04:18 that an SLA is an 'agreement between the service provider and the consumer' that defines guarantees for aspects like availability.

Q4 A · The provider will need to provision more redundant resources for the exam portal, leading to a higher cost. — The lecture (04:24-04:57) uses this exact scenario to explain that higher availability requirements (like for an exam) necessitate more resource tooling and redundancy from the provider, who will then charge based on this higher level of service.

Q5 D · Whether it is more economically beneficial to purchase (build) or hire (use cloud) computing resources. — The lecturer explains Cloudonomics (05:13-05:59) by posing the question of whether it's always beneficial to use the cloud and uses the analogy of purchasing a car versus hiring one to illustrate the 'purchasing or hiring a resources' decision.

Q6 B · The cloud service provider (CSP). — At 06:26, the lecture clearly states that resource management is 'more in the service provider end' or 'cloud service provider' end, focusing on how they manage resources to serve many people.

Q7 B · The primary value and irreplaceability of the user's data. — This statement (10:23-10:52) is used to underscore that while applications and operating systems are replaceable, the data created by the user is unique and its loss is permanent, making data security and backup paramount.

Q8 B · Trust, Reputation, and Risk. — At 12:27, the lecturer explicitly states, 'this trust reputation risk goes somewhere what we say three nodes of a triangle so they are interlinked'.

Q9 C · As an evolution that amalgamates and builds upon previous developments like distributed computing. — The lecturer states (15:26-15:39) that cloud computing 'is not a suddenly new stuff' but has 'evolved' and was helped by 'different other development which is already in place'.

Q10 C · Quantum Computing — The lecture (16:08-16:35) lists distributed, grid, cluster, and utility computing as predecessors or related paradigms. Quantum computing is not mentioned in this context.

Q11 B · Each entity has its own local memory. — The definition provided around 19:02 states that a distributed system has 'several autonomous computational entities each of has its own local memory'.

Q12 D · Centralized systems have a single processing unit, while distributed systems have multiple autonomous processors. — The lecture describes centralized computing (17:10-17:36) as having a 'logically single processing thing' (like a mainframe), whereas distributed computing (18:01-18:13) involves 'different systems distributed over a geographical space'.

Q13 A · The system can continue to work even if some of its nodes fail. — As explained at 22:11, fault tolerance means that 'even with some node failures it works faithfully', perhaps at a lower performance, unlike a centralized system that would go down completely.

Q14 A · Robustness and avoidance of a single point of failure. — The lecture explains (24:08-24:18) that a key reason for using distributed systems is to ensure robustness and that there is 'no single point failure', which is critical for applications like a banking network.

Q15 B · Infrastructure as a Service (IaaS) — The lecture mentions IaaS at 03:08. While not defining it in depth, IaaS is the model that provides fundamental computing infrastructure like servers, storage, and networking.

Q16 C · Purchasing a car for daily commutes vs. hiring one for occasional long trips. — At 05:26, the lecturer uses the specific analogy of commuting 20km daily (justifying a car purchase) versus commuting 100km once a month (making hiring more economical) to explain the economic decisions behind cloud usage.

Q17 A · Optimally managing resources to avoid over-provisioning (waste) or under-provisioning (starvation). — The lecture (08:14-08:33) explicitly describes the challenge of not having 'huge volume of resources without utilization' (over-provisioning) or a situation of 'starving for my store' (under-provisioning), calling the optimal management of resources a 'serious challenge'.

Q18 C · Storage as a Service — Around 09:32, when discussing data management and 'storage as a service', the lecturer says, 'one of the popular thing may be the drop box'.

Q19 C · To provide pointers for future research and projects for students. — The lecturer explains (13:52) that discussing recent trends can give students 'interested in research or even some of projects in pg level ug level... a pointer that what are the different aspects'.

Q20 C · By message passing over a communication network. — The definition cited around 19:16 explicitly states that 'The entities communicate with each other by message passing over a backbone communication network'.

Q21 D · Vehicular Ad Hoc Networks (VANETs). — Around 20:23-20:40, the lecturer discusses ad hoc establishments and gives 'vehicular ad hoc networks' as a specific example where smart vehicles form a temporary, volatile network.

Q22 A · Hire resources from a cloud provider to handle the spike and scale down afterward. — This scenario is analogous to the 'hiring a car for an infrequent hundred kilometer trip' (05:40). For a temporary, high-demand situation, hiring (using the cloud) is more economical than purchasing permanent infrastructure.

Q23 B · Managing backups and redundancy to prevent data loss. — The lecture contrasts (10:57-11:10) having data on a personal system where it is the user's 'responsibility to take backups', with storing data in 'the others place' (the cloud), implying this responsibility is transferred to the provider.

Q24 A · Whether a cloud is public, private, hybrid, or community. — In standard cloud terminology, mentioned at 03:18, 'deployment models' refer to the different ways a cloud environment can be set up and made accessible, such as public (for anyone), private (for a single organization), or a hybrid of the two.

Q25 D · It has a single point of failure. — The lecture contrasts the fault tolerance of distributed systems with centralized ones by stating, 'had it been a centralized system so if down the whole thing is down' (22:34), highlighting its nature as a single point of failure.

Q26 B · Services are designed as independent, interoperable components that can be combined. — Service Oriented Architecture (SOA), mentioned at 03:01, is a design principle where functionality is provided as a set of distinct services that can be used by different applications, promoting reusability and flexibility, which is a core concept of the cloud's IaaS, PaaS, SaaS model.

Q27 D · Workstations, Server Systems, and Personal Assistance Devices — From 21:02 to 21:33, the lecturer explicitly lists and briefly defines these three categories: workstations for end-users, server systems providing resources, and personal assistance devices like handhelds.

Q28 A · They will incur more cost for underutilized resources. — The lecturer explains (08:14-08:20) that if a provider needs to 'manage huge volume of resources without utilization then i have to incur what we say more cost'.

Q29 D · It suggests that a complex problem is broken down and solved collaboratively by multiple nodes. — This statement at 22:56 highlights the collaborative nature of distributed computing. No single node is responsible for the entire task; instead, they work together, each contributing to the overall solution.

Q30 A · Providing infotainment or safety-related information to drivers. — When discussing VANETs (20:55), the lecturer explicitly mentions that they can execute functions like 'safety related things' or

'infotainment related type of things'.

Q31 A · Parallel computing can be considered a subset of distributed systems, focused on high performance. — At 23:38, the lecturer states that 'parallel computing can be considered as a subset of a distributed systems where i can have higher performance'.

Q32 C · Ensuring the correct entities (users, services) can access the appropriate resources and nothing more. — IAM, mentioned from 11:30 to 11:41, is about managing the 'particular identity' of collaborating parties and controlling their 'access' to resources, which is a fundamental part of securing a multi-user environment.

Q33 D · An introduction to cloud computing, including NIST models and characteristics. — The lecturer outlines the initial plan at 02:30, stating 'initial lectures we will have a more... introduction to cloud computing which has gives overview what that nist models says what are the typical properties characteristics'.

Q34 C · The service model is flexible and can be extended to offer potentially any capability as a service. — The lecturer mentions IaaS, PaaS, and SaaS and then adds 'or anything as a service' (03:14), suggesting that the 'as a service' model is a broader paradigm for delivering capabilities, not just a fixed list.

Q35 A · It is straightforward to add more computers (nodes) to the system to increase its capacity. — As explained at 23:23, being 'easy to expand' means 'I can had ah distributed system ah more system into the network as and when I have it', referring to the scalability of adding more nodes.

Q36 D · To have a guarantee of minimal service levels, such as uptime. — The lecture states (03:45) that from the 'consumer end i i would like to have what is the guarantee of minimal services', which is the core function of an SLA for the user.

Q37 C · Resource sharing is about making hardware available, while load sharing is about distributing computational work across that hardware. — The lecture lists them as separate points (23:01-23:13). Resource sharing is the general concept of sharing resources. Load sharing (or load balancing) is the specific action of distributing the 'load' or workload among the available resources to optimize performance and prevent overload.

Q38 B · The need for robustness and the elimination of single points of failure for critical applications. — Towards the end of the lecture (24:08-24:33), the speaker emphasizes that the need for robustness and avoiding 'any single point failure' for 'mission critical things' is a primary motivation for using distributed systems.

Q39 C · Personnel and human resources — Around 07:42, the lecturer lists 'processor or the cpu', 'working memory or popularly the ram', 'hard disk', and 'network connectivity' as typical computing resources to be managed. Personnel are part of running the business but not a computational resource offered to customers.

Q40 A · Students and researchers looking for future project or research topics. — The lecture explicitly states (13:52) that this section is for 'those who are interested in research or even some of projects in pg level ug level' so they can get a 'pointer' on future trends.

Q41 B · Access control — The lecturer mentions 'access control' as a key security issue and then immediately provides 'role base access control mechanism' as a specific example of how it can be implemented (11:46-11:54).

Q42 A · Understanding the resource requirements needed to meet the guaranteed service level. — The lecture explains the provider's view at 03:51, stating the CSP wants to understand 'what are the resource requirement at the back end to surf so much computers' to meet the guarantees, which also ties into their profitability.

Q43 D · Because traditional database management systems may not be suitable for the scale and service model of the cloud. — The lecturer questions (09:14) whether the 'conventional way of approaching normal data or data base management system' is 'still good for cloud', implying that the scale and nature of cloud services present new challenges for data management.

Q44 B · Its core technologies are built upon decades of development in fields like distributed and utility computing. — This statement (15:33) introduces the idea that cloud computing is an evolutionary step, leveraging and combining concepts from pre-existing paradigms like distributed, grid, and utility computing, which are discussed immediately after.

Q45 B · Preventing any single node from being overwhelmed by distributing tasks among multiple nodes. — Load sharing, also known as load balancing (23:07-23:17), is the process of distributing the workload across the system's nodes to ensure efficient use of resources and prevent any one part from becoming a bottleneck.

Q46 B · Both open source and commercial cloud platforms. — At 12:55, the lecturer states the plan is to 'look at some of case studies... on open source comma and on commercial cloud'.

Q47 B · Your data and computation are located in someone else's domain. — The security section begins (09:51) by framing the problem: 'your data is in some other place you are computing in in some other others domain', which is the core issue that gives rise to various security concerns.

Q48 D · It is a computer that primarily provides resources and services to other nodes. — The lecture defines server systems (21:21) as 'computers which provide resources and services', distinguishing them from workstations used by end-users.

Q49 A · Higher availability guarantees require more resources from the provider, leading to a higher price. — The lecturer explicitly connects availability to cost (04:45-04:57), stating that 'based on the availability the resource tooling or resource management will be done... and the provider will charge based on the type of its resources type of availability etcetera'.

Q50 D · The user's confidence in a cloud provider. — The lecture explicitly states that 'Trust, Reputation, and Risk' is 'A critical triad (12:27) that governs a user's confidence in a provider, addressing risks like data loss or unauthorized access.'

Week 2 · L6 — Deployment Models your file w11 · 50 questions

Lecture 6: Cloud Computing: Architecture – Deployment Models · 📖 notes: 2.1 Deployment models in detail (L6)

Q1. According to the lecture, which of the following is a primary characteristic of a public cloud deployment model? Easy

A. The infrastructure is provisioned for open use by the general public.
B. The infrastructure is provisioned for exclusive use by a single organization.
C. The infrastructure is located on the user's premises for maximum control.
D. The infrastructure is shared by a specific community with common concerns.

Q2. A financial services company is concerned about sharing physical hardware with competitors. Which deployment model risk, mentioned in the lecture, is their primary concern? Medium

A. Limited elasticity
B. Network dependency
C. High upfront cost
D. Risk from multi-tenancy

Q3. A small startup with a limited budget wants to deploy its application without a large initial investment. Which feature of the public cloud would be most appealing to them? Easy

A. Negotiable service level agreements (SLAs)
B. Potentially strong security from external threats
C. Complete control over data location
D. Low upfront cost

Q4. What does the lecture mean by 'workload locations are hidden' in a public cloud context? Medium

A. Workloads are encrypted, so their content is hidden from the provider.
B. The cloud provider intentionally moves data to insecure locations.
C. The user does not know the specific physical server or location where their VM or data resides.
D. Only the user has access to the physical address of the data center.

Q5. Which of the following is NOT listed as an example of a public cloud provider in the lecture? Easy

A. Google App Engine
B. Microsoft Azure
C. Amazon EC2
D. Eucalyptus

Q6. A university decides to use a public cloud for its student labs. According to the lecture, what is the most critical dependency they must consider? Medium

A. The availability and quality of their network connectivity.
B. The cost of migrating data from legacy systems.
C. The need to purchase and install all hardware.
D. The physical security of the on-premises data center.

Q7. Which statement best describes a private cloud as defined in the lecture? Easy

A. It must physically exist on the organization's premises.
B. It is always owned, managed, and operated by the organization itself.
C. It is provisioned for exclusive use by a single organization.
D. It offers unlimited resources with no upfront costs.

Q8. The lecture challenges a common belief about private clouds. What is this contrary belief? Medium

A. That private clouds require skilled IT staff to maintain.
B. That private clouds can exist off-premises and be managed by a third party.
C. That private clouds also suffer from multi-tenancy risks.
D. That private clouds can be more expensive than public clouds.

Q9. What is the key difference between an 'on-site private cloud' and an 'outsourced private cloud'? Easy

A. On-site clouds are for single users, while outsourced are for multiple organizations.
B. On-site clouds use public internet, while outsourced clouds use dedicated lines.
C. The physical location of the infrastructure (customer's premises vs. hosting company).
D. On-site clouds have higher elasticity than outsourced clouds.

Q10. An organization is building an on-site private cloud. What is a major financial consideration they will face, according to the lecture? Medium

A. Significant to high upfront costs for migration and setup.
B. Pay-as-you-go pricing for resource consumption.
C. No cost, as open-source software like OpenStack is free.
D. Low upfront cost but high variable operational costs.

Q11. In an 'outsourced private cloud' setup, what does the lecture identify as a critical component for ensuring security? Hard

A. Having a single, unified security perimeter controlled only by the subscriber.
B. Keeping the workload locations hidden from the subscriber.
C. The strength of both the subscriber's and provider's security perimeters and the protected communication link.
D. Relying solely on the default SLA provided by the hosting company.

Q12. Compared to an on-site private cloud, what is a key advantage of an outsourced private cloud regarding resources? Medium

A. It has a fixed, limited computing capacity.
B. Extensive resource availability and better elasticity.
C. It eliminates all risks from multi-tenancy.
D. The subscriber has direct physical access to the servers.

Q13. The lecture mentions IIT Kharagpur's 'Meghamala' as an example of what? Medium

A. A community cloud for Indian universities.
B. A hybrid cloud connecting to Amazon EC2.
C. An on-site private cloud for research purposes.
D. A public cloud service available to anyone.

Q14. What defines a community cloud according to the lecture? Easy

A. It is for exclusive use by a specific community of organizations with shared concerns.
B. It is open to the general public but with stricter access rules.
C. It is a combination of public and private clouds for a single company.
D. It is a private cloud that has been outsourced to a third party.

Q15. A group of government agencies decides to create a cloud to share data related to environmental policy. This is an example of which deployment model? Medium

A. Public Cloud
B. Hybrid Cloud
C. Community Cloud
D. Private Cloud

Q16. Why can the access policy of a community cloud be 'pretty complex'? Hard

A. Because multiple organizations are involved, requiring rules to prevent information leakage between communities.
B. Because resources are unlimited and hard to track.
C. Because it must integrate with public cloud policies.
D. Because it always resides on the premises of every member organization.

Q17. What access control techniques are mentioned as being useful for managing complex community clouds? Medium

A. Firewall and Intrusion Detection Systems
B. Multi-Factor Authentication and Biometrics
C. Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC)
D. Username and Password Authentication

Q18. Which of the following best describes a hybrid cloud? Easy

A. A cloud exclusively for a community of hybrid car manufacturers.
B. A private cloud that is managed by both the organization and a third party.
C. A composition of two or more distinct cloud infrastructures (private, public, or community).
D. A cloud that can be deployed either on-premises or off-premises.

Q19. What is a key technological requirement for a functional hybrid cloud, as stated in the lecture? Hard

A. A standardized or proprietary technology that enables data and application portability.
B. A single, centralized management console for all clouds.
C. All constituent clouds must be from the same provider.
D. The use of only open-source software across all environments.

Q20. An e-commerce site uses its on-site private cloud for processing payments but shifts its public-facing website to a public cloud to handle traffic spikes. This strategy is an example of using which deployment model? Medium

A. Community Cloud
B. Public Cloud only
C. Hybrid Cloud
D. Outsourced Private Cloud

Q21. What is described as a major challenge or disadvantage of implementing a hybrid cloud? Medium

A. It is always more expensive than a pure public cloud solution.
B. It can be extremely complex to architect and manage.
C. It has very limited resource availability.
D. It offers the weakest security of all deployment models.

Q22. The lecture discusses the dynamic nature of hybrid clouds. What does this refer to? Hard

A. The fact that performance and reliability can change unexpectedly.
B. The constant updates and patches required to maintain the cloud.
C. The possibility that constituent clouds may join or leave the hybrid environment over time.
D. The ability to dynamically scale resources up and down.

Q23. An organization suddenly needs more computing power than its on-site private cloud can provide. The lecture suggests they could use a public cloud for a short period. This practice is often referred to as: Medium ⚠

A. Cloud bursting
B. Cloud migration
C. Cloud sourcing
D. Multi-tenancy

Q24. When choosing a deployment model, what does the lecture suggest is the ultimate deciding factor? Medium

A. The organization's specific requirements, including business needs and client demands.
B. The popularity of the cloud provider.
C. The upfront cost, as it is always the most important consideration.
D. The availability of skilled IT staff.

Q25. A data storage provider serves clients in the defense sector. Why might these clients compel the provider to use a private cloud instead of a public one? Hard

A. Because public clouds are more expensive for data storage.
B. Because public clouds lack the elasticity needed for defense applications.
C. Because public clouds do not offer data portability.
D. Because the clients have mission-critical security requirements and want assurance that data is not on shared infrastructure.

Q26. Which deployment model is characterized by restrictive, non-negotiable Service Level Agreements (SLAs) for most users? Medium

A. On-site Private Cloud
B. Public Cloud
C. Outsourced Private Cloud
D. Community Cloud

Q27. Which issue is common to ALL four deployment models (Public, Private, Community, Hybrid) as discussed in the lecture? Hard

A. High upfront costs
B. Network dependency
C. Risk from multi-tenancy with competitors
D. Unlimited resource availability

Q28. What is the primary difference in resource limitation between an on-site private cloud and a public cloud? Medium

A. Both have fixed resources, but the private cloud is more secure.
B. The public cloud has a fixed capacity, while the on-site private cloud is theoretically infinite.
C. Both have unlimited resources, but the public cloud is cheaper.
D. The on-site private cloud has fixed capacity, while the public cloud offers the 'illusion of unlimited resources'.

Q29. Which of the following scenarios describes an outsourced community cloud? Hard

A. A single company pays a hosting company to run its internal HR system.
B. A university uses Amazon EC2 for its public website.
C. Several non-profits pay a third-party provider to host a shared application platform in the provider's data center.
D. Several hospitals build and manage a shared data center on one hospital's campus.

Q30. The lecture states that in a public cloud, 'there can be some attackers, hackers, etc.' among the clients. What is the implication of this? Medium

A. The cloud provider is responsible for all security breaches.
B. Users must implement their own physical security for the data centers.
C. Public clouds are illegal for storing sensitive data.
D. The cloud serves a diverse and not always trustworthy pool of clients.

Q31. Why might an organization still need IT skills even when using an on-site private cloud? Easy

A. Because the organization is responsible for maintaining its own cloud infrastructure.
B. To negotiate SLAs with themselves.
C. Because private clouds do not support automation.
D. To manage the relationship with the public cloud provider.

Q32. What is the security perimeter for an on-site private cloud? Medium

A. It extends around both the subscriber's on-site resources and the private cloud resources.
B. It extends around the cloud provider's global network.
C. It is defined by the public internet.
D. There is no security perimeter; it is completely open internally.

Q33. In a hybrid cloud model, an application runs partly on a private cloud and partly on a public cloud. What specific 'portability' issue is mentioned in the lecture? Hard

A. The physical difficulty of moving servers between data centers.
B. The problem of porting a phone number to the cloud.
C. The legal challenges of moving data across national borders.
D. The need to ensure both data and the application itself can work across the different environments.

Q34. Which statement accurately reflects the lecture's view on control and visibility in a public cloud? Medium

A. Users have no control, but the provider guarantees 100% security.
B. Users have limited visibility and control, relying heavily on the Service Level Agreement (SLA).
C. Users have complete visibility and control over their data's physical location.
D. Visibility is limited, but users can install their own hardware for more control.

Q35. According to the lecture, how can an organization manage workloads with different security requirements? Medium

A. By placing all workloads on the most secure model, the on-site private cloud.
B. By encrypting everything and using only a public cloud to save costs.
C. By using a hybrid model to classify and place sensitive workloads on a private cloud and less sensitive ones on a public cloud.
D. By creating multiple community clouds, one for each security level.

Q36. Which of these is NOT a characteristic of an on-site private cloud mentioned in the lecture? Medium

A. Theoretically infinite elasticity
B. Significant upfront cost
C. Requires subscriber IT skills
D. Limited resources/fixed capacity

Q37. What is a potential downside of the 'hidden workload location' feature in a public cloud? Hard

A. It improves performance by abstracting the hardware.
B. It may conflict with legal or policy matters regarding data residency and security.
C. It makes it easier to comply with data sovereignty laws.
D. It reduces the need for network connectivity.

Q38. What is the relationship between an organization and a community in a community cloud model? Medium

A. An organization can be a part of more than one community.
B. The community must be owned and operated by a single lead organization.
C. Organizations within a community cannot communicate with each other.
D. An organization can only belong to one community at a time.

Q39. The lecture mentions that public cloud infrastructure exists 'in the premise of the cloud provider'. What is the direct implication of this? Medium

A. The user organization has no physical control over the computing and storage infrastructure.
B. The user organization must send its IT staff to the provider's location for maintenance.
C. The provider's premises must be located in the same country as the user.
D. The user organization is responsible for the physical security of the provider's premises.

Q40. How does an outsourced private cloud offer 'extensive resource availability' compared to an on-site one? Medium

A. The outsourced cloud uses more efficient virtualization technology.
B. The on-site cloud is intentionally limited to save on electricity costs.
C. The resources are shared with the public, making the pool larger.
D. The hosting provider has a large backbone of resources that can be provisioned on request.

Q41. Which two deployment models can exist either on-premises or off-premises (outsourced)? Hard

A. Public and Hybrid
B. Public and Private
C. Private and Community
D. Community and Hybrid

Q42. An organization is considering a community cloud. What is a primary financial difference between building it on-premises versus outsourcing it? Hard

A. The on-premises model has no upfront costs, while the outsourced model is expensive.
B. Both models have identical costs, with the only difference being location.
C. The outsourced model is pay-as-you-go, while the on-premises model is a fixed monthly fee.
D. The outsourced model has a 'modest to significant' upfront cost, which is likely less than the 'high variable' cost of an on-premises build.

Q43. What inherent risk of a public cloud is also present within an on-site private cloud, according to the lecture? Hard

A. Risk from multi-tenancy among different internal business units.
B. Restrictive service level agreements.
C. Threat from external state-sponsored hackers.
D. The risk of co-residency with a business competitor.

Q44. A key challenge of a hybrid cloud is that an application 'needs to run over a combination of public and private cloud.' What is the main difficulty here? Medium

A. Ensuring the application runs seamlessly despite a potentially very complex underlying architecture.
B. Training users to access two different systems.
C. Finding a public cloud that is compatible with any private cloud.
D. Paying for two different cloud bills at the same time.

Q45. Which of these is presented as a benefit of a public cloud? Easy

A. Guaranteed data residency in a specific country.
B. Illusion of unlimited resource availability and elasticity.
C. Potentially strong security from external threats.
D. Complete control and visibility over the infrastructure.

Q46. For which deployment model is the subscriber most likely to have full control over the resources and security perimeter? Medium

A. On-site Private Cloud
B. Public Cloud
C. Outsourced Community Cloud
D. Hybrid Cloud

Q47. The lecture discusses a scenario where an organization might classify its applications and data into different categories. What is the purpose of this classification? Medium

A. To determine which data to delete to save space.
B. To decide which applications can be retired.
C. To determine which workloads can go to a public cloud, private cloud, or community cloud.
D. To calculate the total cost of ownership for the IT department.

Q48. What is the primary motivation for forming a community cloud? Easy

A. To create a cloud that is managed entirely by a third-party provider.
B. To serve a group of organizations that have shared concerns, missions, or security requirements.
C. To get the lowest possible cost by opening the infrastructure to the public.
D. To experiment with cloud technology before building a private cloud.

Q49. If a company uses VMware vCloud to bridge its on-premises data center with a public cloud provider, which deployment model is it implementing? Medium

A. Private Cloud
B. Public Cloud
C. Hybrid Cloud
D. Community Cloud

Q50. According to the lecture, what is a significant performance dependency for public cloud users? Easy

A. The availability of specialized hardware within the cloud provider's data center.
B. The speed and reliability of the public internet connection.
C. The organization's internal network infrastructure.
D. The number of other tenants sharing the same physical server.

Answer key — Week 2 · L6 (your file w11)

Q1 A · The infrastructure is provisioned for open use by the general public. — The lecture explicitly states that in a public cloud, the infrastructure is provisioned for open use by the general public, enterprises, and anyone who can pay to use the services (01:44).

Q2 D · Risk from multi-tenancy — The lecture defines the risk of multi-tenancy as the possibility that a workload may be co-resident with the workload of a competitor or adversary on the same physical machine (04:45). This is a key risk in public clouds.

Q3 D · Low upfront cost — The lecture highlights that a major advantage of the public cloud is the very low upfront cost required to migrate and start using services, as opposed to the significant investment needed for a private cloud (06:34).

Q4 C · The user does not know the specific physical server or location where their VM or data resides. — The lecture explains that in a public cloud, users don't know where their virtual machine is or on which server their data is residing, making the physical location hidden (03:20).

Q5 D · Eucalyptus — The lecture lists Microsoft Azure, Amazon EC2, and Google App Engine as popular public clouds (01:24). Eucalyptus is mentioned later as an example of a technology used for private clouds (08:43).

Q6 A · The availability and quality of their network connectivity. — The lecture uses the example of IIT Kharagpur considering a public cloud for its labs and states that the first dependency is the network; connectivity must be always available and up to a certain standard (05:01-05:26).

Q7 C · It is provisioned for exclusive use by a single organization. — The lecture's core definition for a private cloud is that the infrastructure is provisioned for the exclusive use of a single organization, which may comprise multiple consumer or business units (07:52).

Q8 B · That private clouds can exist off-premises and be managed by a third party. — The lecture explicitly states, 'contrary to popular belief, private cloud may exist off premises and can be managed by third party' (09:06), introducing the concept of the outsourced private cloud.

Q9 C · The physical location of the infrastructure (customer's premises vs. hosting company). — The lecture distinguishes between the two scenarios: 'on-site private cloud' applies to clouds on the customer's premises, while 'outsourced private cloud' applies when the server side is outsourced to a hosting company (09:30-10:00).

Q10 A · Significant to high upfront costs for migration and setup. — The lecture points out that for an on-site private cloud, there is a 'significant to high upfront cost to migrate into the cloud' due to the need to purchase, install, and maintain the infrastructure (12:33).

Q11 C · The strength of both the subscriber's and provider's security perimeters and the protected communication link. — The lecture explains that an outsourced private cloud has two security perimeters and the overall security depends on the strength of both perimeters and the secure communication channel connecting them (14:00-14:54).

Q12 B · Extensive resource availability and better elasticity. — The lecture contrasts the 'limited resources' of an on-site private cloud (12:49) with the 'extensive resource availability' of an outsourced private cloud, which is considered an advantage (16:18).

Q13 C · An on-site private cloud for research purposes. — The lecturer cites 'Meghamala' as a private cloud developed by IIT Kharagpur for research, which exists within their network premise, making it a clear example of an on-site private cloud (12:13).

Q14 A · It is for exclusive use by a specific community of organizations with shared concerns. — The lecture defines a community cloud as an infrastructure provisioned for the exclusive use of a specific community of consumers from organizations that have shared concerns like mission, security requirements, or policy (17:09).

Q15 C · Community Cloud — This scenario fits the definition of a community cloud, where multiple organizations with a shared concern (environmental policy, compliance) collaborate and share a cloud infrastructure (17:09).

Q16 A · Because multiple organizations are involved, requiring rules to prevent information leakage between communities. — The lecture notes that access policies can be complex because an organization might be part of multiple communities, and rules are needed to restrict information flow, for instance, preventing information from being passed from one community to another (19:18).

Q17 C · Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) — To handle the complex policies in community clouds, the lecture specifically mentions policy specification techniques like Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) (19:31).

Q18 C · A composition of two or more distinct cloud infrastructures (private, public, or community). — The lecture defines a hybrid cloud as an infrastructure that is a composition of two or more distinct cloud infrastructures, such as private, community, or public clouds (22:32).

Q19 A · A standardized or proprietary technology that enables data and application portability. — The lecture emphasizes that the different clouds in a hybrid model are 'bounded together in a standardize or proprietary technology enables data and application to be portable'. This portability is crucial (24:33).

Q20 C · Hybrid Cloud — This scenario demonstrates a key use case for a hybrid cloud: keeping sensitive operations (payments) on a private cloud while using the elasticity of a public cloud for less critical, high-volume workloads (website traffic). This combines two deployment models (22:58).

Q21 B · It can be extremely complex to architect and manage. — The lecture repeatedly warns that a hybrid cloud can be 'extremely complex', especially in making applications run seamlessly across different environments and managing the joining/leaving of constituent clouds (26:01, 27:40).

Q22 C · The possibility that constituent clouds may join or leave the hybrid environment over time. — A significant complexity of hybrid clouds is that they may change over time, with constituent public or community clouds joining or leaving based on subscriptions or business needs, making the overall process difficult to handle (26:35).

Q23 A · Cloud bursting — The scenario described (24:12), where a private cloud's capacity is temporarily augmented by 'bursting' into a public cloud to handle a sudden increase in demand, is a classic use case for hybrid clouds.

⚠ The lecture describes the idea (using a public cloud for the overflow) without naming it; 'cloud bursting' is the standard term.

Q24 A · The organization's specific requirements, including business needs and client demands. — The lecture concludes that the choice of deployment model 'totally depends on on your requirement' (28:05), citing factors like organization size, business logic, and even the demands of an organization's own clients (e.g., defense sector).

Q25 D · Because the clients have mission-critical security requirements and want assurance that data is not on shared infrastructure. — The lecture uses this exact example, stating that mission-critical clients like the financial or defense sector might demand a private cloud ('your own thing') due to their stringent security and compliance needs (28:43).

Q26 B · Public Cloud — The lecture states that for public clouds, users normally have to conform to 'restrictive default service level agreements' provided by the provider, unless they are a large-scale deployment that can negotiate special terms (06:56).

Q27 B · Network dependency — The lecture lists 'network dependency' as a consideration for public (05:01), on-site private (10:47), outsourced private (15:18), and community clouds (19:43). By extension, a hybrid cloud, being a composition of these, is also network-dependent.

Q28 D · The on-site private cloud has fixed capacity, while the public cloud offers the 'illusion of unlimited resources'. — The lecture contrasts the 'limited resources' and 'fixed computing and storage capacity' of an on-site private cloud (13:14) with the 'elasticity or illusion of unlimited resource availability' in a public cloud (05:54).

Q29 C · Several non-profits pay a third-party provider to host a shared application platform in the provider's data center. — This scenario involves multiple organizations with a shared purpose (non-profits) using an infrastructure that is off-premises and managed by an external entity, fitting the description of an outsourced community cloud (21:07-21:15).

Q30 D · The cloud serves a diverse and not always trustworthy pool of clients. — The statement highlights that the public cloud is open to anybody who can subscribe, and this diverse pool of clients may include malicious actors, which contributes to the overall risk environment (03:02).

Q31 A · Because the organization is responsible for maintaining its own cloud infrastructure. — The lecture points out that for an on-site private cloud, the subscriber 'still needs a IT skill' because their own organization is responsible for installing, running, and maintaining the cloud (10:54).

Q32 A · It extends around both the subscriber's on-site resources and the private cloud resources. — For an on-site private cloud, the lecture states that the security perimeter 'extends around both the subscriber on site resources and private cloud resources,' implying the cloud is within the organization's security boundary (10:06).

Q33 D · The need to ensure both data and the application itself can work across the different environments. — The lecture highlights that portability is key and includes not just data, but also the application. It questions whether an application running on a private cloud will still work when a VM is provisioned in the public domain (24:43-25:21).

Q34 B · Users have limited visibility and control, relying heavily on the Service Level Agreement (SLA). — The lecture states that users have 'limited visibility and control over the data regarding the security' and that their primary assurance comes from the SLA or MOU with the provider (05:31-05:49).

Q35 C · By using a hybrid model to classify and place sensitive workloads on a private cloud and less sensitive ones on a public cloud. — The lecture presents this as a primary reason for using a hybrid cloud: classifying data and applications, then putting critical or vulnerable items in a private cloud and others in a public cloud (22:58-23:29).

Q36 A · Theoretically infinite elasticity — The lecture explicitly associates 'theoretically infinite' elasticity with the public cloud (06:01), while stating that on-site private clouds have 'limited resources' and a 'fixed computing and storage capacity' (12:49, 13:14).

Q37 B · It may conflict with legal or policy matters regarding data residency and security. — The lecture implies that while not caring about location is fine for some, it becomes an issue if you are 'very stringent on the legal and policy matter about the security and other aspects' (03:41), which often include data residency rules.

Q38 A · An organization can be a part of more than one community. — The lecture explicitly states that 'an organization can be more than one community' and uses the real-life analogy of being part of an office group and a residential community simultaneously (18:16-18:35).

Q39 A · The user organization has no physical control over the computing and storage infrastructure. — By stating that the infrastructure resides at the Cloud Service Provider's (CSP's) premises and not the user's premises (02:08-02:33), the lecture implies a separation where the user cedes physical control to the provider.

Q40 D · The hosting provider has a large backbone of resources that can be provisioned on request. — The lecture explains this advantage by stating that the provider usually has 'a lot of resource at their backbone', making it possible to increase resources at the other end upon request (16:18-16:35).

Q41 C · Private and Community — The lecture explicitly details on-premises and off-premises (outsourced) scenarios for both Private Clouds (09:30, 09:45) and Community Clouds (17:39, 21:07). Public clouds are by definition off-premises from the user's perspective.

Q42 D · The outsourced model has a 'modest to significant' upfront cost, which is likely less than the 'high variable' cost of an on-premises build. — The lecture describes the upfront cost for an on-premises community cloud as 'high variable' (20:35) but for an outsourced one as 'modest to significant' (21:42), noting there's a 'chance that the overall upfront cost will be much less' if outsourced.

Q43 A · Risk from multi-tenancy among different internal business units. — The lecture points out that even in an on-site private cloud, the 'risk from multi tenancy again the same issues of within things also come into play' (11:26), where different subunits or clients within the same organization share resources.

Q44 A · Ensuring the application runs seamlessly despite a potentially very complex underlying architecture. — The lecture highlights that making an application 'run seamlessly over the things' can be a major challenge because the 'overall underlining architecture maybe very complex' when combining public and private infrastructures (26:19-26:35).

Q45 B · Illusion of unlimited resource availability and elasticity. — The lecture clearly identifies 'elasticity or illusion of unlimited resource availability' as a key feature and benefit of the public cloud, allowing users to scale up or down as needed (05:54).

Q46 A · On-site Private Cloud — The lecture states that in an on-site private cloud, the security perimeter encompasses the cloud resources, and the subscriber can 'exercise control over the resources' (10:06-10:36), offering the highest degree of control among the options.

Q47 C · To determine which workloads can go to a public cloud, private cloud, or community cloud. — Towards the end, the lecture explains that a key management challenge is to 'classify my application into different things' and then decide that 'this bunch can go to the private this bunch can go to the public this can go to the community' (29:03-29:20).

Q48 B · To serve a group of organizations that have shared concerns, missions, or security requirements. — The fundamental definition of a community cloud is that it caters to a 'specific community of consumer from organization that are shared concerned' (17:09), such as having the same mission or compliance considerations.

Q49 C · Hybrid Cloud — The lecture specifically lists 'vmware v cloud' as an example of a technology capable of creating a hybrid cloud (25:26), which by definition connects two or more different cloud infrastructures.

Q50 B · The speed and reliability of the public internet connection. — The lecture explicitly states under the 'Cons' of Public Cloud that 'Network Dependency: Performance is entirely dependent on the public internet connection.'

Week 3 · L11 — Service Level Agreement your file w12 · 50 questions

Lecture 11: Service Level Agreement · 📖 notes: 3.1 Service Level Agreement (L11)

Q1. What is the fundamental definition of a Service Level Agreement (SLA) as described in the lecture? Easy

A. A marketing brochure outlining the features of a cloud platform.
B. An informal list of performance goals set by the provider.
C. A technical specification document for a single cloud service.
D. A formal contract between a service provider and a service consumer.

Q2. According to the lecture, what is the primary purpose of an SLA for a consumer? Easy

A. To get a list of all other clients using the service.
B. To define a formal, guaranteed basis for the performance and availability of a service.
C. To ensure the provider uses the most expensive hardware available.
D. To receive daily reports on the provider's stock performance.

Q3. What is the relationship between an SLA, Service Level Objectives (SLOs), and Key Performance Indicators (KPIs)? Medium

A. They are interchangeable terms for the same concept.
B. An SLA is composed of KPIs, which are measured by SLOs.
C. SLOs are high-level goals in the SLA, and they are measured by aggregating low-level KPIs.
D. KPIs are the legal agreement, which contains SLOs as optional add-ons.

Q4. A financial firm requires 99.99% uptime during trading hours but can tolerate 98% uptime overnight. This is an example of which concept? Medium

A. A Service Level Objective (SLO) that varies over time.
B. A Key Performance Indicator (KPI).
C. Jurisdictional requirement.
D. A non-negotiable SLA.

Q5. Why is a neutral third-party auditor considered the best solution for monitoring SLA compliance? Hard

A. They eliminate the conflict of interest inherent in a provider self-reporting its own performance.
B. They are always less expensive than the provider's internal team.
C. They can impose legal penalties directly on the provider.
D. They are the only ones with the technical skills to perform monitoring.

Q6. What is a key difference between traditional web service SLAs and modern cloud SLAs regarding Quality of Service (QoS) parameters? Medium

A. Traditional web service SLAs are more dynamic and scalable.
B. Cloud SLAs place a greater emphasis on security, privacy, and trust management compared to traditional web service SLAs.
C. Cloud SLAs ignore response time, while web service SLAs focus on it exclusively.
D. Web service SLAs are more concerned with security and privacy than cloud SLAs.

Q7. An SLA for a public cloud provider is offered as a standard, take-it-or-leave-it contract. What type of SLA is this? Easy

A. Verbally agreed-upon SLA
B. Off-the-shelf or non-negotiable SLA
C. Multi-layer SLA
D. Negotiable SLA via external agent

Q8. The lecture discusses the risk of 'seizure of data' by law enforcement. Why is this a particularly complex issue in a multi-tenant cloud environment? Hard

A. Data in the cloud cannot be legally seized by any authority.
B. Seizing data from one targeted consumer may unintentionally impact or disrupt service for other consumers sharing the same physical infrastructure.
C. Multi-tenant environments make it impossible to locate a specific consumer's data.
D. Cloud providers are required to notify all customers before complying with a law enforcement request.

Q9. Which of the following is an example of a Key Performance Indicator (KPI)? Medium

A. Downtime measured in minutes per month.
B. Database query response time must be under 3 seconds.
C. The service level agreement document itself.
D. The service should be user-friendly.

Q10. A large corporation is migrating its entire financial system to the cloud and has very specific data residency and compliance needs. Which SLA approach should it pursue? Medium

A. A negotiable SLA to fine-tune the agreement.
B. No SLA, relying on the provider's reputation.
C. A standard off-the-shelf SLA.
D. A simple verbal agreement.

Q11. An SLA clause that requires the provider to 'zero out the memory' of a VM after a consumer powers it off falls under which SLA requirement category? Medium

A. Business Continuity
B. Billing and Payment Terms
C. System Redundancy
D. Hardware Erasure and Privacy

Q12. What does the metric 'linearity' evaluate in the context of an SLA? Medium

A. The consistency of the monthly bill.
B. How system performance scales in relation to increases in load.
C. The time it takes to resolve a customer support ticket.
D. The physical path of the network cabling.

Q13. Why might a cloud provider place limits on the frequency of consumer-initiated audits? Medium

A. To encourage consumers to trust them without verification.
B. Because consumer audits are illegal in most countries.
C. Because audits are disruptive, expensive, and can consume significant resources.
D. To hide non-compliance with the SLA.

Q14. What is a 'penalty' or 'remedy' in an SLA? Easy

A. A punishment for a consumer who writes a negative review.
B. A fee the consumer pays for exceeding their usage limits.
C. The automatic termination of the contract.
D. A consequence, such as a service credit, the provider must give the consumer for failing to meet a guaranteed service level.

Q15. The metric 'durability' in a storage SLA is concerned with: Medium

A. The physical ruggedness of the hard drives.
B. The speed of data access.
C. The likelihood of the data being permanently lost.
D. The service's uptime percentage.

Q16. A cloud provider offers a storage service with 99% availability but 99.9999% durability. What is the practical meaning of this for a user? Hard

A. The data will be safe, but accessing it will be very slow.
B. The service is unreliable and data loss is frequent.
C. The service might be temporarily unavailable, but the data itself is extremely unlikely to be lost.
D. Availability and durability are the same, so the numbers are contradictory.

Q17. According to the lecture, why is SLA automation more critical for cloud services than for traditional web services? Medium

A. Because cloud services are inherently less reliable.
B. Because cloud consumers are less technical.
C. Because traditional web service SLAs are too complex to automate.
D. Because cloud environments are highly dynamic and scalable, requiring automated management to keep up with changes.

Q18. What is the role of the Web Service Level Agreement (WS-LA) framework? Medium

A. A legal document that replaces traditional contracts.
B. A marketing tool to promote web services.
C. A formal, XML-based language and runtime interpreter to define, negotiate, and monitor SLAs for web services.
D. A billing system for web service usage.

Q19. An SLA for an IaaS provider states the provider is responsible for the network and server hardware, but the consumer is responsible for patching the OS on their VMs. This is an example of: Hard

A. A provider failure.
B. A well-defined balance of responsibilities.
C. A conflict of interest.
D. A third-party certification.

Q20. A European company must comply with GDPR, which restricts data transfers outside the EU. Which SLA consideration is most critical for them when choosing a cloud provider? Hard

A. Elasticity
B. Agility
C. Location of the data
D. Throughput

Q21. According to the lecture, a service level agreement is applicable to which cloud service models? Easy

A. Only Infrastructure as a Service (IaaS).
B. IaaS, PaaS, SaaS, and other types of services.
C. Only Platform as a Service (PaaS).
D. Only Software as a Service (SaaS).

Q22. What is the primary concern addressed by the 'isolation of consumer data' requirement in a multi-tenant SLA? Medium

A. Making sure all consumer data is stored in the same physical location for efficiency.
B. Simplifying the billing process for the provider.
C. Ensuring that one consumer cannot access or interfere with another's data.
D. Improving the speed of data transfer between tenants.

Q23. What does the lecture identify as a significant challenge with SLAs today? Medium

A. They are too long to read.
B. There is no universally accepted standard format for them.
C. They are only available in English.
D. They are prohibitively expensive for small businesses.

Q24. An SLA clause that specifies that all legal disputes will be handled in the courts of California is an example of what? Easy

A. Auditability
B. Jurisdiction
C. Data Seizure
D. Penalty

Q25. A consumer's business is heavily dependent on their cloud service. Which SLA clause would be most critical in the event of a major earthquake near the provider's primary data center? Medium

A. The clause on third-party monitoring.
B. The clause on billing and payments.
C. The clause on business continuity and disaster recovery.
D. The clause on data deletion policies.

Q26. What is the primary function of UDDI (Universal Description, Discovery, and Integration) in the context of web services? Easy

A. To encrypt all service communications.
B. To serve as a directory for advertising and discovering services.
C. To guarantee 100% uptime for all services.
D. To automatically provision resources.

Q27. A service's 'elasticity,' when defined in an SLA, refers to its ability to: Medium

A. Operate across different legal jurisdictions.
B. Prevent data loss over a long period.
C. Automatically scale resources up or down to match demand.
D. Resist physical damage.

Q28. An SLA for a PaaS provider guarantees platform uptime. A consumer deploys poorly optimized code that makes their application slow. Who is at fault for the poor application performance? Hard ⚠

A. The consumer, for deploying inefficient code on the platform.
B. The provider, for failing to meet the performance guarantee.
C. Both parties are equally responsible.
D. The third-party auditor, for not reviewing the code.

Q29. The lecture lists 'transparency' as an SLA requirement. What does this most likely mean? Medium

A. The provider must be open and honest about its procedures, security, and how it meets its guarantees.
B. The user interface must be invisible.
C. The provider's data centers must have glass walls.
D. The service pricing must be the lowest on the market.

Q30. Which of the following would NOT be a typical component of an SLA's contents? Easy

A. A complete definition of each service provided.
B. The provider's internal employee salary list.
C. Responsibilities of both the provider and the consumer.
D. A set of metrics to measure service quality.

Q31. What is the main reason a consumer migrating from an on-premise system cares deeply about the SLA? Medium

A. To receive free training for all their employees.
B. To force the provider to use the same hardware brands they used on-premise.
C. To get a guarantee that the cloud service will offer a comparable level of reliability, performance, and security.
D. To ensure the cloud provider is more expensive than their old system.

Q32. What is a potential drawback of the Web Service Level Agreement (WS-LA) framework mentioned in the lecture? Hard

A. It is controlled by a single company, creating a monopoly.
B. It doesn't use XML, making it incompatible with web services.
C. It has a lack of formal definitions for the semantics (the meaning) of the metrics.
D. It is too simple and lacks the ability to define metrics.

Q33. An SLA that can be customized through discussion with an intermediary is known as what? Medium

A. Off-the-shelf SLA
B. Non-conducive SLA
C. Direct SLA
D. Negotiable via external agent SLA

Q34. According to the lecture, the provider's responsibility in an SLA is influenced by its business objectives and what other factor? Medium

A. The consumer's budget.
B. The current stock market trends.
C. The technical realities of its infrastructure.
D. The number of employees it has.

Q35. In the context of SLA metrics, what is 'agility' likely to measure? Medium

A. The durability of the storage hardware.
B. The speed and flexibility with which the provider can respond to changes or new service requests.
C. The physical security of the data center.
D. The provider's financial health.

Q36. A consumer's SLA has a 'data retention' clause stating data is kept for 5 years, but no 'data deletion' clause. Why is this a potential problem after the contract ends? Hard

A. The SLA is incomplete, leaving ambiguity about whether, when, and how the consumer's data will be securely destroyed.
B. The retention period of 5 years is too short for any business.
C. The provider might delete the data too soon.
D. The provider might charge the consumer for storage indefinitely.

Q37. The lecture gives an example availability SLO of 'ninety nine point nine percent'. Which of the following is an example of a KPI that would be used to calculate this SLO? Medium

A. The total number of service outages in a month.
B. The total cost of the service for one year.
C. The name of the service provider.
D. The customer's satisfaction rating.

Q38. What is the primary risk of a consumer's action if the SLA does not clearly define their responsibilities? Hard

A. The provider might sue the consumer for breach of contract.
B. The consumer will not be able to scale their services.
C. The consumer might accidentally violate the provider's terms and cause a service failure for themselves or others.
D. The consumer might be overcharged for their service.

Q39. The lecture mentions that a provider having a 'third party certification' is a desirable SLA requirement. What does this provide the consumer? Medium

A. A guarantee of 100% uptime.
B. Independent verification that the provider meets a certain standard of service.
C. Proof that the service is the fastest on the market.
D. A discount on the service.

Q40. What service does Amazon S3 provide, and what is its stated availability in the lecture? Easy ⚠

A. Compute (IaaS), 99.5%
B. Storage as a Service, 99.9%
C. Platform (PaaS), 99.0%
D. Software (SaaS), 100%

Q41. If a cloud provider fails and goes out of business, what SLA consideration becomes critical for the consumer? Hard

A. The uptime guarantee for the last month.
B. The color scheme of the provider's website.
C. The process for data retrieval and migration to another provider.
D. The daily performance metrics.

Q42. What is the primary purpose of the 'Service Definitions' section within a Service Level Agreement (SLA)? Easy

A. To define the mechanisms for auditing performance.
B. To provide a complete and specific description of each service offered.
C. To specify the duties of both the provider and the consumer.
D. To outline the financial penalties for non-compliance.

Q43. The 'Responsibilities' section of an SLA primarily serves to delineate what? Easy

A. The methods for calculating service credits.
B. The geographical location of data centers.
C. The duties of both the service provider and the consumer.
D. The encryption standards for data in transit.

Q44. What is the main function of the 'Remedies & Penalties' section in an SLA? Medium

A. To outline the process for data seizure by law enforcement.
B. To specify the 'payback mechanism' or consequences for unmet terms.
C. To define the service's elasticity and scalability.
D. To describe the encryption key management procedures.

Q45. According to the lecture's SLA hierarchy, how are Service Level Objectives (SLOs) typically derived from Key Performance Indicators (KPIs)? Medium

A. KPIs are aggregated or used in formulas to calculate high-level SLOs.
B. SLOs are the fundamental building blocks used to calculate KPIs.
C. KPIs are abstract guarantees, while SLOs are raw data.
D. SLOs are legal contracts, and KPIs are their informal precursors.

Q46. For which type of consumer is an 'Off-the-Shelf' (non-negotiable) SLA most suitable? Easy

A. Large enterprises requiring extensive customization.
B. Consumers needing specific data residency guarantees.
C. Organizations with complex multi-cloud deployments.
D. Individuals or small-scale operations where customization is not required.

Q47. Which section of an SLA defines the mechanisms for monitoring services and auditing performance? Easy

A. Temporal Conditions
B. Service Definitions
C. Remedies & Penalties
D. Auditing & Monitoring

Q48. What is the purpose of 'Temporal Conditions' in an SLA? Medium

A. To outline how the SLA might change over time, such as stricter requirements during peak hours.
B. To set the schedule for third-party audits.
C. To define the time limits for dispute resolution.
D. To specify the duration of data retention.

Q49. In the context of data governance in cloud environments, what specific requirement must an SLA address regarding data after a VM is powered off or hardware is decommissioned? Medium

A. Confirmation that data is properly erased (e.g., zeroing out memory/disk platters).
B. The encryption standards for data at rest.
C. The protocol for law enforcement data seizure.
D. The right to audit data location.

Q50. The lecture describes the Service Level Agreement (SLA) as the 'master document' governing the relationship between a service provider and a service consumer. What does this signify? Easy

A. It primarily serves as a marketing tool for the provider.
B. It is the only document that needs to be reviewed.
C. It is a flexible guideline with no legal enforceability.
D. It is the foundational, legally binding contract that establishes trust and accountability.

Answer key — Week 3 · L11 (your file w12)

Q1 D · A formal contract between a service provider and a service consumer. — The lecture explicitly defines an SLA as a 'formal contract between the service provider and the service consumer' (02:42), serving as the foundation of trust.

Q2 B · To define a formal, guaranteed basis for the performance and availability of a service. — The purpose of an SLA is 'to define a formal basis for performing and availability of service provider guarantees to the to delivery' (03:19), which builds consumer trust.

Q3 C · SLOs are high-level goals in the SLA, and they are measured by aggregating low-level KPIs. — The lecture presents a hierarchy where low-level KPIs (e.g., downtime) are used to calculate SLOs (e.g., 99.9% availability), and the SLA is the overall agreement that contains these SLOs (25:05).

Q4 A · A Service Level Objective (SLO) that varies over time. — The lecture explains that objectives can vary over time, citing an example of requiring higher availability during peak hours versus off-peak hours (05:39).

Q5 A · They eliminate the conflict of interest inherent in a provider self-reporting its own performance. — The lecture explicitly states that a neutral third party is the best solution 'to eliminate the conflict of interest between the provider and consumer' (23:34).

Q6 B · Cloud SLAs place a greater emphasis on security, privacy, and trust management compared to traditional web service SLAs. — The lecture contrasts the two, noting that while traditional web services focus on parameters like response time, for the cloud, 'QoS is related to security, privacy, trust management, etcetera, have more importance' (13:10).

Q7 B · Off-the-shelf or non-negotiable SLA — The lecture describes this as an 'off-the-shelf or non-negotiable SLA,' where the provider creates a template with all criteria, which is typical of present-day public clouds (14:41, 15:25).

Q8 B · Seizing data from one targeted consumer may unintentionally impact or disrupt service for other consumers sharing the same physical infrastructure. — The lecture highlights that the 'multi-tenant nature of the cloud computing makes it likely that the other consumer will be effected also' when data is seized from one tenant (21:18).

Q9 A · Downtime measured in minutes per month. — KPIs are 'low-level resource metric[s]' that are directly measurable. The lecture gives 'downtime' as a specific example of a KPI (25:41), whereas a response time target is an SLO.

Q10 A · A negotiable SLA to fine-tune the agreement. — The lecture suggests that for a 'large requirement,' a 'negotiable SLA' is more appropriate to accommodate specific needs, analogous to booking an entire hotel versus a single room (16:38).

Q11 D · Hardware Erasure and Privacy — This is a direct example used in the lecture under 'hardware erasure or destruction' (22:31), which is a critical practice for ensuring data privacy and security in a shared environment.

Q12 B · How system performance scales in relation to increases in load. — The lecture defines linearity as 'how the system performs at the load increases.' A linear relationship makes it easier to predict and manage scaling (27:16).

Q13 C · Because audits are disruptive, expensive, and can consume significant resources. — The lecture notes that 'audits are disruptive and expensive,' and as a result, a provider will 'most likely place limits on when' they can occur (24:22).

Q14 D · A consequence, such as a service credit, the provider must give the consumer for failing to meet a guaranteed service level. — The lecture describes this as a 'payback mechanism' or 'penalty' a provider must give to 'compensate' the consumer if a guaranteed service is not provided (07:42, 08:24).

Q15 C · The likelihood of the data being permanently lost. — The lecture defines durability as 'how likely the data to be lost' (27:09), distinguishing it from availability, which is about whether the service is accessible.

Q16 C · The service might be temporarily unavailable, but the data itself is extremely unlikely to be lost. — This tests the understanding between availability (uptime) and durability (data permanence). A service can be temporarily down without losing the underlying data, which is what these two distinct metrics represent.

Q17 D · Because cloud environments are highly dynamic and scalable, requiring automated management to keep up with changes. — The lecture directly connects the need for automation to the nature of cloud, stating that 'cloud SLA automation is required for high dynamic and scalable services' (13:43).

Q18 C · A formal, XML-based language and runtime interpreter to define, negotiate, and monitor SLAs for web services. — The lecture describes the WS-LA framework as a 'formal xml schema based languages to express sla and runtime interpreter' to 'measure and monitor qos parameter' (10:53).

Q19 B · A well-defined balance of responsibilities. — This reflects the shared responsibility model in cloud computing. The lecture emphasizes that the 'balance of responsibility between the provider and consumer will vary according to the type of services' (19:08), and this is a classic IaaS example.

Q20 C · Location of the data — The lecture highlights 'location of the data' as a key consideration, especially 'to enforce data location regulation' (20:44). For a company under GDPR, this is a paramount legal and compliance issue.

Q21 B · IaaS, PaaS, SaaS, and other types of services. — The lecture is clear that SLAs 'are valid for different type of ah service provisioning' including IaaS, PaaS, and SaaS (06:14).

Q22 C · Ensuring that one consumer cannot access or interfere with another's data. — The lecture lists 'isolation of the consumer data in a multi tenant environment' as a key privacy issue (22:16). This is to prevent data breaches between customers sharing resources.

Q23 B · There is no universally accepted standard format for them. — Early in the lecture, it is stated that 'till today they we dont have any very standard format of doing that' (01:53), which means consumers

must carefully compare different providers' unique agreements.

Q24 B · Jurisdiction — This directly relates to the concept of 'jurisdiction,' which the lecture mentions as a consideration for defining 'where [any type of litigation] will be ah addressed' (21:49).

Q25 C · The clause on business continuity and disaster recovery. — The 'business continuity and disaster recovery' clause (20:00) is designed to ensure the provider has adequate protection and recovery plans for disasters, which is essential for the consumer's operational survival.

Q26 B · To serve as a directory for advertising and discovering services. — The lecture explains that UDDI is a protocol used for 'advertising and discovering between the web services' (14:14).

Q27 C · Automatically scale resources up or down to match demand. — The lecture mentions elasticity in the context of when 'new vms are booted or terminated' to handle changes in load (26:59), which is the definition of scaling resources.

Q28 A · The consumer, for deploying inefficient code on the platform. — This scenario highlights the shared responsibility model. While the provider guarantees the platform, the consumer is responsible for the application they build on it. The provider is not responsible for the quality of the consumer's code.

> ⚠️ This is reasoning from the lecture's 'balance of responsibilities' (the consumer owns its code on PaaS), not a statement from the lecture.

Q29 A · The provider must be open and honest about its procedures, security, and how it meets its guarantees. — Transparency in a business and service context refers to openness and clarity. In an SLA, it means the provider is clear with the consumer about its operations and compliance (23:05).

Q30 B · The provider's internal employee salary list. — The lecture lists specific contents like service definitions, responsibilities, and metrics (05:59). Confidential internal financial data like salaries would not be part of a customer-facing SLA.

Q31 C · To get a guarantee that the cloud service will offer a comparable level of reliability, performance, and security. — The lecture notes that consumers switch 'thinking that you will get same type of reliability... performance... security what he was having in his own premises' (01:28). The SLA is the document that formalizes this expectation.

Q32 C · It has a lack of formal definitions for the semantics (the meaning) of the metrics. — The lecture points out that a weakness of WS-LA is the 'lack of formal definitions for semantics of the metrics' (11:08), meaning the framework can describe a metric syntactically but not its standardized meaning.

Q33 D · Negotiable via external agent SLA — The lecture describes a type of negotiable SLA where 'there may be external agents which negotiates between the provider and consumer' (15:46) to customize the terms.

Q34 C · The technical realities of its infrastructure. — From the provider's perspective, decisions are made based on 'business objective and technical realities' (18:35), meaning what they can realistically deliver with their technology.

Q35 B · The speed and flexibility with which the provider can respond to changes or new service requests. — Agility is listed as a metric (27:45). In a services context, it refers to the ability to adapt and react quickly, a key benefit often touted by cloud providers.

Q36 A · The SLA is incomplete, leaving ambiguity about whether, when, and how the consumer's data will be securely destroyed. — The lecture discusses both 'data retention and deletion' (22:26) as related but distinct concepts. An SLA needs to specify not only how long data is kept but also the process for its secure disposal upon contract termination.

Q37 A · The total number of service outages in a month. — To calculate the availability percentage (an SLO), you need low-level, measurable metrics (KPIs) like the number and duration of outages. The lecture

shows availability is calculated from downtime and uptime KPIs (26:01).

Q38 C · The consumer might accidentally violate the provider's terms and cause a service failure for themselves or others. — The lecture gives an example where a consumer sending data too fast can cause an 'overflow of data' that may 'intern violate other slas of the thing' (19:46). This shows how consumer actions can negatively impact service if responsibilities are not clear.

Q39 B · Independent verification that the provider meets a certain standard of service. — Third-party certification (23:05) provides assurance to the consumer that an independent body has audited and confirmed that the provider adheres to a specific set of standards or guarantees.

Q40 B · Storage as a Service, 99.9% — In the lecture's table of provider SLAs, Amazon S3 is the storage-as-a-service example (EC2 is the IaaS one, 99.5 %). The availability the lecture actually reads out for S3 is 99 %, not 99.9 %; see the note.

> ⚠ The lecture says Amazon S3 (storage as a service) = 99 %. The 99.9 % figure belongs to Microsoft/Google/Salesforce, and EC2 = 99.5 %. B is the only option pairing S3 with storage, so it is the keyed answer, but if 99 % appears as an option, choose 99 %.

Q41 C · The process for data retrieval and migration to another provider. — The lecture lists 'failure of the provider' (21:45) as a consideration. In this event, the most critical issue for the consumer is getting their data back so they can continue their business operations elsewhere.

Q42 B · To provide a complete and specific description of each service offered. — The lecture states that 'Service Definitions' include 'A complete and specific definition of each service being provided (e.g., compute instances, storage buckets, database access).'

Q43 C · The duties of both the service provider and the consumer. — The lecture states that 'Responsibilities' 'Clearly delineates the duties of both the provider and the consumer.'

Q44 B · To specify the 'payback mechanism' or consequences for unmet terms. — The lecture defines 'Remedies & Penalties' as specifying the 'payback mechanism' or penalties if the terms are not met, such as service credits or financial compensation.

Q45 A · KPIs are aggregated or used in formulas to calculate high-level SLOs. — The lecture states: 'Key Performance Indicators (KPIs): ...are the fundamental building blocks used to calculate SLOs.' and 'Service Level Objectives (SLOs): These KPIs are then aggregated or used in formulas to calculate the high-level objectives.'

Q46 D · Individuals or small-scale operations where customization is not required. — The lecture states that Off-the-Shelf SLAs are 'Suitable for individuals or small-scale operations where customization is not required.'

Q47 D · Auditing & Monitoring — The 'Anatomy of a Service Level Agreement' section lists 'Auditing & Monitoring' as defining 'the mechanisms for monitoring services and auditing performance.'

Q48 A · To outline how the SLA might change over time, such as stricter requirements during peak hours. — The lecture states that 'Temporal Conditions' 'Outlines how the SLA might change over time, such as having stricter availability requirements during peak business hours versus off-peak hours.'

Q49 A · Confirmation that data is properly erased (e.g., zeroing out memory/disk platters). — Under 'Data Governance - Retention & Deletion,' the lecture states: 'The agreement must specify how long data is retained and confirm that it is properly erased (e.g., zeroing out memory/disk platters) when a VM is powered off or hardware is decommissioned.'

Q50 D · It is the foundational, legally binding contract that establishes trust and accountability. — The 'Core Concepts' table defines SLA as 'The master document governing the relationship' and the introduction states it 'builds trust, sets expectations, and provides a framework for accountability.'

Week 4 · L16 — OpenStack demo (Meghamala) your file w1 · 50 questions

Lecture 16 : Cloud Computing: Open source Cloud–Openstack Demo · 📖 notes: 4.1 OpenStack demo — Meghamala, IIT Kharagpur (L16)

Q1. What is the name of the experimental, OpenStack-based cloud at IIT Kharagpur demonstrated in the lecture? Easy ⚠

A. Meghadoop
B. Meghamala
C. OpenCloud IIT
D. Meghaduta

Q2. In the OpenStack dashboard, which component is responsible for managing disk images for virtual machines? Medium

A. Neutron
B. Cinder
C. Glance
D. Nova

Q3. An administrator launches a new VM and assigns it only to the 'internal' network. A user then tries to connect to this VM from their laptop at home but fails. What is the most likely reason for the failure? Hard

A. The user is using the wrong login credentials.
B. The VM has not been assigned a publicly accessible (external) IP address.
C. The VM is still in the 'Build' state and not yet 'Active'.
D. The VM's firewall is blocking the connection.

Q4. What is the primary purpose of a 'Security Group' in OpenStack as described in the lecture? Medium

A. To bundle a VM with specific storage and compute resources (a flavor).
B. To act as a virtual firewall for VMs, defining traffic rules.
C. To group users with similar access permissions.
D. To create a secure backup snapshot of a VM.

Q5. What was the name given to the virtual machine created during the live demonstration? Easy

A. nptel
B. CentOS-demo
C. Test-VM
D. Meghamala-VM

Q6. According to the demo, what is the role of the 'X2Go' software? Medium

A. It is a command-line tool for managing OpenStack resources.
B. It is a GUI client used to connect to the desktop environment of a remote VM.
C. It is a data storage service similar to Dropbox.
D. It is the hypervisor used by OpenStack to run VMs.

Q7. The lecture mentions several OpenStack services. Which service is responsible for providing persistent block storage to VMs? Medium

A. Cinder
B. Swift
C. Glance
D. Nova

Q8. What is the initial status of a VM immediately after an administrator clicks the 'Launch' button in the OpenStack dashboard? Easy

A. Running
B. Spawning
C. Build
D. Active

Q9. What key difference between the 'Admin' view and the 'Project' (user) view was highlighted in the OpenStack dashboard demo? Medium

A. The Admin view has a dark theme, while the Project view has a light theme.
B. The Admin view shows billing and cost information hidden from users.
C. The Admin view includes infrastructure-level details like Hypervisors.
D. The Admin view can only terminate VMs, while the Project view can create them.

Q10. What action in the OpenStack dashboard releases all resources (CPU, RAM, storage) associated with a virtual machine, making them available for others? Easy

A. Detach Volume
B. Pause Instance
C. Shut Down Instance
D. Terminate Instance

Q11. In the Meghamala cloud, three VM 'flavors' are offered. If a user needs to run a memory-intensive simulation and the 'IITKGP Regular' flavor is insufficient, what is the correct procedure? Hard ⚠

A. Manually increase the RAM of the running 'IITKGP Regular' VM through the dashboard.
B. Re-install the operating system on the current VM to free up memory.
C. Contact the administrator to add more physical RAM to the hypervisor.
D. Terminate the current VM and request a new one with a larger flavor, like 'IITKGP Large'.

Q12. The Meghamala cloud provides 'ephemeral storage' with its VM flavors. What is the key characteristic of this storage type? Medium

A. It is a network-attached storage that can be shared between VMs.
B. It is temporary and is deleted when the VM is terminated.
C. It is an extra-cost option for high-performance applications.
D. It persists even after the VM is terminated.

Q13. According to the presenter, which OpenStack component manages the networking aspects of the cloud, such as IP addressing? Easy

A. Cinder
B. Nova
C. Neutron
D. Keystone

Q14. What is 'Meghadoop', as mentioned in the lecture? Medium

A. A personal cloud storage service.
B. The operating system image for Hadoop-based VMs.
C. The billing and metering component of Meghamala.
D. A Hadoop cluster service running on top of the Meghamala infrastructure.

Q15. What is the primary objective of OpenStack as described at the beginning of the lecture? Easy

A. To be a high-cost, enterprise-only cloud platform.
B. To be an open-source cloud platform you can download and install on your own systems.
C. To be a proprietary software managed by a single company.
D. To provide only container-based virtualization services.

Q16. In the demo, what operating system image was chosen for the new 'nptel' virtual machine? Medium

A. Ubuntu
B. Fedora
C. CentOS
D. Windows Server

Q17. The 'IITKGP Regular' flavor is specified to have 'two b c p u'. What does 'vCPU' stand for? Easy

A. Volatile CPU
B. Verified CPU
C. Vector CPU
D. Virtual CPU

Q18. Why is it beneficial to use Security Groups instead of configuring firewalls on each VM individually? Hard

A. Individual VM firewalls are not supported in OpenStack.
B. Security Groups are the only way to allow outgoing internet traffic.
C. Security Groups provide better performance by offloading firewall rules to the hardware.
D. A pre-defined Security Group can be applied to many VMs, ensuring consistent and automatic rule application.

Q19. What is the purpose of associating a 'new external IP address' with the VM in the demo? Medium

A. To increase the network bandwidth available to the VM.
B. To enable access to the VM from outside the cloud network.
C. To give the VM a static internal IP address.
D. To connect the VM to the persistent storage service.

Q20. Which OpenStack component is associated with compute services, including managing hypervisors and VM lifecycle? Medium

A. Glance
B. Cinder
C. Nova
D. Neutron

Q21. What service is described as a 'personal cloud ah this is a kind of a ah draw box like thing'? Easy ⚠

A. OpenStack Swift
B. Meghadoop
C. Meghaduta
D. Meghamala

Q22. The admin dashboard for Meghamala shows three physical hypervisors. What information is displayed for each hypervisor? Medium

A. The number of vCPUs, RAM, disk usage, and the number of running instances.
B. Only the hostname and IP address.
C. The physical location and temperature of the server.
D. The list of users who have VMs running on it.

Q23. A user wants to save the current state of their VM's disk so they can create identical VMs from it later. Which OpenStack feature, related to Glance, would they use? Hard ⚠

A. Snapshot/Image
B. Flavor
C. Security Group
D. Volume

Q24. What was the final status of the 'nptel' instance after it was successfully provisioned and running? Easy

A. Online
B. Active
C. Build
D. Complete

Q25. What is the primary difference between ephemeral storage and persistent storage (volumes) in OpenStack? Medium

A. Ephemeral storage is deleted with the VM, while persistent storage exists independently.
B. Ephemeral storage has unlimited capacity, while persistent storage is capped at 60 GB.
C. Ephemeral storage is for Linux VMs, and persistent storage is for Windows VMs.
D. Ephemeral storage is faster but more expensive than persistent storage.

Q26. If an administrator terminates a VM, what happens to the external (floating) IP address that was associated with it? Hard ⚠

A. The IP address remains reserved for the user who last used it.
B. The IP address is deleted permanently.
C. The IP address is returned to the pool of available IPs to be used by another VM.
D. The IP address is blacklisted for 24 hours.

Q27. The lecture mentions that the admin user has two components in the dashboard: 'Project' and 'Admin'. What does this imply? Medium

A. The admin must switch between dashboards to perform different tasks.
B. The admin has a dual role: as a regular user/tenant ('Project') and as a cloud administrator ('Admin').
C. The admin must manage two separate clouds.
D. The 'Project' view is for active VMs and the 'Admin' view is for terminated VMs.

Q28. What technology did the demo use to prove the VM had full internet access? Easy

A. Pinging a website and then playing a YouTube video in a browser.
B. Downloading a large file via FTP.
C. Running a speed test from the command line.
D. Connecting to another VM in the same cloud.

Q29. What is a 'flavor' in the context of OpenStack? Medium

A. A color-coding scheme for organizing VMs in the dashboard.
B. A pre-installed software package on a VM image.
C. A template that defines the compute, memory, and storage resources for a VM.
D. A type of operating system, such as Ubuntu or CentOS.

Q30. The presenter states that one goal of the demo is to show how services like compute, storage, and image are 'realized'. What does 'realized' mean in this context? Hard

A. To be understood in a theoretical sense.
B. To be sold for profit.
C. To be deleted and removed from the cloud.
D. To be implemented and made functional in a practical system.

Q31. What is the availability zone named in the 'Launch Instance' dialog during the demo? Easy

A. iitkgp-zone-1
B. default
C. nova
D. meghamala

Q32. If the hypervisor dashboard shows all physical machines are at 99% vCPU utilization, what will likely happen when a user tries to launch a new VM? Hard ⚠

A. The VM launch will fail due to insufficient resources.
B. The oldest running VM will be automatically terminated to make space.
C. The VM will launch, but run very slowly.
D. The request will be queued until resources become available.

Q33. The lecture mentions that Meghamala offers three types of operating systems. Which two were explicitly named? Medium ⚠

A. Red Hat and SUSE
B. Windows and macOS
C. Debian and Arch Linux
D. Ubuntu and Fedora

Q34. What does the 'Terminate Instance' action do that 'Shut Down Instance' does not? Medium

A. It powers off the virtual machine.
B. It reboots the virtual machine.
C. It logs the user out of the VM.
D. It deletes the VM and de-allocates its disk and RAM.

Q35. Who is Rajesh, as introduced in the lecture? Easy

A. The professor teaching the course.
B. The administrator of the Meghamala cloud.
C. The head of the computer science department.
D. A student who developed OpenStack.

Q36. What is the key takeaway the professor wants students to get from this demonstration? Medium

A. That command-line interfaces are obsolete for cloud management.
B. That commercial clouds are always better than open-source ones.
C. That setting up a cloud is too difficult for students.
D. That they can set up their own small-scale OpenStack cloud to gain practical experience.

Q37. The admin overview dashboard showed a 'd c p hour' metric. What does this likely represent? Hard

A. The data center's power usage in hours.
B. The total download capacity per hour.
C. A typo for 'vCPU hour', a unit of computing resource consumption.
D. The number of hours the DHCP server has been running.

Q38. During VM creation, the demonstrator selects a boot source. What are the two main types of boot sources in OpenStack? Hard ⚠

A. Image and Snapshot
B. Flavor and Network
C. Internal and External
D. Image and Volume

Q39. What is the specification for the 'IITKGP regular' flavor mentioned in the slides? Easy

A. 1 vCPU, 2 GB RAM, 20 GB storage
B. 2 vCPU, 2 GB RAM, 40 GB storage
C. 4 vCPU, 8 GB RAM, 60 GB persistent storage
D. 2 vCPU, 4 GB RAM, ephemeral storage

Q40. Why would an administrator use the 'View Log' option for a running instance? Medium ⚠

A. To view the VM's console output for debugging boot or system issues.
B. To see the billing information for that VM.
C. To see a list of users currently logged into the VM.
D. To check the network traffic in real-time.

Q41. The Meghamala portal shows three physical machines, each with 48 vCPUs. If a user requests a VM with the 'IITKGP Regular' flavor (2 vCPUs), the OpenStack scheduler could place this VM on any of the three machines. This scheduling decision is a function of which component? Hard ⚠

A. Glance
B. Neutron
C. Nova
D. Cinder

Q42. Which network type is selected by default for the new VM in the demo to provide it with a private IP address? Easy

A. External
B. Public
C. Internal
D. Management

Q43. What is demonstrated to be running successfully inside the newly created VM, proving it's a fully functional machine? Medium

A. A complex scientific computation.
B. A standard desktop environment with internet access.
C. A database service for other VMs.
D. A web server hosting the Meghamala portal.

Q44. If a user requires a persistent disk that survives the deletion of their VM, what should they request? Medium

A. A Cinder volume (persistent storage).
B. A security group with a 'do not delete' rule.
C. A VM snapshot.
D. A larger flavor with more ephemeral storage.

Q45. The lecture describes Meghamala as an 'experimental ah cloud'. What does this imply about its purpose? Medium

A. It is only accessible to the administrators.
B. It is not stable and crashes frequently.
C. It is used for commercial hosting of production websites.
D. It is a platform for learning, research, and testing cloud concepts.

Q46. The OpenStack dashboard provides a graphical representation of resource usage. What is the benefit of this visualization? Medium

A. It automatically balances the load across hypervisors.
B. It is the only way to launch new VMs.
C. It provides administrators with a quick, at-a-glance understanding of the cloud's overall health and capacity.
D. It enforces user quotas and prevents over-consumption of resources.

Q47. If a security group rule denies all incoming traffic except for port 22 (SSH), what will be the result? Hard ⚠

A. The user can access the VM's web server on port 80 but cannot SSH into it.
B. The VM will be unable to access the internet.
C. All traffic, both incoming and outgoing, will be blocked.
D. The user can only connect to the VM using SSH; all other connections like HTTP will be blocked.

Q48. During the demo, when the administrator needs an external IP, he selects one from a pre-existing list. What does this imply? Medium

A. Each user can only ever have one external IP address.
B. The cloud has a pre-allocated pool of public IP addresses that can be assigned to VMs.
C. External IPs are created on-the-fly for each VM.
D. External IPs can only be assigned by contacting support.

Q49. What is the relationship between OpenStack and Meghamala? Easy

A. OpenStack is the company that owns and operates Meghamala.
B. Meghamala is the specific implementation or instance of an OpenStack cloud at IIT Kharagpur.
C. Meghamala is a competing product to OpenStack.
D. OpenStack is a service running inside the Meghamala cloud.

Q50. In the admin view of hypervisors, one machine is listed as having '14 instances' running on it. What does 'instance' refer to in this context? Easy

A. Software applications
B. Physical servers
C. Virtual machines (VMs)
D. Docker containers

Answer key — Week 4 · L16 (your file w1)

Q1 B · Meghamala — The lecture explicitly introduces the cloud as 'Meghamala' (01:11.590). Meghaduta and Meghadoop are other services mentioned that are built on top of the Meghamala infrastructure.

⚠ The lecture's Dropbox-like service is called Meghadata; 'Meghaduta' is this set's misspelling of it.

Q2 C · Glance — The lecture states that 'images is actually the glance' (09:01.790), which manages the OS images used to boot VMs. Cinder manages block storage, Nova manages compute, and Neutron manages networking.

Q3 B · The VM has not been assigned a publicly accessible (external) IP address. — The demo explicitly shows that a VM on the 'internal' network is not accessible from outside the cloud. An 'external' (or floating) IP must be associated with it for public access (13:26.430 - 13:45.699). The other options are possible but less fundamental to the described setup.

Q4 B · To act as a virtual firewall for VMs, defining traffic rules. — The lecture describes security groups as a 'firewall kind of concept' (09:22.089) where rules are defined to control which type of incoming and outgoing traffic is allowed for a VM.

Q5 A · nptel — During the VM creation process, the administrator explicitly names the instance 'nptel' (11:41.620).

Q6 B · It is a GUI client used to connect to the desktop environment of a remote VM. — Rajesh uses the X2Go client to establish a graphical desktop session with the newly created CentOS VM (14:33.699).

Q7 A · Cinder — The presenter mentions that 'volumes and snapshots of things which are maintained by cinder' (04:02.780) are the persistent storage part of the cloud.

Q8 C · Build — The demo clearly shows that after launching the 'nptel' VM, its status is listed as 'Build' while the system provisions it (13:07.060). It later changes to 'Active'.

Q9 C · The Admin view includes infrastructure-level details like Hypervisors. — The presenter explicitly switches to the 'Admin' page to show the 'Hypervisors' tab, which lists the physical machines in the cloud, noting this is a key difference (10:16.329 - 10:45.620).

Q10 D · Terminate Instance — The lecture demonstrates that clicking 'Terminate Instance' deletes the VM and 'releases the all the resources that that was the allocated' (18:31.741, 19:11.570). 'Shut Down' powers off the VM but keeps its resources allocated.

Q11 D · Terminate the current VM and request a new one with a larger flavor, like 'IITKGP Large'. — VM flavors are fixed templates. One cannot dynamically change core resources like CPU or RAM. The correct procedure is to terminate the inadequate VM and provision a new one using a more powerful flavor ('IITKGP Large' or 'Extra Large') as offered by the cloud (02:27.920, 06:48.330).

⚠ Real OpenStack can also resize a VM to another flavour (with a reboot). The lecture's route, and the key's, is to ask for a new VM with a bigger flavour.

Q12 B · It is temporary and is deleted when the VM is terminated. — The term 'ephemeral' means temporary. In cloud computing, ephemeral storage is non-persistent disk space tied to the lifecycle of the instance. The lecture mentions it as part of the flavor definition (02:27.920) and contrasts it with persistent storage.

Q13 C · Neutron — The lecture explicitly states, 'Neutron is the networking aspects ah of the things' (04:09.050).

Q14 D · A Hadoop cluster service running on top of the Meghamala infrastructure. — The lecture introduces 'meghadoop which is running over meghamala' (03:00.620) and later clarifies it is a 'hard do [Hadoop] cluster' (06:26.550).

Q15 B · To be an open-source cloud platform you can download and install on your own systems. — The speaker states, 'the primary objective is a open source cloud you can easily ah download those this open stack in your local ah systems' (00:32.349).

Q16 C · CentOS — During the instance creation process, the demonstrator selects the image and says, 'say i put sent wise [CentOS]' (12:13.610).

Q17 D · Virtual CPU — In cloud computing and virtualization, vCPU stands for Virtual Central Processing Unit. It represents a share of the processing power of the underlying physical CPUs.

Q18 D · A pre-defined Security Group can be applied to many VMs, ensuring consistent and automatic rule application. — The lecture explains that by assigning a security group, 'you dont have to ah ah configure its firewall independently... automatically the firewall rules will apply' (10:03.450). This simplifies management and ensures consistency.

Q19 B · To enable access to the VM from outside the cloud network. — The demonstrator allocates an external network IP 'so that it can be accessed from outside the cloud' (13:39.821). This is often called a 'floating IP'.

Q20 C · Nova — The lecture mentions 'hypervisors ah nova v c p u' and 'nova compute servers' (04:27.229 - 04:34.030), indicating Nova is the compute component.

Q21 C · Meghaduta — The lecture introduces 'megha data which is a personal cloud ah this is a kind of a ah draw box [Dropbox] like thing' (06:26.550 - 06:33.330).

⚠ The lecture calls it Meghadata (personal cloud, Dropbox-like). Option 'Meghaduta' is the set's misspelling, and it is the intended answer.

Q22 A · The number of vCPUs, RAM, disk usage, and the number of running instances. — The demonstration shows the hypervisor list with details like 'forty eight v c p u s one twenty g b of...ram' and 'fourteen instances such running' (10:57.269 - 11:08.279).

Q23 A · Snapshot/Image — While the demo focuses on pre-existing images, a common OpenStack workflow is to take a 'snapshot' of a running VM's disk, which creates a new 'image' in Glance. This new image can then be used to launch new, identical VMs.

⚠ Snapshots are not shown in the lecture; this is standard OpenStack (a snapshot becomes a new Glance image).

Q24 B · Active — The presenter notes that after the VM is built, 'the status changed to active and the power state is running' (17:58.050 - 18:05.160).

Q25 A · Ephemeral storage is deleted with the VM, while persistent storage exists independently. — The core distinction is lifecycle. Ephemeral storage is tied to the instance lifecycle (02:33.890). Persistent storage, managed by Cinder, can be detached from one VM and attached to another, and it survives VM termination.

Q26 C · The IP address is returned to the pool of available IPs to be used by another VM. — Terminating the instance releases all associated resources. The floating IP is disassociated and returned to the project's pool of available IPs, as seen when the demonstrator allocates one from a list of available IPs (13:45.699).

⚠ Not shown in the lecture; standard OpenStack behaviour (a released floating IP goes back to the pool).

Q27 B · The admin has a dual role: as a regular user/tenant ('Project') and as a cloud administrator ('Admin'). — The speaker clarifies, 'admin user is also ah attainment or a ah attainment of the open stack cloud and also it is an administrator so we got two... components...one is project and one is admin' (10:24.910).

Q28 A · Pinging a website and then playing a YouTube video in a browser. — The demonstrator first checks connectivity ('lets check whether internet' - 16:40.579) and then proceeds to open a browser and stream a YouTube video (18:38.330) to show full functionality.

Q29 C · A template that defines the compute, memory, and storage resources for a VM. — The lecture describes different flavors like 'IITKGP Regular' with specific allocations of 'two b c p u four g b ram' (02:27.920), showing that a flavor is a resource template.

Q30 D · To be implemented and made functional in a practical system. — The presenter wants to show 'how these are realized' (00:52.500), meaning how the abstract concepts of cloud services are turned into a working, demonstrable system using OpenStack.

Q31 C · nova — When launching the instance, the presenter points out, 'currently we have only one availability zone which is nova' (11:35.470).

Q32 A · The VM launch will fail due to insufficient resources. — The OpenStack scheduler checks for available resources on the hypervisors before placing a VM. If the available resources (vCPU, RAM) are below what the requested flavor requires, the scheduling will fail, and the instance will not be created, typically entering an 'ERROR' state.

⚠ Not in the lecture; standard OpenStack scheduler behaviour (no host fits the flavour → launch fails, ERROR state).

Q33 D · Ubuntu and Fedora — The speaker lists the available operating systems as 'whom to [Ubuntu] and ferrora [Fedora]' (07:06.460). The demo itself used CentOS, which is related to this family of OS.

⚠ The lecture names three OS images: Ubuntu, CentOS and Fedora (the demo VM used CentOS). B is the only option taken from that list.

Q34 D · It deletes the VM and de-allocates its disk and RAM. — 'Shut Down' is like turning off a physical computer; the hardware (allocated resources) still belongs to you. 'Terminate' is like returning the computer to the store; it's gone, and all its resources are released (19:11.570).

Q35 B · The administrator of the Meghamala cloud. — The speaker introduces him by saying, 'rajesh is primarily a administrator of meghamala so he will so the how a ah v ms is created' (01:18.270).

Q36 D · That they can set up their own small-scale OpenStack cloud to gain practical experience. — The conclusion of the lecture encourages students, saying 'it is a good exercise to have a open source cloud of you' (20:36.710) to get a feel for both administrator and user roles.

Q37 C · A typo for 'vCPU hour', a unit of computing resource consumption. — The transcript shows 'd c p hour' (08:12.210), which is almost certainly a transcription error for 'vCPU hour'. This is a standard cloud computing metric representing one virtual CPU used for one hour, used for tracking usage and billing.

Q38 D · Image and Volume — While the demo only showed booting from an 'Image' (12:03.180), OpenStack also allows booting from a persistent 'Volume' (managed by Cinder). This is useful for having a persistent root disk that survives termination. 'Snapshot' is a type of image.

⚠ The demo only boots from an image; booting from a volume is general OpenStack knowledge.

Q39 D · 2 vCPU, 4 GB RAM, ephemeral storage — The slide overview explicitly lists the 'i i t k g p regular' flavor with 'two b c p u four g b ram for different g b a ephemeral storage' (02:27.920).

Q40 A · To view the VM's console output for debugging boot or system issues. — The 'View Log' or 'Console Log' option in OpenStack (mentioned at 18:13.300) shows the text output from the VM's virtual

console, which is crucial for troubleshooting problems where the VM is unresponsive over the network.

⚠ The lecture only lists 'view log' among the actions; what it shows (console output) is general knowledge.

Q41 C · Nova — The Nova component is responsible for all compute-related activities. This includes the 'nova-scheduler' service, which decides which physical hypervisor is best suited to host a new VM based on resource availability and scheduling policies.

⚠ 'nova-scheduler' is not named in the lecture; the lecture only ties hypervisors and compute to Nova.

Q42 C · Internal — The demonstrator says, 'currently i will select the internal network and i will come back to this external network later' (12:26.760 - 12:40.029). The internal network provides a private IP accessible only from within the cloud.

Q43 B · A standard desktop environment with internet access. — The demonstrator connects to the VM using X2Go, which opens a graphical desktop. From there, they open a terminal and a web browser to access the internet and play a video (15:55.809 - 17:04.010), demonstrating a complete user environment.

Q44 A · A Cinder volume (persistent storage). — The lecture distinguishes between ephemeral storage and 'persistent storage typically we give twenty g b' (02:39.280), which is managed by Cinder as 'volumes'. These volumes are independent of the VM lifecycle.

Q45 D · It is a platform for learning, research, and testing cloud concepts. — An 'experimental' cloud in an academic institution like IIT Kharagpur (00:55.760) is primarily for educational and research purposes, allowing students and faculty to experiment with cloud technologies in a controlled environment.

Q46 C · It provides administrators with a quick, at-a-glance understanding of the cloud's overall health and capacity. — The lecture mentions the dashboard gives 'a overall ah representation of the resource uses in terms of different graph by graphs' (03:49.320), which is for easy monitoring and summarization.

Q47 D · The user can only connect to the VM using SSH; all other connections like HTTP will be blocked. — Security groups act as a stateful firewall. A rule allowing only incoming traffic on port 22 means that only SSH connections will be accepted by the VM from the outside. Outgoing traffic is typically allowed by default unless explicitly denied.

⚠ Port-level rules are not in the lecture; standard security-group behaviour (inbound allow-list, outbound open by default).

Q48 B · The cloud has a pre-allocated pool of public IP addresses that can be assigned to VMs. — The demonstrator says, 'we have some allocated i p address' and 'new i p address from the pool will be generated' (13:45.699 - 13:54.810), indicating a managed pool of IPs is available for association.

Q49 B · Meghamala is the specific implementation or instance of an OpenStack cloud at IIT Kharagpur. — The lecture clarifies this multiple times, stating Meghamala is an 'experimental ah cloud using open source ah platform open stack' (01:03.320) and 'it is primarily a open stack based cloud' (01:11.590).

Q50 C · Virtual machines (VMs) — In OpenStack and general IaaS terminology, an 'instance' is a virtual machine. The number indicates how many VMs are currently running on that physical host (hypervisor). This is confirmed throughout the demo, e.g., 'coming to instances which is where we will find what are the v ms' (08:18.740).

Week 5 · L21 — SLA tutorial your file w2 · 50 questions

Lecture 21: SLA-Tutorial · 📖 notes: 5.1 SLA tutorial (L21)

Q1. What is the primary purpose of a Service Level Agreement (SLA) in cloud computing? Easy

A. To replace the need for any legal contract between consumer and provider.
B. To standardize the pricing of all cloud services globally.
C. To define a formal basis for the performance and availability of a service.
D. To serve as a marketing brochure for the cloud provider.

Q2. According to the lecture, why is there no single standard SLA across all cloud services? Medium

A. Different services, providers, and consumer scales make a single standard impractical.
B. Cloud providers refuse to cooperate on standardization.
C. The technology changes too rapidly to create a standard.
D. Legal jurisdictions prevent a global standard from being formed.

Q3. The lecture distinguishes between two types of agreements within an SLA. What are they? Easy

A. Software agreements and hardware agreements.
B. Technical agreements and financial agreements.
C. Metric-based agreements and policy-driven agreements.
D. Internal agreements and external agreements.

Q4. In the first problem presented, what was the guaranteed service availability percentage? Easy

A. 97%
B. 99%
C. 99.95%
D. 99.5%

Q5. In the first problem, an application runs 12 hours a day for a 30-day month. An outage of 10.75 hours is observed. What is the total time the application was supposed to run? Easy

A. 720 hours
B. 349.25 hours
C. 360 hours
D. 365 hours

Q6. Using the numbers from the first problem, if the guaranteed availability was 99%, how many hours of downtime would be the maximum allowed in the 360-hour period to meet the SLA? Medium

A. 12 hours
B. 3.6 hours
C. 1 hour
D. 10.75 hours

Q7. Based on the lecturer's calculation in the first problem, what was the final calculated service availability? Medium ⚠

A. 96.92%
B. 99.00%
C. 95.50%
D. 97.01%

Q8. The lecture discusses that availability requirements can vary. What is an example of a scenario where availability requirements might be tiered? Medium

A. A batch processing job that runs once a month.
B. A cloud storage service where all data is archived and rarely accessed.
C. A banking application that requires higher availability during business hours than overnight.
D. A static personal blog that has the same traffic day and night.

Q9. In the second problem, what is the consequence for the provider if the availability guarantee is not met? Easy

A. The contract is immediately terminated.
B. The provider must upgrade the consumer's hardware for free.
C. Service credits are awarded to the consumer.
D. The provider faces legal action.

Q10. In the second problem, what were the two tiers of service credits offered for failing to meet the 99.95% availability guarantee? Medium

A. 5% and 10%
B. 10% and 20%
C. 15% and 30%
D. 10% and 25%

Q11. What was the total initial cost of the cloud service for the 30-day period in the second problem? Easy

A. $1,200
B. $3,750
C. $1,500
D. $50

Q12. In the second problem, five outages were recorded. According to the lecture's calculation, what was the total service downtime? Medium ⚠

A. 9 hours 25 minutes
B. 8 hours 45 minutes
C. 10 hours 15 minutes
D. 9 hours 40 minutes

Q13. Based on the calculated availability of 97.385% in the second problem, which service credit tier was applied? Medium ⚠

A. The 10% credit tier.
B. The 25% credit tier.
C. A special 50% credit tier for major failures.
D. No credit was applied as availability was above 97%.

Q14. What was the final effective cost payable by the consumer in the second problem after the service credit was applied? Medium

A. $1,500
B. $1,125
C. $375
D. $1,350

Q15. If the total downtime in the second problem had been exactly 3 hours, what would the service availability have been, and which credit would apply? Hard

A. 99.5%, no credit
B. 99.17%, 10% credit
C. 98.5%, 25% credit
D. 99.17%, no credit

Q16. In the context of commercial SLAs as discussed for Microsoft Azure, what does a 'Fault Domain' represent? Medium

A. A group of virtual machines with the same software bug.
B. A collection of servers that share common resources like power and network.
C. The department responsible for fixing service failures.
D. A geographic region prone to network outages.

Q17. What is the purpose of deploying virtual machines in an 'Availability Set' across different fault and update domains? Medium

A. To ensure all VMs are updated at the same time.
B. To avoid a single point of failure and improve uptime.
C. To group all development and testing machines together.
D. To lower the cost of the virtual machines.

Q18. According to the lecture, what HTTP status code series typically indicates a server-side failure or 'error code' relevant to SLA downtime calculations? Easy

A. 1xx (Informational)
B. 4xx (Client Error)
C. 5xx (Server Error)
D. 2xx (Success)

Q19. The Cloud Standards Customer Council provides a 7-step guide for evaluating SLAs. What is the first step? Easy

A. Evaluate business-level policies.
B. Identify the cloud actors.
C. Understand the service model (IaaS, PaaS, SaaS).
D. Prepare for failure.

Q20. Which of the following is NOT listed as a cloud actor according to the NIST architecture mentioned in the lecture? Medium

A. Auditor
B. Provider
C. Consumer
D. Regulator

Q21. A startup is considering a cloud provider but is worried about being locked in. Which part of the SLA best practices should they pay close attention to? Hard

A. Identifying the cloud broker.
B. The list of supported success codes.
C. The definition of 'Fault Domain'.
D. The disaster recovery plan and exit clause.

Q22. Evaluating data policies, services not covered, and software licensing falls under which of the seven best practice steps? Medium

A. Step 4: Identify metrics
B. Step 2: Evaluate business-level policies
C. Step 6: Service management requirements
D. Step 1: Identify actors

Q23. Why is it important to understand the service model (IaaS, PaaS, SaaS) when evaluating an SLA? Hard

A. Because the complexity of control variables given to the consumer differs between models.
B. Because only IaaS provides an SLA.
C. Because PaaS does not allow for disaster recovery.
D. Because the price is always higher for SaaS.

Q24. A media company needs to stream video with minimal buffering. Besides availability, which metric category from Step 4 of the best practices would be most important for their SLA? Medium

A. Cost metrics
B. Compliance metrics
C. Security metrics
D. Response metrics

Q25. What is a key consideration under Step 5, 'Consider key security parameters'? Medium

A. The number of available virtual machine instances.
B. The speed of application deployment.
C. Legal and regulatory requirements for data.
D. The cost of the service.

Q26. A rapidly growing e-commerce site needs its cloud environment to scale automatically during flash sales. This relates to which aspect of the 'Service Management Requirements' (Step 6)? Hard

A. The exit clause.
B. How disaster recovery is handled.
C. What should be metered.
D. How rapid provisioning and demand flexibility should be managed.

Q27. What does an 'Update Domain' refer to in the context of the Azure example? Medium

A. A database that logs all system updates.
B. A website for downloading software patches.
C. A set of instances to which platform updates are concurrently applied.
D. A user group that receives notifications about updates.

Q28. The lecture states that the complexity of SLA control variables depends on the service and cloud type. Which combination would likely give the consumer the MOST control variables to manage and monitor? Hard

A. IaaS on a private cloud
B. SaaS on a hybrid cloud
C. SaaS on a public cloud
D. PaaS on a private cloud

Q29. What does the term 'remedy' refer to in Step 7 of the SLA evaluation best practices? Medium

A. The act of rebooting a failed server.
B. A software patch to fix a security vulnerability.
C. The compensation provided for SLA failures, such as service credits.
D. The technical support team that fixes problems.

Q30. Which of the following is an example of a policy-driven agreement in an SLA? Medium

A. A guarantee of 99.9% uptime.
B. A rule that all customer data must reside within the European Union.
C. A target for network latency to be under 50ms.
D. A promise of average CPU utilization below 80%.

Q31. In the context of the lecture, what is an SLO (Service Level Objective)? Easy

A. A legal document that overrides the SLA.
B. An objectively measurable condition for a service, which is a building block for an SLA.
C. A summary of service outages over the last year.
D. The marketing term for a service package.

Q32. A provider's SLA promises 99.5% availability over a 30-day month (720 hours). What is the maximum total downtime allowed in minutes to meet this guarantee? Medium

A. 360 minutes
B. 432 minutes
C. 3.6 minutes
D. 216 minutes

Q33. In problem 2, if the calculated availability was exactly 99.00%, which service credit would be applied? Hard

A. No credit
B. 25%
C. 10%
D. A prorated amount between 10% and 25%

Q34. What is the function of a 'Management Portal' as described in the lecture? Easy

A. A web interface for the consumer to manage their services.
B. A monitoring dashboard used only by the provider's engineers.
C. An API for automated service deployment.
D. A physical location where servers are managed.

Q35. According to the lecture, what is an 'incident' in the context of an SLA? Medium

A. A security breach.
B. Any customer support ticket.
C. A planned maintenance window.
D. Any single or set of events that result in downtime.

Q36. A hospital needs to store patient records in the cloud. Which two SLA evaluation steps are most critical for ensuring compliance and patient privacy? Hard

A. Step 4 (Metrics) and Step 7 (Failure Management)
B. Step 2 (Business Policies) and Step 5 (Security)
C. Step 1 (Actors) and Step 4 (Metrics)
D. Step 3 (Service Models) and Step 6 (Service Management)

Q37. Why would a consumer want to meter their service usage, as mentioned in Step 6 of the best practices? Medium

A. To verify the provider's billing and manage costs.
B. To reduce the number of cloud actors involved.
C. To gain administrator access to the provider's servers.
D. To increase the speed of their application.

Q38. The lecture mentions 'sub contract services' as a consideration in business-level policies. What risk does this address? Hard

A. The risk that the cloud provider uses a third party for part of its service, which may have different security standards.
B. The risk that the main cloud provider might go out of business.
C. The risk of using unlicensed subcontracted software.
D. The risk that the consumer might hire their own subcontractors.

Q39. In the first problem, the provider's guarantee was violated. What is the immediate, direct outcome based on the problem's description? Medium

A. The consumer receives a service credit.
B. The lecturer concludes that the provider has violated the SLA.
C. The contract is automatically renewed with a discount.
D. The provider's CEO must issue a public apology.

Q40. What is the role of a 'broker' in the NIST cloud actor model? Hard ⚠

A. An entity that provides the physical network connection.
B. An entity that manages the performance of a cloud service.
C. An entity that manages the use, performance, and delivery of cloud services and negotiates relationships between providers and consumers.
D. An entity that audits the cloud provider for compliance.

Q41. What distinguishes a 'web role' from a 'worker role' in the context of the Azure definitions provided? Medium ⚠

A. Web roles are for external users, worker roles are for internal employees.
B. Web roles are more expensive than worker roles.
C. Web roles have a guaranteed SLA, while worker roles do not.
D. Web roles run web applications with a UI (like a website), while worker roles typically perform background processing.

Q42. If a cloud provider guarantees 99.8% monthly uptime, how much downtime per month (30 days) is implicitly accepted by the consumer? Medium

A. Approximately 1.44 hours
B. Approximately 7.2 hours
C. Approximately 14.4 hours
D. Approximately 3.6 hours

Q43. The lecture mentions that a 'disaster recovery plan' should be part of the failure management strategy. What is the primary goal of such a plan? Medium

A. To define the procedures for restoring service after a major outage or disaster.
B. To exit the contract with the cloud provider.
C. To determine who was at fault for the disaster.
D. To calculate the service credit owed after a failure.

Q44. A consumer signs an SLA for a VM with 99.9% uptime. The provider performs scheduled maintenance, taking the VM offline for 2 hours. This maintenance was announced weeks in advance. Does this typically count against the uptime guarantee? Hard ⚠

A. Only if the maintenance takes longer than one hour.
B. Yes, but it only counts as half the actual time.
C. Yes, all downtime is counted regardless of the cause.
D. No, scheduled maintenance is usually excluded from uptime calculations in the SLA's fine print.

Q45. What is meant by 'External Connectivity' in the list of SLA definitions? Medium

A. The consumer's ability to connect to the service from a public IP address.
B. The ability to connect a laptop to an external monitor.
C. The provider's connection to the internet backbone.
D. The connection between two virtual machines in the same VNet.

Q46. In problem 2, what would the effective cost be if the calculated availability was 99.2%? Hard

A. $1,500
B. $1,200
C. $1,125
D. $1,350

Q47. The lecture emphasizes the importance of an 'exit clause'. What is the most significant challenge a consumer faces when exiting an SLA without a clear exit clause? Hard

A. Incurring financial penalties for early termination.
B. Migrating their data and applications out of the provider's environment.
C. Finding a new provider.
D. Recalculating all previous service credits.

Q48. Which of the following is the BEST example of a 'Tenant' as defined in the lecture? Medium

A. A physical server rack in the data center.
B. A customer's entire deployment, consisting of one or more roles in a single package.
C. The cloud provider company itself.
D. A single virtual machine.

Q49. Why is it important for an SLA to define 'Success Codes' (e.g., HTTP 2xx)? Medium

A. To provide a baseline for what constitutes a successful, non-downtime transaction.
B. To trigger billing events for the customer.
C. To count the number of successful user logins.
D. To confirm that an operation has failed.

Q50. A company is using a cloud provider's IaaS offering. They are responsible for patching the operating system of their VMs. If a VM is down because they failed to apply a critical patch, would this typically count against the provider's SLA uptime guarantee? Hard ⚠

A. Only if the provider failed to notify the company about the patch.
B. Yes, the provider is responsible for all downtime.
C. No, the provider is only responsible for the infrastructure; failures caused by the consumer's software or configuration are excluded.
D. Yes, but the provider only has to pay 50% of the service credit.

Answer key — Week 5 · L21 (your file w2)

Q1 C · To define a formal basis for the performance and availability of a service. — As stated in the lecture (around 03:54), the purpose of an SLA is to define a formal basis for the performance and availability of the service, where the provider guarantees delivery.

Q2 A · Different services, providers, and consumer scales make a single standard impractical. — The lecturer mentions (around 01:25) that a major reason for the lack of a standard SLA is the variety of service types, different providers, and different scales of consumer requirements.

Q3 C · Metric-based agreements and policy-driven agreements. — Around 02:31, the lecture explains that some SLA components are metric-based (like uptime or CPU usage), while others are policy-driven (like data residency or backup policies).

Q4 B · 99% — The first problem (04:36) begins with the premise that the cloud provider guarantees service availability of 'ninety nine percent of of time'.

Q5 C · 360 hours — The calculation shown at 05:40 is 12 hours/day multiplied by 30 days, which equals 360 hours for the month.

Q6 B · 3.6 hours — To meet a 99% availability guarantee over 360 hours, the maximum allowed downtime is 1% of 360 hours. 0.01 * 360 = 3.6 hours. Since the observed outage was 10.75 hours, the SLA was violated.

Q7 A · 96.92% — At 07:23, the lecturer concludes the calculation, stating the percentage availability is 96.92%. This value is less than the 99% guarantee, indicating a violation.

⚠ The lecture divides 10.75 by the uptime (349.25 h) and gets 96.92 %. The standard formula (360 – 10.75)/360 gives 97.01 %, which is option B here. Keys follow the lecture, so pick 96.92 %; the verdict (violated) is the same either way.

Q8 C · A banking application that requires higher availability during business hours than overnight. — The lecturer provides a specific example of a banking organization (09:52) that might require 99.9% availability during peak hours (9:00-17:00) but a lower percentage like 97% overnight.

Q9 C · Service credits are awarded to the consumer. — The problem states (13:17) that 'service credits are awarded to the consumer if availability guarantees are not satisfied', which acts as a penalty for the provider.

Q10 D · 10% and 25% — The problem outlines two penalty tiers (13:58): a 10% credit for availability between 99% and 99.95%, and a 25% credit for availability less than 99%.

Q11 C · $1,500 — The cost is given as $50 per day (12:48). For a 30-day service period, the total cost at the time of negotiation is $50 * 30 = $1,500 (17:20).

Q12 A · 9 hours 25 minutes — Although a direct sum of the outages (5h 30m, 1h 30m, 15m, 2h 25m) is 9 hours 40 minutes, the lecturer explicitly states the sum is 'nine hours twenty five minutes' at 18:38. The question asks for the value used in the lecture's calculation.

⚠ Only four outages are listed, and they add up to 9 h 40 min (option B). The lecture says 9 h 25 min and the key follows it. The 25 % credit and $1,125 payable are the same either way.

Q13 B · The 25% credit tier. — The calculated availability of 97.385% is less than 99%. As per the SLA terms (20:30), this falls into the category that qualifies for a 25% service credit.

⚠️ 97.385 % uses the lecture's 9 h 25 min. With the true sum (9 h 40 min) it is 97.31 %, still below 99 %, so the credit is still 25 %.

Q14 B · $1,125 — The initial cost was $1,500. A 25% credit amounts to $375 (0.25 * 1500). The effective cost is the initial cost minus the credit: $1500 - $375 = $1,125 (22:02).

Q15 B · 99.17%, 10% credit — With 3 hours of downtime over 360 hours, the uptime is 357 hours. Availability = (357 / 360) * 100 = 99.17%. This is less than the 99.95% guarantee but greater than 99%, so it qualifies for the 10% service credit.

Q16 B · A collection of servers that share common resources like power and network. — The lecture defines a Fault Domain (26:54) as a collection of servers sharing common resources such as power and network connectivity. Deploying across fault domains helps avoid single points of failure.

Q17 B · To avoid a single point of failure and improve uptime. — As defined at 26:25, an Availability Set involves deploying two or more VMs across different fault domains to avoid downtime from a single point of failure (e.g., a rack losing power).

Q18 C · 5xx (Server Error) — The lecture mentions (around 24:57) that an 'error code' can be an indication of a failed operation, such as an HTTP status code in the 5xx series, which signifies a server error.

Q19 B · Identify the cloud actors. — The first step listed from the Cloud Standards Customer Council's guide (29:44) is to 'identify the cloud actors,' which according to the NIST architecture are the consumer, provider, carrier, broker, and auditor.

Q20 D · Regulator — At 30:07, the lecturer lists the five actors from the NIST architecture as consumer, producer (provider), carrier, broker, and auditor. 'Regulator' is not on this list, although they influence policies.

Q21 D · The disaster recovery plan and exit clause. — Step 7 of the best practices, 'Prepare for and manage for the failure' (33:54), explicitly mentions that an 'exit clause should be a part of every cloud sla' (34:29) for when a consumer or provider wants to terminate the relationship.

Q22 B · Step 2: Evaluate business-level policies — Step 2, 'evaluate the business level policies' (30:15), covers aspects like data policies, services not covered, payment penalties, and licensing, which are crucial for defining the business relationship.

Q23 A · Because the complexity of control variables given to the consumer differs between models. — As stated at 31:48, the terms and conditions in an SLA depend on the complexity of control variables that the provider gives to the consumer, which varies significantly between IaaS, PaaS, and SaaS.

Q24 D · Response metrics — Step 4 (32:40) mentions identifying metrics to achieve performance objectives, giving 'availability' and 'response metrics' as examples. For video streaming, low latency (a response metric) is as critical as availability.

Q25 C · Legal and regulatory requirements for data. — Step 5 (33:03) involves considering security parameters, including data sensitivity and importantly, 'legal regulatory requirements,' such as data residency rules.

Q26 D · How rapid provisioning and demand flexibility should be managed. — Step 6, Service Management Requirements (33:25), includes considerations for 'how rapid provisioning should be' and 'demand flexibility', which are essential for handling variable loads like a flash sale.

Q27 C · A set of instances to which platform updates are concurrently applied. — The lecture defines an Update Domain (27:15) as a set of Microsoft Azure instances where platform updates are applied

concurrently. Spreading VMs across update domains prevents them all from being rebooted simultaneously for an update.

Q28 A · IaaS on a private cloud — IaaS (Infrastructure as a Service) provides the most control over the underlying infrastructure compared to PaaS and SaaS. A private cloud typically offers more control and customization than a public cloud. Therefore, IaaS on a private cloud would offer the most control variables.

Q29 C · The compensation provided for SLA failures, such as service credits. — In Step 7, 'Prepare for and manage for the failure' (33:54), the lecturer asks 'what remedy should provided like for example service credits.' This refers to the compensation for service failures.

Q30 B · A rule that all customer data must reside within the European Union. — Around 02:34, the lecture gives 'where your data through reside' as an example of a policy-driven agreement, as opposed to a purely numerical metric like uptime or CPU usage.

Q31 B · An objectively measurable condition for a service, which is a building block for an SLA. — The lecture briefly recaps (04:08) that SLOs are 'objectively measurable condition for services' and form the basis for creating the guarantees within an SLA.

Q32 D · 216 minutes — The total time is 720 hours. The allowed downtime is 0.5% (100% - 99.5%). 0.005 * 720 hours = 3.6 hours. To convert to minutes: 3.6 hours * 60 minutes/hour = 216 minutes.

Q33 C · 10% — The first penalty tier is for availability 'less than ninety nine point nine five percent but more than ninety nine percent greater than equal to ninety nine percent' (14:05). Since 99.00% meets the 'greater than or equal to' condition, the 10% credit would apply.

Q34 A · A web interface for the consumer to manage their services. — The 'management portal' is defined (25:25) as the web interface provided (in this case, by Microsoft Azure) through which the consumer can manage their services.

Q35 D · Any single or set of events that result in downtime. — An 'incident' is defined around 25:25 as 'any single or a set of incident that result in down time.' This is a key term for tracking events that impact availability.

Q36 B · Step 2 (Business Policies) and Step 5 (Security) — Step 2 covers data policies, and Step 5 covers security, legal, and regulatory requirements (33:03). For a hospital, ensuring data is handled according to legal regulations (like HIPAA) and secured properly is paramount.

Q37 A · To verify the provider's billing and manage costs. — Step 6 discusses what should be monitored, reported, and 'metered' (33:30). Metering allows a consumer to track their resource consumption, which is essential for cost management and for verifying that the provider's bills are accurate.

Q38 A · The risk that the cloud provider uses a third party for part of its service, which may have different security standards. — When evaluating business-level policies (Step 2, 30:27), understanding if a provider subcontracts parts of their service is crucial. The consumer's data or service might depend on a third party whose SLA and security posture are unknown or weaker than the main provider's.

Q39 B · The lecturer concludes that the provider has violated the SLA. — The first problem (04:36 - 08:40) is a simple calculation to determine if a violation occurred. Unlike the second problem, it does not specify any penalties or service credits. The only outcome described is the conclusion that the CSP 'has violated the sla' (08:27).

Q40 C · An entity that manages the use, performance, and delivery of cloud services and negotiates relationships between providers and consumers. — While the lecture only lists the actors (30:07), the standard NIST definition of a cloud broker is an intermediary that provides value-added services on top of cloud infrastructure, often by aggregating services or enhancing security.

⚠ The lecture only lists the five NIST actors; the broker definition comes from NIST SP 500-292.

Q41 D · Web roles run web applications with a UI (like a website), while worker roles typically perform background processing. — The lecture mentions web and worker roles (26:47, 27:28). This is a classic Azure concept where a Web Role is designed to host a web application front-end (IIS), and a Worker Role is for general-purpose background tasks without a direct UI.

⚠ The lecture only lists the Azure terms; web role = front end, worker role = background processing (see Week 4 Azure notes).

Q42 A · Approximately 1.44 hours — A 30-day month has 30 * 24 = 720 hours. A 99.8% uptime guarantee allows for 0.2% downtime. 0.002 * 720 hours = 1.44 hours of downtime.

Q43 A · To define the procedures for restoring service after a major outage or disaster. — As part of managing failure (Step 7), the lecture asks 'how the disaster recovery plan will work when needed' (34:22). The purpose of a DR plan is to enable the recovery of IT infrastructure and services following a disaster.

Q44 D · No, scheduled maintenance is usually excluded from uptime calculations in the SLA's fine print. — While the lecture does not detail this, standard industry SLAs almost always have clauses that exclude pre-announced scheduled maintenance from the downtime calculation. This is a critical nuance of SLA evaluation.

⚠ Not in the lecture; this is common industry SLA practice (announced maintenance is usually excluded).

Q45 A · The consumer's ability to connect to the service from a public IP address. — The lecture defines 'External Connectivity' (25:14) as bidirectional network traffic over protocols like HTTP/HTTPS that can be received from a public IP. It's about the service being reachable from the outside world.

Q46 D · $1,350 — An availability of 99.2% falls in the tier 'less than 99.95% but greater than or equal to 99%,' which warrants a 10% service credit. The credit would be 10% of $1,500, which is $150. The effective cost would be $1,500 - $150 = $1,350.

Q47 B · Migrating their data and applications out of the provider's environment. — While financial penalties can be an issue, a major technical and logistical challenge of leaving a provider is vendor lock-in. A good exit clause (34:29) should define the provider's responsibilities in facilitating data export and a smooth transition, which is often a complex process.

Q48 B · A customer's entire deployment, consisting of one or more roles in a single package. — A 'tenant' is defined (27:01) as representing 'one or more roles... deployed in a single packages'. This refers to a customer's application or deployment instance running on the shared cloud infrastructure.

Q49 A · To provide a baseline for what constitutes a successful, non-downtime transaction. — Just as error codes (5xx) define failure, success codes (2xx) (26:03) are crucial for defining what constitutes a successful operation. Monitoring systems use these to distinguish between successful requests and errors when calculating availability.

Q50 C · No, the provider is only responsible for the infrastructure; failures caused by the consumer's software or configuration are excluded. — In an IaaS model, there is a shared responsibility. The provider guarantees the infrastructure (network, power, hypervisor), but the consumer is responsible for what's inside the VM (OS, applications). A failure caused by the consumer's mismanagement would not be a violation of the provider's SLA.

⚠ Not in the lecture; this is the shared-responsibility idea (provider covers infrastructure, consumer covers OS and apps in IaaS).

Week 6 · L26 — Security I

your file w3 · 50 questions

Lecture 26: Cloud Computing: Security I · 📖 notes: 6.1 Security fundamentals (L26)

Q1. According to the lecture, what is a primary reason security is a major concern in cloud computing? Easy

A. Users are relying on a third-party service provider.
B. Cloud computing is a new and untested technology.
C. Cloud services use outdated hardware.
D. The internet protocols used are inherently insecure.

Q2. The lecture identifies one of the major hindrances to cloud adoption as being more significant than technology itself. What is this hindrance? Medium

A. The lack of skilled cloud engineers.
B. The complexity of the user interface for cloud dashboards.
C. Concerns about security, policies, and data handling.
D. The high cost of cloud migration.

Q3. What are the three basic components of the security model often referred to as the 'CIA triad'? Easy

A. Control, Identification, Authorization
B. Confidentiality, Integrity, Availability
C. Cybersecurity, Information, Assurance
D. Confidentiality, Integrity, Authentication

Q4. A Distributed Denial of Service (DDoS) attack primarily compromises which component of the CIA triad? Medium

A. Availability
B. Confidentiality
C. Integrity
D. Authenticity

Q5. An attacker intercepts a message between a source and destination but does not alter it, only listens. Which type of attack is this, and which security principle does it violate? Medium

A. Interruption, violating Availability
B. Modification, violating Integrity
C. Fabrication, violating Authenticity
D. Interception, violating Confidentiality

Q6. In the context of security attacks, what is 'Fabrication'? Medium

A. An unauthorized party tampers with an asset.
B. An unauthorized party gains access to an asset.
C. An unauthorized party inserts counterfeit objects into the system.
D. An asset of the system is destroyed or becomes unavailable.

Q7. A 'man-in-the-middle' attack where an intruder intercepts a bank transfer instruction, changes the destination account number, and forwards the altered message is an example of which type of attack? Hard

A. Modification
B. Interruption
C. Fabrication
D. Replay

Q8. What is the fundamental difference between security 'Policies' and 'Mechanisms' as described in the lecture? Medium

A. Policies state what is allowed, while mechanisms enforce those statements.
B. Policies are created by managers, while mechanisms are created by engineers.
C. Policies are hardware-based, while mechanisms are software-based.
D. Policies are for networks, while mechanisms are for individual computers.

Q9. The lecture mentions that composing multiple security policies can create vulnerabilities. Why does this happen? Hard

A. Conflicts or discrepancies between policies can be exploited.
B. Mechanisms can only enforce one policy at a time.
C. Each new policy reduces the overall system performance.
D. Policies are written in different programming languages.

Q10. Which of the following is NOT one of the three major security goals discussed in the lecture? Easy

A. Prevention
B. Detection
C. Retaliation
D. Recovery

Q11. What is the primary objective of the 'Recovery' security goal? Medium

A. To stop an attack, assess damage, and continue to function correctly.
B. To punish the attacker legally.
C. To prevent any future attacks of the same type.
D. To detect an attack as soon as it begins.

Q12. The lecture describes a system as 'fully secure' under what condition? Hard

A. When the set of reachable states is a subset of the set of secured states.
B. When the set of secured states is empty.
C. When the set of reachable states is larger than the set of secured states.
D. When there are no reachable states.

Q13. What does the concept of 'Assurance' in security involve? Medium

A. Only focusing on post-attack investigation and forensics.
B. Guaranteeing 100% prevention of all possible attacks.
C. A process involving specification, design, and implementation to ensure security functionalities.
D. Buying insurance against cyber attacks.

Q14. In a lab environment where no data is stored on local machines, the lecture suggests that it might be cheaper to recover from an attack (by reinstalling an image) than to prevent it. This is an example of what kind of analysis? Medium

A. Penetration testing
B. Threat analysis
C. Vulnerability scanning
D. Cost-benefit analysis

Q15. A threat of 'usurpation' involves which of the following attack types? Hard

A. Only denial of receipt
B. Traffic analysis
C. Snooping only
D. Modification, spoofing, delay, or denial of service

Q16. Which of the following best describes a 'passive attack'? Easy

A. An attack that modifies the data stream.
B. An attack that obtains information being transmitted without altering system resources.
C. An attack that creates a false data stream.
D. An attack that prevents the normal use of communication facilities.

Q17. An attacker observing the volume and timing of network traffic between two parties, without reading the content, is performing what kind of analysis? Medium

A. Content analysis
B. Traffic analysis
C. Denial of service
D. Modification analysis

Q18. An attack where an entity pretends to be a different, legitimate entity is known as: Medium

A. Modification
B. Denial of Service
C. Masquerade
D. Replay

Q19. What is a 'Replay' attack? Medium

A. An attacker passively captures a data unit and retransmits it later to produce an unauthorized effect.
B. An attacker sends the same message over and over to flood a server.
C. An attacker pretends to be a different user.
D. An attacker alters part of a legitimate message.

Q20. The security service that ensures a sender cannot falsely deny having sent a message is called: Medium

A. Non-repudiation
B. Integrity
C. Confidentiality
D. Authentication

Q21. What is the role of 'Authorization' in security services? Medium

A. To determine if a user is allowed to perform a certain operation.
B. To keep data secret from unauthorized users.
C. To ensure data is not altered.
D. To verify the identity of a user.

Q22. What is the first step in the network security lifecycle presented in the lecture? Easy

A. Vulnerability scanning
B. Penetration testing
C. Determination of network security policies
D. Implementation of firewalls

Q23. In network security, what is the purpose of 'reconnaissance'? Medium

A. To install and configure firewalls and IDS.
B. To learn about the network, such as IP addresses and key servers.
C. To actively attack the system to find weaknesses.
D. To repair damage after an attack.

Q24. What is the purpose of a 'Honeypot' or 'Honeynet' in a network security setup? Hard ⚠

A. To perform high-speed data processing.
B. To act as a primary firewall for the internal network.
C. To store the most critical company data.
D. To be a deliberately vulnerable system to attract and study attackers.

Q25. The lecture mentions using tools like 'Nmap' for what purpose? Medium

A. Vulnerability scanning.
B. Post-attack forensic analysis.
C. Writing security policies.
D. Encrypting network traffic.

Q26. What is 'Penetration Testing'? Medium

A. A safe, authorized attempt to exploit a computer system's vulnerabilities.
B. A process of documenting security policies.
C. The investigation that occurs after a security breach.
D. The daily monitoring of network logs for suspicious activity.

Q27. The process of 'post-attack investigation' is also known as: Easy

A. Penetration testing
B. Risk analysis
C. Forensics
D. Reconnaissance

Q28. The lecture mentions the National Vulnerability Database (NVD). What is its purpose? Medium

A. A real-time intrusion detection system.
B. To prosecute cybercriminals.
C. A tool for performing penetration tests.
D. A repository that lists known security vulnerabilities.

Q29. Why is it important for vulnerability scanners to be updatable? Medium

A. To improve their user interface.
B. To incorporate information about newly discovered vulnerabilities and attack signatures.
C. To make them compatible with new operating systems.
D. To reduce their cost over time.

Q30. Which class of threat is associated with 'snooping'? Hard

A. Usurpation
B. Deception
C. Disclosure
D. Disruption

Q31. A threat of 'deception' includes which of the following? Hard

A. Only delaying messages
B. Only modifying the service
C. Modification, spoofing, repudiation of origin, denial of receipt
D. Only snooping and traffic analysis

Q32. A company's central IT department has a policy that its implementation is distributed across various switches and routers in different departments. According to the lecture, what is a major challenge in this scenario? Hard

A. The central policy is always weaker than local policies.
B. Guaranteeing that the distributed implementation conforms to the central policy.
C. Distributed devices cannot be managed remotely.
D. The physical distance between devices causes network latency.

Q33. Which security role is responsible for protecting against data alteration or corruption? Easy

A. Availability
B. Integrity
C. Authorization
D. Confidentiality

Q34. The lecture states, 'policies unambiguously partition system states.' What does this mean? Hard

A. Policies should clearly define whether a given system state is secure or not, with no ambiguity.
B. Policies make the system run slower.
C. Policies force the system into a single, static state.
D. Policies can only be applied to a portion of the system at any time.

Q35. According to the lecture, human issues like 'organizational problems or people problems' are part of the overall security landscape. This implies that: Medium

A. Humans are always more trustworthy than automated systems.
B. Security is purely a technical problem.
C. Technology can solve all security issues without human intervention.
D. The human element is often a factor in security vulnerabilities and must be considered.

Q36. If an active attacker prevents the normal use of communication facilities, which specific type of attack are they committing? Easy

A. Replay
B. Masquerade
C. Modification
D. Denial of Service

Q37. Which of the following attack types are generally considered 'difficult to detect' because they 'don't do direct harm'? Medium

A. Modification attacks
B. Passive attacks
C. Active attacks
D. Denial of Service attacks

Q38. The integrated security model shows 'operational issues' can feed back into several stages. What is one way operational issues act as a feedback loop? Hard

A. They only provide feedback to the end-users.
B. They can become a new type of threat.
C. They automatically fix bugs in the implementation.
D. They have no impact on the security model.

Q39. When discussing the trust and assumptions in security, what example does the lecturer provide from day-to-day life? Medium

A. Trusting that a security person guarding a premise can be trusted.
B. Trusting that your computer's operating system has no bugs.
C. Trusting that email is a secure form of communication.
D. Trusting that a bank will secure your money.

Q40. Why is 'Access Control' considered a crucial security service? Medium

A. It encrypts all data on the hard drive.
B. It prevents the misuse of resources by unauthorized or improperly authorized users.
C. It ensures that users cannot deny their actions.
D. It guarantees 100% uptime and availability.

Q41. The lecture gives an example of critical data that requires serious security consideration. Which of the following was NOT mentioned in that context? Easy

A. Social media posts
B. Defense data
C. Academic data (student results)
D. Banking data

Q42. If a system's set of reachable states and its set of secured states perfectly overlap, the security is described as: Hard

A. Unstable
B. Precise
C. Broad
D. Insecure

Q43. The lecture mentions several specific attack methods, such as 'phishing' and 'buffer overflow'. These are categorized under what general heading? Easy

A. Operational Issues
B. Security Policies
C. Security Services
D. Series of Attacks

Q44. A network design that includes a 'Demilitarized Zone' (DMZ) with two firewalls is an example of what? Medium

A. Implementing a security policy
B. Determining security policy
C. Post-attack investigation
D. Passive reconnaissance

Q45. Which of the following is an example of a 'human issue' in security? Medium

A. An employee tricked by a phishing email into revealing their password.
B. A server failure due to a power outage.
C. A software bug that leads to a buffer overflow.
D. A firewall rule that is too permissive.

Q46. What is the key characteristic of an 'active attack' that distinguishes it from a passive one? Medium

A. It is impossible to detect.
B. It can only be performed by a human, not a bot.
C. It is always successful.
D. It involves some modification of the data stream or creation of a false stream.

Q47. The term 'integrity' in information security can refer to data integrity and what other type of integrity mentioned in the lecture? Hard

A. Network integrity
B. Hardware integrity
C. Origin or source integrity
D. System integrity

Q48. What is a key activity during the 'post-attack investigation' phase of network security? Medium

A. Retaining the chain of evidence for legal purposes.
B. Implementing a new firewall.
C. Writing a new security policy from scratch.
D. Scanning the network for new vulnerabilities.

Q49. According to the lecture, the concepts of security threats, policies, and attack types discussed are: Easy

A. Only applicable to network security and not cloud security.
B. Outdated and no longer used in modern systems.
C. Only relevant for personal computers, not large systems.
D. Generic concepts that are also relevant to cloud computing.

Q50. A security policy that is enforced by a firewall's rule set is an example of a policy being enacted by a: Medium

A. Mechanism
B. Human Issue
C. Threat
D. Vulnerability

Answer key — Week 6 · L26 (your file w3)

Q1 A · Users are relying on a third-party service provider. — The lecture emphasizes that when using cloud services (IaaS, PaaS, SaaS), you are entrusting your applications, data, and processes to a third party, which inherently raises security and trust issues.

Q2 C · Concerns about security, policies, and data handling. — The lecturer states that a major hindrance to cloud adoption is not the technology, but rather the concerns surrounding security and data policies.

Q3 B · Confidentiality, Integrity, Availability — The lecture explicitly defines the three basic security components as Confidentiality, Integrity, and Availability, or the 'CIA components'.

Q4 A · Availability — The lecture explains that denial of service attacks compromise 'Availability' by preventing legitimate users from accessing data and resources.

Q5 D · Interception, violating Confidentiality — This scenario describes an 'Interception' attack, also known as eavesdropping or snooping. It is an attack on 'Confidentiality' because an unauthorized party gains access to the information.

Q6 C · An unauthorized party inserts counterfeit objects into the system. — The lecture describes 'Fabrication' as an attack on authenticity where an intruder pretends to be a legitimate source and sends a false message, effectively inserting a counterfeit object into the communication.

Q7 A · Modification — This is a classic 'Modification' attack. The attacker intercepts a legitimate message and alters its content, compromising the integrity of the data.

Q8 A · Policies state what is allowed, while mechanisms enforce those statements. — The lecture clearly distinguishes the two: 'Policy says what is and what is not allowed,' whereas 'mechanisms enforce policies.'

Q9 A · Conflicts or discrepancies between policies can be exploited. — As stated in the lecture, 'if the policies conflict, discrepancies may create security vulnerabilities.' An example given is one policy allowing traffic that another policy denies, creating ambiguity.

Q10 C · Retaliation — The lecture outlines three primary security goals: Prevention (preventing violations), Detection (detecting violations), and Recovery (recovering from attacks). Retaliation is not mentioned as a security goal.

Q11 A · To stop an attack, assess damage, and continue to function correctly. — The lecture describes recovery as the process of stopping the attack, assessing and repairing damage, and ensuring the system can continue to function, even if the attack was partially successful.

Q12 A · When the set of reachable states is a subset of the set of secured states. — The lecturer explains with a diagram that a system is considered fully secure if all of its possible 'reachable states' are contained within the boundary of what is defined as 'secured states'.

Q13 C · A process involving specification, design, and implementation to ensure security functionalities. — Assurance is presented as a structured process that includes requirement analysis (specification), designing how to meet them, and the actual implementation, all to ensure the system meets desired security functionalities.

Q14 D · Cost-benefit analysis — The lecturer uses this scenario to illustrate 'cost-benefit analysis,' where an organization weighs the cost of prevention against the cost of recovery to decide on the most economical security strategy.

Q15 D · Modification, spoofing, delay, or denial of service — The lecture categorizes threats and lists the 'threat of usurpation' as including modification, spoofing, delay, and denial of services.

Q16 B · An attack that obtains information being transmitted without altering system resources. — Passive attacks, like eavesdropping and traffic analysis, involve obtaining information without changing it or disrupting the system. The lecture notes they are difficult to detect for this reason.

Q17 B · Traffic analysis — The lecture defines 'traffic analysis' as a type of passive attack where the attacker analyzes patterns in communication (like volume and timing) to infer information, even if the content itself is encrypted.

Q18 C · Masquerade — The lecture defines 'Masquerade' as one of the four categories of active attacks, where one entity pretends to be a different entity.

Q19 A · An attacker passively captures a data unit and retransmits it later to produce an unauthorized effect. — A 'Replay' attack involves the capture of a valid data transmission (like a login authentication)

and its subsequent retransmission to gain unauthorized access or produce an unauthorized effect.

Q20 A · Non-repudiation — The lecture explains 'Non-repudiation' with the example of a bank transaction, stating that this service provides proof of origin and delivery so that the sender or receiver cannot deny their involvement. It is summarized as 'the order is final'.

Q21 A · To determine if a user is allowed to perform a certain operation. — Authorization follows authentication. Once a user's identity is verified (authentication), authorization determines what specific actions or resources that user is permitted to access or use.

Q22 C · Determination of network security policies — The lecture outlines the network security process starting with the 'determination of the network security policies,' which serves as the roadmap for all subsequent steps like implementation and testing.

Q23 B · To learn about the network, such as IP addresses and key servers. — Reconnaissance is the information-gathering phase. An attacker (or a security professional) performs reconnaissance to learn about the target network's structure, hosts, and critical assets before taking further action.

Q24 D · To be a deliberately vulnerable system to attract and study attackers. — A honeypot is a decoy system set up to be an attractive target for attackers. Security personnel can then monitor attacks against the honeypot to learn about attacker methods and signatures, which helps in fine-tuning actual defense mechanisms like IDS and firewalls.

⚠ Honeypots/honeynets do not appear in the L26 transcript; this is the standard definition.

Q25 A · Vulnerability scanning. — Nmap is mentioned as an example of an open-source tool used for vulnerability scanning, which helps in identifying potential weaknesses in a network like open ports and running services.

Q26 A · A safe, authorized attempt to exploit a computer system's vulnerabilities. — The lecture describes penetration testing as a 'safe attacking scenario' where security professionals try to penetrate the system to find vulnerabilities before a malicious attacker does.

Q27 C · Forensics — The lecture explicitly refers to post-attack investigation as the 'forensics of the attacks,' which involves a process guided by laws to understand what happened and retain a chain of evidence.

Q28 D · A repository that lists known security vulnerabilities. — The NVD is mentioned as a security vulnerability database. Scanners and security professionals use such databases to identify known vulnerabilities in software and systems.

Q29 B · To incorporate information about newly discovered vulnerabilities and attack signatures. — The lecture draws a parallel with antivirus software, stating that scanners need to be updated with new signatures to effectively detect the latest vulnerabilities and threats.

Q30 C · Disclosure — The lecture classifies 'snooping' as a threat of 'Disclosure', which relates to the unauthorized release of information, violating confidentiality.

Q31 C · Modification, spoofing, repudiation of origin, denial of receipt — The lecture lists 'modifications, spoofing, repudiation of origin, denial of receipt' as examples under the 'threat of deception'.

Q32 B · Guaranteeing that the distributed implementation conforms to the central policy. — The lecture highlights the challenge of ensuring that a centrally formulated policy is implemented correctly and consistently across many distributed devices, which is a 'strong research problem'.

Q33 B · Integrity — The lecture defines 'Integrity' as the protection against data alteration or corruption, ensuring that the data remains accurate and trustworthy.

Q34 A · Policies should clearly define whether a given system state is secure or not, with no ambiguity. — This phrase means that a well-defined policy should create a clear separation between different system

states (e.g., secure, insecure, authorized, unauthorized) so there is no confusion about the system's current security status.

Q35 D · The human element is often a factor in security vulnerabilities and must be considered. — By mentioning 'human issues' and 'people problems', the lecture acknowledges that security is not just about technology but also involves people, their responsibilities, and their potential to be a weak link (e.g., via social engineering).

Q36 D · Denial of Service — The lecture defines a 'Denial of Service' (DoS) attack as one that 'prevents the normal use of communication facilities,' directly targeting the availability of the service.

Q37 B · Passive attacks — The lecture states that 'passive attackers are difficult to detect because they don't do direct harm.' They observe and collect information without altering the system's state or data.

Q38 B · They can become a new type of threat. — The diagram in the lecture shows a feedback loop where 'operational issues' can be identified and either fed back into the design/specification phase for improvement or can themselves emerge as a new 'threat'.

Q39 A · Trusting that a security person guarding a premise can be trusted. — The lecturer illustrates the concept of trust and assumptions by giving the real-world example of trusting that the human security guard at a facility is doing their job correctly and is trustworthy.

Q40 B · It prevents the misuse of resources by unauthorized or improperly authorized users. — The lecture defines access control as a service that prevents the misuse of resources by ensuring that users only have access to the specific resources and actions they are explicitly permitted.

Q41 A · Social media posts — The lecturer highlights the importance of securing mission-critical data by listing examples such as banking data, defense data, and even academic data like student results. Social media posts were not mentioned in this context of critical data.

Q42 B · Precise — The lecturer describes three scenarios for the relationship between reachable and secure states. The 'precise' scenario is when the set of reachable states and the set of secure states are a perfect match.

Q43 D · Series of Attacks — After discussing the roles of a security infrastructure, the lecture presents a slide titled 'Series of Attacks' which lists various specific attack vectors, including social engineering, phishing, and buffer overflow.

Q44 A · Implementing a security policy — The lecture shows a diagram of a network with a DMZ and firewalls as a 'big picture' example of how security policies are implemented in practice through network architecture and security appliances.

Q45 A · An employee tricked by a phishing email into revealing their password. — This is a direct example of a 'human issue' or 'people problem' as mentioned in the lecture. The vulnerability is not in the technology itself but in the human user who was manipulated (social engineering).

Q46 D · It involves some modification of the data stream or creation of a false stream. — The lecture defines active attacks as those that 'involve some modification of the data stream or creation of false streams,' fundamentally altering the state of the system or communication, unlike passive attacks which only observe.

Q47 C · Origin or source integrity — The lecturer explains that integrity is not just about the data being unchanged but also about 'origin or the source integrity,' which is related to authenticating that the message is truly from the claimed source.

Q48 A · Retaining the chain of evidence for legal purposes. — In the discussion on post-attack investigation (forensics), the lecture highlights that the process is heavily guided by laws and a key part is to 'retain chain of evidences' to understand the sequence of events.

Q49 D · Generic concepts that are also relevant to cloud computing. — The lecturer concludes by stating that these general security concepts are true for computer, information, and network security, and that they will 'also come in different in same or different forms' in the context of cloud computing.

Q50 A · Mechanism — The lecture defines mechanisms as the tools that 'enforce policies.' A firewall is a technical mechanism used to enforce access control policies defined by an organization.

Week 7 · L31 — Mobile Cloud Computing I

your file w4 · 50 questions

Lecture 31: Mobile Cloud Computing -I · 📖 notes: 7.1 Mobile Cloud Computing — basics (L31)

Q1. What is the primary definition of Mobile Cloud Computing (MCC) as presented in the lecture? Easy

A. A new type of mobile device with built-in cloud storage capabilities.
B. A software that allows mobile devices to function as standalone servers.
C. A method for compressing mobile data to improve network speeds.
D. A combination of cloud computing, mobile computing, and wireless networks to bring rich computational resources to mobile users.

Q2. Which of the following is NOT mentioned as a primary motivation for the emergence of Mobile Cloud Computing? Medium

A. The huge growth in the use of smartphones and smart applications.
B. Resource challenges on mobile devices, such as limited battery life and storage.
C. The low cost and elastic availability of cloud computing resources.
D. The development of mobile devices that are more resourceful than desktop servers.

Q3. The lecture discusses the concept of offloading. In the context of MCC, what does 'offloading' primarily refer to? Easy

A. Backing up user photos and contacts to a cloud storage service.
B. Transferring the entire mobile operating system to run on the cloud.
C. Deleting unnecessary files from the mobile device to free up space.
D. Moving computationally intensive parts of an application from the mobile device to be executed on cloud servers.

Q4. What is a key trade-off or 'flip side' of computational offloading mentioned in the lecture? Medium

A. Offloading always makes the application run slower.
B. Sending program state and data to the cloud consumes battery power, partially offsetting the power saved from not computing locally.
C. Cloud servers are less secure than the mobile device itself.
D. It makes the mobile device physically heavier.

Q5. According to the MCC workflow described, what is the role of the 'Profiler' component? Medium

A. To decide which part of the app to offload.
B. To execute the offloaded code on the cloud infrastructure.
C. To combine the results from the mobile device and the cloud.
D. To monitor an application's execution to collect data on time, power consumption, and network traffic.

Q6. In the context of the MCC architecture, what is the primary responsibility of the 'Solver'? Medium

A. To manage billing and pricing for the cloud resources used.
B. To select which parts of an application should run locally on the mobile device and which should be offloaded to the cloud.
C. To solve the computational problem sent by the mobile device.
D. To resolve any software bugs that occur during split execution.

Q7. A developer is implementing a complex image recognition feature in a mobile app using MCC. What is the role of the 'Synchronizer' component after the cloud has processed the image? Hard

A. To profile the network usage of the data transfer.
B. To update the app on the app store.
C. To collect the results of the split execution, combine them, and make the process transparent to the user.
D. To decide if the image processing should have been done on the device instead.

Q8. Which of the following is presented as a major advantage of using MCC for mobile applications? Easy

A. It extends battery life by offloading resource-hungry computations.
B. It eliminates all security risks for the mobile user.
C. It guarantees the lowest possible network latency at all times.
D. It allows apps to function perfectly without any internet connection.

Q9. How does MCC improve the 'reliability and availability' of user data? Medium

A. By creating multiple copies of the data on the mobile device's SD card.
B. By storing data in the cloud, which reduces the chance of loss if the mobile device is damaged or lost.
C. By encrypting the data so strongly that it can never be lost.
D. By ensuring the mobile device's battery never runs out.

Q10. The lecture mentions several real-world examples of technologies augmenting mobile devices with cloud computing. Which of the following was cited? Easy

A. Amazon Silk browser and Apple Siri
B. Adobe Photoshop for desktop
C. Google Chromecast
D. Microsoft Office 365

Q11. What is considered a major bottleneck and challenge for MCC that is dependent on the intermediate wireless network? Medium

A. The version of the mobile operating system
B. Processing speed of the cloud servers
C. Network latency and bandwidth
D. Screen resolution of the mobile device

Q12. Which of the following is NOT listed as a key feature of MCC? Medium

A. Requires developers to have specific knowledge of underlying network technologies.
B. Improves reliability as information can be backed up on the cloud.
C. Uses fewer device resources because applications are cloud-supported.
D. Facilitates quick development and delivery of mobile apps.

Q13. The lecture describes 'dynamic runtime offloading' as being particularly challenging. What is one of the tricky issues involved? Hard

A. Designing the user interface of the app.
B. Runtime application partitioning and continuous synchronization.
C. Choosing a cloud provider.
D. Installing the mobile application.

Q14. What is a major security challenge in MCC that affects consumer trust? Medium

A. The complexity of the application's user interface.
B. The high cost of cloud storage.
C. The risk of the mobile device overheating.
D. Protecting user privacy and data secrecy from adversaries.

Q15. In addition to data secrecy, what is a specific privacy concern for mobile users in MCC mentioned in the lecture? Medium

A. The user might receive too many notifications.
B. The app might use too much of the phone's memory.
C. The app's color scheme might be unappealing.
D. The user's identity, mobility, and GPS footprints could be tracked.

Q16. What does the concept of 'Mobile Backend as a Service' (MBaaS) aim to provide for developers? Medium

A. A way to connect mobile apps to cloud storage and processing, abstracting away backend complexity.
B. A consulting service for marketing mobile applications.
C. A service that automatically writes the front-end code for mobile apps.
D. A physical server delivered to the developer's location.

Q17. How can MCC lead to increased 'flexibility' in mobile application development, as discussed in the lecture? Hard

A. By allowing the application to run on any operating system without changes.
B. By allowing developers to quickly revise backend algorithms on the cloud without updating the front-end app on the device.
C. By ensuring the app is always free for the end-user.
D. By making the user interface automatically adapt to any screen size.

Q18. What role does the wireless network provider play in the MCC ecosystem? Easy

A. They provide the computational resources in the cloud.
B. They provide the essential connectivity and backbone between the mobile device and the internet/cloud.
C. They develop the mobile applications.
D. They manufacture the mobile devices.

Q19. The lecture mentions 'Context-Aware Mobile Cloud Services'. What is the goal of these services? Medium

A. To utilize local context (like network status, user preferences) to improve the quality of service.
B. To make all mobile apps look and feel the same.
C. To provide services only in a single language and context.
D. To block services based on the user's location.

Q20. Which of the following is an example of an application that would greatly benefit from MCC, as described in the lecture? Hard

A. A text-based note-taking app.
B. A simple calculator app.
C. An offline single-player game.
D. A real-time disaster management system collecting and processing sensor data from multiple mobile sources.

Q21. What does 'multi-tenancy' in the context of MCC advantages refer to? Medium

A. An application that can be used by ten people at once.
B. A mobile device that can run ten different applications simultaneously.
C. A service provider sharing resources and costs to support a variety of applications and a large number of users.
D. A cloud service that is hosted in ten different countries.

Q22. Why is 'pricing' considered a tricky issue and challenge in MCC? Hard

A. Because all cloud services must be offered for free.
B. Because multiple parties are involved (cloud provider, mobile service provider), and a clear business model is needed.
C. Because it's difficult to measure how much battery power an app uses.
D. Because users are unwilling to pay for any mobile application.

Q23. The lecture discusses a 'win-win situation' for MCC. Who are the parties that benefit? Easy

A. Mobile users, network operators, and cloud providers.
B. Only the mobile users.
C. Only the cloud providers.
D. Only the network operators.

Q24. What is the key difference between how a resource-hungry application runs with MCC versus without it? Medium

A. With MCC, the application code is automatically optimized to be less resource-hungry.
B. Without MCC, applications cannot use the internet.
C. With MCC, parts of the application can be partitioned to run on powerful cloud servers instead of the limited device.
D. With MCC, the application runs entirely on the cloud and is just streamed to the phone.

Q25. What is a proposed approach mentioned in the lecture to handle security threats like malicious code on a mobile device using MCC? Hard

A. Preventing the user from installing any new applications.
B. Running a resource-intensive antivirus program constantly on the mobile device.
C. Moving threat detection capabilities to the cloud, where a remote server performs the analysis.
D. Requiring the user to manually scan every file they download.

Q26. The lecture describes an MCC architecture with three main components. What are they? Easy

A. User, App, and Battery
B. CPU, RAM, and Storage
C. Profiler, Solver, and Synchronizer
D. Mobile Device, Wireless Network, and Computational Cloud

Q27. Why is 'ease of integration' considered an advantage of MCC? Medium

A. It physically integrates a keyboard and mouse with the mobile device.
B. It forces all applications to use the same user interface.
C. Multiple services from different providers can be integrated through the cloud to meet user demands.
D. It allows developers to easily copy code from other applications.

Q28. A challenge for MCC is 'service convergence'. What does this concept entail? Hard

A. Making all mobile apps converge to a single design standard.
B. Forcing all mobile network operators to merge into one company.
C. A scheme where a mobile user can utilize multiple different clouds in a unified and seamless fashion.
D. All cloud services eventually becoming one single service.

Q29. If a mobile game developer uses MCC to render complex 3D graphics on a cloud server, what is the primary benefit for the user's device? Medium

A. The device will consume less battery and generate less heat.
B. The game will have a better story.
C. The game can be played offline.
D. The game will take up more storage space on the device.

Q30. What is the primary function of the 'Mobile Load Manager' in the MCC workflow diagram? Medium

A. To manage the load on the mobile device's CPU.
B. To load and unload applications from the device's memory.
C. To send data to the cloud and manage the offloaded task from the device's side.
D. To manage the electrical load on the battery.

Q31. Which of these is a direct consequence of high network latency in an MCC application? Easy

A. Increased security.
B. Execution delays, making the application feel unresponsive.
C. Improved battery life.
D. Faster application performance.

Q32. The lecture states that in an MCC scenario, a user's request is delivered to the cloud through the internet. Who processes this request in the cloud? Medium

A. A peer-to-peer network of other mobile users.
B. The mobile network operator.
C. The Cloud Controller.
D. The user's mobile device.

Q33. What is a key requirement for APIs in the MCC environment to encourage adoption by developers? Easy

A. They must be unique for every single device model.
B. They should be complex and highly technical.
C. They should be simple and not require deep knowledge of network technologies.
D. They must be written in a proprietary programming language.

Q34. The proliferation of what two factors, working together, created the 'natural need' for MCC? Medium

A. The growth of social media and online shopping.
B. The increasing resource-hungriness of apps and the limited resources of mobile devices.
C. Larger screens and faster processors in mobile devices.
D. The desire for longer phone calls and better cameras.

Q35. Which cloud computing characteristic is most relevant to MCC's ability to handle a massive number of users and fluctuating demand? Medium

A. Measured Service
B. On-demand self-service
C. Rapid elasticity and scalability
D. Broad network access

Q36. How does the use of MCC impact the reusability of algorithms and applications? Hard

A. It has no impact on reusability.
B. It makes algorithms less efficient.
C. It decreases reusability because code is tied to a specific device.
D. It increases reusability, as a single computing service on the cloud can support numerous different apps and devices.

Q37. What is the primary goal of the 'amalgamation' of mobile and cloud technologies? Easy

A. To replace mobile phones with cloud-based terminals.
B. To decrease the number of apps available to users.
C. To increase the profits of mobile network operators exclusively.
D. To augment resource-constrained mobile devices by leveraging the power of the cloud.

Q38. A user is running a mobile augmented reality (AR) app that overlays information on a live camera feed. This requires intense computation. Why is MCC a good fit for this application? Hard

A. MCC can turn off the phone's camera to save power.
B. MCC provides a higher resolution screen for the AR app.
C. The AR app can offload the object recognition and rendering tasks to the cloud, making it run smoothly on the phone.
D. MCC guarantees the app will work without an internet connection.

Q39. What is a significant challenge related to 'standard interfaces' in MCC? Medium

A. The need to create a standard interface that allows different applications and devices to work with the cloud services.
B. There are too many existing standards to choose from.
C. Standard interfaces use too much battery.
D. Users find standard interfaces boring.

Q40. According to the lecture, the feeling that an MCC-powered application is running seamlessly on the device itself is crucial. What component is responsible for making the 'split execution' transparent to the user? Medium

A. The Cloud Controller
B. The Profiler
C. The Synchronizer
D. The Solver

Q41. The lecture discusses security issues in MCC as having two major categories. What are they? Medium

A. Beginner-level security and expert-level security.
B. Security of mobile users and security of data in the cloud.
C. Physical security and network security.
D. Hardware security and software security.

Q42. Why does the 'detachment' of processing from the device to the cloud offer greater speed, according to the lecture? Easy

A. Because data travels faster through the air than through wires.
B. Because it reduces the amount of code in the application.
C. Because cloud servers are vastly more resourceful and can perform computations much faster than a mobile device.
D. Because the processing is done physically closer to the user.

Q43. In the MCC architecture, base stations are mentioned. What is their function? Easy

A. To manufacture the mobile devices.
B. To store all the user's data.
C. To perform the cloud computations.
D. To establish and control the connection between the mobile network and the mobile devices.

Q44. A user is concerned that their new fitness app, which uses MCC for route planning, is tracking their location even when the app is closed. This scenario relates to which specific MCC challenge? Hard

A. Security and Privacy of the mobile user
B. Network Access Management
C. Service Convergence
D. Pricing

Q45. Which statement best summarizes the relationship between a mobile device's resources and MCC? Medium

A. MCC augments the resources of a mobile device, regardless of how resourceful it is, because the cloud's resources are comparatively vast.
B. MCC replaces all resources on the mobile device, turning it into a 'dumb terminal'.
C. MCC is only for old phones with very few resources.
D. As mobile devices become more resourceful, the need for MCC decreases.

Q46. What does the term 'resource hungry applications' refer to in the context of this lecture? Easy

A. Applications that are very popular and in high demand.
B. Applications that require significant computational power, memory, or energy to run.
C. Applications that display a lot of advertisements.
D. Applications that require a paid subscription.

Q47. The lecture mentions 'network access management' as a challenge. What is its goal beyond just improving link performance? Medium

A. To optimize bandwidth usage.
B. To increase the cost of network access.
C. To restrict access to only certain users.
D. To slow down the network for security.

Q48. What is the end goal of the entire MCC process, from the user's perspective? Easy

A. To pay more for their mobile service.
B. To manually manage data transfer between their phone and the cloud.
C. To be able to run enriched, more resourceful applications on their mobile device with a seamless experience.
D. To understand the complex architecture of cloud computing.

Q49. What is a major factor that makes the 'dynamic application profiling' component of runtime offloading so difficult? Hard

A. It has to analyze the application's behavior in real-time on the resource-constrained mobile device itself.
B. Cloud providers charge a high fee for profiling.
C. The process has to be done manually by the user.
D. It requires a complete rewrite of the application code.

Q50. The lecture states that an advantage of MCC is 'dynamic provisioning'. What does this mean? Medium

A. Supplying the mobile device with a dynamic IP address.
B. The ability to provision or allocate cloud resources dynamically based on the application's real-time needs.
C. Providing the user with a dynamic and changing user interface.
D. The ability to sell the mobile device and get a new one dynamically.

Answer key — Week 7 · L31 (your file w4)

Q1 D · A combination of cloud computing, mobile computing, and wireless networks to bring rich computational resources to mobile users. — The lecture explicitly defines MCC (at 09:27.900) as the amalgamation of these three technologies to augment the capabilities of mobile devices.

Q2 D · The development of mobile devices that are more resourceful than desktop servers. — The lecture states that while mobile devices are becoming more resourceful, they still do not match the resources of desktops or servers (02:14.790), which is a key reason for needing MCC.

Q3 D · Moving computationally intensive parts of an application from the mobile device to be executed on cloud servers. — As described around 03:18.049, offloading involves sending computing modules or phases to the cloud for processing to overcome the resource limitations of the mobile device.

Q4 B · Sending program state and data to the cloud consumes battery power, partially offsetting the power saved from not computing locally. — The lecture explicitly states (at 15:49.140) that while offloading saves battery from computation, the act of transferring the data and state to the server also consumes battery, creating a trade-off.

Q5 D · To monitor an application's execution to collect data on time, power consumption, and network traffic. — The lecture defines the Profiler's role (at 19:19.460) as monitoring the application to gather performance metrics, which are then used by other components like the Solver.

Q6 B · To select which parts of an application should run locally on the mobile device and which should be offloaded to the cloud. — As stated at 19:32.340, the Solver's task is to make the crucial decision of how to partition the application for execution between the mobile device and the cloud.

Q7 C · To collect the results of the split execution, combine them, and make the process transparent to the user. — The lecture explains (at 19:49.899) that the Synchronizer's job is to 'stitch together' the results from the split execution and give the user a seamless experience.

Q8 A · It extends battery life by offloading resource-hungry computations. — The lecture repeatedly emphasizes extending battery life (23:06.690) as a key advantage of MCC, as less computation is performed on the device itself.

Q9 B · By storing data in the cloud, which reduces the chance of loss if the mobile device is damaged or lost. — The lecture points out (at 24:29.399) that keeping data in the cloud, which has its own reliability measures, protects it from device-specific failures.

Q10 A · Amazon Silk browser and Apple Siri — At 07:31.680, the lecture specifically lists Amazon Silk, Apple Siri, and Apple iCloud as examples of augmenting mobile devices with cloud computing.

Q11 C · Network latency and bandwidth — The lecture highlights network latency as a potential cause for execution delays (16:01.740) and a major bottleneck (30:59.539) in the MCC framework.

Q12 A · Requires developers to have specific knowledge of underlying network technologies. — The lecture states (at 20:44.009) that a requirement for MCC is simple APIs 'requiring no specific knowledge of underlining network technologies,' making it easier for developers.

Q13 B · Runtime application partitioning and continuous synchronization. — At 18:03.440, the lecture explicitly lists 'runtime application partitioning' and 'continuous synchronization' as very tricky issues in dynamic runtime offloading.

Q14 D · Protecting user privacy and data secrecy from adversaries. — The lecture emphasizes (at 26:44.450) that protecting user privacy and data is a key challenge to establish and maintain consumer trust in MCC platforms.

Q15 D · The user's identity, mobility, and GPS footprints could be tracked. — The lecture highlights (around 28:08.039) that location-based services and GPS data expose the user to privacy issues, as their location and movements can be tracked.

Q16 A · A way to connect mobile apps to cloud storage and processing, abstracting away backend complexity. — The lecture explains MBaaS (at 06:10.400) as a model that allows developers to focus on front-end development by providing a ready-made backend infrastructure on the cloud.

Q17 B · By allowing developers to quickly revise backend algorithms on the cloud without updating the front-end app on the device. — The lecture provides an example (at 11:00.070) of changing a pathfinding algorithm on the cloud. This change is seamless for the mobile app, demonstrating the flexibility of detaching the processing from the device.

Q18 B · They provide the essential connectivity and backbone between the mobile device and the internet/cloud. — The lecture's architecture diagrams and descriptions (e.g., 21:18.590) place the mobile/wireless network as the crucial intermediary connecting the user's device to the cloud services.

Q19 A · To utilize local context (like network status, user preferences) to improve the quality of service. — As described at 30:26.029, context-aware services aim to monitor the user's environment and preferences to provide more satisfying and appropriate services.

Q20 D · A real-time disaster management system collecting and processing sensor data from multiple mobile sources. — The lecture uses a disaster management scenario (at 12:32.040) to illustrate how MCC can quickly collect and integrate data from various mobile sources for complex, time-sensitive analysis.

Q21 C · A service provider sharing resources and costs to support a variety of applications and a large number of users. — The lecture explains multi-tenancy (at 26:05.330) as a core cloud property where resources are shared among many users (tenants), which makes the service cost-effective.

Q22 B · Because multiple parties are involved (cloud provider, mobile service provider), and a clear business model is needed. — The lecture points out (at 31:14.619) that the involvement of multiple parties makes pricing complex, requiring clear business models and SLAs to determine who pays, who benefits, and what is guaranteed.

Q23 A · Mobile users, network operators, and cloud providers. — At 10:18.810, the lecture states that MCC brings benefits to all three parties, creating a 'win-win situation' for the entire ecosystem.

Q24 C · With MCC, parts of the application can be partitioned to run on powerful cloud servers instead of the limited device. — The core concept of MCC is to augment the mobile device (not replace it) by offloading specific, demanding tasks to the cloud, as described throughout the lecture (e.g., 15:02.220).

Q25 C · Moving threat detection capabilities to the cloud, where a remote server performs the analysis. — The lecture suggests (at 28:41.489) an approach where threat detection is moved to the cloud. The phone sends a trace to a security server, which then performs the attack detection.

Q26 D · Mobile Device, Wireless Network, and Computational Cloud — The lecture presents a high-level architecture (at 14:42.750) consisting of the smart mobile device at one end, the computational cloud at the other, and the wireless network in between.

Q27 C · Multiple services from different providers can be integrated through the cloud to meet user demands. — As mentioned at 26:16.070, MCC facilitates the integration of services from various providers through the centralized cloud platform, enabling more comprehensive solutions for users.

Q28 C · A scheme where a mobile user can utilize multiple different clouds in a unified and seamless fashion. — The lecture defines service convergence (at 31:51.110) as the need for a new scheme that allows a user to transparently leverage services from multiple clouds in a unified way.

Q29 A · The device will consume less battery and generate less heat. — By offloading the heavy graphics rendering, the device's own processor and GPU are used less. The lecture states (at 13:33.190) that this results in less battery power consumption and less heating up.

Q30 C · To send data to the cloud and manage the offloaded task from the device's side. — In the workflow diagram (around 16:56.750), the mobile load manager is positioned as the component on the device that handles the interaction with the cloud, sending data and managing the process.

Q31 B · Execution delays, making the application feel unresponsive. — The lecture explicitly states (16:01.740) that 'network latency can lead with execution and delay,' which would negatively impact the user experience.

Q32 C · The Cloud Controller. — According to the lecture (at 22:39.519), the 'cloud controller' processes the incoming requests to provide the corresponding cloud services to the mobile user.

Q33 C · They should be simple and not require deep knowledge of network technologies. — The lecture highlights (at 20:29.190) that simple APIs are a key requirement so that developers are inclined to use them without being burdened by underlying complexities.

Q34 B · The increasing resource-hungriness of apps and the limited resources of mobile devices. — The lecture builds the case for MCC by highlighting the tension between increasingly demanding applications and the inherent resource constraints (battery, processing) of mobile devices (05:03.920), creating a need for external computation.

Q35 C · Rapid elasticity and scalability — The lecture mentions scalability (25:59.889) as a key advantage inherited from the cloud, allowing the backend to scale resources up or down to support a large, variable number of mobile users.

Q36 D · It increases reusability, as a single computing service on the cloud can support numerous different apps and devices. — The lecture suggests (around 13:54.990) that because the computing is

done at the cloud end, the same standardized algorithm or service can be reused by many different applications, increasing reusability.

Q37 D · To augment resource-constrained mobile devices by leveraging the power of the cloud. — The lecture introduces MCC as an 'amalgamation' (00:32.390) and a framework 'to augment resource-constrained mobile devices' (15:02.220), which is the central theme of the discussion.

Q38 C · The AR app can offload the object recognition and rendering tasks to the cloud, making it run smoothly on the phone. — The lecture mentions image recognition and augmented reality apps (07:46.900) as use cases. These are computationally intensive and benefit greatly from offloading processing to the cloud to avoid overtaxing the mobile device.

Q39 A · The need to create a standard interface that allows different applications and devices to work with the cloud services. — The lecture lists 'standard interfaces' as a challenge (31:41.840), implying the difficulty of creating a unified way for a diverse ecosystem of devices and apps to interact with cloud backends.

Q40 C · The Synchronizer — The lecture states the Synchronizer's role is to combine results and 'make the execution details transparent to the user' (19:55.470), ensuring a seamless experience.

Q41 B · Security of mobile users and security of data in the cloud. — At 27:45.749, the lecture explicitly divides MCC security issues into two major categories: 'security of mobile users' and 'securing data of [the] cloud.'

Q42 C · Because cloud servers are vastly more resourceful and can perform computations much faster than a mobile device. — The lecture states (at 10:38.870 and 10:50.291) that speed is an advantage because clouds are 'more resourceful things' and can perform the computing 'immediately' (assuming low latency).

Q43 D · To establish and control the connection between the mobile network and the mobile devices. — The lecture explains (at 22:01.799) that mobile devices connect to the mobile network 'via base stations,' which control this functional interface.

Q44 A · Security and Privacy of the mobile user — This is a direct example of the privacy issue discussed in the lecture (at 28:16.760), where location-based services can cause privacy issues for subscribers by tracking their location (GPS footprints).

Q45 A · MCC augments the resources of a mobile device, regardless of how resourceful it is, because the cloud's resources are comparatively vast. — The lecture acknowledges the 'upliftment of the resources' in modern devices (02:14.790) but immediately notes there is 'no match' with cloud servers (15:17.890), establishing that augmentation is still highly beneficial.

Q46 B · Applications that require significant computational power, memory, or energy to run. — The lecture uses this term (e.g., at 05:15.110) to describe applications that are heavy and 'require more resource to execute,' which are prime candidates for MCC.

Q47 A · To optimize bandwidth usage. — As stated at 30:49.389, network access management aims not only to improve performance but also to 'optimize bandwidth usage,' as it is a shared and valuable resource.

Q48 C · To be able to run enriched, more resourceful applications on their mobile device with a seamless experience. — The lecture concludes (around 33:20.429) by summarizing the goal as enabling 'enriched...or more resourceful applications running on devices' transparently for the user.

Q49 A · It has to analyze the application's behavior in real-time on the resource-constrained mobile device itself. — The lecture lists 'dynamic application profiling and solver on smd' (smart mobile device)

as a key issue (18:09.250). The challenge lies in performing this complex analysis dynamically on the device that you are trying to offload from.

Q50 B · The ability to provision or allocate cloud resources dynamically based on the application's real-time needs. — Dynamic provisioning is mentioned at 25:59.889 as an advantage. This is a core cloud concept where resources can be allocated on-the-fly, which is inherited by MCC applications.

Week 8 · L36 — Introduction to Docker your file w5 · 50 questions

Lecture 36 : Introduction to DOCKER Container · 📖 notes: 8.1 Introduction to Docker containers (L36)

Q1. What is the primary motivation for using container technology like Docker, as described in the lecture? Easy

A. To reduce the cost of physical server hardware.
B. To solve the problem of recompilation and reconfiguration when moving software between environments.
C. To replace cloud computing with a more efficient technology.
D. To create a new programming language for web development.

Q2. According to the lecture, what is the relationship between cloud computing and container technology? Medium

A. Cloud computing can act as a platform to host containers, enhancing the cloud's capabilities.
B. Cloud computing and containers are mutually exclusive and cannot be used together.
C. Containers are only useful for on-premise data centers, not for the cloud.
D. Container technology is a competitor that will eventually replace cloud computing.

Q3. What is the core philosophy of Docker, summarized by the keywords mentioned in the lecture? Easy

A. Build, Host, Scale, On-demand
B. Code, Compile, Configure, Cloud
C. Design, Test, Secure, Anywhere
D. Develop, Ship, Run, Anywhere

Q4. In the shipping analogy used in the lecture, what does the standardized shipping container represent? Medium

A. The network used to transfer data.
B. The Docker container that packages an application and its dependencies.
C. The application's source code.
D. The physical server or infrastructure.

Q5. What is the fundamental architectural difference between Docker containers and traditional virtual machines (VMs)? Medium

A. Containers share the host operating system's kernel, while each VM runs its own complete guest OS.
B. Containers run a full guest operating system for each application, while VMs share a single guest OS.
C. Containers can only run on Linux, whereas VMs are platform-independent.
D. VMs are lightweight and start instantly, while containers are resource-intensive and slow to boot.

Q6. What is a Docker 'Image' as defined in the lecture? Easy

A. A backup file of the host operating system.
B. A graphical user interface for managing containers.
C. A lightweight, standalone, executable package that includes everything needed to run a piece of software.
D. A running instance of an application.

Q7. What is a Docker 'Container' in relation to an 'Image'? Medium

A. A container and an image are two terms for the same thing.
B. A container is a compressed version of an image stored in a registry.
C. A container is a script used to build an image.
D. A container is a runtime instance of an image.

Q8. Which of the following is a key advantage of Docker containers being 'lightweight'? Medium

A. They require a powerful, dedicated hypervisor to run.
B. They are inherently more secure than virtual machines.
C. They are easily scalable and start almost instantaneously.
D. They can only run simple, non-demanding applications.

Q9. What is the function of the 'Docker Engine'? Easy

A. It is the guest operating system inside a container.
B. It is a public cloud service for hosting websites.
C. It is a version control system similar to Git.
D. It is used for building Docker images and creating Docker containers.

Q10. What is 'Docker Hub' according to the lecture? Easy

A. A registry service used to host and share various Docker images.
B. A local folder on a developer's machine.
C. A command-line tool for writing Dockerfiles.
D. A physical hardware device for running containers.

Q11. A development team is struggling because an application works on one developer's laptop but fails on another's. How does Docker address this 'works on my machine' problem? Hard

A. By providing a universal debugging tool.
B. By packaging the application and all its dependencies into a container, ensuring a consistent environment.
C. By forcing all developers to use the exact same hardware.
D. By slowing down all machines to the speed of the slowest one.

Q12. What is the purpose of 'Docker Compose'? Medium

A. To automatically test the security of a container.
B. To write the source code for an application.
C. To create a single, monolithic container from multiple images.
D. To define and run applications that use multiple Docker containers.

Q13. How do containers achieve better performance compared to virtual machines? Medium

A. By pre-compiling all code into a single binary.
B. By running applications natively on the host machine's kernel.
C. By having dedicated physical hardware for each container.
D. By using a more efficient programming language.

Q14. What is a 'Dockerfile'? Easy

A. A build script used to create a Docker image automatically.
B. A running instance of a container.
C. A configuration file for the Docker Engine.
D. A log file that records all container activity.

Q15. Why is application portability a major challenge in modern computing, leading to the rise of containers? Medium

A. Because all modern devices use the same operating system.
B. Because applications often need to be recompiled or reconfigured to run on different devices and cloud platforms.
C. Because network speeds are too slow to transfer applications.
D. Because developers refuse to share their code.

Q16. An enterprise wants to ship new features faster and more securely. How does Docker help them achieve this, according to the lecture? Hard

A. By building agile software delivery pipelines.
B. By providing free physical servers.
C. By replacing the need for system operators.
D. By automatically writing application code.

Q17. In the context of Docker, what does 'isolation' mean? Medium

A. The container has no access to the internet.
B. The container runs in a process that is separate from the host and other containers by default.
C. The container's source code is encrypted and cannot be viewed.
D. The container can only be run on a specific, isolated physical machine.

Q18. Which of the following components is NOT part of the traditional virtualization stack but IS central to the Docker architecture? Medium

A. Host OS
B. Hypervisor
C. Docker Engine
D. Physical Server

Q19. How can Docker run on an IaaS (Infrastructure as a Service) cloud? Hard

A. By directly controlling the cloud provider's physical hardware.
B. By replacing the entire IaaS infrastructure.
C. It can't; Docker is a PaaS (Platform as a Service) solution.
D. By running the Docker Engine and containers on top of a virtual machine provided by the IaaS cloud.

Q20. What problem from the perspective of a system operator does Docker help solve? Hard

A. Getting better compute density by running and managing apps side-by-side in isolated containers.
B. Eliminating the need for servers entirely.
C. Designing user interfaces.
D. Writing application code.

Q21. What is the key difference in resource consumption between a VM and a container? Medium

A. Both consume the exact same amount of resources.
B. VMs consume less memory because they don't have dependencies.
C. Containers are more resource-intensive as they duplicate the OS kernel.
D. Containers are lightweight (tens of MBs), while VMs are resource-intensive (GBs) due to the full guest OS.

Q22. The lecture mentions several Docker commands like `ps`, `start`, `stop`, and `rm`. What is a notable characteristic of many of these commands? Easy

A. They are similar to common Linux commands.
B. They are designed to manage virtual machines, not containers.
C. They can only be run from Docker Hub.
D. They are all graphical and require a mouse.

Q23. In the shipping analogy, what does the 'infrastructure' (trucks, cranes, ships) correspond to in the software world? Medium

A. The various computing environments (laptops, servers, cloud platforms).
B. The application being deployed.
C. The programming language used.
D. The end-user of the software.

Q24. Which component in the Docker architecture is analogous to a hypervisor in traditional virtualization, but is much more lightweight? Medium

A. The Guest OS
B. The Docker Engine
C. Docker Hub
D. A Dockerfile

Q25. Why does the lecture describe Docker as a 'buzz word for modern world development especially in the face of agile based projects'? Hard

A. Because it is a complex and poorly understood technology.
B. Because it enables rapid and consistent deployment, which is a core tenet of agile development.
C. Because it is expensive and exclusive.
D. Because it is only used by large, old-fashioned companies.

Q26. What is the function of the `docker pull` command mentioned in the lecture? Easy

A. To push a local image to a repository.
B. To create a new container from an image.
C. To download an image from a repository.
D. To delete a local image.

Q27. If a developer wants to create a new image based on the current state of a running container, which command would they use? Medium

A. docker stop
B. docker ps
C. docker run
D. docker commit

Q28. How does Docker facilitate collaboration between development, QA, and operations teams? Medium

A. By providing a shared chat application.
B. By automatically translating code between different programming languages.
C. By ensuring all teams work with an identical, containerized application environment.
D. By requiring all teams to be in the same physical location.

Q29. According to the lecture, a container runs its processes in isolation. Under what condition can it access host files and ports? Hard

A. Only if it is explicitly configured to do so.
B. Only when running in 'developer mode'.
C. It automatically has access to all host resources.
D. It can never access host resources.

Q30. What kind of 'footprint' does Docker provide for the operating system within a container? Easy

A. No operating system components at all.
B. A larger, more feature-rich version of the operating system.
C. A smaller footprint of the operating system.
D. A complete, full-sized copy of the operating system.

Q31. Which of the following is NOT an environment where Docker containers can be deployed, according to the lecture? Medium

A. A physical or virtual machine.
B. A system without an underlying host operating system.
C. A developer's desktop or laptop.
D. A cloud platform like AWS or Azure.

Q32. What is the primary benefit of the 'configuration entanglement' being absent in containerized applications? Hard

A. The application requires more memory to run.
B. The application is automatically configured for the user.
C. The application becomes dependent on a specific host OS version.
D. The application can run anywhere without conflicting with host system dependencies.

Q33. The lecture mentions a variety of applications like static websites, background workers, and databases. What challenge related to this variety does Docker address? Medium

A. It can only run static websites, not databases.
B. It forces all applications to be written in a single language.
C. It provides a standardized way to package and run any of these different applications on any system.
D. It combines all applications into a single large program.

Q34. What does the 'application level virtualization' enabled by Docker allow a single host to do? Medium

A. Run only one application at a time.
B. Run several specialized applications for better resource utilization.
C. Virtualize the hardware for other users.
D. Act as a hypervisor for traditional VMs.

Q35. What happens when a small modification is made to an application running in a Docker container, compared to a VM? Hard

A. A completely new, full-sized VM must be created.
B. The entire host OS needs to be rebuilt.
C. The modification is not possible without recompiling the Docker Engine.
D. Docker allows you to save only the differences between the old and new container versions.

Q36. What is the role of the Docker client-server model in its overall workflow? Medium

A. It is a model for billing users, not for technical operations.
B. The server is a remote database, and the client is a web browser.
C. The client (user) searches for an image, pulls it, and the server (Docker Engine) runs it as a container.
D. The client builds the image, and the server stores it permanently.

Q37. The lecture describes Docker as an 'open source container'. What does 'open source' imply? Easy

A. It is a closed system that cannot be integrated with other tools.
B. It is owned and controlled by a single corporation.
C. Its source code is publicly available for anyone to view, use, or modify.
D. It can only be used on open-air, un-firewalled networks.

Q38. How does Docker improve resource utilization on a physical server compared to traditional virtualization? Medium

A. By running each application on a separate physical server.
B. By eliminating the overhead of running a separate Guest OS for each application.
C. By requiring more physical RAM for each application.
D. By using a proprietary and more efficient CPU architecture.

Q39. Which Docker command would you use to see a list of all containers, including those that are not currently running? Easy

A. ps -a
B. ps
C. top
D. images

Q40. If you want to create a Docker image that can be versioned in a system like Git alongside your source code, what would you create? Medium

A. A Docker Compose file.
B. A Dockerfile.
C. A shell script.
D. A Docker Hub account.

Q41. A team is developing a microservices application with a web front-end, a user database, and an analytics service. Which Docker tool is specifically designed to define and manage this multi-container setup? Hard

A. Dockerfile
B. Docker Hub
C. Docker Engine
D. Docker Compose

Q42. What is the primary content of a container image, distinguishing it from a VM image? Medium

A. A full operating system and the application.
B. Only the host operating system's kernel.
C. The hypervisor and the application.
D. Only the application executable and its specific package dependencies.

Q43. The lecture mentions that Docker Hub can automatically build images. What is this automatic build process typically based on? Hard

A. Manual uploads of binary files.
B. The current CPU usage of the server.
C. Dockerfiles located in a GitHub repository.
D. A schedule set by the system administrator.

Q44. What was the initial release date of Docker mentioned in the lecture? Easy

A. December 2010
B. March 2013
C. June 2015
D. January 2008

Q45. When comparing virtualization architectures, where does the Guest OS reside? Easy

A. Directly on the physical server.
B. On top of the Docker Engine.
C. Inside the application code.
D. On top of the hypervisor, inside a virtual machine.

Q46. Why does the lecture assert that containers offer 'better compute density' for operators? Hard

A. Because containers are physically larger.
B. Because they require fewer operators to manage.
C. Because their low resource footprint allows more isolated applications to run on a single host.
D. Because they can perform more complex mathematical computations.

Q47. Which problem is NOT a primary focus of the Docker platform? Medium

A. Packaging an application with its code, runtime, and dependencies.
B. Hardware abstraction and emulation.
C. Creating a consistent environment for development, testing, and production.
D. Application portability between different environments.

Q48. According to the lecture, a developer can run a container created on their desktop on a cloud platform like Azure or AWS. What does this demonstrate? Medium

A. That Docker only works on Windows.
B. The core principle of 'develop, ship, run anywhere'.
C. The inefficiency of cloud platforms.
D. The requirement for all platforms to have identical hardware.

Q49. In the lecture's comparison of a modified app (Container A vs. Container A'), what is the key efficiency of Docker? Hard

A. It automatically reverts the changes if they introduce a bug.
B. It allows saving only the layer representing the changes, not a full new image.
C. It requires a full rebuild of the host OS.
D. It creates a complete, independent copy, which is safer.

Q50. What is the ultimate goal of the 'matrix' of applications and systems shown in the lecture? Medium

A. To find the single best system for running all applications.
B. To show that no application can run on all systems.
C. To prove that only one application is needed.
D. To have 'all ticks on this matrix', meaning any application can run on any system with minimal effort.

Answer key — Week 8 · L36 (your file w5)

Q1 B · To solve the problem of recompilation and reconfiguration when moving software between environments. — The lecture highlights that a major challenge is the need to 're-deployment or reconfiguration or recompilation of the software whenever we want to ship one package from one environment to other' (01:57). Container technology bundles the application with its dependencies to ensure it runs consistently anywhere.

Q2 A · Cloud computing can act as a platform to host containers, enhancing the cloud's capabilities. — The speaker clarifies that while some see containers as a contender to the cloud, the modern view is that 'cloud can act as a platform to host these container and basically cloud can enhance its capability by adapting this container technology' (03:36 - 03:49).

Q3 D · Develop, Ship, Run, Anywhere — The lecture explicitly states, 'main feature of docker are develop ship run anywhere so this is this key words' (04:30 - 04:37). This philosophy emphasizes the portability and consistency Docker provides across different environments.

Q4 B · The Docker container that packages an application and its dependencies. — The analogy compares a software application and its dependencies to products being shipped (like a car or piano). The standardized shipping container, which can be handled by any truck or crane, represents the Docker container that standardizes the 'shipping' of software across different infrastructures (11:08 - 11:59).

Q5 A · Containers share the host operating system's kernel, while each VM runs its own complete guest OS. — The lecture contrasts the two architectures, showing that VMs have a hypervisor and a guest OS for each VM, making them resource-intensive. Containers, on the other hand, run on the Docker Engine and share the host OS kernel, making them lightweight (24:36 - 25:37).

Q6 C · A lightweight, standalone, executable package that includes everything needed to run a piece of software. — The lecture defines an image as 'a light weight standalone executable package that includes everything needed to run a piece of software including code runtime libraries environment variables config files etcetera' (22:53 - 23:04).

Q7 D · A container is a runtime instance of an image. — The lecture explains, 'a container is a runtime instance of an image... what the image becomes in memory when actually is executed' (23:24 - 23:32).

Q8 C · They are easily scalable and start almost instantaneously. — The lecture states, 'since docker containers are pretty lightweight they are easily scalable' (16:42 - 16:47) and that they 'start almost ah instantaneous' (22:14 - 22:21).

Q9 D · It is used for building Docker images and creating Docker containers. — The lecture mentions that the Docker Engine 'is used for building docker images and creating docker containers' (17:40 - 17:46) and sits between the host OS and the containers.

Q10 A · A registry service used to host and share various Docker images. — The lecture identifies Docker Hub as 'a registry... which is use to host various docker images' (18:00 - 18:05). It is a central place to find and distribute images.

Q11 B · By packaging the application and all its dependencies into a container, ensuring a consistent environment. — The lecture explains that developers use Docker to 'eliminate work on my machine

problem' (32:06 - 32:11). By bundling the application with all its libraries and configurations into an image, Docker ensures that the application runs identically everywhere.

Q12 D · To define and run applications that use multiple Docker containers. — The lecture states that Docker Compose 'is used to define application using multiple docker containers' (18:20 - 18:24), which is essential for complex applications with multiple services (e.g., a web server, a database, a queue).

Q13 B · By running applications natively on the host machine's kernel. — The lecture explains that containers 'runs apps natively on the host machines kernel' and have 'better performance characteristics than virtual machine that only get virtual access to the host resources through a hypervisor' (24:00 - 24:10).

Q14 A · A build script used to create a Docker image automatically. — The lecture describes a Dockerfile as a 'build script' used to 'create an image automatically' (31:02 - 31:05). This file contains instructions for assembling an image.

Q15 B · Because applications often need to be recompiled or reconfigured to run on different devices and cloud platforms. — The lecture emphasizes that the 'portability of one application from one environment to another is becoming a major challenge' (02:47 - 02:52) due to the variety of devices and platforms, requiring recompilation and reconfiguration.

Q16 A · By building agile software delivery pipelines. — The lecture states that enterprises use Docker 'to build agile software delivery pipelines to ship new features faster more securely with confident' (32:33 - 32:41).

Q17 B · The container runs in a process that is separate from the host and other containers by default. — The lecture notes that a container 'runs completely isolated from host environment by default' (23:43 - 23:47) and that multiple containers run in 'isolated processes in the user space' (22:04 - 22:06).

Q18 C · Docker Engine — The architectural diagrams (18:41 - 21:13) show that traditional virtualization uses a Hypervisor, while the Docker architecture replaces this layer (and the Guest OS) with the Docker Engine.

Q19 D · By running the Docker Engine and containers on top of a virtual machine provided by the IaaS cloud. — The lecture shows a diagram (27:44) and explains that on an IaaS cloud, you can have a VM (Infrastructure -> Hypervisor -> Guest OS) and then run Docker services on that VM. It states, 'on some vms that is in a iaas type of cloud we can run this sort of services' (28:15 - 28:25).

Q20 A · Getting better compute density by running and managing apps side-by-side in isolated containers. — The lecture mentions that 'operators use docker to run and manage apps side by side in isolated container to get better compute density' (32:22 - 32:33), meaning they can run more applications on the same hardware.

Q21 D · Containers are lightweight (tens of MBs), while VMs are resource-intensive (GBs) due to the full guest OS. — The lecture highlights that container images are 'typically tens of mbs in size,' whereas VMs run a full guest OS, making them 'resource ah intensive' and their disk images much larger (22:14, 25:10).

Q22 A · They are similar to common Linux commands. — After listing several commands, the speaker notes, 'what you may notice that these are the most of the many of them things are some sort of linux commands right already we are used to it' (30:11 - 30:24).

Q23 A · The various computing environments (laptops, servers, cloud platforms). — The lecture explains that 'outside the container the infrastructure... dictates' (12:20-12:25). Just as different vehicles (trucks, ships) transport the physical container, different computing platforms (desktops, cloud servers) run the Docker container.

Q24 B · The Docker Engine — The lecture presents the Docker architecture with the Docker Engine sitting on the Host OS, in a position analogous to the hypervisor. It also draws a direct comparison: 'some sort of analogy with the hypervisors of a particular virtualization system' (17:50-17:57).

Q25 B · Because it enables rapid and consistent deployment, which is a core tenet of agile development. — The speaker connects Docker to agile projects (05:37) because its 'develop, ship, run anywhere' philosophy facilitates the fast, iterative, and reliable delivery cycles that are characteristic of agile methodologies.

Q26 C · To download an image from a repository. — The lecture provides a list of terminologies, stating that `pull` is used to 'download an image from a repository' (29:32 - 29:40).

Q27 D · docker commit — The lecture lists `commit` as the command to 'create an image on the container' (30:11 - 30:17), which means saving the changes made inside a running container as a new image.

Q28 C · By ensuring all teams work with an identical, containerized application environment. — The lecture states that with containers, it becomes 'easier for software teams such as development qa teams operations to work seamlessly across applications' (16:00 - 16:07) because they are all using the same portable container, eliminating environment inconsistencies.

Q29 A · Only if it is explicitly configured to do so. — The lecture clarifies that a container 'runs completely isolated from host environment by default only accessing the host file and ports if considered to do so' (23:43 - 23:55), meaning access must be granted explicitly.

Q30 C · A smaller footprint of the operating system. — The lecture mentions that Docker has the ability to 'reduce the size of development by providing a smaller footprint of the operating system by a containers' (15:48 - 15:54).

Q31 B · A system without an underlying host operating system. — The Docker architecture shown (20:08) clearly indicates that the Docker Engine runs on top of a 'Host OS' on a server. Containers share this host kernel, so a host OS is a prerequisite.

Q32 D · The application can run anywhere without conflicting with host system dependencies. — The lecture states that because containers contain their own dependencies, 'there is no configuration entanglement and so containerized applications runs anywhere' (25:56 - 26:02). This avoids conflicts with libraries or settings on the host machine.

Q33 C · It provides a standardized way to package and run any of these different applications on any system. — The lecture presents a matrix of different applications and systems (07:17 - 08:39) and poses the challenge of getting 'all ticks on this matrix like anything can run anywhere'. Docker's containerization is the solution to this challenge.

Q34 B · Run several specialized applications for better resource utilization. — One of the needs met by Docker is 'application level virtualization... a single host can run several special application for utilization of resources' (28:33 - 28:44).

Q35 D · Docker allows you to save only the differences between the old and new container versions. — The lecture explains that while a slight modification in a VM context might require a new virtual environment, in Docker's case, a 'slight modification ah can be done allows us only to save the difference between container a and container a dash' (26:41 - 26:50), highlighting its efficiency.

Q36 C · The client (user) searches for an image, pulls it, and the server (Docker Engine) runs it as a container. — The workflow described (27:24 - 27:36) shows a user/consumer searching a registry, pulling an image, and then running it. The lecture mentions this is a 'some sort of a client server mode operation' where the user's commands (client) are executed by the Docker Engine (server/daemon).

Q37 C · Its source code is publicly available for anyone to view, use, or modify. — The lecture identifies Docker as 'a open source container' (03:51 - 03:56). The term 'open source' generally means the underlying code is publicly accessible, encouraging community collaboration and transparency.

Q38 B · By eliminating the overhead of running a separate Guest OS for each application. — The core difference highlighted is that VMs require a resource-intensive Guest OS per instance (25:10), while containers share the host kernel. By removing this redundant OS layer for each app, Docker allows more applications to run on the same hardware, improving utilization.

Q39 A · ps -a — The lecture explicitly states that `ps` lists running containers, while `ps minus a` is used to 'list all containers ah including the stopped one' (29:55 - 30:04).

Q40 B · A Dockerfile. — The lecture explains that a Dockerfile is a build script that 'can be versioned in a version control system like git or svn along with other dependency' (31:02 - 31:12).

Q41 D · Docker Compose — The lecture defines Docker Compose as being 'used to define application using multiple docker containers' (18:20 - 18:24), which is the exact scenario described by a microservices architecture.

Q42 D · Only the application executable and its specific package dependencies. — The lecture clarifies that 'the only information that needs to be in a container image is the executable and its package dependence' (25:31 - 25:37), unlike a VM which contains a full OS.

Q43 C · Dockerfiles located in a GitHub repository. — The lecture states that 'docker hub can automatically build images based on docker files on github' (31:12 - 31:22), describing a common CI/CD practice.

Q44 B · March 2013 — The lecture specifies, 'if you look at ah that docker it is somewhere initial release where march twenty thirteen' (04:23 - 04:30).

Q45 D · On top of the hypervisor, inside a virtual machine. — The diagram and explanation of traditional virtualization (18:41 - 19:23) show the stack as Server -> Host OS -> Hypervisor -> Guest OS, placing the Guest OS within each VM on top of the hypervisor.

Q46 C · Because their low resource footprint allows more isolated applications to run on a single host. — The lecture states operators use Docker to 'get better compute density' (32:27 - 32:33). This is a direct result of containers being lightweight, which allows more of them to be packed onto the same hardware compared to resource-heavy VMs.

Q47 B · Hardware abstraction and emulation. — Docker is described as 'abstraction at the app layer' (21:30). Traditional virtual machines and hypervisors are responsible for hardware abstraction and emulation. Docker focuses on OS-level virtualization, abstracting the application environment.

Q48 B · The core principle of 'develop, ship, run anywhere'. — This scenario is a perfect example of Docker's main philosophy: 'develop once and ah ship to any environment and run to this' (04:48 - 04:57). The container abstracts away the differences between the desktop and the cloud environment.

Q49 B · It allows saving only the layer representing the changes, not a full new image. — The speaker notes that with a slight modification, Docker 'allows us only to save the difference between container a and container a dash' (26:41-26:50). This refers to Docker's layered filesystem, where changes are stored as new layers, making updates very efficient.

Q50 D · To have 'all ticks on this matrix', meaning any application can run on any system with minimal effort. — The lecturer states the 'major objective is there with minimal ah investment... i should be able to have all ticks on this matrix like anything can run anywhere type of situations' (08:26 - 08:40), which is the problem Docker aims to solve.

Practice MCQs — check yourself (answers with explanations)

Pick an option to check it instantly. Multi-select questions have a **Check answer** button. These are practice questions I wrote from the lecture notes (all 12 weeks). They are **not** NPTEL assignment questions; your own sets are in the Your practice sets tab.

Week 1 — practice set

Q1. Which of the following is NOT one of the five essential characteristics of cloud computing in the NIST definition? theory

- **A.** Rapid elasticity
- **B.** Measured service
- **C.** Virtualization
- **D.** Broad network access

Q2. Which of the following statements is/are correct according to the lectures? (Select all that apply) theory · select all

- **A.** Utility computing is a concept that cloud computing practically implements
- **B.** Utility computing charges a flat rate irrespective of usage
- **C.** Grid computing harnesses unused processing cycles of computers in a network
- **D.** A cluster is more tightly coupled than an SMP

Q3. A tax-filing application sees huge demand for a few weeks each year and little demand otherwise. Which service model does the lecture suggest is especially suitable for this kind of software? concept

- **A.** On-premise dedicated deployment
- **B.** SaaS
- **C.** A high-availability cluster only
- **D.** None — cloud is unsuitable for demand spikes

Q4. In which of the following situations is PaaS NOT an ideal choice, according to the lecture? theory

A. Multiple developers working on the same project

B. Developers wish to automate testing and deployment

C. A proprietary language would hinder a later move to another provider

D. Agile software development

Q5. In an IaaS model, up to which layer is the cloud provider responsible (as per the lecture's responsibility stack)? concept

A. Application

B. Runtime/middleware

C. Virtualization (providing VMs)

D. Only physical networking

Q6. Which statement about scale-up vs scale-out is correct? theory

A. Scale-up adds more machines; scale-out adds resources to one box

B. For large scenarios, scale-out is the only practical solution because one 64-way server is far costlier than 64 one-way servers

C. Scale-out always introduces a single point of failure

D. Scale-up is always cheaper regardless of scale

Q7. As per the lecture's comparison, which of the following is TRUE of the client–server model but NOT of cloud computing? theory

A. Supports IaaS, PaaS and SaaS

B. Has no concept of virtualization

C. Is theoretically infinitely scalable

D. Is always load balanced

Q8. A group of banks shares a cloud managed through RBI as the nodal agency to exchange core-banking data. Which deployment model is this? concept

A. Private cloud

B. Public cloud

C. Community cloud

D. Hybrid cloud

Answers — Week 1

Q1: C — NIST's five are on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service. Virtualization is an enabling/common characteristic, not one of the five.

Q2: A, C — Utility computing charges for specific usage rather than a flat rate, and clusters are more loosely coupled than SMPs. Options A and C are lecture statements.

Q3: B — The lecture lists 'software where demand spikes significantly, e.g., tax and billing' as a SaaS use case. The pay-as-you-go model absorbs the spikes.

Q4: C — Vendor lock-in through proprietary languages is listed as a scenario where PaaS may not be ideal. The other three are listed as situations where PaaS is useful.

Q5: C — In IaaS the provider manages up to virtualization (VMs). The guest OS may be provided or chosen by the user, and everything above is the consumer's responsibility. PaaS goes up to the runtime and SaaS up to the application.

Q6: B — Scale-up (vertical) adds resources to the same box and is cheaper in small scenarios. Scale-out (horizontal) adds machines, is the only solution at large scale, and avoids a single point of failure.

Q7: B — Client–server has no concept of virtualization, while virtualization is the core concept of cloud. Cloud supports the complex service models and is theoretically infinitely scalable.

Q8: C — A cloud shared by several organizations with common goals, such as a banking cloud, is a community cloud. The lecture describes it as 'semi-public, semi-private'.

Week 2 — practice set

Q1. Which of the following is listed in the lecture as a characteristic of an on-site private cloud but NOT of an outsourced private cloud? theory

A. Network dependency

B. Limited resources (fixed capacity sized for anticipated workloads)

C. Workload locations hidden from clients

D. Potentially strong security from external threats

Q2. Which statement(s) about deployment models is/are correct? (Select all that apply) theory · select all

A. A private cloud may exist off premises and be managed by a third party

B. An outsourced private cloud has two security perimeters joined by a protected link

C. Public cloud typically has high up-front cost to migrate

D. In a hybrid cloud, the constituent clouds remain unique entities

Q3. According to Popek and Goldberg, a VMM satisfying equivalence, resource control and efficiency can be constructed if: theory

A. Privileged instructions are a subset of sensitive instructions

B. Sensitive instructions are a subset of privileged instructions

C. There are no privileged instructions

D. All instructions are emulated by the VMM

Q4. In which virtualization approach is the guest OS kernel modified/recompiled so that it uses a specialized API to talk to the VMM? concept

A. Full virtualization (binary translation)

B. Para-virtualization

C. Hardware-assisted virtualization

D. Emulation with QEMU/Bochs

Q5. In the 3-level IaaS architecture, which component determines whether part or all of a resource-allocation command can be satisfied using its cluster's computers, and is connected to Persistent Local Storage? theory

A. Cloud manager

B. Cluster manager

C. Computer manager

D. Data Object Storage

Q6. Which of the following is TRUE about XML? theory

A. Every well-formed XML document is valid

B. XML Schema (XSD) is written in XML and can declare entities

C. A non-validating parser stops processing DTD content at the first entity it cannot find

D. XML tags are predefined like HTML tags

Q7. An application must process a very large XML stream with minimal memory and does not need to modify the document. Which parser is most suitable? concept

A. DOM

B. JDOM

C. SAX

D. XSLT processor

Q8. Match correctly: (i) SOAP (ii) WSDL (iii) UDDI with (a) registry to register/locate services (b) describing services offered (c) XML messaging protocol theory

A. i-a, ii-b, iii-c

B. i-c, ii-b, iii-a

C. i-b, ii-c, iii-a

D. i-c, ii-a, iii-b

Q9. Why can web services traverse firewalls more easily than CORBA/DCOM, according to the lecture? theory

A. They encrypt all traffic

B. They run over HTTP (port 80), which most firewalls allow, whereas CORBA/DCOM used non-standard ports

C. They do not use any network protocol

D. They use binary encoding

Answers — Week 2

Q1: B — An on-site private cloud has a fixed computing and storage capacity, so its resources are limited. An outsourced private cloud offers extensive resource availability. The other three options apply to both.

Q2: A, B, D — Public cloud has LOW up-front cost to migrate. The other three are direct lecture statements.

Q3: B — If every sensitive instruction is privileged, it traps to the VMM in user mode and can be emulated. All other instructions run directly on the CPU, which gives efficiency.

Q4: B — Para-virtualization (cooperative virtualization, e.g., Xen) modifies the guest. Full virtualization and hardware-assisted virtualization both run unmodified guests.

Q5: B — The cluster manager receives commands from the cloud manager, queries the computer managers and checks feasibility. PLS gives VMs persistent disk-like storage at this level. DOS (subscriber metadata) belongs to the cloud manager.

Q6: C — Valid implies well-formed, not the other way round. XSD cannot declare entities (only a DTD can). XML tags are user-defined.

Q7: C — SAX is event-based, very fast and has a low memory footprint. DOM and JDOM build an in-memory tree, which is slower and memory-intensive.

Q8: B — SOAP is the messaging protocol (stateless, one-way, over HTTP). WSDL describes services through port types, messages and bindings. UDDI is the registry.

Q9: B — Firewall traversal is one of the three problems web services address, along with interoperability and complexity.

Week 3 — practice set

Q1. Arrange from lowest to highest level: SLA, KPI, SLO. theory

- **A.** SLA → SLO → KPI
- **B.** KPI → SLO → SLA
- **C.** SLO → KPI → SLA
- **D.** KPI → SLA → SLO

Q2. Which of the following is/are correct about cloud SLAs compared with traditional web-service SLAs? (Select all that apply) theory · select all

- **A.** In the cloud, security, privacy and trust management have more importance
- **B.** In traditional web services, SLA negotiation and monitoring are not automated
- **C.** Cloud resource allocation relies on a single central UDDI directory
- **D.** Cloud needs SLA automation for highly dynamic, scalable services

Q3. An organization's demand has a peak of 400 servers and an average of 100 servers. Owning costs ₹10 per server-hour and the cloud costs ₹35 per server-hour. Which option is cheaper? numerical

A. Owning, because the cloud unit cost is higher

B. Cloud, because U = 3.5 < P/A = 4

C. Owning, because U = 3.5 > A/P = 0.25

D. Both cost the same

Q4. The demands of 64 independent customers each have the same coefficient of variation Cv = 0.8. What is the Cv of the aggregated demand? numerical

A. 0.8

B. 0.1

C. 0.0125

D. 6.4

Q5. A cluster has 500 components, each failing independently with probability 0.002 in a given period. What is the approximate probability that at least one component fails? numerical

A. 0.002

B. ≈0.37

C. ≈0.63

D. ≈1.0

Q6. In Dynamo with N = 5 replicas, which (R, W) setting does NOT guarantee quorum consistency? concept

A. R = 3, W = 3

B. R = 1, W = 5

C. R = 2, W = 3

D. R = 5, W = 1

Q7. In a GFS write/append, which entity chooses the offset at which data is appended? theory

- **A.** The client
- **B.** The master
- **C.** The primary chunk server designated by the master
- **D.** Any replica independently

Q8. For word counting with n = 20,000 documents, f = 2 average occurrences per word per document and p = 40 processors, what is the parallel efficiency when each processor handles all words over n/p documents? numerical

- **A.** 1/40 = 0.025
- **B.** ≈0.998
- **C.** ≈0.5
- **D.** ≈0.9

Q9. Which statement about MapReduce is correct? theory

- **A.** Each mapper writes one file to GFS for all reducers
- **B.** Each mapper writes one key-sorted file per reducer to its local disk, and the master tracks file locations
- **C.** If a reducer fails, all its completed work must be redone
- **D.** Reducers push data to mappers using RPC

Q10. Match the OpenStack projects: Swift, Cinder, Glance, Ceilometer. theory

- **A.** Block storage, object storage, telemetry, images
- **B.** Object storage, block storage, images, telemetry
- **C.** Images, block storage, object storage, identity
- **D.** Object storage, images, block storage, dashboard

Answers — Week 3

Q1: B — KPIs are low-level resource metrics. They are composed or aggregated into SLOs, and SLOs are integrated into the SLA.

Q2: A, B, D — In the cloud, resources are allocated and distributed globally without any central directory. UDDI is the traditional web-service mechanism.

Q3: B — U = C/B = 35/10 = 3.5 and P/A = 400/100 = 4, so U < P/A and the cloud is cheaper. Check: cloud = 35×100 = 3500/h, owned = 10×400 = 4000/h.

Q4: B — Aggregating n independent demands reduces Cv by $1/\sqrt{n}$: $0.8/\sqrt{64} = 0.8/8 = 0.1$.

Q5: C — $P = 1 - (1 - 0.002)^{500} \approx 1 - e^{(-1)} \approx 0.632$. Failures are near-inevitable at scale, which is why GFS/HDFS are fault tolerant.

Q6: C — Quorum consistency requires $R + W > N$. Here $2 + 3 = 5$ is not greater than 5, so the read and write sets might not overlap.

Q7: C — The master designates a primary chunk server. The primary chooses the offset (possibly beyond EOF) and sends it to the replicas, and the client retries if any replica fails.

Q8: B — $\varepsilon = 1/(1 + 2p/(nf)) = 1/(1 + 80/40{,}000) = 1/1.002 \approx 0.998$. Splitting the words across processors would give only $1/p = 0.025$.

Q9: B — Mapper output is local and partitioned per reducer. Reducers pull it with RPC after the master tells them where it is. Completed reduce output is already in GFS, so only unfinished reduce tasks are reassigned.

Q10: B — Swift is object storage (REST/HTTP), Cinder is persistent block storage, Glance is VM image storage and Ceilometer is telemetry/metering.

Week 4 — practice set

Q1. In the OpenStack dashboard demo, what is a 'security group'? theory

- **A.** A group of users sharing one project quota
- **B.** A set of firewall-like rules defining allowed incoming and outgoing traffic, assigned to VMs
- **C.** A Keystone token group
- **D.** A set of physical hypervisors

Q2. A newly launched OpenStack VM has only an internal IP. What must be done so it can be reached from outside the cloud? concept

- **A.** Terminate and relaunch it
- **B.** Allocate and associate an external (floating) IP from the pool
- **C.** Attach a Cinder volume
- **D.** Upload a new Glance image

Q3. In Windows Azure, how does a worker role communicate with a web role? theory

- **A.** Through direct TCP sockets only
- **B.** Through Azure storage queues
- **C.** Through SQL Azure tables only
- **D.** Through AppFabric only

Q4. Which statement(s) about Azure roles is/are correct? (Select all that apply) theory · select all

- **A.** A web role is a running environment that users access over the Internet
- **B.** A worker role handles distributed or complex processing tasks
- **C.** Every service must contain at least one web role AND at least one worker role
- **D.** A service may contain any number of web or worker roles

Q5. In the Azure deployment demo, what is a 'resource group'? theory

- **A.** The location, size and features of the hosting web server
- **B.** A logical container into which Azure resources such as web apps, databases and storage accounts are deployed and managed
- **C.** The deployment user used for FTP/Git
- **D.** The URL of the web app

Q6. Match the GCP services to their type: App Engine, Compute Engine, Cloud SQL, Cloud Datastore. theory

- **A.** IaaS, PaaS, NoSQL, MySQL
- **B.** PaaS, IaaS, MySQL, NoSQL
- **C.** PaaS, IaaS, NoSQL, object storage
- **D.** SaaS, IaaS, MySQL, NoSQL

Q7. To host a static website on GCP as shown in the demo, which steps are needed? concept

- **A.** Create a Compute Engine VM and install Apache
- **B.** Create a Cloud Storage bucket, upload index.html and 404.html (plus other files), and share them publicly
- **C.** Create a Cloud SQL instance
- **D.** Run gcloud app deploy with app.yaml

Q8. In the App Engine demo, what does 'threadsafe: true' in app.yaml indicate? theory

- **A.** The app uses Python 3
- **B.** The app can handle many simultaneous requests
- **C.** The app is deployed to multiple regions
- **D.** The app is read-only

Answers — Week 4

Q1: B — A security group holds port/service-based rules for both inbound and outbound traffic. Assigning one to a VM applies the firewall rules automatically.

Q2: B — The internal network IP is assigned automatically. External access needs an external IP associated with the VM.

Q3: B — Worker roles talk to web roles through Azure storage queues. Each VM also runs an agent that lets the application interact with the Azure fabric.

Q4: A, B, D — A service must include at least one role of EITHER type, not one of each.

Q5: B — Option A describes the App Service plan, C the deployment user and D the web app's hosting space and URL.

Q6: B — App Engine is a managed platform (PaaS), Compute Engine gives raw VMs (IaaS), Cloud SQL is MySQL and Cloud Datastore is NoSQL.

Q7: B — The static-site demo used only Cloud Storage. The App Engine flow with app.yaml was the second demo, which builds a web application.

Q8: B — app.yaml is the configuration file. threadsafe: true allows concurrent request handling, and the handlers map URL regexes to main.app.

Week 5 — practice set

Q1. A cloud guarantees 99.5% availability. A service is scheduled for 10 hours/day for 30 days, and total outage in the month is 2 hours. What is the measured availability (downtime/expected time), and is the SLA violated? numerical

- **A.** 99.33%, violated
- **B.** 99.72%, not violated
- **C.** 99.17%, violated
- **D.** 98.00%, violated

Q2. An SLA bills $40/day for 30 days. The credit is 10% if 99% ≤ uptime < 99.95% and 25% if uptime < 99%. Measured uptime is 99.2%. What is the effective cost payable? numerical

A. $1200

B. $1080

C. $900

D. $1170

Q3. What is the maximum downtime allowed in a 30-day month (24 h/day) for a 99.9% availability guarantee? numerical

A. 4.32 min

B. 43.2 min

C. 7.2 h

D. 72 min

Q4. An in-house server costs ₹8,76,000 for a 3-year life and has 10 cores. What is its price per core-hour (ignoring power and management)? numerical

A. ₹3.33

B. ₹33.3

C. ₹2.92

D. ₹0.33

Q5. Using the lecture's in-house vs cloud example (in-house ₹22.83/h at 40% efficiency + ₹22 power/cooling + ₹6 management; cloud ₹42/h at 80% efficiency + ₹1 management), what is the ratio of total cost per effective hour (in-house : cloud)? numerical

A. 0.63

B. 1.07

C. 1.59

D. 2.17

Q6. With HDFS block size 128 MB and replication factor 3, how many block replicas are stored for files of 100 MB, 130 MB and 300 MB? numerical

A. 5

B. 6

C. 15

D. 18

Q7. To compute the average of a large integer set with several mappers and one reducer, what should each mapper emit so the reducer gets the correct global average? concept

A. Only its local average

B. Its local average (or sum) together with the count of elements

C. Only the count

D. The maximum element

Q8. 4 identical servers each draw 105 W idle and 170 W fully loaded (8 VMs). If 8 VMs are consolidated onto one server and the other three are switched off, what is the saving compared with keeping all 4 on at 2 VMs each (138 W per server)? numerical

A. 67 W

B. 382 W

C. 315 W

D. 0 W

Q9. Which resource-management aspect is defined as 'negotiation of resources through an agent to ensure necessary resources are available at the right time to complete the objectives'? theory

A. Resource mapping

B. Resource brokering

C. Resource adaptation

D. Resource estimation

Q10. According to the CSCC guidance for evaluating SLAs, which are the NIST cloud actors to identify? theory

A. Consumer, provider, carrier, broker, auditor

B. User, developer, admin, auditor

C. Consumer, provider, regulator, insurer

D. Tenant, landlord, broker, carrier

Answers — Week 5

Q1: A — Expected time = 10 × 30 = 300 h. Availability = 1 − 2/300 = 0.9933 = 99.33%, which is below 99.5%, so the SLA is violated.

Q2: B — Bill = 40 × 30 = $1200. 99.2% falls in the [99%, 99.95%) band, so the credit is 10% = $120. Payable = $1080.

Q3: B — Allowed downtime = (1 − 0.999) × 720 h = 0.72 h = 43.2 minutes.

Q4: A — 3 years = 26,280 h. Cost per hour = 876000/26280 = ₹33.33, and per core = 33.33/10 = ₹3.33.

Q5: C — In-house = 22.83/0.4 + 22 + 6 = 85.075. Cloud = 42/0.8 + 1 = 53.5. Ratio = 85.075/53.5 ≈ 1.59.

Q6: D — ⌈100/128⌉ = 1, ⌈130/128⌉ = 2 and ⌈300/128⌉ = 3, giving 6 blocks. With 3 replicas that is 18 stored blocks.

Q7: B — The reducer must weight each partial result by its count. Averaging the averages, e.g. (15+35+50)/3 = 33.3, is wrong when the counts differ; the correct answer in the lecture example is 30.

Q8: B — Spread: 4 × 138 = 552 W. Consolidated with the rest off: 170 W. Saving = 382 W. If the idle servers stay on, consolidation draws 485 W, a saving of only 67 W.

Q9: B — Brokering is agent-based negotiation. Mapping matches required resources to available ones, adaptation adjusts them dynamically and estimation guesses requirements.

Q10: A — Step 1 of the 7 CSCC steps is to identify the five NIST actors: consumer, provider, carrier, broker and auditor.

Week 6 — practice set

Q1. An intruder captures a message, alters it and forwards it so the receiver believes it came unaltered from the source. Which attack is this, and which security property does it violate? theory

A. Interception, confidentiality

B. Modification, integrity

C. Fabrication, authenticity

D. Interruption, availability

Q2. Which of the following are ACTIVE attacks? (Select all that apply) theory · select all

- **A.** Traffic analysis
- **B.** Replay
- **C.** Masquerade
- **D.** Release of message contents

Q3. Let R be the set of reachable states and Q the set of secure states. A security mechanism is 'precise' when: theory

- **A.** R ⊂ Q strictly
- **B.** R = Q
- **C.** R contains states not in Q
- **D.** Q is empty

Q4. Which of the following is NOT one of Gartner's seven cloud computing security risks? theory

- **A.** Data segregation
- **B.** Investigative support
- **C.** Long-term viability
- **D.** Pricing transparency

Q5. 'The maximum amount of data that may be lost following an interruption or disaster' defines: theory

- **A.** RTO
- **B.** RPO
- **C.** MTTR
- **D.** SLO

Q6. In the 'Hey, You, Get Off of My Cloud' study, which observations suggest two EC2 instances are co-resident? (Select all that apply) concept · select all

- **A.** Matching Dom0 IP address
- **B.** Small packet round-trip time
- **C.** Numerically close internal IP addresses (typically within 7)
- **D.** Being launched simultaneously from the same account

Q7. What does the threat model of the EC2 co-residency study assume? theory

- **A.** The provider is malicious
- **B.** Adversaries exploit hypervisor (VMM) vulnerabilities
- **C.** The provider and infrastructure are trusted, and adversaries are ordinary non-provider-affiliated customers
- **D.** Victims run only public, non-confidential services

Q8. The Inter-Domain Role Mapping (IDRM) problem in collaborative SaaS clouds is to: theory

- **A.** Map every user to all roles
- **B.** Find a minimal set of local roles that encompasses the requested permission set (with minimal excess permissions)
- **C.** Encrypt roles across domains
- **D.** Remove all roles from a domain

Q9. A user can indirectly obtain write access to an object through a cycle of cross-domain role inheritances, although their own role allows only viewing. This is an example of: concept

- **A.** SoD constraint violation
- **B.** Cyclic inheritance conflict
- **C.** Replay attack
- **D.** VM escape

Q10. In the fuzzy cloud-marketplace broker, what are the inputs and output of the provider-selection fuzzy inference engine? theory

A. Inputs: cost only; output: migration decision

B. Inputs: QoS offered by the provider and its trustworthiness; output: suitability of the provider

C. Inputs: degree of SLA satisfaction; output: trust

D. Inputs: customer identity; output: VM placement

Answers — Week 6

Q1: B — Modification (m changed to m′ in transit) violates integrity. Interception violates confidentiality, fabrication violates authenticity and interruption violates availability.

Q2: B, C — Replay and masquerade modify or create data streams, so they are active. Traffic analysis and release of message contents are passive eavesdropping attacks.

Q3: B — Secure means $R \subseteq Q$, precise means $R = Q$, and broad means some reachable states are not secure.

Q4: D — The seven are privileged user access, regulatory compliance and audit, data location, data segregation, recovery, investigative support and long-term viability.

Q5: B — RPO is the Recovery Point Objective (how much data may be lost). RTO is the time allowed between the disaster and activation of the secondary site.

Q6: A, B, C — Instances launched simultaneously from the same account are NOT placed on the same machine. The other three are the network-based co-residency checks.

Q7: C — The study assumes a trusted provider, no subversion of administrative functions or VMM exploits, and adversaries who are ordinary customers able to run many instances.

Q8: B — No polynomial-time solution is known, so greedy heuristics are used. Multiple minimal role sets, or none that match exactly, may exist.

Q9: B — This is a cyclic inheritance conflict. An SoD violation is when one subject gets two duties that must be separated, such as issuing and verifying a draft.

Q10: B — The migration decider is the engine whose output is the degree of SLA satisfaction. It migrates the service when that falls below a threshold.

Week 7 — practice set

Q1. In a mobile cloud computing framework, which component selects which parts of an application run on the mobile device and which run on the cloud? theory

A. Profiler

B. Solver

C. Synchronizer

D. Mobile load manager

Q2. Which statements about MAUI and COMET are correct? (Select all that apply) theory · select all

- **A.** MAUI requires the programmer to mark methods as remoteable
- **B.** In MAUI, native methods can be remoteable
- **C.** COMET works on unmodified applications using only binaries
- **D.** COMET uses distributed shared memory to migrate threads

Q3. A task needs C = 2×10^9 instructions. Mobile speed M = 10^9 instr/s, the cloud is F = 20 times faster, Pc = 1 W, Pi = 0.2 W, Ptr = 1.5 W, data D = 4 MB and bandwidth B = 4 MB/s. What is the energy saved by offloading? numerical

- **A.** 0.48 J
- **B.** −0.02 J
- **C.** 2.0 J
- **D.** 0.5 J

Q4. According to the offloading energy model, offloading is most beneficial when: concept

- **A.** Little computation and a lot of data transfer
- **B.** Large computation and relatively little data transfer
- **C.** The cloud is slower than the mobile device
- **D.** Bandwidth is very low

Q5. What is a cloudlet? theory

- **A.** A small public cloud region run by Amazon
- **B.** A trusted, resource-rich computer or cluster, well connected to the Internet and available to nearby mobile devices; the middle tier between mobile and cloud
- **C.** A mobile app that caches web pages
- **D.** A container image

Q6. Which characterizes FOG computing compared with cloud computing, according to the lecture's comparison table? theory

A. Multiple hops between client and server

B. Centralized geographical distribution

C. Location awareness and one-hop distance to the client

D. Leased-line last-mile connectivity

Q7. The term 'fog computing' was first introduced by: theory

A. Google

B. Amazon

C. Cisco

D. IBM

Q8. Which of the following are enabling technologies for fog computing named in the lecture? (Select all that apply) theory · select all

A. Virtualization

B. Containers (e.g., Docker)

C. Software-defined networking (e.g., OpenFlow)

D. Leased-line WAN links

Q9. Ten temperature sensors in a lab report readings; a sink node sends only the average to the cloud. This illustrates which fog benefit? concept

A. Increasing core congestion

B. Reducing bandwidth by aggregating data at the edge

C. Eliminating the cloud entirely

D. Increasing latency

Q10. In the geospatial cloud lecture, interoperability challenges are identified at which levels? theory

A. Data, service and security levels

B. Physical, link and network levels

C. Only the data level

D. Hardware and OS levels

Answers — Week 7

Q1: B — The profiler collects time, power and traffic data. The solver makes the placement decision. The synchronizer combines the results of the split execution.

Q2: A, C, D — In MAUI, native methods cannot be remoteable. COMET migrates threads via DSM and sends only the modified parts of the heap.

Q3: A — E = (C/M)(Pc − Pi/F) − Ptr·D/B = 2 × (1 − 0.01) − 1.5 × 1 = 1.98 − 1.5 = 0.48 J. This is positive, so offloading saves energy.

Q4: B — Energy is saved when D/B is small compared with C/M and the speed-up F is large.

Q5: B — A cloudlet cuts latency through single-hop access and can lower battery use, since it is reached over Wi-Fi or short-range radio.

Q6: C — Fog servers sit at the edge of the local network, one hop away, and are location-aware. They are also distributed, very numerous, support mobility and usually use wireless last-mile links.

Q7: C — Cisco introduced it to ease wireless data transfer to distributed IoT devices and to run applications directly on network-edge devices.

Q8: A, B, C — The enablers are virtualization, containers, SOA and SDN. Leased lines are the cloud's typical last-mile connectivity, not a fog enabler.

Q9: B — Fog aggregates or pre-processes data at access points. This reduces the bandwidth needed and the load on the cloud. Fog complements the cloud; it does not replace it.

Q10: A — Data-level interoperability (matching data sets), service-level (services talking to each other) and security-level (different policies on different data sets).

Week 8 — practice set

Q1. In Docker terminology, what is a 'container'? theory

A. A lightweight, stand-alone executable package including code, runtime and libraries

B. A runtime instance of an image

C. A registry hosting images

D. A tool to define multi-container apps

Q2. Which statements about containers vs virtual machines are correct? (Select all that apply) theory · select all

A. Containers share the host OS kernel

B. Each VM runs its own guest OS

C. Container images are typically several GBs and slow to start

D. Containers get native access to the host kernel and so tend to perform better

Q3. Which Docker component is described as the 'registry used to host various Docker images'? theory

A. Docker Engine

B. Docker Compose

C. Docker Hub

D. Dockerfile

Q4. A data center draws 1.5 MW in total, of which 600 kW goes to IT equipment. What is its PUE? numerical

A. 0.4

B. 2.5

C. 1.5

D. 0.9

Q5. According to the green cloud lecture, which component consumes the largest share of data-center energy? theory

A. IT equipment (40%)

B. Cooling system (45%)

C. Power distribution (15%)

D. Network switches (60%)

Q6. An idle server consumes about 66% of its peak power. What does this imply for energy-efficient VM placement? concept

- **A.** Spread VMs evenly across all servers
- **B.** Consolidate VMs onto fewer servers and put idle servers in sleep/off mode
- **C.** Keep all servers idle
- **D.** Energy does not depend on placement

Q7. In a sensor cloud, which virtual-sensor configuration has a virtual sensor built from a combination of multiple different types of physical sensors (e.g., to answer 'is a wildlife habitat safe?')? theory

- **A.** One-to-many
- **B.** Many-to-one
- **C.** Many-to-many
- **D.** Derived

Q8. In the one-to-many virtual sensor configuration, what computes the physical sensor's sampling duration and frequency? theory

- **A.** Each end user independently
- **B.** The middleware, considering all users
- **C.** The physical sensor's firmware only
- **D.** The cloud billing module

Q9. Which of the following are listed as limitations of sensor networks motivating the sensor cloud? (Select all that apply) theory · select all

- **A.** Hard to scale to large sizes
- **B.** Proprietary vendor-specific designs hinder interconnection
- **C.** Sensor data is easily shared between user groups
- **D.** Insufficient computation and storage for large-scale applications

Q10. Which connectivity protocols does the lecture list on the IoT side of an IoT cloud system? theory

A. MQTT, CoAP, HTTP

B. SOAP, WSDL, UDDI

C. FTP, SMTP, POP3

D. BGP, OSPF, RIP

Answers — Week 8

Q1: B — Option A defines an image, C is Docker Hub and D is Docker Compose.

Q2: A, B, D — Container images are typically tens of MBs and start almost instantly.

Q3: C — Docker Engine builds images and creates containers. Compose defines multi-container applications. A Dockerfile is the build script.

Q4: B — PUE = total facility power / IT power = 1500/600 = 2.5. DCiE = 1/PUE = 40%. An ideal PUE is 1.0.

Q5: B — The breakdown is cooling ≈ 45%, IT equipment ≈ 40% and power distribution ≈ 15%.

Q6: B — Idle servers still burn most of their peak power. Packing VMs together and switching off the idle machines saves the most energy.

Q7: D — The derived configuration generalizes the other three by combining different types of physical sensors.

Q8: B — One physical sensor is shared by many virtual sensors. The middleware works out the sampling schedule that satisfies all users.

Q9: A, B, D — In sensor networks, data can NOT be easily shared between groups of users, which is one of the limitations.

Q10: A — The IoT side uses MQTT (Message Queue Telemetry Transport), CoAP (Constrained Application Protocol) and HTTP. The cloud side lists MQTT, AMQP and HTTP.

Week 9 — practice set

Q1. In the health cloud–fog case study, which module was kept in the CLOUD even in the fog-based placement, and why? concept

A. Data filtering module, because it is compute-heavy

B. Confirmatory module, because it needs a global view

C. Client module, because it needs the display

D. Event handling module, because it needs storage

Q2. Which simulator was developed jointly by IIT Kharagpur and the University of Melbourne, as mentioned in the lecture? theory

A. CloudSim

B. iFogSim

C. NS-3

D. OMNeT++

Q3. In the resource-management taxonomy for fog/edge computing, which components fall under 'Architecture'? theory

A. Hardware, system software, middleware

B. Discovery, benchmarking, load balancing, placement

C. Data flow, control, tenancy

D. Latency, cost, energy

Q4. In the fog environment model, what happens when the fog layer has no or limited resources for a client request? theory

A. The request is dropped

B. The request is passed to the cloud layer

C. The edge device must process it

D. The request waits indefinitely

Q5. Which of the following are application-level constraints in service placement? (Select all that apply) theory · select all

A. Locality requirement (service must execute in a specific location)

B. Delay sensitivity / deadlines

C. Finite CPU and RAM of a node

D. Link bandwidth

Q6. Which control approaches does the lecture list for DISTRIBUTED service-placement control? theory

A. Solver-based and graph-matching

B. Blockchain-based, game-theoretic and genetic-algorithm-based

C. Only round robin

D. ILP solvers only

Q7. Match each federation architecture to its coupling level: cloud bursting, broker, aggregated, multi-tier. theory

A. Tight, loose, partial, loose

B. Loose, loose, partial, tight

C. Partial, tight, loose, loose

D. Loose, partial, tight, partial

Q8. Which feature is typically available ONLY in a tightly coupled federation? theory

A. Basic VM deployment in a remote cloud

B. Monitoring of CPU/memory/disk usage

C. Cross-site VM migration and deciding the exact placement of remote VMs

D. Using a broker to choose a public cloud

Q9. In post-copy live migration, what is transferred first? theory

A. All memory pages iteratively

B. The CPU (processor) state, after which memory pages are fetched on demand

C. Only the disk

D. Dirty pages first

Q10. The pre-copy phase of live migration is terminated when: (Select all that apply) theory · select all

A. The number of iterations exceeds a threshold

B. The total memory transmitted exceeds a threshold

C. The number of dirty pages in the previous round drops below a threshold

D. The VM's CPU usage reaches 100%

Answers — Week 9

Q1: B — Filtering, processing and event handling ran at the area gateway. The confirmatory module stayed in the cloud because it needs the global view.

Q2: B — iFogSim was developed jointly by IIT Kharagpur and the University of Melbourne. CloudSim comes from the University of Melbourne.

Q3: C — Option A lists the Infrastructure pillar and option B the Algorithms pillar.

Q4: B — The fog layer serves client requirements and forwards requests to the cloud (or to collaborating fog nodes) when it lacks resources.

Q5: A, B — CPU and RAM limits are resource-level constraints, and bandwidth and latency are network-level constraints.

Q6: B — Centralized control uses solver-based and graph-matching approaches. Distributed control uses blockchain, game theory or genetic algorithms.

Q7: B — Bursting/hybrid and broker are loosely coupled, aggregated is partially coupled and multi-tier (hierarchical, same corporation) is tightly coupled.

Q8: C — Loosely coupled federations offer limited monitoring and no advanced features such as VM migration. Tight coupling allows cross-site networks, VM migration, high availability and virtual storage across sites.

Q9: B — Post-copy stops the VM, copies the CPU state, restarts it at the destination and demand-pages memory. Pre-copy transfers memory first and the CPU state last.

Q10: A, B, C — These three thresholds trigger the stop-and-copy phase.

Week 10 — practice set

Q1. A VM with 2048 MB of memory is migrated non-live over a link of 128 MB/s. What are the migration time and downtime? numerical

A. 16 s and ≈0 s

B. 16 s and 16 s

C. 8 s and 8 s

D. 32 s and 16 s

Q2. Pre-copy live migration: V_m = 1000 MB, R = 100 MB/s, dirtying ratio r = 0.5, n = 2 iterations, restart time 0.5 s. What is the downtime? numerical

A. 10.5 s

B. 3.0 s

C. 2.5 s

D. 17.5 s

Q3. In pre-copy migration, what happens if the memory dirtying rate becomes equal to or larger than the transmission rate (r ≥ 1)? concept

A. Migration finishes faster

B. The remaining dirty memory never shrinks, so iterations stop only at N_max and downtime becomes large

C. Downtime becomes zero

D. Post-copy is automatically used by KVM

Q4. In parallel migration of m equal-sized VMs from one host to another, roughly what transmission rate does each VM get, and how is downtime determined? theory

A. R each; downtime = sum of all stop-and-copy phases

B. R/m each; downtime ≈ stop-and-copy time of any one VM + resume time

C. m·R each; downtime = 0

D. R/2 each; downtime = migration time of first VM

Q5. Which statement about containers vs virtual machines is correct? theory

A. Containers virtualize at the hardware level

B. Containers share the host OS kernel and virtualize at the OS level; VMs virtualize at the hardware level

C. Each container needs its own guest OS

D. Containers need more memory than VMs

Q6. Which are the three main benefits of containers listed in the lecture? (Select all that apply) theory · select all

- **A.** Separation of responsibilities
- **B.** Workload portability
- **C.** Application isolation
- **D.** Mandatory hardware virtualization

Q7. In Kubernetes, what runs on the worker nodes and what manages them? theory

- **A.** Pods run on worker nodes; the control plane manages nodes and Pods
- **B.** VMs run on control plane; Pods manage nodes
- **C.** Docker Hub runs on nodes; kubelet manages the cluster
- **D.** Clusters run on Pods; nodes manage the control plane

Q8. In Docker's architecture, which component receives client requests over HTTP and manages images and containers? theory

- **A.** Docker client
- **B.** Docker daemon (with a RESTful API)
- **C.** Docker Hub
- **D.** Dockerfile

Q9. Which Docker command saves a running container's current state (e.g., a modified database) as a new image? theory

- **A.** docker save
- **B.** docker load
- **C.** docker commit
- **D.** docker tag

Q10. Which statement best defines 'continuous delivery' as described in the Docker lecture? theory

A. Delivering software once a year

B. A software delivery paradigm based on a pipeline that rebuilds the system on every change and delivers it to production through an automated or partly automated process

C. Manually configuring each server

D. Running only on virtual machines

Answers — Week 10

Q1: B — In non-live migration $T_{mig} = V_m/R = 2048/128 = 16$ s. The VM is down for the whole transfer, so $T_{down} \approx T_{mig} = 16$ s.

Q2: B — $T_{down} = r^n \cdot V_m/R + T_{res} = 0.25 \times 10 + 0.5 = 3.0$ s. Total migration time $= 10 \times (1 - 0.125)/0.5 + 0.5 = 17.5 + 0.5 = 18.0$ s.

Q3: B — The geometric series converges only for $r < 1$. With $r \geq 1$ each round sends at least as much as the previous one.

Q4: B — All m VMs share the bandwidth and, being the same size, enter stop-and-copy together. In serial migration, the first VM uses the full rate R and the others are suspended when it enters stop-and-copy.

Q5: B — Containers have relaxed isolation (a shared kernel), so they are lightweight and use a fraction of a VM's memory.

Q6: A, B, C — Developers own the app logic while ops own deployment. Containers run virtually anywhere and are isolated at the OS level.

Q7: A — A cluster is a set of worker nodes, with at least one, that host Pods. In production the control plane runs on multiple machines for fault tolerance and high availability.

Q8: B — The client is only an intermediary to the daemon's RESTful API. Docker Hub is the public registry, and a private registry sits on the internal network.

Q9: C — docker commit makes an image from a container. docker save exports an image to a tar file and docker load imports it at the destination.

Q10: B — Docker builds are reproducible and replicable, which makes continuous delivery easier to implement.

Week 11 — practice set

Q1. Which two key features define dew computing? theory

A. Elasticity and multi-tenancy

B. Independence and collaboration

C. Scalability and metering

D. Virtualization and isolation

Q2. Dropbox keeps a local copy of your folder that you can edit offline and syncs it with the cloud later. Which dew category is this? concept

A. Platform-in-Dew

B. Storage-in-Dew

C. Software-in-Dew

D. Web-in-Dew

Q3. In Database-in-Dew, the local device and the cloud both store copies of the same database. What is a key benefit? theory

A. Eliminates the need for a DBA

B. Increased reliability, since one copy can act as a backup of the other

C. Removes the need for synchronization

D. Makes the database stateless

Q4. Which statement about serverless computing is correct? theory

A. It runs code without any servers at all

B. It lets users run event-driven, granular applications without handling operational logic; the provider provisions servers and bills only for use

C. Developers must provision and patch VMs

D. Functions run continuously and are billed per hour regardless of use

Q5. Which is an example of Backend-as-a-Service (BaaS) rather than FaaS? theory

A. AWS Lambda

B. Google Cloud Functions

C. Google Firebase (managed database/auth used directly from apps)

D. Azure Functions

Q6. Which of the following are drawbacks of serverless computing discussed in the lecture? (Select all that apply) theory · select all

A. Functions calling other functions increase debugging complexity and cost

B. Risk of hitting image size limits and longer warm-up for bigger images

C. Temptation to create too many functions, lowering maintainability

D. Developers must manage OS patching

Q7. In AWS Lambda, what are 'Lambda layers'? theory

A. Network layers used by Lambda

B. A distribution mechanism for libraries, custom runtimes and other function dependencies

C. Logs of each invocation

D. The AWS service that triggers the function

Q8. In Azure Functions, a function consists of which two pieces? theory

A. A VM and a container

B. Code and configuration (function.json)

C. A trigger and a queue only

D. A database and a storage account

Q9. In the sustainable cloud model, the power manager uses grid energy for which kind of workload? theory

A. Workloads with no deadlines

B. Deadline-oriented workloads, to maintain reliability

C. Only idle servers

D. Only cooling devices

Q10. Why can DVFS-based energy management hurt service quality? concept

A. It increases carbon emissions

B. Switching between high and low frequency/voltage modes increases response time and service delay, risking SLA violations

C. It disables virtualization

D. It requires renewable energy

Answers — Week 11

Q1: B — Independence means the local device provides service without a continuous Internet connection. Collaboration means it connects and synchronizes with the cloud when appropriate.

Q2: B — Storage-in-dew means the local device's storage is partially or fully copied to the cloud, which meets both independence and collaboration. GitHub is the platform-in-dew example and App Store/Google Play the software-in-dew example.

Q3: B — One copy is designated the main version and the other acts as a backup, which increases reliability.

Q4: B — Servers still exist but are hidden and managed by the provider. Functions are event-triggered, stateless and billed when they run.

Q5: C — BaaS replaces server-side components with off-the-shelf services such as authentication, databases and storage. The other three options are FaaS platforms.

Q6: A, B, C — OS patching is handled by the provider in serverless, which is one of its benefits.

Q7: B — The trigger origin is the event source, and custom logging goes to log streams.

Q8: B — For compiled languages the configuration is generated from annotations. For scripting languages you must provide function.json yourself.

Q9: B — Renewable energy can serve flexible work. Grid energy keeps deadline-oriented jobs reliable. An ATS switches between sources and PDUs distribute the power.

Q10: B — This is the energy-versus-reliability trade-off in sustainable cloud computing.

Week 12 — practice set

Q1. Which 5G service class is meant for applications like autonomous vehicles, remote surgery and smart-grid control? theory

A. Enhanced Mobile Broadband (eMBB)

B. Ultra-Reliable Low-Latency Communication (URLLC)

C. Massive Machine-Type Communication (mMTC)

D. Fixed wireless access

Q2. Which of the following are the two key aspects of the relationship between 5G and cloud computing given in the lecture? (Select all that apply) theory · select all

A. Cloud computing's further development must meet 5G needs (growing role of edge/MEC/fog)

B. 5G is undergoing cloudification through softwarization (NFV, SDN)

C. 5G replaces the need for cloud data centers

D. Cloud computing no longer needs connectivity

Q3. The term 'cyber-physical systems' was coined around 2006 by: theory

A. Vint Cerf at DARPA

B. Helen Gill at the US National Science Foundation

C. Cisco Systems

D. NIST

Q4. Zero-net-energy buildings that switch lights and AC by occupancy and daylight are an example of which CPS application area? concept

A. Intervention

B. Precision

C. Efficiency

D. Coordination

Q5. A cyber-physical cloud computing (CPCC) framework is best described as: theory

A. A cloud with no sensors

B. A system environment that can rapidly build, modify and provision CPS composed of cloud-based sensor, processing, control and data services

C. A purely physical control loop

D. A storage-only cloud

Q6. Tobler's First Law of Geography, used in the spatial cloud lecture, states that: theory

A. Distant things are more related than near things

B. Everything is related to everything else, but near things are more related than distant things

C. Spatial data has no temporal aspect

D. All maps use the same projection

Q7. In the IoHT iFogSim case study, which link had the highest latency setting? theory

A. Body sensor → mobile (1)

B. Mobile → area gateway (2)

C. Area gateway → ISP gateway (2)

D. ISP gateway → cloud (100)

Q8. In the IoHT simulation, which metric stayed about the same in both cloud-only and fog–edge placements? concept

A. Network usage

B. Cost of execution in cloud

C. Mobile (edge) device energy

D. Average latency

Q9. An accelerometer magnitude series is 3, 5, 9, 7, 6 (centre value 9). What is the 5-point smoothed value at the centre? numerical

A. 9

B. 6

C. 7

D. 5

Q10. Which classifier was used for activity detection in the IoHT hardware implementation? theory

A. Support Vector Machine

B. k-Nearest Neighbours

C. Decision tree

D. Naive Bayes

Answers — Week 12

Q1: B — These are mission-critical and need ultra-reliable, low-latency links. eMBB covers AR/VR and broadband. mMTC covers massive IoT such as smart agriculture and asset tracking.

Q2: A, B — Each influences the other's development, leading to an era of hyper-connectivity.

Q3: B — In a CPS, embedded computers monitor and control physical processes through feedback loops. It is about the intersection, not the union, of the physical and the cyber.

Q4: C — Intervention covers collision avoidance, precision covers robotic surgery and nano-manufacturing, and coordination covers air-traffic control and war-fighting.

Q5: B — Its benefits are efficient resource use, modular composition, rapid development, scalability, smart adaptation and a reliable, resilient architecture.

Q6: B — The lecture's example is that Kharagpur's temperature is closer to Midnapore's than to Darjeeling's.

Q7: D — The cloud hop dominates latency, so moving filtering, processing and event handling to the area gateway keeps latency roughly constant.

Q8: C — The mobile's load does not change between placements. Network usage, cloud cost and latency all rise sharply in the cloud-only placement.

Q9: B — 5-point smoothing averages the 2 preceding values, the value itself and the 2 succeeding values: (3+5+9+7+6)/5 = 30/5 = 6.

Q10: B — The pipeline was magnitude $\sqrt{(x^2+y^2+z^2)}$, then 5-point smoothing, then features (min/max/mean amplitude, standard deviation, time- and frequency-domain energy), then a kNN classifier.

GAAND FATT RAHI HAII😬.....

⚠️ **Still no official NPTEL assignment questions.** No assignment PDFs were uploaded, so nothing in this tab is an NPTEL assignment question.

Your practice sets are now included. 8 of your 12 files are on-syllabus: **400 questions**, one lecture each, every answer checked against the transcripts.

- Each week below ends with **all of its practice answers in one line each**; tap 🎯 to open the list.
- Questions whose key follows a lecture slip or goes past the lecture carry a ⚠️ note.
- The interactive bank, with saved progress and a retry-wrong mode, is in Your practice sets.

Files **w6–w9** turned out to be from the NPTEL *Smart Grid* course and are left out. The worked problems below are the ones **solved in the lectures and tutorials**, each with a variant using changed values.

Course at a glance

Week	Key topics	Your practice set
1	Evolution (distributed→grid/cluster→utility→cloud) · NIST 5-3-4 · IaaS/PaaS/SaaS/XaaS · client–server vs cloud · architecture (goals→attributes→tactics) · scale-up vs scale-out · NFV	L1 · 50 Q
2	Deployment models (costs, resources, perimeters) · IaaS manager hierarchy · Popek–Goldberg · full/para/HW-assisted · XML (well-formed/valid, DTD/XSD, SAX/DOM) · SOAP/WSDL/UDDI, SOA	L6 · 50 Q
3	SLA (KPI→SLO→SLA) · cloudonomics (C_v, $U < P/A$, penalty) · GFS/HDFS/BigTable/Dynamo · MapReduce & efficiency · OpenStack components	L11 · 50 Q
4	Meghamala/OpenStack demo · Azure (web/worker roles, deployment) · GCP (App Engine vs Compute Engine, bucket hosting)	L16 · 50 Q
5	**Numerical tutorials**: SLA credits, economics, HDFS blocks, MapReduce average · resource management (green, consolidation, RM aspects)	L21 · 50 Q
6	Security basics · cloud-specific risks, Gartner 7, RPO/RTO · EC2 co-residency case · collaborative SaaS security · fuzzy broker	L26 · 50 Q
7	MCC (profiler/solver/synchronizer, MAUI vs COMET, cloudlet, **offload energy**) · fog (Cisco, cloud vs fog table) · geospatial cloud	L31 · 50 Q
8	Docker intro · green cloud (energy %, PUE) · sensor cloud (virtual sensors) · IoT cloud (MQTT/CoAP) · research areas	L36 · 50 Q
9	Cloud–fog health case (iFogSim) · fog RM taxonomy · service placement · federation (coupling, architectures) · VM migration basics	—
10	**Migration formulas** · containers vs VMs · Kubernetes · Docker architecture & demo	—
11	Dew computing · serverless (FaaS/BaaS, Lambda/Cloud Functions/Azure Functions) · sustainable cloud	—
12	5G (eMBB/URLLC/mMTC) · CPS & CPCC · spatial cloud & Tobler · IoHT case (latencies, placement, kNN)	—

Week 1 — must-remember

→ open the full Week 1 notes

- NIST: **5** characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service) · **3** service models · **4** deployment models.
- **Utility computing** = metered, charge per use (concept) → **cloud** realizes it. Five utility risks: backup, security, competence, SLA definition, charge-back.

- **Grid** = unused cycles / single system image (power-grid analogy); **cluster** = stand-alone computers as one resource (HA, load-balancing, parallel).
- Client–server: **no virtualization**; cloud: virtualization is core, theoretically infinite scaling.
- Provider responsibility: IaaS → **virtualization**; PaaS → **runtime**; SaaS → **application**. OSI: IaaS → data link; PaaS → presentation; SaaS → all.
- **Scale-out** is the only solution at large scale (64 × 1-way ≪ one 64-way server).
- Lecture classifications: **Azure = PaaS**, App Engine = PaaS, EC2 = IaaS, Salesforce/Google Docs = SaaS.
- SaaS **not** for real-time; PaaS risk = **vendor lock-in**; IaaS not when highest dedicated performance is needed.

Likely question types (from the lectures): pick the non-NIST characteristic · classify a product into IaaS/PaaS/SaaS · where is X *not* suitable · scale-up vs scale-out.

🎯 Your practice set — L1 Cloud Computing — Overview (file w10): 50 answers in one line

1. According to the lecture, what is the primary objective of this course on cloud computing? → To give an overall picture of what cloud computing is, its components, and recent trends.
2. The lecture cites the ACM's 2005 definition of 'computing'. What is the core idea of this definition? → Any goal-oriented activity that can be solved through computation.
3. What is the primary purpose of a Service Level Agreement (SLA) as described in the lecture? → An agreement between the service provider and consumer on the guarantee of services.
4. A university needs 100% service availability for its online exam portal but only 90% for practice sessions. What is the direct implication for the cloud provider, based on the lecture's explanation? → The provider will need to provision more redundant resources for the exam portal, leading to a higher cost. hard
5. What key economic question does the topic of 'Cloudonomics' address for an organization? → Whether it is more economically beneficial to purchase (build) or hire (use cloud) computing resources.
6. From which perspective is 'resource management' primarily a concern, according to the lecture? → The cloud service provider (CSP).
7. The lecture highlights a fundamental security principle by stating, 'you can... reinstall an application but you cannot reinstall your data.' What critical aspect of cloud usage does this emphasize? → The primary value and irreplaceability of the user's data. hard
8. The lecture mentions 'three nodes of a triangle' that are interlinked in the context of cloud security. What are these three concepts? → Trust, Reputation, and Risk.
9. How does the lecture characterize the emergence of cloud computing in the history of technology? → As an evolution that amalgamates and builds upon previous developments like distributed computing.
10. Which of the following is NOT listed as a computing paradigm that contributed to the rise of cloud computing? → Quantum Computing
11. According to the lecture, what is a key characteristic of the 'autonomous computational entities' in a distributed system? → Each entity has its own local memory.
12. The lecture contrasts distributed computing with the older mainframe-based centralized computing model. What is the main difference highlighted? → Centralized systems have a single processing unit, while distributed systems have multiple autonomous processors.
13. What is the primary benefit of 'fault tolerance' in a distributed system? → The system can continue to work even if some of its nodes fail.
14. A bank wants to ensure its ATM network continues to operate even if a regional data center fails. Which property of distributed systems is most crucial for this requirement? → Robustness and avoidance of a single point of failure. hard

15. Which of the following service models involves providing access to operating systems, storage, and networking resources over the internet? → Infrastructure as a Service (IaaS)
16. What does the lecturer use as an analogy to explain the 'buy vs. hire' decision in Cloudonomics? → Purchasing a car for daily commutes vs. hiring one for occasional long trips.
17. What is described as a 'serious challenge' in resource management for a cloud provider? → Optimally managing resources to avoid over-provisioning (waste) or under-provisioning (starvation). hard
18. The lecture uses Dropbox as a real-world example of what concept? → Storage as a Service
19. What is the stated motivation in the lecture for covering recent trends like 'Fog Computing'? → To provide pointers for future research and projects for students.
20. How do the 'autonomous computational entities' in a distributed system communicate with each other, according to the lecture? → By message passing over a communication network.
21. Which of these is mentioned as an example of a modern, ad hoc distributed network that is highly volatile? → Vehicular Ad Hoc Networks (VANETs).
22. A startup is launching a mobile game and anticipates a massive, but temporary, spike in users during the first week. Based on the lecture's Cloudonomics principles, what is their most financially sound strategy? → Hire resources from a cloud provider to handle the spike and scale down afterward. hard
23. When a user moves their data from a personal laptop to a cloud storage service, what key responsibility shifts from the user to the provider? → Managing backups and redundancy to prevent data loss.
24. The lecture's course outline mentions 'deployment models'. What does this term likely refer to in a cloud context? → Whether a cloud is public, private, hybrid, or community.
25. What is the main vulnerability of a purely centralized computing system like an old mainframe, as implied by the discussion on distributed systems? → It has a single point of failure.
26. The course outline mentions that the cloud computing stack is 'moving towards a service oriented architecture (SOA)'. What does this imply about cloud services? → Services are designed as independent, interoperable components that can be combined. hard
27. What are the three broad types of computers (nodes) in a distributed system, as categorized in the lecture? → Workstations, Server Systems, and Personal Assistance Devices
28. According to the lecture, what is a potential negative consequence for a cloud provider of over-provisioning resources? → They will incur more cost for underutilized resources.
29. What is the significance of the statement that 'each node in the distributed system plays its partial role'? → It suggests that a complex problem is broken down and solved collaboratively by multiple nodes.
30. Vehicular Ad Hoc Networks (VANETs) are used as an example of a distributed system. Based on the lecture, which of the following is a plausible function for a VANET? → Providing infotainment or safety-related information to drivers.
31. How does the lecture relate parallel computing to distributed systems? → Parallel computing can be considered a subset of distributed systems, focused on high performance.
32. What concern does 'Identity and Access Management' (IAM) address in a cloud security context? → Ensuring the correct entities (users, services) can access the appropriate resources and nothing more.
33. The initial lectures of the course are planned to focus primarily on which topic? → An introduction to cloud computing, including NIST models and characteristics.
34. The concept of 'Anything as a Service' or 'XaaS' implies what about the cloud model? → The service model is flexible and can be extended to offer potentially any capability as a service.
35. The lecture describes distributed systems as 'easy to expand'. What does this mean in a practical sense? → It is straightforward to add more computers (nodes) to the system to increase its capacity.
36. From a consumer's point of view, what is the main purpose of an SLA for a critical application? → To have a guarantee of minimal service levels, such as uptime.
37. What is the primary difference between resource sharing and load sharing as implied in the lecture? → Resource sharing is about making hardware available, while load sharing is about distributing computational work across that hardware. hard
38. According to the lecture, which of the following is a primary motivation for the development of distributed systems? → The need for robustness and the elimination of single points of failure for critical applications. hard

39. The lecture mentions that a cloud provider's resource management must handle several types of resources. Which of the following was NOT mentioned as a typical resource? → Personnel and human resources
40. Who is the primary intended audience for the lecture module on 'Recent Trends in Cloud Computing'? → Students and researchers looking for future project or research topics.
41. The lecture cites 'role base access control mechanism' as an example of what broader security concept? → Access control
42. From a cloud service provider's (CSP) perspective, what is a primary concern related to an SLA they offer? → Understanding the resource requirements needed to meet the guaranteed service level. hard
43. Why is data management considered 'tricky' in a cloud environment, according to the lecture? → Because traditional database management systems may not be suitable for the scale and service model of the cloud.
44. The lecture states that cloud computing 'is not a a suddenly new stuff which came into the play.' What does this imply about its technological origins? → Its core technologies are built upon decades of development in fields like distributed and utility computing.
45. In the context of distributed systems, what does 'load sharing' help to achieve? → Preventing any single node from being overwhelmed by distributing tasks among multiple nodes.
46. The lecture mentions that the course will cover 'case studies'. What kind of clouds will these case studies examine? → Both open source and commercial cloud platforms.
47. What is the fundamental security risk highlighted when you use a cloud service? → Your data and computation are located in someone else's domain.
48. What is the primary characteristic of a 'server system' node within a distributed system? → It is a computer that primarily provides resources and services to other nodes.
49. According to the lecture, what is the relationship between the availability guarantee in an SLA and the price of the service? → Higher availability guarantees require more resources from the provider, leading to a higher price. hard
50. The lecture identifies 'Trust, Reputation, and Risk' as a critical triad in cloud security. What does this triad primarily govern? → The user's confidence in a cloud provider.

Week 2 — must-remember

→ open the full Week 2 notes

- Public: **low** up-front cost, multi-tenancy risk, restrictive default SLA. On-site private: **limited** resources, **significant–high** cost. Outsourced private: **two perimeters**, **extensive** resources. On-site community: **highly variable** cost.
- Hybrid constituents **remain unique entities**; hybrid can be extremely complex; members join/leave.
- IaaS hierarchy: **Cloud manager (+DOS) → Cluster manager (+PLS) → Computer manager** (per host).
- Popek–Goldberg: equivalence, resource control, efficiency; VMM possible if **sensitive ⊆ privileged**.
- Full (binary translation, QEMU/Bochs, portable) · Para (**modified guest**, Xen, faster) · HW-assisted (**Intel VT/AMD-V**, unmodified guest, ring 0).
- XML: **valid ⇒ well-formed**; XSD (2001, written in XML, no entities); `xmlns`; **SAX** events/fast vs **DOM** tree/memory.
- **SOAP** (stateless, one-way, HTTP) · **WSDL** (describe) · **UDDI** (registry); SOA = publish–find–bind (IBM).

Likely question types: deployment-model consideration matching · which component does X in IaaS hierarchy · virtualization type from a description · well-formed vs valid · SOAP/WSDL/UDDI matching.

🎯 Your practice set — L6 Deployment Models (file w11): 50 answers in one line · ⚠️ 1

1. According to the lecture, which of the following is a primary characteristic of a public cloud deployment model? → The infrastructure is provisioned for open use by the general public.
2. A financial services company is concerned about sharing physical hardware with competitors. Which deployment model risk, mentioned in the lecture, is their primary concern? → Risk from multi-tenancy
3. A small startup with a limited budget wants to deploy its application without a large initial investment. Which feature of the public cloud would be most appealing to them? → Low upfront cost
4. What does the lecture mean by 'workload locations are hidden' in a public cloud context? → The user does not know the specific physical server or location where their VM or data resides.
5. Which of the following is NOT listed as an example of a public cloud provider in the lecture? → Eucalyptus
6. A university decides to use a public cloud for its student labs. According to the lecture, what is the most critical dependency they must consider? → The availability and quality of their network connectivity.
7. Which statement best describes a private cloud as defined in the lecture? → It is provisioned for exclusive use by a single organization.
8. The lecture challenges a common belief about private clouds. What is this contrary belief? → That private clouds can exist off-premises and be managed by a third party.
9. What is the key difference between an 'on-site private cloud' and an 'outsourced private cloud'? → The physical location of the infrastructure (customer's premises vs. hosting company).
10. An organization is building an on-site private cloud. What is a major financial consideration they will face, according to the lecture? → Significant to high upfront costs for migration and setup.
11. In an 'outsourced private cloud' setup, what does the lecture identify as a critical component for ensuring security? → The strength of both the subscriber's and provider's security perimeters and the protected communication link. hard
12. Compared to an on-site private cloud, what is a key advantage of an outsourced private cloud regarding resources? → Extensive resource availability and better elasticity.
13. The lecture mentions IIT Kharagpur's 'Meghamala' as an example of what? → An on-site private cloud for research purposes.
14. What defines a community cloud according to the lecture? → It is for exclusive use by a specific community of organizations with shared concerns.
15. A group of government agencies decides to create a cloud to share data related to environmental policy. This is an example of which deployment model? → Community Cloud
16. Why can the access policy of a community cloud be 'pretty complex'? → Because multiple organizations are involved, requiring rules to prevent information leakage between communities. hard
17. What access control techniques are mentioned as being useful for managing complex community clouds? → Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC)
18. Which of the following best describes a hybrid cloud? → A composition of two or more distinct cloud infrastructures (private, public, or community).
19. What is a key technological requirement for a functional hybrid cloud, as stated in the lecture? → A standardized or proprietary technology that enables data and application portability. hard
20. An e-commerce site uses its on-site private cloud for processing payments but shifts its public-facing website to a public cloud to handle traffic spikes. This strategy is an example of using which deployment model? → Hybrid Cloud
21. What is described as a major challenge or disadvantage of implementing a hybrid cloud? → It can be extremely complex to architect and manage.
22. The lecture discusses the dynamic nature of hybrid clouds. What does this refer to? → The possibility that constituent clouds may join or leave the hybrid environment over time. hard

23. An organization suddenly needs more computing power than its on-site private cloud can provide. The lecture suggests they could use a public cloud for a short period. This practice is often referred to as: → Cloud bursting
    ⚠ The lecture describes the idea (using a public cloud for the overflow) without naming it; 'cloud bursting' is the standard term.
24. When choosing a deployment model, what does the lecture suggest is the ultimate deciding factor? → The organization's specific requirements, including business needs and client demands.
25. A data storage provider serves clients in the defense sector. Why might these clients compel the provider to use a private cloud instead of a public one? → Because the clients have mission-critical security requirements and want assurance that data is not on shared infrastructure. hard
26. Which deployment model is characterized by restrictive, non-negotiable Service Level Agreements (SLAs) for most users? → Public Cloud
27. Which issue is common to ALL four deployment models (Public, Private, Community, Hybrid) as discussed in the lecture? → Network dependency hard
28. What is the primary difference in resource limitation between an on-site private cloud and a public cloud? → The on-site private cloud has fixed capacity, while the public cloud offers the 'illusion of unlimited resources'.
29. Which of the following scenarios describes an outsourced community cloud? → Several non-profits pay a third-party provider to host a shared application platform in the provider's data center. hard
30. The lecture states that in a public cloud, 'there can be some attackers, hackers, etc.' among the clients. What is the implication of this? → The cloud serves a diverse and not always trustworthy pool of clients.
31. Why might an organization still need IT skills even when using an on-site private cloud? → Because the organization is responsible for maintaining its own cloud infrastructure.
32. What is the security perimeter for an on-site private cloud? → It extends around both the subscriber's on-site resources and the private cloud resources.
33. In a hybrid cloud model, an application runs partly on a private cloud and partly on a public cloud. What specific 'portability' issue is mentioned in the lecture? → The need to ensure both data and the application itself can work across the different environments. hard
34. Which statement accurately reflects the lecture's view on control and visibility in a public cloud? → Users have limited visibility and control, relying heavily on the Service Level Agreement (SLA).
35. According to the lecture, how can an organization manage workloads with different security requirements? → By using a hybrid model to classify and place sensitive workloads on a private cloud and less sensitive ones on a public cloud.
36. Which of these is NOT a characteristic of an on-site private cloud mentioned in the lecture? → Theoretically infinite elasticity
37. What is a potential downside of the 'hidden workload location' feature in a public cloud? → It may conflict with legal or policy matters regarding data residency and security. hard
38. What is the relationship between an organization and a community in a community cloud model? → An organization can be a part of more than one community.
39. The lecture mentions that public cloud infrastructure exists 'in the premise of the cloud provider'. What is the direct implication of this? → The user organization has no physical control over the computing and storage infrastructure.
40. How does an outsourced private cloud offer 'extensive resource availability' compared to an on-site one? → The hosting provider has a large backbone of resources that can be provisioned on request.
41. Which two deployment models can exist either on-premises or off-premises (outsourced)? → Private and Community hard
42. An organization is considering a community cloud. What is a primary financial difference between building it on-premises versus outsourcing it? → The outsourced model has a 'modest to significant' upfront cost, which is likely less than the 'high variable' cost of an on-premises build. hard
43. What inherent risk of a public cloud is also present within an on-site private cloud, according to the lecture? → Risk from multi-tenancy among different internal business units. hard

44. A key challenge of a hybrid cloud is that an application 'needs to run over a combination of public and private cloud.' What is the main difficulty here? → Ensuring the application runs seamlessly despite a potentially very complex underlying architecture.
45. Which of these is presented as a benefit of a public cloud? → Illusion of unlimited resource availability and elasticity.
46. For which deployment model is the subscriber most likely to have full control over the resources and security perimeter? → On-site Private Cloud
47. The lecture discusses a scenario where an organization might classify its applications and data into different categories. What is the purpose of this classification? → To determine which workloads can go to a public cloud, private cloud, or community cloud.
48. What is the primary motivation for forming a community cloud? → To serve a group of organizations that have shared concerns, missions, or security requirements.
49. If a company uses VMware vCloud to bridge its on-premises data center with a public cloud provider, which deployment model is it implementing? → Hybrid Cloud
50. According to the lecture, what is a significant performance dependency for public cloud users? → The speed and reliability of the public internet connection.

Week 3 — must-remember

→ open the full Week 3 notes

- KPI → SLO → SLA; off-the-shelf SLA unsuitable for mission-critical; cloud SLA needs **automation**; neutral **third-party** monitoring.
- $C_v = \sigma/|\mu|$; n independent loads → $C_v/\sqrt{n}$.
- Cloud cheaper ⇔ $U = C/B < P/A$; penalty $\propto \int |D - R|$; exponential demand ⇒ exponential penalty.
- $P(\text{fail}) = 1 - (1-p)^N$ → fault tolerance essential.
- GFS read: **one** replica's metadata; append: **all** replicas, **primary** chooses offset; master = NameNode, chunk server = DataNode; **3** replicas.
- Dynamo: key–value, MD5 ring, $R + W > N$; BigTable: row/column/timestamp, tablets, SSTables.
- Word count: split words $1/p$ (not scalable) vs split docs $1/(1 + 2p/nf)$; MR $1/(1 + 2c\sigma/w)$.
- OpenStack: Nova compute, Neutron network, **Swift object**, **Cinder block**, Keystone identity, Glance image, Ceilometer metering, Horizon dashboard.

Lecture-solved problems & variants

- **Quorum (L13).** $N = 3$: is $(R, W) = (2, 2)$ consistent? Yes: $2 + 2 = 4 > 3$. *Variant:* $N = 5$, $(R, W) = (2, 3)$? No: $5 \not> 5$; need e.g. $(3, 3)$.
- **Aggregation (C_v).** 100 independent workloads with $C_v = 0.5$ each. Aggregate C_v? $0.5/\sqrt{100} = 0.05$. *Variant:* 49 workloads, $C_v = 0.7$? $0.7/7 = 0.1$.
- **Utility pricing.** $P = 100$, $A = 25$, $C = 0.30$, $B = 0.10$. Cloud or own? $U = 3 < P/A = 4$ → cloud. *Variant:* $P = 60$, $A = 40$, $U = 2$? $P/A = 1.5 < 2$ → own.
- **Failure probability.** $p = 0.001$, $N = 1000$? $1 - 0.999^{1000} \approx 0.63$.

🎯 Your practice set — L11 Service Level Agreement (file w12): 50 answers in one line · ⚠️ 2

1. What is the fundamental definition of a Service Level Agreement (SLA) as described in the lecture? → A formal contract between a service provider and a service consumer.
2. According to the lecture, what is the primary purpose of an SLA for a consumer? → To define a formal, guaranteed basis for the performance and availability of a service.
3. What is the relationship between an SLA, Service Level Objectives (SLOs), and Key Performance Indicators (KPIs)? → SLOs are high-level goals in the SLA, and they are measured by aggregating low-level KPIs.
4. A financial firm requires 99.99% uptime during trading hours but can tolerate 98% uptime overnight. This is an example of which concept? → A Service Level Objective (SLO) that varies over time.
5. Why is a neutral third-party auditor considered the best solution for monitoring SLA compliance? → They eliminate the conflict of interest inherent in a provider self-reporting its own performance. hard
6. What is a key difference between traditional web service SLAs and modern cloud SLAs regarding Quality of Service (QoS) parameters? → Cloud SLAs place a greater emphasis on security, privacy, and trust management compared to traditional web service SLAs.
7. An SLA for a public cloud provider is offered as a standard, take-it-or-leave-it contract. What type of SLA is this? → Off-the-shelf or non-negotiable SLA
8. The lecture discusses the risk of 'seizure of data' by law enforcement. Why is this a particularly complex issue in a multi-tenant cloud environment? → Seizing data from one targeted consumer may unintentionally impact or disrupt service for other consumers sharing the same physical infrastructure. hard
9. Which of the following is an example of a Key Performance Indicator (KPI)? → Downtime measured in minutes per month.
10. A large corporation is migrating its entire financial system to the cloud and has very specific data residency and compliance needs. Which SLA approach should it pursue? → A negotiable SLA to fine-tune the agreement.
11. An SLA clause that requires the provider to 'zero out the memory' of a VM after a consumer powers it off falls under which SLA requirement category? → Hardware Erasure and Privacy
12. What does the metric 'linearity' evaluate in the context of an SLA? → How system performance scales in relation to increases in load.
13. Why might a cloud provider place limits on the frequency of consumer-initiated audits? → Because audits are disruptive, expensive, and can consume significant resources.
14. What is a 'penalty' or 'remedy' in an SLA? → A consequence, such as a service credit, the provider must give the consumer for failing to meet a guaranteed service level.
15. The metric 'durability' in a storage SLA is concerned with: → The likelihood of the data being permanently lost.
16. A cloud provider offers a storage service with 99% availability but 99.9999% durability. What is the practical meaning of this for a user? → The service might be temporarily unavailable, but the data itself is extremely unlikely to be lost. hard
17. According to the lecture, why is SLA automation more critical for cloud services than for traditional web services? → Because cloud environments are highly dynamic and scalable, requiring automated management to keep up with changes.
18. What is the role of the Web Service Level Agreement (WS-LA) framework? → A formal, XML-based language and runtime interpreter to define, negotiate, and monitor SLAs for web services.
19. An SLA for an IaaS provider states the provider is responsible for the network and server hardware, but the consumer is responsible for patching the OS on their VMs. This is an example of: → A well-defined balance of responsibilities. hard
20. A European company must comply with GDPR, which restricts data transfers outside the EU. Which SLA consideration is most critical for them when choosing a cloud provider? → Location of the data hard
21. According to the lecture, a service level agreement is applicable to which cloud service models? → IaaS, PaaS, SaaS, and other types of services.

22. What is the primary concern addressed by the 'isolation of consumer data' requirement in a multi-tenant SLA? → Ensuring that one consumer cannot access or interfere with another's data.
23. What does the lecture identify as a significant challenge with SLAs today? → There is no universally accepted standard format for them.
24. An SLA clause that specifies that all legal disputes will be handled in the courts of California is an example of what? → Jurisdiction
25. A consumer's business is heavily dependent on their cloud service. Which SLA clause would be most critical in the event of a major earthquake near the provider's primary data center? → The clause on business continuity and disaster recovery.
26. What is the primary function of UDDI (Universal Description, Discovery, and Integration) in the context of web services? → To serve as a directory for advertising and discovering services.
27. A service's 'elasticity,' when defined in an SLA, refers to its ability to: → Automatically scale resources up or down to match demand.
28. An SLA for a PaaS provider guarantees platform uptime. A consumer deploys poorly optimized code that makes their application slow. Who is at fault for the poor application performance? → The consumer, for deploying inefficient code on the platform. hard

    ⚠ This is reasoning from the lecture's 'balance of responsibilities' (the consumer owns its code on PaaS), not a statement from the lecture.
29. The lecture lists 'transparency' as an SLA requirement. What does this most likely mean? → The provider must be open and honest about its procedures, security, and how it meets its guarantees.
30. Which of the following would NOT be a typical component of an SLA's contents? → The provider's internal employee salary list.
31. What is the main reason a consumer migrating from an on-premise system cares deeply about the SLA? → To get a guarantee that the cloud service will offer a comparable level of reliability, performance, and security.
32. What is a potential drawback of the Web Service Level Agreement (WS-LA) framework mentioned in the lecture? → It has a lack of formal definitions for the semantics (the meaning) of the metrics. hard
33. An SLA that can be customized through discussion with an intermediary is known as what? → Negotiable via external agent SLA
34. According to the lecture, the provider's responsibility in an SLA is influenced by its business objectives and what other factor? → The technical realities of its infrastructure.
35. In the context of SLA metrics, what is 'agility' likely to measure? → The speed and flexibility with which the provider can respond to changes or new service requests.
36. A consumer's SLA has a 'data retention' clause stating data is kept for 5 years, but no 'data deletion' clause. Why is this a potential problem after the contract ends? → The SLA is incomplete, leaving ambiguity about whether, when, and how the consumer's data will be securely destroyed. hard
37. The lecture gives an example availability SLO of 'ninety nine point nine percent'. Which of the following is an example of a KPI that would be used to calculate this SLO? → The total number of service outages in a month.
38. What is the primary risk of a consumer's action if the SLA does not clearly define their responsibilities? → The consumer might accidentally violate the provider's terms and cause a service failure for themselves or others. hard
39. The lecture mentions that a provider having a 'third party certification' is a desirable SLA requirement. What does this provide the consumer? → Independent verification that the provider meets a certain standard of service.
40. What service does Amazon S3 provide, and what is its stated availability in the lecture? → Storage as a Service, 99.9%

    ⚠ The lecture says Amazon S3 (storage as a service) = 99 %. The 99.9 % figure belongs to Microsoft/Google/Salesforce, and EC2 = 99.5 %. B is the only option pairing S3 with storage, so it is the keyed answer, but if 99 % appears as an option, choose 99 %.
41. If a cloud provider fails and goes out of business, what SLA consideration becomes critical for the consumer? → The process for data retrieval and migration to another provider. hard

42. What is the primary purpose of the 'Service Definitions' section within a Service Level Agreement (SLA)? → To provide a complete and specific description of each service offered.
43. The 'Responsibilities' section of an SLA primarily serves to delineate what? → The duties of both the service provider and the consumer.
44. What is the main function of the 'Remedies & Penalties' section in an SLA? → To specify the 'payback mechanism' or consequences for unmet terms.
45. According to the lecture's SLA hierarchy, how are Service Level Objectives (SLOs) typically derived from Key Performance Indicators (KPIs)? → KPIs are aggregated or used in formulas to calculate high-level SLOs.
46. For which type of consumer is an 'Off-the-Shelf' (non-negotiable) SLA most suitable? → Individuals or small-scale operations where customization is not required.
47. Which section of an SLA defines the mechanisms for monitoring services and auditing performance? → Auditing & Monitoring
48. What is the purpose of 'Temporal Conditions' in an SLA? → To outline how the SLA might change over time, such as stricter requirements during peak hours.
49. In the context of data governance in cloud environments, what specific requirement must an SLA address regarding data after a VM is powered off or hardware is decommissioned? → Confirmation that data is properly erased (e.g., zeroing out memory/disk platters).
50. The lecture describes the Service Level Agreement (SLA) as the 'master document' governing the relationship between a service provider and a service consumer. What does this signify? → It is the foundational, legally binding contract that establishes trust and accountability.

Week 4 — must-remember

→ open the full Week 4 notes

- Security group = **in + out firewall rules** per VM; VM gets an internal IP, **associate an external (floating) IP** for outside access; **terminate** deletes & releases resources.
- Meghamala: 3 flavours (Regular/Large/Extra Large), 3 OS (Ubuntu/CentOS/Fedora); Meghadoop (Hadoop), Meghadata (Dropbox-like).
- Azure: Windows Azure + SQL Azure + AppFabric; **web role** (front end) vs **worker role** (background) via **storage queues**; ≥ 1 role of either type.
- Azure deploy: deployment user → **resource group** (logical container) → **App Service plan** (location/size/features) → web app (`*.azurewebsites.net`) → git push.
- GCP: **App Engine = PaaS**, **Compute Engine = IaaS**; Cloud Storage (object), Cloud SQL (MySQL), Datastore (NoSQL).
- Static site = **Cloud Storage bucket** + index.html + 404.html + public sharing; App Engine app = `app.yaml` (+ `threadsafe`) + `main.py` → `gcloud app deploy` .

Likely question types: which OpenStack service backs a dashboard item · resource group vs App Service plan · web vs worker role · GCP service ↔ model.

🎯 Your practice set — L16 OpenStack demo (Meghamala) (file w1): 50 answers in one line · ⚠️ 11

1. What is the name of the experimental, OpenStack-based cloud at IIT Kharagpur demonstrated in the lecture? → Meghamala

⚠ The lecture's Dropbox-like service is called Meghadata; 'Meghaduta' is this set's misspelling of it.

2. In the OpenStack dashboard, which component is responsible for managing disk images for virtual machines? → Glance
3. An administrator launches a new VM and assigns it only to the 'internal' network. A user then tries to connect to this VM from their laptop at home but fails. What is the most likely reason for the failure? → The VM has not been assigned a publicly accessible (external) IP address. hard
4. What is the primary purpose of a 'Security Group' in OpenStack as described in the lecture? → To act as a virtual firewall for VMs, defining traffic rules.
5. What was the name given to the virtual machine created during the live demonstration? → nptel
6. According to the demo, what is the role of the 'X2Go' software? → It is a GUI client used to connect to the desktop environment of a remote VM.
7. The lecture mentions several OpenStack services. Which service is responsible for providing persistent block storage to VMs? → Cinder
8. What is the initial status of a VM immediately after an administrator clicks the 'Launch' button in the OpenStack dashboard? → Build
9. What key difference between the 'Admin' view and the 'Project' (user) view was highlighted in the OpenStack dashboard demo? → The Admin view includes infrastructure-level details like Hypervisors.
10. What action in the OpenStack dashboard releases all resources (CPU, RAM, storage) associated with a virtual machine, making them available for others? → Terminate Instance
11. In the Meghamala cloud, three VM 'flavors' are offered. If a user needs to run a memory-intensive simulation and the 'IITKGP Regular' flavor is insufficient, what is the correct procedure? → Terminate the current VM and request a new one with a larger flavor, like 'IITKGP Large'. hard

    ⚠ Real OpenStack can also resize a VM to another flavour (with a reboot). The lecture's route, and the key's, is to ask for a new VM with a bigger flavour.
12. The Meghamala cloud provides 'ephemeral storage' with its VM flavors. What is the key characteristic of this storage type? → It is temporary and is deleted when the VM is terminated.
13. According to the presenter, which OpenStack component manages the networking aspects of the cloud, such as IP addressing? → Neutron
14. What is 'Meghadoop', as mentioned in the lecture? → A Hadoop cluster service running on top of the Meghamala infrastructure.
15. What is the primary objective of OpenStack as described at the beginning of the lecture? → To be an open-source cloud platform you can download and install on your own systems.
16. In the demo, what operating system image was chosen for the new 'nptel' virtual machine? → CentOS
17. The 'IITKGP Regular' flavor is specified to have 'two b c p u'. What does 'vCPU' stand for? → Virtual CPU
18. Why is it beneficial to use Security Groups instead of configuring firewalls on each VM individually? → A pre-defined Security Group can be applied to many VMs, ensuring consistent and automatic rule application. hard
19. What is the purpose of associating a 'new external IP address' with the VM in the demo? → To enable access to the VM from outside the cloud network.
20. Which OpenStack component is associated with compute services, including managing hypervisors and VM lifecycle? → Nova
21. What service is described as a 'personal cloud ah this is a kind of a ah draw box like thing'? → Meghaduta

    ⚠ The lecture calls it Meghadata (personal cloud, Dropbox-like). Option 'Meghaduta' is the set's misspelling, and it is the intended answer.
22. The admin dashboard for Meghamala shows three physical hypervisors. What information is displayed for each hypervisor? → The number of vCPUs, RAM, disk usage, and the number of running instances.
23. A user wants to save the current state of their VM's disk so they can create identical VMs from it later. Which OpenStack feature, related to Glance, would they use? → Snapshot/Image hard

    ⚠ Snapshots are not shown in the lecture; this is standard OpenStack (a snapshot becomes a new Glance image).

24. What was the final status of the 'nptel' instance after it was successfully provisioned and running? → Active
25. What is the primary difference between ephemeral storage and persistent storage (volumes) in OpenStack? → Ephemeral storage is deleted with the VM, while persistent storage exists independently.
26. If an administrator terminates a VM, what happens to the external (floating) IP address that was associated with it? → The IP address is returned to the pool of available IPs to be used by another VM. hard
    ⚠ Not shown in the lecture; standard OpenStack behaviour (a released floating IP goes back to the pool).
27. The lecture mentions that the admin user has two components in the dashboard: 'Project' and 'Admin'. What does this imply? → The admin has a dual role: as a regular user/tenant ('Project') and as a cloud administrator ('Admin').
28. What technology did the demo use to prove the VM had full internet access? → Pinging a website and then playing a YouTube video in a browser.
29. What is a 'flavor' in the context of OpenStack? → A template that defines the compute, memory, and storage resources for a VM.
30. The presenter states that one goal of the demo is to show how services like compute, storage, and image are 'realized'. What does 'realized' mean in this context? → To be implemented and made functional in a practical system. hard
31. What is the availability zone named in the 'Launch Instance' dialog during the demo? → nova
32. If the hypervisor dashboard shows all physical machines are at 99% vCPU utilization, what will likely happen when a user tries to launch a new VM? → The VM launch will fail due to insufficient resources. hard
    ⚠ Not in the lecture; standard OpenStack scheduler behaviour (no host fits the flavour → launch fails, ERROR state).
33. The lecture mentions that Meghamala offers three types of operating systems. Which two were explicitly named? → Ubuntu and Fedora
    ⚠ The lecture names three OS images: Ubuntu, CentOS and Fedora (the demo VM used CentOS). B is the only option taken from that list.
34. What does the 'Terminate Instance' action do that 'Shut Down Instance' does not? → It deletes the VM and de-allocates its disk and RAM.
35. Who is Rajesh, as introduced in the lecture? → The administrator of the Meghamala cloud.
36. What is the key takeaway the professor wants students to get from this demonstration? → That they can set up their own small-scale OpenStack cloud to gain practical experience.
37. The admin overview dashboard showed a 'd c p hour' metric. What does this likely represent? → A typo for 'vCPU hour', a unit of computing resource consumption. hard
38. During VM creation, the demonstrator selects a boot source. What are the two main types of boot sources in OpenStack? → Image and Volume hard
    ⚠ The demo only boots from an image; booting from a volume is general OpenStack knowledge.
39. What is the specification for the 'IITKGP regular' flavor mentioned in the slides? → 2 vCPU, 4 GB RAM, ephemeral storage
40. Why would an administrator use the 'View Log' option for a running instance? → To view the VM's console output for debugging boot or system issues.
    ⚠ The lecture only lists 'view log' among the actions; what it shows (console output) is general knowledge.
41. The Meghamala portal shows three physical machines, each with 48 vCPUs. If a user requests a VM with the 'IITKGP Regular' flavor (2 vCPUs), the OpenStack scheduler could place this VM on any of the three machines. This scheduling decision is a function of which component? → Nova hard
    ⚠ 'nova-scheduler' is not named in the lecture; the lecture only ties hypervisors and compute to Nova.
42. Which network type is selected by default for the new VM in the demo to provide it with a private IP address? → Internal
43. What is demonstrated to be running successfully inside the newly created VM, proving it's a fully functional machine? → A standard desktop environment with internet access.

44. If a user requires a persistent disk that survives the deletion of their VM, what should they request? → A Cinder volume (persistent storage).
45. The lecture describes Meghamala as an 'experimental ah cloud'. What does this imply about its purpose? → It is a platform for learning, research, and testing cloud concepts.
46. The OpenStack dashboard provides a graphical representation of resource usage. What is the benefit of this visualization? → It provides administrators with a quick, at-a-glance understanding of the cloud's overall health and capacity.
47. If a security group rule denies all incoming traffic except for port 22 (SSH), what will be the result? → The user can only connect to the VM using SSH; all other connections like HTTP will be blocked. hard
    ⚠ Port-level rules are not in the lecture; standard security-group behaviour (inbound allow-list, outbound open by default).
48. During the demo, when the administrator needs an external IP, he selects one from a pre-existing list. What does this imply? → The cloud has a pre-allocated pool of public IP addresses that can be assigned to VMs.
49. What is the relationship between OpenStack and Meghamala? → Meghamala is the specific implementation or instance of an OpenStack cloud at IIT Kharagpur.
50. In the admin view of hypervisors, one machine is listed as having '14 instances' running on it. What does 'instance' refer to in this context? → Virtual machines (VMs)

Week 5 — must-remember

→ open the full Week 5 notes

- Availability $= (1 - d/T) \times 100$ with T = **scheduled** hours; credit from the **band**; payable = bill − credit.
- CSCC 7 steps; NIST 5 actors (consumer, provider, carrier, broker, auditor); fault domain vs update domain vs availability set.
- HDFS blocks = ⌈size/block⌉ × replication.
- MapReduce average: mappers emit **(avg, count)**.
- Consolidate VMs; idle servers waste power; Wake-on-LAN; minimal VM images (38 → 8 s; 4 GB → 600 MB).
- RM aspects: provisioning, allocation, adaptation, mapping, modelling, estimation, discovery, brokering, scheduling.

Lecture-solved problems & variants

- **SLA P1 (L21).** Guarantee 99 %, 12 h/day × 30 days, outage 10.75 h. Violated? $T = 360$ h; lecture: $1 - 10.75/349.25 = 96.92\%$ (standard 97.01 %) < 99 % → **violated**. *Variant:* guarantee 99.5 %, 10 h/day × 30 days, outage 2 h? $1 - 2/300 = 99.33\% < 99.5\%$ → violated.
- **SLA P2 (L21).** $50/day, 30 days, 12 h/day, credit 10 % if 99–99.95 %, 25 % if < 99 %; outages total 9 h 25 min. Payable? $A = 97.385\%$ → 25 % of $1500 = $375 → **$1125**. *Variant:* $40/day, measured 99.2 %? Bill $1200, credit 10 % = $120 → **$1080**.
- **Allowed downtime.** 99.9 % over a 30-day, 24 h month? $0.001 \times 720 = 0.72$ h = **43.2 min**.

- **Economics P1 (L22).** $D = 50 \sin t \,/\, 20 \sin t$, $R = D + \delta D'$, $\delta = \pi/12$. Penalty? Lecture: $30\pi/12 \approx$ **7.86** (strict $\int |D - R| = 70\pi/12 \approx 18.33$).
- **Economics P2 (L22).** $D = 50(1 + e^{-t})$, $P = 100$, $B = 120$, $C = 200$, $T = 100$? $B_T = 1.2$ M, $C_T \approx 1.01$ M, ratio 0.84 → **cloud cheaper**.
- **Economics P3 (L22).** In-house ₹6 lakh/3 yr, 12 cores, 40 %, +₹22 power, +₹6 mgmt; cloud ₹42/h, 8 cores, 80 %, +₹1. Core-hour & ratio? ₹1.90 vs ₹5.25 per core-h; totals 85.075 vs 53.5 → **1.59**; in-house wins only at ≈ 90 % efficiency. *Variant:* in-house efficiency 60 %? $22.83/0.6 + 28 = 66.05$ vs 53.5 → cloud still cheaper.
- **HDFS (L23).** 64 MB blocks, ×3; files 64 KB, 65 MB, 127 MB? 1 + 2 + 2 = 5 blocks → **15** stored. *Variant:* 128 MB blocks, ×3; files 100 MB, 130 MB, 300 MB? 1 + 2 + 3 = 6 → **18**.
- **MapReduce average (L23).** {10, 20}, {30, 40}, {50}? (15·2 + 35·2 + 50·1)/5 = **30**, not 33.3.
- **Consolidation (L24).** Idle 105 W, full (8 VMs) 170 W, 2 VMs 138 W; 4 nodes, 8 VMs? Spread 552 W; packed 485 W; packed + 3 off **170 W**.

From your L21 practice set (file w2): same template, new values

- **Allowed downtime at 99 % of 360 h** (Q6)? $0.01 \times 360 = 3.6$ h, so the 10.75 h outage violates the SLA.
- **Problem 2 with only 3 h of downtime** (Q15)? $357/360 = 99.17\%$ → between 99 % and 99.95 % → **10 % credit**.
- **Exactly 99.00 %** (Q33)? The band is "≥ 99 % and < 99.95 %" → **10 %**: the boundary belongs to the upper band.
- **99.2 % on the $1500 bill** (Q46)? 10 % credit = $150 → **$1350**.
- **99.5 % over 720 h, in minutes** (Q32)? $0.005 \times 720 = 3.6$ h = **216 min**.
- **99.8 % over a 30-day month** (Q42)? $0.002 \times 720 = 1.44$ h (≈ 86 min). *Variants:* 99.95 % over 720 h? **21.6 min**. · Exactly 99.95 % in Problem 2? Not below 99.95 % → **no credit**. · 98.9 % on $60/day × 30 days? Bill $1800, 25 % credit = $450 → **$1350**.

🎯 Your practice set — L21 SLA tutorial (file w2): 50 answers in one line · ⚠️ 7

1. What is the primary purpose of a Service Level Agreement (SLA) in cloud computing? → To define a formal basis for the performance and availability of a service.
2. According to the lecture, why is there no single standard SLA across all cloud services? → Different services, providers, and consumer scales make a single standard impractical.
3. The lecture distinguishes between two types of agreements within an SLA. What are they? → Metric-based agreements and policy-driven agreements.
4. In the first problem presented, what was the guaranteed service availability percentage? → 99%
5. In the first problem, an application runs 12 hours a day for a 30-day month. An outage of 10.75 hours is observed. What is the total time the application was supposed to run? → 360 hours
6. Using the numbers from the first problem, if the guaranteed availability was 99%, how many hours of downtime would be the maximum allowed in the 360-hour period to meet the SLA? → 3.6 hours
7. Based on the lecturer's calculation in the first problem, what was the final calculated service availability? → 96.92%

   ⚠️ The lecture divides 10.75 by the uptime (349.25 h) and gets 96.92 %. The standard formula (360 – 10.75)/360 gives 97.01 %, which is option B here. Keys follow the lecture, so pick 96.92 %; the verdict

(violated) is the same either way.

8. The lecture discusses that availability requirements can vary. What is an example of a scenario where availability requirements might be tiered? → A banking application that requires higher availability during business hours than overnight.
9. In the second problem, what is the consequence for the provider if the availability guarantee is not met? → Service credits are awarded to the consumer.
10. In the second problem, what were the two tiers of service credits offered for failing to meet the 99.95% availability guarantee? → 10% and 25%
11. What was the total initial cost of the cloud service for the 30-day period in the second problem? → $1,500
12. In the second problem, five outages were recorded. According to the lecture's calculation, what was the total service downtime? → 9 hours 25 minutes

    ⚠ Only four outages are listed, and they add up to 9 h 40 min (option B). The lecture says 9 h 25 min and the key follows it. The 25 % credit and $1,125 payable are the same either way.
13. Based on the calculated availability of 97.385% in the second problem, which service credit tier was applied? → The 25% credit tier.

    ⚠ 97.385 % uses the lecture's 9 h 25 min. With the true sum (9 h 40 min) it is 97.31 %, still below 99 %, so the credit is still 25 %.
14. What was the final effective cost payable by the consumer in the second problem after the service credit was applied? → $1,125
15. If the total downtime in the second problem had been exactly 3 hours, what would the service availability have been, and which credit would apply? → 99.17%, 10% credit hard
16. In the context of commercial SLAs as discussed for Microsoft Azure, what does a 'Fault Domain' represent? → A collection of servers that share common resources like power and network.
17. What is the purpose of deploying virtual machines in an 'Availability Set' across different fault and update domains? → To avoid a single point of failure and improve uptime.
18. According to the lecture, what HTTP status code series typically indicates a server-side failure or 'error code' relevant to SLA downtime calculations? → 5xx (Server Error)
19. The Cloud Standards Customer Council provides a 7-step guide for evaluating SLAs. What is the first step? → Identify the cloud actors.
20. Which of the following is NOT listed as a cloud actor according to the NIST architecture mentioned in the lecture? → Regulator
21. A startup is considering a cloud provider but is worried about being locked in. Which part of the SLA best practices should they pay close attention to? → The disaster recovery plan and exit clause. hard
22. Evaluating data policies, services not covered, and software licensing falls under which of the seven best practice steps? → Step 2: Evaluate business-level policies
23. Why is it important to understand the service model (IaaS, PaaS, SaaS) when evaluating an SLA? → Because the complexity of control variables given to the consumer differs between models. hard
24. A media company needs to stream video with minimal buffering. Besides availability, which metric category from Step 4 of the best practices would be most important for their SLA? → Response metrics
25. What is a key consideration under Step 5, 'Consider key security parameters'? → Legal and regulatory requirements for data.
26. A rapidly growing e-commerce site needs its cloud environment to scale automatically during flash sales. This relates to which aspect of the 'Service Management Requirements' (Step 6)? → How rapid provisioning and demand flexibility should be managed. hard
27. What does an 'Update Domain' refer to in the context of the Azure example? → A set of instances to which platform updates are concurrently applied.
28. The lecture states that the complexity of SLA control variables depends on the service and cloud type. Which combination would likely give the consumer the MOST control variables to manage and monitor? → IaaS on a private cloud hard
29. What does the term 'remedy' refer to in Step 7 of the SLA evaluation best practices? → The compensation provided for SLA failures, such as service credits.

30. Which of the following is an example of a policy-driven agreement in an SLA? → A rule that all customer data must reside within the European Union.
31. In the context of the lecture, what is an SLO (Service Level Objective)? → An objectively measurable condition for a service, which is a building block for an SLA.
32. A provider's SLA promises 99.5% availability over a 30-day month (720 hours). What is the maximum total downtime allowed in minutes to meet this guarantee? → 216 minutes
33. In problem 2, if the calculated availability was exactly 99.00%, which service credit would be applied? → 10% hard
34. What is the function of a 'Management Portal' as described in the lecture? → A web interface for the consumer to manage their services.
35. According to the lecture, what is an 'incident' in the context of an SLA? → Any single or set of events that result in downtime.
36. A hospital needs to store patient records in the cloud. Which two SLA evaluation steps are most critical for ensuring compliance and patient privacy? → Step 2 (Business Policies) and Step 5 (Security) hard
37. Why would a consumer want to meter their service usage, as mentioned in Step 6 of the best practices? → To verify the provider's billing and manage costs.
38. The lecture mentions 'sub contract services' as a consideration in business-level policies. What risk does this address? → The risk that the cloud provider uses a third party for part of its service, which may have different security standards. hard
39. In the first problem, the provider's guarantee was violated. What is the immediate, direct outcome based on the problem's description? → The lecturer concludes that the provider has violated the SLA.
40. What is the role of a 'broker' in the NIST cloud actor model? → An entity that manages the use, performance, and delivery of cloud services and negotiates relationships between providers and consumers. hard

    ⚠ The lecture only lists the five NIST actors; the broker definition comes from NIST SP 500-292.
41. What distinguishes a 'web role' from a 'worker role' in the context of the Azure definitions provided? → Web roles run web applications with a UI (like a website), while worker roles typically perform background processing.

    ⚠ The lecture only lists the Azure terms; web role = front end, worker role = background processing (see Week 4 Azure notes).
42. If a cloud provider guarantees 99.8% monthly uptime, how much downtime per month (30 days) is implicitly accepted by the consumer? → Approximately 1.44 hours
43. The lecture mentions that a 'disaster recovery plan' should be part of the failure management strategy. What is the primary goal of such a plan? → To define the procedures for restoring service after a major outage or disaster.
44. A consumer signs an SLA for a VM with 99.9% uptime. The provider performs scheduled maintenance, taking the VM offline for 2 hours. This maintenance was announced weeks in advance. Does this typically count against the uptime guarantee? → No, scheduled maintenance is usually excluded from uptime calculations in the SLA's fine print. hard

    ⚠ Not in the lecture; this is common industry SLA practice (announced maintenance is usually excluded).
45. What is meant by 'External Connectivity' in the list of SLA definitions? → The consumer's ability to connect to the service from a public IP address.
46. In problem 2, what would the effective cost be if the calculated availability was 99.2%? → $1,350 hard
47. The lecture emphasizes the importance of an 'exit clause'. What is the most significant challenge a consumer faces when exiting an SLA without a clear exit clause? → Migrating their data and applications out of the provider's environment. hard
48. Which of the following is the BEST example of a 'Tenant' as defined in the lecture? → A customer's entire deployment, consisting of one or more roles in a single package.
49. Why is it important for an SLA to define 'Success Codes' (e.g., HTTP 2xx)? → To provide a baseline for what constitutes a successful, non-downtime transaction.
50. A company is using a cloud provider's IaaS offering. They are responsible for patching the operating system of their VMs. If a VM is down because they failed to apply a critical patch, would this typically

count against the provider's SLA uptime guarantee? → No, the provider is only responsible for the infrastructure; failures caused by the consumer's software or configuration are excluded. hard

⚠️ Not in the lecture; this is the shared-responsibility idea (provider covers infrastructure, consumer covers OS and apps in IaaS).

Week 6 — must-remember

→ open the full Week 6 notes

- Interruption→availability · interception→confidentiality · modification→integrity · fabrication→authenticity.
- Disclosure (snooping) · deception (spoofing, repudiation, denial of receipt) · disruption · usurpation (delay, DoS).
- Passive (release of contents, **traffic analysis**, hard to detect) vs active (masquerade, **replay**, modification, DoS).
- Secure $R \subseteq Q$ · precise $R = Q$ · broad otherwise.
- Cloud-specific: **co-tenancy**, lack of control, reputation fate-sharing; IDC 2008: security top concern.
- **Gartner 7**: privileged user access, compliance & audit, data location, data segregation, recovery, investigative support, long-term viability. **RPO** = data loss; **RTO** = recovery time.
- EC2 study: **placement → extraction**; co-residency = same **Dom0**, small RTT, IPs within **7**; ~**40 %** success; probes on ports 80/443.
- Collaborative SaaS: SelCSP, risk-based access (fuzzy), **IDRM** (NP-hard → greedy), conflicts (**cyclic inheritance, SoD**). Broker: fuzzy (QoS + trust → suitability); migrate if SLA satisfaction < threshold.

Likely question types: attack ↔ property · threat class · passive vs active · Gartner list "which is NOT" · RPO vs RTO · co-residency indicators.

🎯 Your practice set — L26 Security I (file w3): 50 answers in one line · ⚠️ 1

1. According to the lecture, what is a primary reason security is a major concern in cloud computing? → Users are relying on a third-party service provider.
2. The lecture identifies one of the major hindrances to cloud adoption as being more significant than technology itself. What is this hindrance? → Concerns about security, policies, and data handling.
3. What are the three basic components of the security model often referred to as the 'CIA triad'? → Confidentiality, Integrity, Availability
4. A Distributed Denial of Service (DDoS) attack primarily compromises which component of the CIA triad? → Availability
5. An attacker intercepts a message between a source and destination but does not alter it, only listens. Which type of attack is this, and which security principle does it violate? → Interception, violating Confidentiality
6. In the context of security attacks, what is 'Fabrication'? → An unauthorized party inserts counterfeit objects into the system.
7. A 'man-in-the-middle' attack where an intruder intercepts a bank transfer instruction, changes the destination account number, and forwards the altered message is an example of which type of attack? → Modification hard

8. What is the fundamental difference between security 'Policies' and 'Mechanisms' as described in the lecture? → Policies state what is allowed, while mechanisms enforce those statements.
9. The lecture mentions that composing multiple security policies can create vulnerabilities. Why does this happen? → Conflicts or discrepancies between policies can be exploited. hard
10. Which of the following is NOT one of the three major security goals discussed in the lecture? → Retaliation
11. What is the primary objective of the 'Recovery' security goal? → To stop an attack, assess damage, and continue to function correctly.
12. The lecture describes a system as 'fully secure' under what condition? → When the set of reachable states is a subset of the set of secured states. hard
13. What does the concept of 'Assurance' in security involve? → A process involving specification, design, and implementation to ensure security functionalities.
14. In a lab environment where no data is stored on local machines, the lecture suggests that it might be cheaper to recover from an attack (by reinstalling an image) than to prevent it. This is an example of what kind of analysis? → Cost-benefit analysis
15. A threat of 'usurpation' involves which of the following attack types? → Modification, spoofing, delay, or denial of service hard
16. Which of the following best describes a 'passive attack'? → An attack that obtains information being transmitted without altering system resources.
17. An attacker observing the volume and timing of network traffic between two parties, without reading the content, is performing what kind of analysis? → Traffic analysis
18. An attack where an entity pretends to be a different, legitimate entity is known as: → Masquerade
19. What is a 'Replay' attack? → An attacker passively captures a data unit and retransmits it later to produce an unauthorized effect.
20. The security service that ensures a sender cannot falsely deny having sent a message is called: → Non-repudiation
21. What is the role of 'Authorization' in security services? → To determine if a user is allowed to perform a certain operation.
22. What is the first step in the network security lifecycle presented in the lecture? → Determination of network security policies
23. In network security, what is the purpose of 'reconnaissance'? → To learn about the network, such as IP addresses and key servers.
24. What is the purpose of a 'Honeypot' or 'Honeynet' in a network security setup? → To be a deliberately vulnerable system to attract and study attackers. hard
    ⚠ Honeypots/honeynets do not appear in the L26 transcript; this is the standard definition.
25. The lecture mentions using tools like 'Nmap' for what purpose? → Vulnerability scanning.
26. What is 'Penetration Testing'? → A safe, authorized attempt to exploit a computer system's vulnerabilities.
27. The process of 'post-attack investigation' is also known as: → Forensics
28. The lecture mentions the National Vulnerability Database (NVD). What is its purpose? → A repository that lists known security vulnerabilities.
29. Why is it important for vulnerability scanners to be updatable? → To incorporate information about newly discovered vulnerabilities and attack signatures.
30. Which class of threat is associated with 'snooping'? → Disclosure hard
31. A threat of 'deception' includes which of the following? → Modification, spoofing, repudiation of origin, denial of receipt hard
32. A company's central IT department has a policy that its implementation is distributed across various switches and routers in different departments. According to the lecture, what is a major challenge in this scenario? → Guaranteeing that the distributed implementation conforms to the central policy. hard
33. Which security role is responsible for protecting against data alteration or corruption? → Integrity
34. The lecture states, 'policies unambiguously partition system states.' What does this mean? → Policies should clearly define whether a given system state is secure or not, with no ambiguity. hard

35. According to the lecture, human issues like 'organizational problems or people problems' are part of the overall security landscape. This implies that: → The human element is often a factor in security vulnerabilities and must be considered.
36. If an active attacker prevents the normal use of communication facilities, which specific type of attack are they committing? → Denial of Service
37. Which of the following attack types are generally considered 'difficult to detect' because they 'don't do direct harm'? → Passive attacks
38. The integrated security model shows 'operational issues' can feed back into several stages. What is one way operational issues act as a feedback loop? → They can become a new type of threat. hard
39. When discussing the trust and assumptions in security, what example does the lecturer provide from day-to-day life? → Trusting that a security person guarding a premise can be trusted.
40. Why is 'Access Control' considered a crucial security service? → It prevents the misuse of resources by unauthorized or improperly authorized users.
41. The lecture gives an example of critical data that requires serious security consideration. Which of the following was NOT mentioned in that context? → Social media posts
42. If a system's set of reachable states and its set of secured states perfectly overlap, the security is described as: → Precise hard
43. The lecture mentions several specific attack methods, such as 'phishing' and 'buffer overflow'. These are categorized under what general heading? → Series of Attacks
44. A network design that includes a 'Demilitarized Zone' (DMZ) with two firewalls is an example of what? → Implementing a security policy
45. Which of the following is an example of a 'human issue' in security? → An employee tricked by a phishing email into revealing their password.
46. What is the key characteristic of an 'active attack' that distinguishes it from a passive one? → It involves some modification of the data stream or creation of a false stream.
47. The term 'integrity' in information security can refer to data integrity and what other type of integrity mentioned in the lecture? → Origin or source integrity hard
48. What is a key activity during the 'post-attack investigation' phase of network security? → Retaining the chain of evidence for legal purposes.
49. According to the lecture, the concepts of security threats, policies, and attack types discussed are: → Generic concepts that are also relevant to cloud computing.
50. A security policy that is enforced by a firewall's rule set is an example of a policy being enacted by a: → Mechanism

---

Week 7 — must-remember

→ open the full Week 7 notes

- MCC = cloud + mobile + wireless; profiler (time/power/traffic) → solver (placement) → synchronizer (merge).
- **MAUI** (annotate remoteable methods; native stays local; RPC) vs **COMET** (binaries only; DSM thread migration).
- **Cloudlet** = trusted, resource-rich, nearby, single-hop, middle tier.
- $E_{saved} = \frac{C}{M}(P_c - P_i/F) - P_{tr}\frac{D}{B}$ — offload when compute-heavy, data-light.
- Fog = **Cisco** term; enablers: virtualization, containers, SOA, **SDN (OpenFlow)**.
- Cloud vs fog: latency high/low, hops multi/one, location awareness no/yes, centralized/distributed, few/very many nodes, mobility limited/supported, leased line/wireless.

- Geospatial cloud: SDI, interoperability at data/service/security levels; Bankura road & water merges.

Lecture-based problem & variant

- **Offloading.** $C/M = 2$ s, $P_c = 1$ W, $P_i = 0.2$ W, $F = 20$, $P_{tr} = 1.5$ W, $D/B = 1$ s? $2(1 - 0.01) - 1.5 = $ **0.48 J saved** → offload. *Variant:* same but $D/B = 2$ s? $1.98 - 3.0 = -1.02$ J → don't offload.

🎯 Your practice set — L31 Mobile Cloud Computing I (file w4): 50 answers in one line

1. What is the primary definition of Mobile Cloud Computing (MCC) as presented in the lecture? → A combination of cloud computing, mobile computing, and wireless networks to bring rich computational resources to mobile users.
2. Which of the following is NOT mentioned as a primary motivation for the emergence of Mobile Cloud Computing? → The development of mobile devices that are more resourceful than desktop servers.
3. The lecture discusses the concept of offloading. In the context of MCC, what does 'offloading' primarily refer to? → Moving computationally intensive parts of an application from the mobile device to be executed on cloud servers.
4. What is a key trade-off or 'flip side' of computational offloading mentioned in the lecture? → Sending program state and data to the cloud consumes battery power, partially offsetting the power saved from not computing locally.
5. According to the MCC workflow described, what is the role of the 'Profiler' component? → To monitor an application's execution to collect data on time, power consumption, and network traffic.
6. In the context of the MCC architecture, what is the primary responsibility of the 'Solver'? → To select which parts of an application should run locally on the mobile device and which should be offloaded to the cloud.
7. A developer is implementing a complex image recognition feature in a mobile app using MCC. What is the role of the 'Synchronizer' component after the cloud has processed the image? → To collect the results of the split execution, combine them, and make the process transparent to the user. hard
8. Which of the following is presented as a major advantage of using MCC for mobile applications? → It extends battery life by offloading resource-hungry computations.
9. How does MCC improve the 'reliability and availability' of user data? → By storing data in the cloud, which reduces the chance of loss if the mobile device is damaged or lost.
10. The lecture mentions several real-world examples of technologies augmenting mobile devices with cloud computing. Which of the following was cited? → Amazon Silk browser and Apple Siri
11. What is considered a major bottleneck and challenge for MCC that is dependent on the intermediate wireless network? → Network latency and bandwidth
12. Which of the following is NOT listed as a key feature of MCC? → Requires developers to have specific knowledge of underlying network technologies.
13. The lecture describes 'dynamic runtime offloading' as being particularly challenging. What is one of the tricky issues involved? → Runtime application partitioning and continuous synchronization. hard
14. What is a major security challenge in MCC that affects consumer trust? → Protecting user privacy and data secrecy from adversaries.
15. In addition to data secrecy, what is a specific privacy concern for mobile users in MCC mentioned in the lecture? → The user's identity, mobility, and GPS footprints could be tracked.
16. What does the concept of 'Mobile Backend as a Service' (MBaaS) aim to provide for developers? → A way to connect mobile apps to cloud storage and processing, abstracting away backend complexity.
17. How can MCC lead to increased 'flexibility' in mobile application development, as discussed in the lecture? → By allowing developers to quickly revise backend algorithms on the cloud without updating the front-end app on the device. hard
18. What role does the wireless network provider play in the MCC ecosystem? → They provide the essential connectivity and backbone between the mobile device and the internet/cloud.

19. The lecture mentions 'Context-Aware Mobile Cloud Services'. What is the goal of these services? → To utilize local context (like network status, user preferences) to improve the quality of service.
20. Which of the following is an example of an application that would greatly benefit from MCC, as described in the lecture? → A real-time disaster management system collecting and processing sensor data from multiple mobile sources. hard
21. What does 'multi-tenancy' in the context of MCC advantages refer to? → A service provider sharing resources and costs to support a variety of applications and a large number of users.
22. Why is 'pricing' considered a tricky issue and challenge in MCC? → Because multiple parties are involved (cloud provider, mobile service provider), and a clear business model is needed. hard
23. The lecture discusses a 'win-win situation' for MCC. Who are the parties that benefit? → Mobile users, network operators, and cloud providers.
24. What is the key difference between how a resource-hungry application runs with MCC versus without it? → With MCC, parts of the application can be partitioned to run on powerful cloud servers instead of the limited device.
25. What is a proposed approach mentioned in the lecture to handle security threats like malicious code on a mobile device using MCC? → Moving threat detection capabilities to the cloud, where a remote server performs the analysis. hard
26. The lecture describes an MCC architecture with three main components. What are they? → Mobile Device, Wireless Network, and Computational Cloud
27. Why is 'ease of integration' considered an advantage of MCC? → Multiple services from different providers can be integrated through the cloud to meet user demands.
28. A challenge for MCC is 'service convergence'. What does this concept entail? → A scheme where a mobile user can utilize multiple different clouds in a unified and seamless fashion. hard
29. If a mobile game developer uses MCC to render complex 3D graphics on a cloud server, what is the primary benefit for the user's device? → The device will consume less battery and generate less heat.
30. What is the primary function of the 'Mobile Load Manager' in the MCC workflow diagram? → To send data to the cloud and manage the offloaded task from the device's side.
31. Which of these is a direct consequence of high network latency in an MCC application? → Execution delays, making the application feel unresponsive.
32. The lecture states that in an MCC scenario, a user's request is delivered to the cloud through the internet. Who processes this request in the cloud? → The Cloud Controller.
33. What is a key requirement for APIs in the MCC environment to encourage adoption by developers? → They should be simple and not require deep knowledge of network technologies.
34. The proliferation of what two factors, working together, created the 'natural need' for MCC? → The increasing resource-hungriness of apps and the limited resources of mobile devices.
35. Which cloud computing characteristic is most relevant to MCC's ability to handle a massive number of users and fluctuating demand? → Rapid elasticity and scalability
36. How does the use of MCC impact the reusability of algorithms and applications? → It increases reusability, as a single computing service on the cloud can support numerous different apps and devices. hard
37. What is the primary goal of the 'amalgamation' of mobile and cloud technologies? → To augment resource-constrained mobile devices by leveraging the power of the cloud.
38. A user is running a mobile augmented reality (AR) app that overlays information on a live camera feed. This requires intense computation. Why is MCC a good fit for this application? → The AR app can offload the object recognition and rendering tasks to the cloud, making it run smoothly on the phone. hard
39. What is a significant challenge related to 'standard interfaces' in MCC? → The need to create a standard interface that allows different applications and devices to work with the cloud services.
40. According to the lecture, the feeling that an MCC-powered application is running seamlessly on the device itself is crucial. What component is responsible for making the 'split execution' transparent to the user? → The Synchronizer
41. The lecture discusses security issues in MCC as having two major categories. What are they? → Security of mobile users and security of data in the cloud.

42. Why does the 'detachment' of processing from the device to the cloud offer greater speed, according to the lecture? → Because cloud servers are vastly more resourceful and can perform computations much faster than a mobile device.
43. In the MCC architecture, base stations are mentioned. What is their function? → To establish and control the connection between the mobile network and the mobile devices.
44. A user is concerned that their new fitness app, which uses MCC for route planning, is tracking their location even when the app is closed. This scenario relates to which specific MCC challenge? → Security and Privacy of the mobile user hard
45. Which statement best summarizes the relationship between a mobile device's resources and MCC? → MCC augments the resources of a mobile device, regardless of how resourceful it is, because the cloud's resources are comparatively vast.
46. What does the term 'resource hungry applications' refer to in the context of this lecture? → Applications that require significant computational power, memory, or energy to run.
47. The lecture mentions 'network access management' as a challenge. What is its goal beyond just improving link performance? → To optimize bandwidth usage.
48. What is the end goal of the entire MCC process, from the user's perspective? → To be able to run enriched, more resourceful applications on their mobile device with a seamless experience.
49. What is a major factor that makes the 'dynamic application profiling' component of runtime offloading so difficult? → It has to analyze the application's behavior in real-time on the resource-constrained mobile device itself. hard
50. The lecture states that an advantage of MCC is 'dynamic provisioning'. What does this mean? → The ability to provision or allocate cloud resources dynamically based on the application's real-time needs.

Week 8 — must-remember

→ open the full Week 8 notes

- Docker (Mar 2013): develop–ship–run anywhere; **image** = package, **container** = running instance; Engine / **Hub** (registry) / Compose.
- Containers share the **host kernel**, tens of MB, near-instant start.
- DC energy: **cooling 45 %, IT 40 %, power distribution 15 %**; idle server ≈ **66 %** of peak; switch: chassis 36 %, line cards 43 %, ports 11 %.
- **PUE = total facility / IT** (ideal 1.0).
- Sensor cloud: sensors = interface, clusters = backbone, Internet = medium; virtual sensors **one-to-many, many-to-one, many-to-many, derived**.
- IoT: unique IDs, no human interaction; things → gateway → cloud; **MQTT/CoAP/HTTP** (IoT), **MQTT/AMQP/HTTP** (cloud).

Problem & variant

- **PUE.** Total 1.5 MW, IT 600 kW? **2.5** (DCiE 40 %). *Variant:* total 1.2 MW, IT 1.0 MW? **1.2**.

🎯 Your practice set — L36 Introduction to Docker (file w5): 50 answers in one line

1. What is the primary motivation for using container technology like Docker, as described in the lecture? → To solve the problem of recompilation and reconfiguration when moving software between environments.

2. According to the lecture, what is the relationship between cloud computing and container technology? → Cloud computing can act as a platform to host containers, enhancing the cloud's capabilities.
3. What is the core philosophy of Docker, summarized by the keywords mentioned in the lecture? → Develop, Ship, Run, Anywhere
4. In the shipping analogy used in the lecture, what does the standardized shipping container represent? → The Docker container that packages an application and its dependencies.
5. What is the fundamental architectural difference between Docker containers and traditional virtual machines (VMs)? → Containers share the host operating system's kernel, while each VM runs its own complete guest OS.
6. What is a Docker 'Image' as defined in the lecture? → A lightweight, standalone, executable package that includes everything needed to run a piece of software.
7. What is a Docker 'Container' in relation to an 'Image'? → A container is a runtime instance of an image.
8. Which of the following is a key advantage of Docker containers being 'lightweight'? → They are easily scalable and start almost instantaneously.
9. What is the function of the 'Docker Engine'? → It is used for building Docker images and creating Docker containers.
10. What is 'Docker Hub' according to the lecture? → A registry service used to host and share various Docker images.
11. A development team is struggling because an application works on one developer's laptop but fails on another's. How does Docker address this 'works on my machine' problem? → By packaging the application and all its dependencies into a container, ensuring a consistent environment. hard
12. What is the purpose of 'Docker Compose'? → To define and run applications that use multiple Docker containers.
13. How do containers achieve better performance compared to virtual machines? → By running applications natively on the host machine's kernel.
14. What is a 'Dockerfile'? → A build script used to create a Docker image automatically.
15. Why is application portability a major challenge in modern computing, leading to the rise of containers? → Because applications often need to be recompiled or reconfigured to run on different devices and cloud platforms.
16. An enterprise wants to ship new features faster and more securely. How does Docker help them achieve this, according to the lecture? → By building agile software delivery pipelines. hard
17. In the context of Docker, what does 'isolation' mean? → The container runs in a process that is separate from the host and other containers by default.
18. Which of the following components is NOT part of the traditional virtualization stack but IS central to the Docker architecture? → Docker Engine
19. How can Docker run on an IaaS (Infrastructure as a Service) cloud? → By running the Docker Engine and containers on top of a virtual machine provided by the IaaS cloud. hard
20. What problem from the perspective of a system operator does Docker help solve? → Getting better compute density by running and managing apps side-by-side in isolated containers. hard
21. What is the key difference in resource consumption between a VM and a container? → Containers are lightweight (tens of MBs), while VMs are resource-intensive (GBs) due to the full guest OS.
22. The lecture mentions several Docker commands like `ps`, `start`, `stop`, and `rm`. What is a notable characteristic of many of these commands? → They are similar to common Linux commands.
23. In the shipping analogy, what does the 'infrastructure' (trucks, cranes, ships) correspond to in the software world? → The various computing environments (laptops, servers, cloud platforms).
24. Which component in the Docker architecture is analogous to a hypervisor in traditional virtualization, but is much more lightweight? → The Docker Engine
25. Why does the lecture describe Docker as a 'buzz word for modern world development especially in the face of agile based projects'? → Because it enables rapid and consistent deployment, which is a core tenet of agile development. hard
26. What is the function of the `docker pull` command mentioned in the lecture? → To download an image from a repository.

27. If a developer wants to create a new image based on the current state of a running container, which command would they use? → docker commit
28. How does Docker facilitate collaboration between development, QA, and operations teams? → By ensuring all teams work with an identical, containerized application environment.
29. According to the lecture, a container runs its processes in isolation. Under what condition can it access host files and ports? → Only if it is explicitly configured to do so. hard
30. What kind of 'footprint' does Docker provide for the operating system within a container? → A smaller footprint of the operating system.
31. Which of the following is NOT an environment where Docker containers can be deployed, according to the lecture? → A system without an underlying host operating system.
32. What is the primary benefit of the 'configuration entanglement' being absent in containerized applications? → The application can run anywhere without conflicting with host system dependencies. hard
33. The lecture mentions a variety of applications like static websites, background workers, and databases. What challenge related to this variety does Docker address? → It provides a standardized way to package and run any of these different applications on any system.
34. What does the 'application level virtualization' enabled by Docker allow a single host to do? → Run several specialized applications for better resource utilization.
35. What happens when a small modification is made to an application running in a Docker container, compared to a VM? → Docker allows you to save only the differences between the old and new container versions. hard
36. What is the role of the Docker client-server model in its overall workflow? → The client (user) searches for an image, pulls it, and the server (Docker Engine) runs it as a container.
37. The lecture describes Docker as an 'open source container'. What does 'open source' imply? → Its source code is publicly available for anyone to view, use, or modify.
38. How does Docker improve resource utilization on a physical server compared to traditional virtualization? → By eliminating the overhead of running a separate Guest OS for each application.
39. Which Docker command would you use to see a list of all containers, including those that are not currently running? → ps -a
40. If you want to create a Docker image that can be versioned in a system like Git alongside your source code, what would you create? → A Dockerfile.
41. A team is developing a microservices application with a web front-end, a user database, and an analytics service. Which Docker tool is specifically designed to define and manage this multi-container setup? → Docker Compose hard
42. What is the primary content of a container image, distinguishing it from a VM image? → Only the application executable and its specific package dependencies.
43. The lecture mentions that Docker Hub can automatically build images. What is this automatic build process typically based on? → Dockerfiles located in a GitHub repository. hard
44. What was the initial release date of Docker mentioned in the lecture? → March 2013
45. When comparing virtualization architectures, where does the Guest OS reside? → On top of the hypervisor, inside a virtual machine.
46. Why does the lecture assert that containers offer 'better compute density' for operators? → Because their low resource footprint allows more isolated applications to run on a single host. hard
47. Which problem is NOT a primary focus of the Docker platform? → Hardware abstraction and emulation.
48. According to the lecture, a developer can run a container created on their desktop on a cloud platform like Azure or AWS. What does this demonstrate? → The core principle of 'develop, ship, run anywhere'.
49. In the lecture's comparison of a modified app (Container A vs. Container A'), what is the key efficiency of Docker? → It allows saving only the layer representing the changes, not a full new image. hard
50. What is the ultimate goal of the 'matrix' of applications and systems shown in the lecture? → To have 'all ticks on this matrix', meaning any application can run on any system with minimal effort.

Week 9 — must-remember

→ open the full Week 9 notes

- Health case: levels **cloud (0) – ISP (1) – area gateway (2) – mobile (3) – sensors (4)**; fog placement: filtering/processing/event handling at **area gateway**, **confirmatory in cloud**; fog → low network use, low cloud cost, stable latency. CloudSim (Melbourne); **iFogSim (IIT KGP + Melbourne)**.
- Fog environment: client, fog, cloud; fog forwards to cloud if short of resources.
- RM taxonomy: **Architecture** (data flow, control, tenancy) · **Infrastructure** (hardware, system software, middleware) · **Algorithms** (discovery, benchmarking, load balancing, placement).
- Placement constraints: resource / network / application (locality, deadlines). Control: centralized (solver, graph matching) vs distributed (blockchain, game theory, GA).
- Federation: **bursting & broker = loose**, **aggregated = partial**, **multi-tier = tight**; cross-site VM migration needs tight coupling.
- Migration reasons: load balancing, maintenance, power, QoS, fault tolerance. **Cold** vs **live**; **pre-copy** (memory first) vs **post-copy** (CPU first).

Likely question types: module placement in fog vs cloud · taxonomy pillar matching · federation architecture ↔ coupling · pre-copy termination conditions.

Week 10 — must-remember

→ open the full Week 10 notes

- $r = PD/R$; $T_{mig} = \frac{V_m}{R}\frac{1-r^{n+1}}{1-r} + T_{res}$; $T_{down} = r^n \frac{V_m}{R} + T_{res}$; cold: $T_{down} = T_{mig} = V_m/R$.
- KVM, Xen, VMware use **pre-copy**. Parallel multi-VM: each gets R/m; serial: others suspended at first VM's stop-copy.
- Containers: OS-level, relaxed isolation, shared kernel; benefits = separation of responsibilities, portability, isolation.
- **Kubernetes** (Google, "helmsman"): cluster → worker nodes → **Pods**; control plane; not monolithic.
- Docker: client + **daemon** (REST/HTTP), private registry vs Docker Hub; `build`, `run`, `commit`, `tag`, `save` / `load`.

Problems & variants

- **Cold migration.** 2048 MB at 128 MB/s? $T_{mig} = T_{down} = 16$ s.
- **Pre-copy.** $V_m = 1000$ MB, $R = 100$ MB/s, $r = 0.5$, $n = 2$, $T_{res} = 0.5$ s? $T_{down} = 0.25 \times 10 + 0.5 = 3.0$ s; $T_{mig} = 10 \times 1.75 + 0.5 = 18.0$ s. *Variant:* $V_m = 1024$ MB, $R = 100$ MB/s, $r = 0.2$, $n = 3$, $T_{res} = 0.1$ s? $T_{mig} \approx 12.88$ s; $T_{down} \approx 0.18$ s.

Week 11 — must-remember

→ open the full Week 11 notes

- Dew = **independence + collaboration**; Dropbox/Drive = storage-in-dew; App Store/Play = software-in-dew; GitHub = platform-in-dew; iCloud = IaD; DBiD = two DB copies (reliability).
- Serverless: event-driven, granular, provider manages servers, pay per use; **FaaS** (your stateless functions; Lambda ~15 min) vs **BaaS** (Firebase-style back ends).
- Drawbacks: async calls, function chaining cost, shared code, image size (~50 MB) & warm-up, too many libraries/tech/functions.
- Lambda: **event source, runtime, layers, log streams**; Google: **trigger = event + function**; Azure: **code + function.json**, Consumption plan.
- Sustainable cloud: minimize energy & carbon **while keeping reliability**; cooling manager, power manager (**ATS, PDU**, grid for deadline jobs), remote CDCs; **DVFS** switching delays; component energy **processor 45 %, cooling 20 %**.

Likely question types: dew category ↔ example · FaaS vs BaaS · serverless drawback "which is NOT" · Lambda term ↔ meaning · sustainable-model component ↔ role.

Week 12 — must-remember

→ open the full Week 12 notes

- 5G: ~10 Gbps, ~1 ms; **eMBB** (AR/VR) · **URLLC** (autonomous vehicles, telesurgery, smart grid) · **mMTC** (massive IoT). 5G–cloud: cloud must meet 5G (edge/MEC/fog); 5G cloudified via **NFV/SDN**.
- CPS: computing + physical with feedback loops; **Helen Gill, NSF, ~2006**; intersection not union; CPCC = rapidly build/modify/provision CPS from cloud sensor/processing/control/data services.
- Spatial: geospatial = location + attributes + time; **Tobler's first law**; OGC; trajectory cloud on GCP (BigQuery, Cloud SQL, Compute Engine).
- IoHT: latencies 1/2/2/**100**/1; mobile energy unchanged; magnitude → **5-point smoothing** → features → **kNN**.

Problem & variant

- **5-point smoothing.** 3, 5, 9, 7, 6 at centre? $30/5 = $ **6**. *Variant:* 2, 4, 6, 8, 10? **6**.
- **Accelerometer magnitude.** $(3, 4, 12)$? $\sqrt{9+16+144} = $ **13**.

Recurring numerical patterns

Pattern	Template
SLA availability / credit	T = days × service hours → d = Σ outages → $A = 1 - d/T$ → band → credit → payable
Allowed downtime	$d_{max} = (1 - A)T$
Own vs cloud	$U = C/B$ vs P/A (or compare PBT with $C \int D$)
Core-hour economics	capex ÷ (years × 8760) → ÷ cores; ÷ efficiency; + power + management
Demand smoothing	$C_v/\sqrt{n}$
System failure	$1 - (1 - p)^N$
HDFS storage	Σ⌈size/block⌉ × replication
Quorum	check $R + W > N$
Parallel efficiency	$T_1/(pT_p)$; split-docs $1/(1 + 2p/nf)$
MapReduce aggregate	emit (value, count); reducer weights
Pre-copy migration	$r = PD/R$; geometric sum; downtime = last term + resume
Offloading	$\frac{C}{M}(P_c - P_i/F) - P_{tr}D/B > 0$?
PUE	total ÷ IT
Consolidation power	Σ node power; switch idle nodes off

Formula strip

→ full formula sheet

- $A = 1 - d/T$ · $d_{max} = (1 - A)T$ · Payable = bill – credit%·bill
- $C_v = \sigma/|\mu|$ · $C_v^{agg} = C_v/\sqrt{n} \cdot U = C/B < P/A$ · Penalty $\propto \int |D - R|\, dt$
- $P_{fail} = 1 - (1 - p)^N$ · blocks $= \lceil s/b \rceil \times r$ · $R + W > N$
- $\varepsilon = T_1/(pT_p) \cdot 1/p$ vs $1/(1 + 2p/nf)$ · $\varepsilon_{MR} = 1/(1 + 2c\sigma/w)$
- $T_{mig}^{cold} = V_m/R$ · $T_{mig} = \frac{V_m}{R}\frac{1-r^{n+1}}{1-r} + T_{res}$ · $T_{down} = r^n \frac{V_m}{R} + T_{res}$
- $E_{saved} = \frac{C}{M}(P_c - P_i/F) - P_{tr}D/B$ · PUE = total/IT · $P(u) = P_{idle} + (P_{max} - P_{idle})u$
- $a = \sqrt{x^2 + y^2 + z^2}$ · 5-point mean

Top traps

→ classification sheet · → high-yield list · → practice MCQs · → your practice sets

1. NIST has **5** characteristics — virtualization/multi-tenancy are **not** among them.
2. **Azure = PaaS** in the lecture; App Engine PaaS; Compute Engine IaaS.

3. Public = **low** up-front cost; on-site private = **limited** resources; outsourced private = **two** perimeters.
4. **Para**-virtualization modifies the guest; Popek–Goldberg needs **sensitive ⊆ privileged**.
5. **Valid ⇒ well-formed**; XSD **cannot** declare entities.
6. U compares with **P/A** (peak ÷ average), and C_v shrinks by $1/\sqrt{n}$.
7. GFS read = **one** replica; append = **all**, **primary** picks offset; quorum is **strictly** $>$.
8. Mapper fails → redo all; reducer fails → only unfinished.
9. **Swift = object, Cinder = block.**
10. Use **scheduled** hours for availability; lower availability → **bigger** credit.
11. **Traffic analysis** is passive; **replay** is active.
12. Gartner = **7** risks; RPO = data, RTO = time.
13. Offload only when **compute-heavy & data-light**.
14. Fog: **one hop, wireless, location-aware**; cloud: multi-hop, leased line.
15. **Bursting/broker loose, aggregated partial, multi-tier tight.**
16. Live migration: total time ↑, **downtime ↓**; **post-copy sends CPU first**.
17. **Image ≠ container**; `rmi` images vs `rm` containers; `commit` container → image.
18. Two energy splits: **DC (cooling 45 %)** vs **components (processor 45 %)**.
19. **FaaS ≠ BaaS**; serverless still uses servers.
20. In the health case the **confirmatory module stays in the cloud**.
21. **S3 = 99 %** in the lecture's SLA table (EC2 99.5 %, Microsoft 99.9 %). Your L11 set keys 99.9 %; choose 99 % whenever it is offered.
22. SLA P1: the lecture's **96.92 %** and the standard **97.01 %** can both appear as options. The key follows the lecture.
23. Meghamala's Dropbox-like service is **Meghadata** (not "Meghaduta"); its 3 OS images are **Ubuntu, CentOS, Fedora**.
24. Security goals = **prevention, detection, recovery**. "Retaliation" is a distractor.